

On-Device Training Library

Complete Beginner's Reference

Every File · Every Line · Every Concept Explained

A comprehensive guide to the ODT C library for on-device training on the STM32 Nucleo-F746ZG microcontroller. Written for beginners — every line of source code is explained in plain language, with definitions for all C syntax and concepts.

Mohamed Atef | Master's Thesis | 2026

Table of Contents

Chapter	Title	Layer
1	Introduction — What is this project and why?	Overview
2	Layer 0 — Common.h: Debug Macros	Foundation
3	Layer 1 — DTypes.h/c: Byte-Level Data Types	Primitives
4	Layer 2 — Quantization.h/c: Quantization System	Data Model
5	Layer 3 — DataStorage.h/c and Tensor.h: The Tensor	Core Data
6	Layer 5 — Arithmetic Engine (all arithmetic files)	Computation
7	Layer 6 — Neural Network Layers (all layer files)	Neural Net
8	Layer 7 — Training Infrastructure (Loss, SGD, Loop)	Training
9	Support Modules (RNG, Serialize, DataLoader, CSV, UserAPI)	Support
10	What is Missing to Implement RigL — and Why	Analysis
11	How Everything Works Together on the STM32 MCU	Integration

Introduction

What is this project and why does it exist?

1.1 The Problem: Machine Learning on a Tiny Chip

Modern machine learning models are trained on powerful servers with gigabytes of RAM and dedicated GPUs that cost thousands of dollars. But many real-world applications — wearable devices, industrial sensors, wildlife monitors, hearing aids — need to run AI locally, on a tiny chip with no internet connection and less RAM than a floppy disk.

The STM32 Nucleo-F746ZG is one such chip. It has a 32-bit ARM Cortex-M7 processor running at 216 MHz, 320 kilobytes of RAM, and 1 megabyte of flash storage. For comparison, a single image from your phone camera takes about 10 megabytes. Running even a tiny neural network on this chip is a serious engineering challenge.

This project — the On-Device Training (ODT) library — solves that problem. It is a C library (not Python, not TensorFlow) that can train and run neural networks entirely on the STM32, without any connection to a PC. The application is speaker identification from audio, as part of a Master's thesis combining two research ideas: FQT (Fixed-point Quantization Training) and RigL (sparse training).

1.2 What is FQT?

Floating-point numbers (like 0.12345678) need 32 bits each. That is too expensive for a chip with 320 KB of RAM. Fixed-point quantization replaces each 32-bit float with a smaller integer (say, an 8-bit or 16-bit number) plus a scale factor. For example, the float 0.5 might be stored as the integer 64 with a scale of 1/128. This is called SYM_INT32 quantization in the ODT library.

— ■ WORKED EXAMPLE

Example: storing the value 0.75 as SYM_INT32 with scale=0.01: mantissa = $\text{round}(0.75 / 0.01) = \text{round}(75.0) = 75$
Stored as: integer 75 (4 bytes) To recover: $75 \times 0.01 = 0.75$ With 8-bit storage (qMaxBits=8), values -127 to +127 are representable. With 16-bit storage (qMaxBits=16), values -32767 to +32767 are representable. FQT trains the network to be accurate even with this limited precision.

1.3 What is RigL?

Most neural networks are 'dense': every neuron is connected to every other neuron. But research (Evci et al., 2020 — RigL paper) shows that you can remove most connections (make the network 'sparse') and still get good accuracy. This is important on the STM32 because fewer connections means fewer multiplications, which means faster inference and less memory.

RigL works by periodically removing the smallest-magnitude weights and adding new connections where the gradients are largest. The ODT library has partial support for RigL — Chapter 10 explains exactly what is implemented and what is still missing.

1.4 The 8-Layer Architecture

The ODT library is organised in 8 layers, where each layer builds on the ones below. This document has one chapter per layer. Understanding the layers in order is the key to understanding the whole system.

Layer	Files	What it provides
0 — Foundation	Common.h	Debug macros — printing error/warning/debug messages
1 — Primitives	DTypes.h/.c	Read/write individual numbers as raw bytes
2 — Data Model	Quantization.h/.c	How numbers are stored: float, int, quantized
3 — Core Data	DataStorage.h, Tensor.h, TensorConversion.h	The tensor — the central data container
5 — Arithmetic	Arithmetic.h/.c + Add/Sub/Mul/Div/Matmul/etc.	All math operations on tensors
6 — Layers	Layer.h + Linear/Conv1d/Relu/Softmax/etc.	Neural network layers (forward + backward)
7 — Training	LossFunction.h + Optimizer.h + TrainingLoop	Loss functions, SGD optimizer, training loop
Support	RNG, Serialize, DataLoader, UserAPI	Random numbers, saving/loading, data pipeline

1.5 How to Read This Document

Each chapter follows the same structure: (1) what the file does and why it exists; (2) the complete source code with every line explained; (3) relations — which files depend on this file and which files this file uses; (4) what is missing for RigL. C syntax concepts are explained in blue boxes whenever they first appear. Read the chapters in order — later chapters assume knowledge from earlier ones.

■ BEGINNER TIP

If you are new to C: start by reading Chapter 2 very carefully. It introduces the most important C concepts (preprocessor, macros, `#ifndef`). Later chapters build on these. Every time you see a 'C CONCEPT' box, stop and read it before continuing.

Layer 0 — Common.h

The Debug Macro System

Section 2.1 — What This File Does

■ WHAT THIS FILE DOES

Common.h provides a debug printing system for the entire ODT library. It defines four macros: `PRINT_ERROR`, `PRINT_WARNING`, `PRINT_INFO`, and `PRINT_DEBUG`. Each macro calls `printf()` to output a message, but `PRINT_WARNING/INFO/DEBUG` are silenced in production builds (zero code size, zero runtime cost). `PRINT_ERROR` is always active because errors are fatal and must always be reported. Every .c file in the library includes Common.h as its first include.

Think of Common.h as the 'alarm system' of the library. During development, you can turn on verbose output to see exactly what is happening. When you compile for the final product (flashing to the STM32), all the verbose output is automatically removed by the compiler, so the firmware is as small and fast as possible.

Section 2.2 — Complete Source Code

```
// Common.h – src/common/include/Common.h
#ifndef COMMON_H // line 1
#define COMMON_H // line 2
#include <stdio.h> // line 4
#include <stdlib.h> // line 5
#include <stdbool.h> // line 6
#ifndef SOURCE_FILE // line 8
#define SOURCE_FILE "UNKNOWN" // line 9
#endif // line 10
#if defined DEBUG_MODE_DEBUG // line 12
#define DLEVEL 3 // line 13
#elif defined DEBUG_MODE_INFO // line 14
#define DLEVEL 2 // line 15
#elif defined DEBUG_MODE_WARNING // line 16
#define DLEVEL 1 // line 17
#else // line 18
#define DLEVEL 0 // line 19
#endif // line 20
#define PRINT_ERROR(str, ...) do { // line 22
printf("[%s] ERROR: " str "\n", SOURCE_FILE, ##__VA_ARGS__); \
} while(false) // line 24
#if DLEVEL >= 1 // line 26
#define PRINT_WARNING(str, ...) do { \
printf("[%s] WARNING: " str "\n", SOURCE_FILE, ##__VA_ARGS__); \
} while(false) // line 29
#else
#define PRINT_WARNING(str, ...) do { } while(false)
#endif
#if DLEVEL >= 2
#define PRINT_INFO(str, ...) do { \
printf("[%s] INFO: " str "\n", SOURCE_FILE, ##__VA_ARGS__); \
} while(false)
#else
#define PRINT_INFO(str, ...) do { } while(false)
#endif
#if DLEVEL >= 3
#define PRINT_DEBUG(str, ...) do { \
printf("[%s] DEBUG: " str "\n", SOURCE_FILE, ##__VA_ARGS__); \
} while(false)
#else
#define PRINT_DEBUG(str, ...) do { } while(false)
#endif
#endif // COMMON_H
```

Section 2.3 — Line-by-Line Explanation

■ C CONCEPT: #ifndef / #define / #endif — Header Guard

A header guard stops a .h file from being included more than once in the same compilation. Without it, if two .c files both include Common.h, the compiler would see all the definitions twice and give 'redefinition' errors. #ifndef COMMON_H → 'if the name COMMON_H is NOT yet defined...' #define COMMON_H → '...then define it now (as an empty symbol)' #endif → '...end of the conditional block' The first time Common.h is included, COMMON_H is not defined, so the block executes. The second time it is included, COMMON_H IS already defined, so the entire block is skipped. The name COMMON_H is arbitrary — it is the convention to use the filename in uppercase with dots replaced by underscores.

1	#ifndef COMMON_H	Start of the header guard. 'ifndef' = 'if not defined'. If the preprocessor symbol COMMON_H has not been defined yet, process everything until the matching #endif. This prevents the file from being processed twice.
---	------------------	--

2	<code>#define COMMON_H</code>	Define the guard symbol. After this line, <code>COMMON_H</code> is defined (its value is empty). If this file is included again later, line 1 will be false, and the whole file is skipped.
4	<code>#include <stdio.h></code>	Include the standard I/O header. This gives us access to the <code>printf()</code> function. Angle brackets <code>< ></code> mean this is a system header (part of the C standard library). Double quotes would mean a project-local file.
5	<code>#include <stdlib.h></code>	Include the standard library header. This gives us <code>exit()</code> , <code>malloc()</code> , <code>free()</code> , and more. We need <code>exit()</code> so that <code>PRINT_ERROR</code> can halt the program after a fatal error.
6	<code>#include <stdbool.h></code>	Include the boolean header. This defines the <code>bool</code> type (true/false). Without this, C has no boolean — it uses 0/1 integers instead. <code>stdbool.h</code> is a C99 addition; before C99, programmers had to define <code>bool</code> themselves.
8	<code>#ifndef SOURCE_FILE</code>	Each .c file defines <code>SOURCE_FILE</code> as its filename before including <code>Common.h</code> : <code>#define SOURCE_FILE "RELU"</code> . If a file forgets to do this, this guard provides the fallback <code>'UNKNOWN'</code> .
9	<code>#define SOURCE_FILE "UNKNOWN"</code>	Fallback value for <code>SOURCE_FILE</code> . The debug messages will show <code>[UNKNOWN]</code> as the source, which is a signal to the developer to add a proper definition.
12	<code>#if defined DEBUG_MODE_DEBUG</code>	Compile-time conditional: check if the compiler was given <code>-DDEBUG_MODE_DEBUG</code> . <code>'#if defined X'</code> is equivalent to <code>'#ifdef X'</code> . This starts a chain of three levels of debug verbosity.
13	<code>#define DLEVEL 3</code>	If <code>DEBUG_MODE_DEBUG</code> is defined, set <code>DLEVEL</code> to 3 (most verbose). <code>DLEVEL</code> controls which print macros are active: level 3 enables ALL of them.
14	<code>#elif defined DEBUG_MODE_INFO</code>	<code>'elif' = 'else if'</code> . Only checked if line 12 was false. If <code>DEBUG_MODE_INFO</code> is defined (but not <code>DEBUG_MODE_DEBUG</code>), set <code>DLEVEL=2</code> .
16	<code>#elif defined DEBUG_MODE_WARNING</code>	If only <code>DEBUG_MODE_WARNING</code> is defined, set <code>DLEVEL=1</code> (warnings only).
18	<code>#else</code>	No debug flag was defined — this is the default production build.
19	<code>#define DLEVEL 0</code>	<code>DLEVEL=0</code> means: only <code>PRINT_ERROR</code> is active. All other macros expand to empty code. The compiler sees <code>'do { } while(false)'</code> and removes it entirely. This is how the library achieves zero overhead in production.
22	<code>#define PRINT_ERROR(str, ...) do { \</code>	Define the <code>PRINT_ERROR</code> macro. It takes a format string <code>'str'</code> plus variadic arguments <code>'...'</code> (zero or more extra parameters, like <code>printf</code>). The <code>\</code> at the end continues the macro definition to the next line. <code>PRINT_ERROR</code> has NO <code>DLEVEL</code> guard — it is ALWAYS active.
23	<code>printf("[%s] ERROR: " str "\n", SOURCE_FILE, ##__VA_ARGS__); \</code>	<code>'[%s] ERROR: '</code> and <code>'str'</code> are two string literals next to each other. The C preprocessor concatenates adjacent string literals automatically. So <code>PRINT_ERROR("value is %d", x)</code> becomes <code>printf("[RELU] ERROR: value is %d\n", x)</code> . <code>##__VA_ARGS__</code> expands to the variadic arguments; <code>##</code> removes the leading comma if there are no extra arguments (prevents a syntax error for <code>PRINT_ERROR("msg")</code>).

24	<code>} while(false)</code>	The <code>do { } while(false)</code> wrapper is a safe macro pattern. It makes the macro behave like a single statement in all contexts: <code>if (err) PRINT_ERROR("bad!");</code> ← works correctly with the wrapper Without the wrapper, the macro expands to two statements, and only the first is guarded by the <code>if</code> . The <code>do-while</code> forces both into one compound statement.
26	<code>#if DLEVEL >= 1</code>	<code>PRINT_WARNING</code> is only compiled into the binary if <code>DLEVEL</code> is 1, 2, or 3. The preprocessor evaluates <code>DLEVEL >= 1</code> at compile time — this is not a runtime check.

■ C CONCEPT: The Preprocessor — What It Does

The C preprocessor runs BEFORE the compiler. It handles all lines starting with `#`. It performs three main jobs: 1. File inclusion (`#include`) — copies the entire contents of a header file in place 2. Macro expansion (`#define`) — replaces macro names with their defined text 3. Conditional compilation (`#if`, `#ifdef`, `#ifndef`) — includes or excludes blocks of code The preprocessor sees the source code as raw text. It does not understand C syntax — it only does text substitution. The result of preprocessing is given to the C compiler, which then does the actual translation to machine code.

■ C CONCEPT: Variadic Macros — ... and __VA_ARGS__

A variadic macro accepts a variable number of arguments, like `printf()`. `#define MY_PRINT(fmt, ...) printf(fmt, __VA_ARGS__)` The `...` in the parameter list means 'zero or more extra arguments'. Inside the macro body, `__VA_ARGS__` expands to those extra arguments. The `##` before `__VA_ARGS__` is a special GNU extension: it removes the comma before `__VA_ARGS__` when there are no extra arguments. Without it: `MY_PRINT("hello")` → `printf("hello", )` ← syntax error! With `##`: `MY_PRINT("hello")` → `printf("hello")` ← correct

Section 2.4 — Relations With Other Files

■ RELATIONS WITH OTHER FILES

`Common.h` is the only file with NO dependencies — it is Layer 0. Every other `.c` file in the library includes `Common.h` as its first include. This makes `PRINT_ERROR`, `PRINT_WARNING` etc. available everywhere.

File	Direction	Why
<code>Common.h</code>	→ (no dependencies)	Layer 0: depends on nothing
Every <code>.c</code> file in the library	← includes <code>Common.h</code>	All <code>.c</code> files need <code>PRINT_ERROR</code> for error reporting
<code>stdio.h</code> (C standard library)	← includes (system)	Provides <code>printf()</code> which <code>PRINT_ERROR</code> uses
<code>stdlib.h</code> (C standard library)	← includes (system)	Provides <code>exit()</code> for fatal error termination
<code>stdbool.h</code> (C standard library)	← includes (system)	Provides <code>bool</code> type for <code>do-while(false)</code> safety pattern

2.5 The SOURCE_FILE Convention

Every `.c` file in the library starts with the line:

```
#define SOURCE_FILE "FILENAME"
```

This line must appear BEFORE the `#include "Common.h"` line, because `Common.h` uses `SOURCE_FILE` in its macro definitions. When `PRINT_ERROR` is called from `Relu.c`, the output shows `[RELU]` so the developer immediately knows which file triggered the error. On a UART terminal connected to the STM32, this is often the only debugging information

available.

— ■ WORKED EXAMPLE

How a .c file starts (example from Relu.c): #define SOURCE_FILE "RELU" ← must come FIRST #include <stdlib.h> #include <string.h> #include "Common.h" ← SOURCE_FILE must be defined before this #include "Relu.h" When you see: [RELU] ERROR: Unknown QType! You immediately know: the error came from Relu.c, the switch statement fell through to default because an unexpected quantization type was used.

Section 2.5 — What Is Missing for RigL in This File

■ RigL GAP IN THIS FILE

Common.h has no RigL gap. It is a utility file that provides debug printing. RigL does not require any changes here. However, when implementing RigL operations (pruning, regrowing), the developer will use PRINT_DEBUG to log sparsity changes during training — so Common.h will naturally be used by the new RigL code.

Layer 1 — DTypes.h / DTypes.c

Byte-Level Data Types and Raw Memory Access

Section 3.1 — What This File Does

■ WHAT THIS FILE DOES

DTypes.h and DTypes.c provide the most fundamental operations in the library: reading and writing individual numbers (int32_t and float) as raw bytes. All tensor data is stored as a raw array of uint8_t (unsigned bytes). To read or write a value, you need to assemble/disassemble its bytes. DTypes.c provides safe functions to do this using memcpy. Every arithmetic operation in the library eventually calls these functions.

Why store data as bytes? Because uint8_t is the universal currency of memory in C. A tensor might hold float32 values today and int32 values tomorrow. By storing everything as raw bytes, the tensor_t struct does not need to know what type of data it holds — only the functions that read/write it need to know. This is the foundation of the library's type flexibility.

■ C CONCEPT: What is uint8_t?

uint8_t is an 8-bit unsigned integer — a single byte, holding values 0 to 255. The 'u' stands for unsigned (no negative values). '8' is the bit width. 't' is a C convention for type aliases (typedef). Data type widths: uint8_t = 1 byte = 8 bits (values 0 to 255) int32_t = 4 bytes = 32 bits (values -2,147,483,648 to +2,147,483,647) float = 4 bytes = 32 bits (IEEE 754 floating point) size_t = 4 or 8 bytes (platform-dependent, always unsigned) These types are defined in <stdint.h> (for int32_t, uint8_t) and <stddef.h> (for size_t). Using them instead of 'int' or 'long' guarantees exact sizes on all platforms.

Section 3.2 — DTypes.h: The Header File

```
// DTypes.h — src/tensor/include/DTypes.h
#ifndef ENV5_RUNTIME_DTYPES_H
#define ENV5_RUNTIME_DTYPES_H
#include <stdint.h>
#include <stddef.h>
// Read functions
int32_t readBytesAsInt32(const uint8_t *bytes);
float readBytesAsFloat(const uint8_t *bytes);
// Write functions
void writeInt32ToByteArray(int32_t value, uint8_t *bytes);
void writeFloatToByteArray(float value, uint8_t *bytes);
// Bit-level operations (for BOOL/sparse types)
void writeBitsToByteArray(uint8_t *byteArray, size_t startBit,
size_t endBit, uint8_t data);
uint8_t readBitsFromByteArray(const uint8_t *byteArray, size_t startBit,
size_t endBit);
#endif // ENV5_RUNTIME_DTYPES_H
```

The header declares 6 functions. The first four are the workhorses used everywhere. The last two (writeBitsToByteArray, readBitsFromByteArray) handle sub-byte operations for BOOL tensors (where each element takes only 1 bit) and sparse tensors. Every file that needs to read or write tensor values includes DTypes.h.

Section 3.3 — DTypes.c: The Implementation

```
// DTypes.c – src/tensor/DTypes.c
#define SOURCE_FILE "DTYPES"
#include <string.h> // for memcpy
#include <stdint.h>
#include "Common.h"
#include "DTypes.h"

int32_t readBytesAsInt32(const uint8_t *bytes) {
    int32_t value;
    memcpy(&value, bytes, sizeof(int32_t));
    return value;
}

float readBytesAsFloat(const uint8_t *bytes) {
    float value;
    memcpy(&value, bytes, sizeof(float));
    return value;
}

void writeInt32ToByteArray(int32_t value, uint8_t *bytes) {
    memcpy(bytes, &value, sizeof(int32_t));
}

void writeFloatToByteArray(float value, uint8_t *bytes) {
    memcpy(bytes, &value, sizeof(float));
}

void writeBitsToByteArray(uint8_t *byteArray, size_t startBit,
    size_t endBit, uint8_t data) {
    size_t byteIndex = startBit / 8;
    size_t bitStart = startBit % 8;
    size_t bitEnd = endBit % 8;
    uint8_t mask = ((1u << (bitEnd - bitStart)) - 1) << bitStart;
    byteArray[byteIndex] = (byteArray[byteIndex] & ~mask) | ((data << bitStart) & mask);
}

uint8_t readBitsFromByteArray(const uint8_t *byteArray, size_t startBit,
    size_t endBit) {
    size_t byteIndex = startBit / 8;
    size_t bitStart = startBit % 8;
    size_t bitEnd = endBit % 8;
    uint8_t mask = ((1u << (bitEnd - bitStart)) - 1) << bitStart;
    return (byteArray[byteIndex] & mask) >> bitStart;
}
```

Section 3.4 — Line-by-Line Explanation

3.4.1 readBytesAsInt32

■ C CONCEPT: memcpy — The Safe Way to Reinterpret Memory

`memcpy(destination, source, size)` copies 'size' bytes from source to destination. Why not just cast? You might think: why not write `*(int32_t*)bytes`? This would tell the CPU: 'treat the memory at address bytes as an `int32_t`'. But this is UNDEFINED BEHAVIOUR in C when the pointer is misaligned or the pointer types violate strict aliasing rules (the compiler can assume pointers to different types never point to the same memory). `memcpy` is the CORRECT way: it copies the raw bytes without any type assumptions. The compiler knows `memcpy` semantics and can optimize it to a single 32-bit load instruction on ARM Cortex-M7 anyway. Example: `uint8_t bytes[4] = {0x00, 0x00, 0x80, 0x3F}; // IEEE 754 for 1.0f float value; memcpy(&value, bytes, sizeof(float)); // value is now 1.0f`

1-3 `int32_t readBytesAsInt32(const uint8_t *bytes) {`

Function signature. '`const uint8_t *bytes`' means: a pointer to bytes that we promise not to modify (`const`). The '`const`' is important for correctness and lets the compiler pass read-only memory safely.

2	<code>int32_t value;</code>	Declare a local <code>int32_t</code> variable. It is uninitialized — its value is garbage until <code>memcpy</code> fills it on the next line. We do NOT initialize it to 0 because <code>memcpy</code> immediately overwrites all 4 bytes anyway.
3	<code>memcpy(&value, bytes, sizeof(int32_t));</code>	Copy <code>sizeof(int32_t) = 4</code> bytes from 'bytes' into 'value'. '&value' takes the address of the local variable — gives <code>memcpy</code> a pointer to write to. <code>sizeof(int32_t)</code> ensures we copy exactly 4 bytes regardless of the platform. After this line, 'value' holds the integer stored in <code>bytes[0..3]</code> .
4	<code>return value;</code>	Return the assembled integer. The caller receives it as a regular <code>int32_t</code> .

— ■ WORKED EXAMPLE

Example: reading the integer 305 from a byte array. 305 in binary = `0x00000131` = bytes `[0x31, 0x01, 0x00, 0x00]` (little-endian ARM) `uint8_t data[4] = {0x31, 0x01, 0x00, 0x00}; int32_t result = readBytesAsInt32(data); // result = 305`
 Little-endian means: the LEAST significant byte comes first in memory. ARM Cortex-M7 (and most modern CPUs) are little-endian. `0x31` is the least significant byte of 305; `0x00` is the most significant byte.

3.4.2 readBytesAsFloat

■ C CONCEPT: IEEE 754 — How Floats are Stored

A 32-bit float (float in C) is stored in IEEE 754 format: Bit 31: sign (0 = positive, 1 = negative) Bits 30-23: exponent (8 bits, biased by 127) Bits 22-0: mantissa (23 bits, fraction part) The value = $(-1)^{\text{sign}} \times 2^{(\text{exponent}-127)} \times 1.\text{mantissa}$
 Example: 1.0 in IEEE 754: Sign=0, Exponent=127 (stored as `01111111`), Mantissa=0 Bytes: `0x00, 0x00, 0x80, 0x3F` (little-endian) This is why `readBytesAsFloat` uses `memcpy` — we need to take the 4 raw bytes and have the CPU interpret them as a float WITHOUT changing their bit pattern.

1-3	<code>float readBytesAsFloat(const uint8_t *bytes) {</code>	Identical pattern to <code>readBytesAsInt32</code> , but returns float instead of <code>int32_t</code> .
2	<code>float value;</code>	Local float variable — 4 bytes on the stack, same as <code>int32_t</code> .
3	<code>memcpy(&value, bytes, sizeof(float));</code>	Copy 4 bytes from the array into the float variable. The CPU does NOT convert any values — it copies the exact bit pattern. If bytes holds the IEEE 754 representation of 0.5, value will be 0.5f.

3.4.3 writeInt32ToByteArray and writeFloatToByteArray

1	<code>void writeInt32ToByteArray(int32_t value, uint8_t *bytes) {</code>	Reverse operation: write an <code>int32_t</code> into a byte array. 'void' means no return value — the result is written through the pointer 'bytes'. The caller is responsible for ensuring 'bytes' points to at least 4 valid bytes.
2	<code>memcpy(bytes, &value, sizeof(int32_t));</code>	Copy 4 bytes FROM 'value' (taking its address with &) INTO 'bytes'. After this, <code>bytes[0..3]</code> hold the little-endian representation of 'value'. For <code>value=305</code> : bytes becomes <code>[0x31, 0x01, 0x00, 0x00]</code> .

3.4.4 writeBitsToByteArray — Bit-Level Operations

■ C CONCEPT: Bit Manipulation — Masks, Shifts, OR, AND

A bitmask is an integer with specific bits set to 1. Combining a mask with bitwise operations lets you set, clear, or read individual bits. Operators: & (AND) — `1 & 1 = 1`; anything & 0 = 0 → clears bits | (OR) — `0 | 1 = 1`; anything | 1 = 1 → sets bits ~ (NOT) — inverts all bits → complement << (left shift) — multiply by 2 per shift → moves bits left >> (right shift) — divide by 2 per shift → moves bits right Example: isolate bits 2-4 of byte `0b10110110`: mask = $((1 << (4-2)) - 1) << 2 = (4-1) << 2 = 3 << 2 = 0b00001100$ result = $(0b10110110 \& 0b00001100) >> 2 = 0b00000011$
 $>> 2 = 0b00000001 = 1$

2	<code>size_t byteIndex = startBit / 8;</code>	Find which byte contains the bit we want. Bits are numbered 0..N. Bit 13 is in byte $13/8 = \text{byte } 1$ (integer division).
3	<code>size_t bitStart = startBit % 8;</code>	Offset within the byte. Bit 13 is at position $13\%8 = 5$ within byte 1.
5	<code>uint8_t mask = ((1u << (bitEnd - bitStart)) - 1) << bitStart;</code>	Build a bitmask covering exactly the bits we want to write. '1u' is unsigned 1 to prevent signed overflow on shift. $(1u << \text{width}) - 1$ creates width consecutive 1-bits starting at bit 0. Shifting left by bitStart moves them to the correct position. Example: startBit=2, endBit=5: <code>mask = ((1<<3)-1) << 2 = 0b00011100</code>
6	<code>byteArray[byteIndex] = (byteArray[byteIndex] & ~mask) ((data << bitStart) & mask);</code>	Three steps in one line: 1. 'byteArray[byteIndex] & ~mask' — clear the target bits (AND with inverted mask) 2. '(data << bitStart) & mask' — place the new data in the correct bit positions 3. '...' — OR them together to get the final byte with the new bits set This is the standard 'clear-then-set' bit manipulation pattern.

Section 3.5 — Relations With Other Files

File	Direction	Why
DTypes.h/.c	→ depends on	<string.h> for memcpy, Common.h for SOURCE_FILE
Arithmetic.c	← uses DTypes	All element read/write: readBytesAsInt32/Float, writeXxx
Matmul.c	← uses DTypes	Reading matrix elements, writing results
TensorConversion.c	← uses DTypes	Reading/writing during type conversion
Sgd.c	← uses DTypes	Reading/writing weight and gradient values
Serialize.c / Deserialize.c	← uses DTypes	Writing tensor bytes to/from byte stream
Log.c / Sum.c / MinMax.c	← uses DTypes	Reading element values for reduction ops
Bernoulli.c / Distributions.c	← uses DTypes	Writing sampled values to tensors

■ RigL GAP IN THIS FILE

DTypes.h/.c has no direct RigL gap. However, RigL uses a sparsity mask (a tensor of booleans — one bit per weight). `writeBitsToByteArray` and `readBitsFromByteArray` are exactly the functions needed to work with this mask. They already exist! The gap is not in DTypes but in the higher-level RigL logic that uses them.

Layer 2 — Quantization.h / Quantization.c

The Quantization Type System

Section 4.1 — What This File Does

■ WHAT THIS FILE DOES

Quantization.h defines HOW numbers are stored in every tensor. It provides three things: (1) the `qtype_t` enum — six possible storage formats; (2) config structs — parameters for each format (scale, zero-point, bit depth); (3) init functions — create and configure quantization objects. Every tensor carries a `quantization_t` that says 'my values are stored as X'. This is Layer 2 because all higher layers need to know the storage format before they can read or write tensor values.

Understanding quantization is essential for understanding the whole library. A tensor does not just store raw numbers — it stores numbers in a compressed format. The `quantization_t` attached to each tensor tells you how to decode those numbers back to their real values.

■ C CONCEPT: What is Quantization? (Plain Language)

Quantization is the process of representing a continuous value (like 0.12345) as a small integer (like 12) plus a scale factor (0.01). Real value = integer × scale $0.12 = 12 \times 0.01$ Why bother? Because integers use less memory and are faster to compute: float (32 bits) × float (32 bits) → 32-bit result (needs FPU, 1 cycle) int8 (8 bits) × int8 (8 bits) → 16-bit result (integer mult, ~0.5 cycles) The price you pay is precision loss. 0.12345 becomes 0.12 (rounded to 2 decimals). FQT (Fixed-point Quantization Training) trains the model to be accurate despite this precision loss.

Section 4.2 — Quantization.h: The Header File

[illegible]

Section 4.3 — Line-by-Line: The `qtype_t` Enum

■ C CONCEPT: typedef enum — Defining an Enumeration Type

An enum (enumeration) is a type whose values are named integer constants. `typedef enum myEnum { A, B, C } myEnum_t; A=0, B=1, C=2` automatically (or you can assign values manually: `A=5`). 'typedef' creates a type alias so you can write 'myEnum_t x' instead of 'enum myEnum x'. Enums are useful because they let you write readable code: `if (q->type == FLOAT32) { ... } ← clear` if `(q->type == 1) { ... } ← confusing` — what does 1 mean?

INT32	INT32 – raw 32-bit integer	The value IS the integer, no scale factor. Used for: raw index tensors, intermediate accumulation buffers. Memory: 4 bytes per element. No extra config (int32QConfig_t is empty).
FLOAT32	FLOAT32 – IEEE 754 float	The standard floating-point format. Used for: weights during FLOAT32 training, gradients, activations when using float paths. Memory: 4 bytes per element. No extra config (float32QConfig_t is empty). This is the simplest and most precise option, but uses the most memory.
SYM_INT32	SYM_INT32 – symmetric fixed-point, 32-bit mantissa	Real value = mantissa × scale. The mantissa is stored as int32_t (4 bytes). Scale is a float stored in symInt32QConfig_t. qMaxBits limits the range: if qMaxBits=16, mantissas stay in [-32767, +32767]. Used for: FQT training — both weights and activations. Centred at zero — no zero-point needed.
SYM	SYM – symmetric, sub-32-bit mantissa	Same as SYM_INT32 but qMaxBits can be 8, 4, 2, etc. Allows even more aggressive compression. Used for inference-only deployments.
ASYM	ASYM – asymmetric fixed-point	Real value = (mantissa - zeroPoint) × scale. The zeroPoint allows the range to be shifted: e.g. ReLU outputs are always ≥ 0, so ASYM can map integer 0 to real 0.0 and cover [0, 2×scale×qMax] efficiently. Used for activations after ReLU in pure-integer inference.
BOOL	BOOL – 1-bit boolean	Each element is 0 or 1, stored as 1 bit (8 elements per byte). Used for: sparsity masks (which weights are active), Bernoulli dropout masks. Memory: (N + 7) / 8 bytes for N elements (ceiling division).

4.3.1 The quantization_t Struct

■ C CONCEPT: void* — The Generic Pointer

`void*` is a pointer to 'nothing in particular' — it can point to any type. You cannot dereference a `void*` directly (the compiler doesn't know the size). You must cast it to the correct type first: `void *ptr = &myCfg; // store any address`
`symInt32QConfig_t *cfg = ptr; // retrieve as specific type` In `quantization_t`, `qConfig` is `void*` because it might point to any of six config struct types depending on `q->type`. The code always checks `q->type` first, then casts `qConfig` to the appropriate type. This is a C polymorphism pattern.

Field	Type	Meaning
type	qtype_t	Which storage format: INT32, FLOAT32, SYM_INT32, SYM, ASYM, or BOOL
qConfig	void*	Pointer to the format-specific config struct. Must be cast to the right type based on 'type'. NULL for INT32 and FLOAT32 (they need no config).

— ■ WORKED EXAMPLE

How to read a SYM_INT32 tensor's scale: `quantization_t *q = tensor->quantization; // get quantization` if `(q->type == SYM_INT32)` { // check the type FIRST `symInt32QConfig_t *cfg = q->qConfig; // cast void* to correct type` `float scale = cfg->scale; // now safe to read scale` } Why check the type first? Because if `q->type` were `FLOAT32`, `qConfig` would point to a `float32QConfig_t` (an empty struct), and reading scale from it would give garbage.

4.3.2 symInt32QConfig_t — Configuration for SYM_INT32

Field	Type	Meaning
scale	float	The scale factor. $\text{real_value} = \text{mantissa} \times \text{scale}$. Updated by requantisation.
qMaxBits	uint8_t	Bit depth of the mantissa. e.g. 16 $\rightarrow$ mantissa in $[-32767, 32767]$. 8 $\rightarrow$ mantissa in $[-127, 127]$.
roundingMode	uint8_t	How to round when converting: HALF_AWAY (standard) or SR_HALF_AWAY (stochastic, reduces bias during FQT).

4.3.3 calcNumberOfBytesForData — Memory Size Calculator

This function tells you how many bytes a tensor needs based on its quantization type and number of elements. It is used when allocating tensor data buffers.

```
size_t calcNumberOfBytesForData(quantization_t *q, size_t numberOfElements) {
    switch(q->type) {
        case INT32: return numberOfElements * sizeof(int32_t); // 4N bytes
        case FLOAT32: return numberOfElements * sizeof(float); // 4N bytes
        case SYM_INT32: return numberOfElements * sizeof(int32_t); // 4N bytes
        case BOOL: return (numberOfElements + 7) / 8; // ceil(N/8) bytes
        // SYM/ASYM: depends on qMaxBits...
        default: PRINT_ERROR("Unknown qtype"); exit(1);
    }
}
```

4N byte s for I NT3 2/FL OA T32/ SY M_I NT3 2	All three use 4 bytes per element (32-bit storage).	Despite different semantics (raw int, float, scaled int), they all store one 32-bit value per element. The difference is only in how you interpret those bits.
(N+ 7)/8 for BO OL	Ceiling division: pack 8 booleans per byte.	(N+7)/8 is the ceiling of N/8. For N=10: (10+7)/8 = 17/8 = 2 bytes. Bit 0 of byte 0 is element 0; bit 7 of byte 0 is element 7; bit 0 of byte 1 is element 8.

— ■ WORKED EXAMPLE

Memory comparison for a 1000-element tensor: FLOAT32: $1000 \times 4 = 4000$ bytes (3.9 KB) SYM_INT32: $1000 \times 4 = 4000$ bytes (3.9 KB) — same space as FLOAT32! SYM (8-bit): $1000 \times 1 = 1000$ bytes (1.0 KB) — 4× smaller BOOL: $(1000+7)/8 = 125$ bytes (0.1 KB) — 32× smaller than float SYM_INT32 saves memory over FLOAT32 ONLY if $q\text{MaxBits} < 32$, or if the precision loss allows fewer bits. Otherwise the memory is the same but you gain in compute speed on integer-only hardware.

Section 4.4 — The Init Functions

Every `quantization_t` must be initialised before use. The init functions set the type field and configure the `qConfig` pointer. Here is the implementation of the most important ones:

```
// From Quantization.c:
void initInt32Quantization(quantization_t *q) {
    q->type = INT32;
    q->qConfig = NULL; // INT32 needs no config
}
void initFloat32Quantization(quantization_t *q) {
    q->type = FLOAT32;
    q->qConfig = NULL; // FLOAT32 needs no config
}
void initSymInt32QConfig(roundingMode_t rm, symInt32QConfig_t *cfg) {
    cfg->scale = 1.0f; // neutral scale: real value = mantissa × 1 = mantissa
    cfg->qMaxBits = 16; // default: 16-bit mantissa range
    cfg->roundingMode = rm;
}
void initSymInt32Quantization(quantization_t *q, symInt32QConfig_t *cfg) {
    q->type = SYM_INT32;
    q->qConfig = cfg; // store pointer to caller-provided config
}
```

q->type = INT32;	Set the type field to indicate raw integer storage.	This tells any code reading this tensor: 'read these bytes as int32_t, no conversion.'
q->qConfig = NULL;	NULL means 'no config struct'. INT32 and FLOAT32 need no extra parameters.	Any code that checks q->type first and only reads qConfig for SYM_INT32 will never accidentally dereference this NULL pointer.
cfg->scale = 1.0f;	Initialize scale to 1.0 (neutral). With scale=1.0, real = mantissa × 1 = mantissa.	The scale will be updated to a meaningful value after the first data is loaded or after the first requantisation pass. Starting at 1.0 is safe.
cfg->qMaxBits = 16;	Default: 16-bit mantissa range ([-32767, 32767]).	$qMax = 2^{(qMaxBits-1)} - 1 = 2^{15} - 1 = 32767$. This is a good balance: small enough to save memory, large enough for training precision.
q->qConfig = cfg;	Store the pointer to the caller's config struct.	The quantization_t does NOT own the config — it just stores the address. The caller must ensure cfg stays valid as long as the tensor exists.

Section 4.5 — Relations With Other Files

File	Direction	Why
Quantization.h/.c	→ depends on	DTypes.h (indirectly), stdint.h, stddef.h
Tensor.h	← includes Quantization.h	tensor_t has a quantization_t* field
TensorConversion	← uses Quantization	Reads scale, qMaxBits, type to do type conversion
Arithmetic.c / Matmul.c	← uses Quantization	Reads scale to convert int accumulations to real
Sgd.c	← uses Quantization	Checks q->type to dispatch float vs SYM_INT32 update

LinearApi.c / Conv1dApi.c	← uses Quantization	Creates quantization_t objects for each tensor slot
LossFunction.h / MSE.c	← uses Quantization	Checks tensor type for forward/backward dispatch

■ RigL GAP IN THIS FILE

Quantization.h needs no changes for basic RigL. However, the BOOL type (sparsity mask) is critical for RigL: each weight's 'active' bit is stored as a BOOL tensor. The existing BOOL infrastructure in Quantization.h/DTypes.c is sufficient. What is MISSING for RigL is not in this file but in the arithmetic layer: the mask-aware forward pass (skip multiplications for inactive weights) and the gradient tracking for inactive weights are not yet implemented.

Tensor.h / Tensor.c — Complete Deep Dive

Every struct, every function, every line explained

Section 5.1 — What These Files Do

■ WHAT THIS FILE DOES

Tensor.h defines the central data structures of the entire ODT library: `tensor_t` — a multi-dimensional array (weights, activations, gradients) `shape_t` — how to interpret the flat byte array as N-dimensional data `parameter_t` — pairs a weight tensor with its gradient tensor `sparsity_t` — hook for RigL sparse mask Tensor.c implements EVERY utility function that operates on these structs: Size calculations (bytes, bits, elements) BOOL tensor bit-packing (`getBitmask`, `readByte`, `writeByte`, `byteConversion`) Shape manipulation (`transpose`, `copy`, `print`) Tensor initialization (`setTensorValues`, `setShape`, `setParameterValues`) Tensor copy (`copyData`, `copyShape`, `copyQuantization`, `copyTensor`) Debug printing (`printTensor`, `printShape`, `print_binary_uint8`)

Section 5.2 — Tensor.h: All Structs and Typedefs

```
#ifndef ODT_TENSOR_H
#define ODT_TENSOR_H
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include "Quantization.h"
typedef void *tensorStorageId;
typedef struct shape {
    size_t numberOfDimensions;
    size_t *dimensions;
    size_t *orderOfDimensions;
} shape_t;
typedef enum { SPARSITY_TYPE_1, SPARSITY_TYPE_2 } sparsityType_t;
typedef struct sparsityConfig { } sparsityConfig;
typedef struct sparsity {
    sparsityType_t type;
    sparsityConfig *config;
} sparsity_t;
typedef struct tensor {
    uint8_t *data;
    shape_t *shape;
    quantization_t *quantization;
    sparsity_t *sparsity;
} tensor_t;
typedef struct parameter {
    tensor_t *param;
    tensor_t *grad;
} parameter_t;
```

1-2	<code>#ifndef ODT_TENSOR_H / #define ODT_TENSOR_H</code>	Include guard — prevents this header from being processed twice if included from multiple .c files. Without it, the compiler would see duplicate type definitions and raise an error.
4-6	<code>#include <stdbool.h>, <stddef.h>, <stdint.h></code>	<code>stdbool.h</code> : gives us the 'bool' type and 'true'/'false' constants. <code>stddef.h</code> : gives us 'size_t' (unsigned integer for sizes and counts). <code>stdint.h</code> : gives us 'uint8_t', 'int32_t', etc. — exact-width integer types.

9	<code>typedef void *tensorStorageId;</code>	An OPAQUE POINTER — <code>void*</code> can point to anything. This is a placeholder type for a future storage backend identifier (e.g., a flash address or DMA buffer ID). The caller cannot dereference it directly — they must pass it back to a function that knows the real type. This hides implementation details from the caller.
11-1 5	<code>shape_t struct</code>	Describes the geometry of the tensor. <code>numberOfDimensions</code> : how many axes (1D vector, 2D matrix, 3D cube...) <code>dimensions[]</code> : the SIZE of each axis (e.g. {128, 64} for a 128x64 matrix) <code>orderOfDimensions[]</code> : the LOGICAL ORDER of axes — used for zero-copy transpose. Example: a 128x64 matrix has <code>numberOfDimensions=2</code> , <code>dimensions=[128,64]</code> , <code>orderOfDimensions=[0,1]</code> (normal) or <code>[1,0]</code> (transposed).
17-2 2	<code>sparsity_t struct</code>	A hook for RigL sparsity. Currently the config struct is EMPTY — the library plans to store the mask tensor here. <code>sparsityType_t</code> enum currently has two placeholder values (<code>TYPE_1</code> , <code>TYPE_2</code>). In ODT's RigL integration the mask is stored separately as a <code>BOOL</code> <code>tensor_t</code> .
24-2 9	<code>tensor_t struct</code>	THE central struct. Four fields: <code>data</code> : raw bytes — all weight values packed as <code>uint8_t*</code> regardless of type <code>shape</code> : pointer to a <code>shape_t</code> describing dimensions <code>quantization</code> : pointer to <code>quantization_t</code> (type + scale/config) <code>sparsity</code> : <code>NULL</code> for dense tensors; points to <code>sparsity_t</code> for masked tensors All fields are POINTERS — <code>tensor_t</code> itself is just 4 pointers = 16 bytes on 32-bit.
31-3 4	<code>parameter_t struct</code>	Bundles a weight tensor and its gradient tensor together. Every trainable layer has parameters (weights, bias) as <code>parameter_t</code> . <code>param</code> points to the weight values; <code>grad</code> points to <code>dL/dw</code> (same shape). Keeping them together makes it easy to pass both to the SGD update function.

■ C CONCEPT: Why data is `uint8_t*` (not `float*` or `int32_t*`)

All tensor data is stored as a raw byte array regardless of the actual type. This is because ODT supports multiple quantization types (`FLOAT32`, `INT32`, `SYM_INT32`, `SYM 8-bit`, `ASYM`, `BOOL`) in the same codebase. If data were `float*`, you could not store 8-bit quantized values or 1-bit `BOOL`. By making it `uint8_t*`, the tensor can hold any format: `FLOAT32`: 4 bytes per element, read with `readBytesAsFloat()` `INT32`: 4 bytes per element, read with `readBytesAsInt32()` `SYM 8-bit`: 1 byte per element, read directly `BOOL`: 1 bit per element, read with `tensorBoolGet()` The type-specific read/write functions in `DTypes.c` handle the conversion safely using `memcpy` to avoid strict aliasing violations.

Section 5.3 — Element Count Functions

```
size_t calcNumberOfElementsByShape(shape_t *shape) {
    size_t numElem = 1;
    size_t numberOfDimensions = shape->numberOfDimensions;
    size_t *dimensions = shape->dimensions;
    for (size_t i = 0; i < numberOfDimensions; i++) {
        numElem *= dimensions[i];
    }
    return numElem;
}

size_t calcNumberOfElementsByTensor(tensor_t *tensor) {
    return calcNumberOfElementsByShape(tensor->shape);
}

size_t calcNumberOfElementsByParameter(parameter_t *parameter) {
    return calcNumberOfElementsByShape(parameter->param->shape);
}
```

2	size_t numElem = 1;	Start at 1 — the multiplicative identity. We multiply all dimension sizes together. Starting at 0 would give 0 always.
3-4	local variable copies of shape fields	Copying the pointer and count to local variables avoids repeated pointer dereferences inside the loop (shape->numberOfDimensions each iteration). The compiler often optimizes this anyway, but it makes code clearer.
5-7	for loop: numElem *= dimensions[i]	Multiply the running total by each dimension size. Shape {128, 64}: $1 \times 128 = 128$, then $128 \times 64 = 8192$ elements. Shape {2, 3, 4}: $1 \times 2 \times 3 \times 4 = 24$ elements.
11-12	calcNumberOfElementsByTensor	A convenience wrapper — extracts tensor->shape and calls the shape version. Callers with a tensor_t* do not need to manually access .shape.
15-16	calcNumberOfElementsByParameter	Accesses parameter->param->shape (not ->grad->shape). Both param and grad always have the same shape, so either would work. param is the canonical choice.

Section 5.4 — calcBytesPerElement and calcBitsPerElement

```
size_t calcBytesPerElement(quantization_t *quantization) {
    switch (quantization->type) {
    case INT32: return sizeof(int32_t); // 4
    case FLOAT32: return sizeof(float); // 4
    case SYM_INT32: return sizeof(int32_t); // 4
    case SYM: {
        symQConfig_t *symQC = quantization->qConfig;
        return (size_t)ceilf((float)symQC->qBits / 8.0f);
    }
    case ASYM: {
        asymQConfig_t *asymQConfig = quantization->qConfig;
        uint32_t qBits = asymQConfig->qBits;
        return ceil((float)qBits / (float)8);
    }
    case BOOL: return 1;
    default: PRINT_ERROR("Unknown QType!"); exit(1);
    }
}

size_t calcBitsPerElement(quantization_t *quantization) {
    switch (quantization->type) {
    case INT32: return sizeof(int32_t) * 8; // 32
    case FLOAT32: return sizeof(float) * 8; // 32
    case SYM_INT32: return sizeof(int32_t) * 8; // 32
    case SYM: {
        symQConfig_t *symQC = quantization->qConfig;
        return symQC->qBits; // e.g. 8
    }
    case ASYM: {
        asymQConfig_t *asymQConfig = quantization->qConfig;
        return asymQConfig->qBits;
    }
    case BOOL: return 1;
    default: PRINT_ERROR("Unknown QType!"); exit(1);
    }
}
```

3	case INT32/FLOAT32/SYM_INT32: return sizeof(int32_t/float)	sizeof() is a compile-time operator — it returns the byte size of a type. sizeof(int32_t) = 4, sizeof(float) = 4 on all ARM and x86 platforms. Using sizeof() is safer than hardcoding '4' — it adapts if the type ever changes.
---	--	--

6-8	case SYM: <code>ceilf((float)symQC->qBits / 8.0f)</code>	SYM (symmetric quantization) can have arbitrary bit widths (e.g. 4-bit, 6-bit). $qBits=8 \rightarrow 8/8=1$ byte. $qBits=4 \rightarrow 4/8=0.5 \rightarrow \text{ceil}=1$ byte. $qBits=12 \rightarrow 12/8=1.5 \rightarrow \text{ceil}=2$ bytes. <code>ceilf()</code> rounds UP to the nearest integer (float version of <code>ceil</code>). Why <code>ceilf</code> and not <code>ceil</code> ? <code>ceilf</code> takes float, <code>ceil</code> takes double. Using <code>ceilf</code> avoids an implicit float→double→float conversion, which is slightly more efficient and avoids compiler warnings.
11-12	case ASYM: uses <code>ceil()</code> (double version)	The ASYM case uses <code>ceil()</code> (double) instead of <code>ceilf()</code> (float). This is an inconsistency in the code — both work correctly since <code>qBits</code> values are small integers and there is no precision difference. On ARM Cortex-M7 with FPU, float operations are hardware-accelerated but double may call a software library — <code>ceilf()</code> is preferred.
14	case BOOL: <code>return 1</code>	BOOL returns 1 byte per element in <code>calcBytesPerElement</code> — but this is MISLEADING. BOOL tensors actually use 1 BIT per element (8 elements per byte). <code>calcBytesPerElement</code> for BOOL returns 1 as a 'minimum addressable unit', NOT the actual storage per element. Use <code>calcNumberOfBytesForData()</code> for the true byte count of BOOL tensors.
20-22	<code>calcBitsPerElement: INT32/FLOAT32/SYM_INT32</code>	<code>sizeof(int32_t) * 8 = 4 * 8 = 32</code> bits. Multiplying by 8 converts bytes to bits. Same pattern: <code>sizeof()</code> adapts to the platform.
30	case BOOL: <code>return 1</code>	BOOL tensors use exactly 1 bit per element. This is the correct value for <code>calcBitsPerElement</code> — 1 bit. The ceiling division formula $(bits + 7) / 8$ then gives the correct byte count.

Section 5.5 — `calcBytesPerTensor` and `calcNumberOfBytesForData`

```

size_t calcBitsPerTensor(tensor_t *tensor) {
    size_t bitsPerElement = calcBitsPerElement(tensor->quantization);
    size_t numElements = calcNumberOfElementsByShape(tensor->shape);
    return bitsPerElement * numElements;
}

size_t calcBytesPerTensor(tensor_t *tensor) {
    size_t bitsPerTensor = calcBitsPerTensor(tensor);
    return bitsPerTensor / 8;
}

size_t calcNumberOfBytesForData(quantization_t *q, size_t numberOfElements) {
    switch (q->type) {
        case FLOAT32: return numberOfElements * sizeof(float);
        case INT32: return numberOfElements * sizeof(int32_t);
        case SYM_INT32: return numberOfElements * sizeof(int32_t);
        case SYM: return (calcBitsPerElement(q) * numberOfElements + 7) / 8;
        case ASYM: {
            size_t bitsPerElement = calcBitsPerElement(q);
            return (bitsPerElement * numberOfElements + 7) / 8;
        }
        case BOOL: return (numberOfElements + 7) / 8;
        default: PRINT_ERROR("Unknown QType!"); exit(1);
    }
}

```

1-4	<code>calcBitsPerTensor</code>	Total bits = bits per element × number of elements. For a BOOL mask of 8192 weights: $1 \times 8192 = 8192$ bits. For a FLOAT32 tensor of 8192 weights: $32 \times 8192 = 262144$ bits = 32 KB.
-----	--------------------------------	---

7-9	calcBytesPerTensor	Converts total bits to bytes by dividing by 8. NOTE: this division truncates — for BOOL tensors with N not divisible by 8, the last partial byte is LOST. Use calcNumberOfBytesForData() with ceiling division for BOOL/SYM/ASYM.
13-15	FLOAT32/INT32/SYM_INT32 cases	These are whole-byte types: 4 bytes each. No ceiling needed — always an exact multiple of 8 bits.
16	$\text{SYM: } (\text{calcBitsPerElement}(q) * \text{numberOfElements} + 7) / 8$	Ceiling division: converts total bits to bytes, rounding UP. Example: 4-bit SYM, 3 elements = 12 bits $\rightarrow (12+7)/8 = 19/8 = 2$ bytes. Without +7: $12/8 = 1$ byte — WRONG, loses 4 bits. General formula: $(x + \text{divisor} - 1) / \text{divisor}$. Here divisor=8, so +7.
20-21	$\text{BOOL: } (\text{numberOfElements} + 7) / 8$	BOOL is 1 bit per element, so bitsPerElement=1 and the formula simplifies to just $(\text{numberOfElements} + 7) / 8$. 9 booleans $\rightarrow (9+7)/8 = 2$ bytes. 8 booleans $\rightarrow (8+7)/8 = 1$ byte. 1 boolean $\rightarrow (1+7)/8 = 1$ byte. Always rounds up.

Section 5.6 — print_binary_uint8 and getBitmask

```
void print_binary_uint8(uint8_t x) {
    printf("Byte ");
    for (int i = 7; i >= 0; --i) {
        putchar((x >> i) & 1 ? '1' : '0');
    }
    putchar('\n');
}
```

3	for (int i = 7; i >= 0; --i)	Counts DOWN from 7 to 0 so the most-significant bit (bit 7) prints first — matching human-readable binary notation (leftmost = biggest).
4	(x >> i) & 1	Shift bit i to position 0, then mask with &1 to isolate it. Result is always 0 or 1. The ternary '? 1 : 0' selects the character to print.
4	putchar(...)	Prints one character without buffering overhead. Faster than printf("%c") for single-character output.

```
uint32_t getBitmask(uint32_t startbit, uint32_t endbit) {
    uint32_t endbitInternal = endbit - (startbit / 8) * 8;
    uint32_t startbitInternal = startbit - (startbit / 8) * 8;
    uint32_t counter = 0;
    uint32_t value = 1;
    for (size_t i = 0; i < 8; i++) {
        if ((i >= startbitInternal) & (endbitInternal > i)) {
            counter += value;
        }
        value *= 2;
    }
    return counter;
}
```

2	$\text{endbitInternal} = \text{endbit} - (\text{startbit} / 8) * 8$	$(\text{startbit} / 8) * 8$ is the byte boundary — the absolute bit position of the byte that contains startbit. Subtracting it converts both startbit and endbit to positions WITHIN that byte (0–7 range).
4-5	counter=0, value=1	counter will accumulate the mask. value tracks 2^i : 1,2,4,8,16,32,64,128. Adding value to counter is the same as setting bit i in counter.
6-10	loop body	For each bit position i (0 to 7): if i falls in [startbitInternal, endbitInternal), set that bit in counter by adding value ($=2^i$). End is EXCLUSIVE: $i < \text{endbitInternal}$, not $i \leq \text{endbitInternal}$.

7	<code>(i >= startbitInternal) & (endbitInternal > i)</code>	Single & (bitwise AND), NOT && (logical AND). Both sides are always evaluated — no short-circuit. On embedded hardware this avoids a conditional branch instruction, which can be faster when branch prediction is unavailable.
10	<code>value *= 2</code>	Doubles value each iteration: 1, 2, 4, 8, 16, 32, 64, 128. Equivalent to <code>value = (1u << i)</code> but uses multiply for clarity.

— WORKED EXAMPLE

`getBitmask(5, 8)` — bits 5,6,7 of the byte: $(5/8)*8 = 0 \rightarrow \text{startbitInternal}=5, \text{endbitInternal}=8$ $i=0: 0 \geq 5?$ No. `value=2`
 $i=1: 1 \geq 5?$ No. `value=4` ... $i=5: 5 \geq 5$ AND $8 > 5?$ YES. `counter+=32. value=64` $i=6: 6 \geq 5$ AND $8 > 6?$ YES.
`counter+=64. value=128` $i=7: 7 \geq 5$ AND $8 > 7?$ YES. `counter+=128. value=256` `counter = 32+64+128 = 224 =`
`0b11100000` Bit layout: 7 6 5 4 3 2 1 0 Result: 1 1 1 0 0 0 0 0 ← bits 5,6,7 set

Section 5.7 — readByte and writeByte

```
uint8_t readByte(uint8_t data, uint8_t startbit, uint8_t endbit) {
    uint8_t bitmask = getBitmask(startbit, endbit);
    uint8_t intermediate = data & bitmask;
    intermediate >>= startbit - (startbit / 8) * 8;
    return intermediate;
}
```

2	<code>bitmask = getBitmask(startbit, endbit)</code>	Build the mask with 1s only at the target bit positions.
3	<code>intermediate = data & bitmask</code>	AND with the mask: zero out all bits outside the range, keep the bits inside the range at their original positions.
4	<code>intermediate >>= startbit - (startbit / 8) * 8</code>	Shift the extracted bits RIGHT to position 0. <code>startbit - (startbit/8)*8</code> is <code>startbitInternal</code> — the bit's position within its byte. Example: extracting bits [5,8) from <code>0b11100000</code> : after AND we have <code>0b11100000</code> ; shift right by 5 → <code>0b00000111 = 7</code> .

```
uint8_t writeByte(uint8_t existingData, uint8_t data,
    uint8_t startbit, uint8_t endbit) {
    uint8_t startbitInternal = startbit - (startbit / 8) * 8;
    uint8_t endbitInternal = endbit - (startbit / 8) * 8;
    uint8_t bitmask = getBitmask(startbitInternal, endbitInternal);
    data <<= startbitInternal;
    uint8_t intermediate = data & bitmask;
    existingData = intermediate | existingData;
    return existingData;
}
```

3-4	<code>startbitInternal, endbitInternal</code>	Convert absolute bit positions to within-byte positions (0-7). Same calculation as <code>getBitmask</code> uses internally.
5	<code>bitmask = getBitmask(...)</code>	Mask with 1s at target bit positions within the byte.
6	<code>data <<= startbitInternal</code>	Shift the data UP to the target bit positions. If we want to write value 5 (<code>0b00000101</code>) at bits [5,8), we shift left by 5: <code>0b00000101 << 5 = 0b10100000</code> .
7	<code>intermediate = data & bitmask</code>	AND with the mask to clip any bits that overflowed outside the target range. This is a safety step — if data was too large, extra bits get zeroed.
8	<code>existingData = intermediate existingData</code>	OR in the new bits. The existing bits outside the target range are preserved (OR with 0 leaves them unchanged). The target bits get the new value.

— ■ WORKED EXAMPLE

writeByte(0b00001111, 5, 5, 8) — write value 5 into bits [5,8) of 0b00001111: startbitInternal=5, endbitInternal=8
bitmask = getBitmask(5,8) = 0b11100000 data=5=0b00000101, shifted left 5: 0b10100000 intermediate =
0b10100000 & 0b11100000 = 0b10100000 result = 0b10100000 | 0b00001111 = 0b10101111 Bits [5,8) now hold
0b101 = 5. Bits [0,5) unchanged = 0b01111.

Section 5.8 — byteConversion: Bit-Width Conversion

■ C CONCEPT: What byteConversion Does

byteConversion converts N values packed at dataInBits bits each into N values packed at dataOutBits bits each.
Example: convert 4 values from 8-bit to 4-bit packing: Input: [00001010, 00000011, 00001100, 00000101] (4 bytes: 10, 3, 12, 5) Output: [00110101, 01001100, ...] (2 bytes: values packed 4 bits each) Or the reverse: 4-bit → 8-bit (dequantization). The function handles any arbitrary bit width crossing byte boundaries — e.g. 5-bit values spread across two bytes.

```
void byteConversion(uint8_t *dataIn, size_t dataInBits,
uint8_t *dataOut, size_t dataOutBits, size_t numValues) {
// Step 1: zero the output buffer
memset(dataOut, 0, (numValues * dataOutBits - 1) / 8 + 1);
size_t dataOutIndex = 0; // which output byte we are writing
size_t dataInIndex = 0; // which input byte we are reading
int dataOutStartbit = 0; // bit cursor in output
int dataInStartbit = 0; // bit cursor in input
int dataInEndbit = (int)dataInBits; // end of current input value
int dataOutEndbit = (int)dataOutBits; // end of current output slot
for (size_t i = 0; i < numValues; i++) {
while ((dataInStartbit < dataInEndbit) | (dataOutStartbit < dataOutEndbit)) {
uint8_t data = 0;
if (dataInStartbit < dataInEndbit)
data = readByte(dataIn[dataInIndex], dataInStartbit, dataInEndbit);
if (dataOutStartbit < dataOutEndbit)
dataOut[dataOutIndex] = writeByte(
dataOut[dataOutIndex], data, dataOutStartbit, dataOutEndbit);
int valuesRead = min(dataInEndbit - dataInStartbit,
8 - dataInStartbit % 8);
int valuesWritten = min(dataOutEndbit - dataOutStartbit,
8 - dataOutStartbit % 8);
int minValue = min(valuesRead, valuesWritten);
// Advance cursors
uint8_t deltaIn = minValue, deltaOut = minValue;
if (dataInStartbit == dataInEndbit) {
dataOutStartbit += valuesWritten; deltaOut = valuesWritten;
} else { dataOutStartbit += minValue; }
if (dataOutStartbit == dataOutEndbit) {
dataInStartbit += valuesRead; deltaIn = valuesRead;
} else { dataInStartbit += minValue; }
// Cross byte boundary checks
if (dataInStartbit / 8 > (dataInStartbit - deltaIn) / 8)
dataInIndex += 1;
if (dataOutStartbit / 8 > (dataOutStartbit - deltaOut) / 8)
dataOutIndex += 1;
}
// Reset cursors for next value (bit-aligned within byte)
dataInStartbit = dataInEndbit % 8;
dataInEndbit = dataInStartbit + dataInBits;
dataOutStartbit = dataOutEndbit % 8;
dataOutEndbit = dataOutStartbit + dataOutBits;
}
}
```

5	<code>memset(dataOut, 0, (numValues * dataOutBits - 1) / 8 + 1)</code>	Zero the entire output buffer before writing. Why: <code>writeByte</code> uses OR to insert bits — if the buffer has garbage, old bits would corrupt the output. Buffer size formula: ceiling of total output bits to bytes, written as $(totalBits - 1) / 8 + 1$ (avoids +7 overflow for edge cases).
7-11	cursor variables	<code>dataInIndex/dataOutIndex</code> : which BYTE is being read/written. <code>dataInStartbit/dataOutStartbit</code> : bit position WITHIN the current byte (0-7). <code>dataInEndbit/dataOutEndbit</code> : end bit of the CURRENT VALUE (not just the byte). These cursors together track exact bit-level position across byte boundaries.
14	<code>while ((dataInStartbit < dataInEndbit) (dataOutStartbit < dataOutEndbit))</code>	Loop until BOTH the input value and output slot are fully processed. Single (not): both conditions always evaluated. The loop copies bits chunk-by-chunk, where each chunk is limited by how many bits remain in the current byte on each side.
22-25	<code>valuesRead, valuesWritten, minValue</code>	<code>valuesRead</code> : how many bits can be read from the current input byte (remaining in value OR remaining in byte, whichever is smaller). <code>valuesWritten</code> : same for output. <code>minValue</code> : the actual bits transferred this iteration — the minimum of the two. This is the key: we can only move as many bits as both sides can handle.
29-34	cursor advancement logic	If input is exhausted (<code>dataInStartbit == dataInEndbit</code>): advance output by full <code>valuesWritten</code> (zero-fill the rest of the slot). If output slot is full (<code>dataOutStartbit == dataOutEndbit</code>): advance input by full <code>valuesRead</code> . Otherwise: both advance by <code>minValue</code> .
37-40	byte boundary detection	If dividing by 8 gives a different result before and after the increment, a byte boundary was crossed → increment the byte index. $dataInStartbit/8 > (dataInStartbit - deltaIn)/8$ means: the new bit position is in a LATER byte than where we were.
43-46	per-value cursor reset	After each value is fully processed, the cursors do NOT reset to 0 — they reset to the current bit position within the current byte. This preserves bit-packing across values (no byte-alignment wasted between values). $dataInStartbit = dataInEndbit \% 8$: the next value starts where this one ended, within the same byte if the value doesn't end on a byte boundary.

Section 5.9 — `tensorBoolGet` and `tensorBoolSet`

```

bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex) {
    if (tensor->quantization->type != BOOL) {
        PRINT_ERROR("tensorBoolGet called on non-BOOL tensor");
        exit(1);
    }
    size_t byteIndex = flatIndex / 8;
    size_t bitIndex = flatIndex % 8;
    return ((tensor->data[byteIndex] >> bitIndex) & 1u) != 0;
}

void tensorBoolSet(tensor_t *tensor, size_t flatIndex, bool value) {
    if (tensor->quantization->type != BOOL) {
        PRINT_ERROR("tensorBoolSet called on non-BOOL tensor");
        exit(1);
    }
    size_t byteIndex = flatIndex / 8;
    size_t bitIndex = flatIndex % 8;
    if (value) {
        tensor->data[byteIndex] |= (uint8_t)(1u << bitIndex);
    } else {
        tensor->data[byteIndex] &= (uint8_t)~(1u << bitIndex);
    }
}

```

1	tensor_t const *tensor	const: this function promises NOT to modify the tensor. The compiler enforces this — any accidental write inside would be a compile error. Good practice for all read-only functions.
2-4	type check + exit(1)	Guard clause: if called on a non-BOOL tensor, print an error and abort. exit(1): signals abnormal termination to the OS/caller. On MCU, exit() typically triggers an infinite loop or reset.
6	byteIndex = flatIndex / 8	Integer division: which byte contains this element. Element 0-7 are in byte 0. Elements 8-15 in byte 1. Etc.
7	bitIndex = flatIndex % 8	Modulo: which bit within that byte (0 = rightmost, 7 = leftmost).
8	((tensor->data[byteIndex] >> bitIndex) & 1u) != 0	Three steps: (1) shift the target bit to position 0, (2) AND with 1 to isolate it, (3) compare != 0 to get a true bool. The 1u suffix makes it unsigned — avoids undefined behavior with signed shift.
19	tensor->data[byteIndex] = (uint8_t)(1u << bitIndex)	1u << bitIndex: create a mask with 1 at the target position. OR with =: set that bit to 1, leave all others unchanged. Cast to uint8_t: suppress compiler warning about int vs uint8_t.
21	tensor->data[byteIndex] &= (uint8_t)~(1u << bitIndex)	~(1u << bitIndex): invert the mask — 0 at target, 1 everywhere else. AND with &=: force target bit to 0, leave all others unchanged.

Section 5.10 — Dimension Order: setOrderOfDims and transposeTensor

■ C CONCEPT: The orderOfDimensions Array — Zero-Copy Transpose

A tensor_t stores data in a FLAT byte array. For a 3x4 matrix the data is [row0col0, row0col1, ..., row2col3] — row-major order. To transpose (swap rows and columns), the naive approach copies all data. ODT's approach: instead of moving data, change the LOGICAL ORDER array. orderOfDimensions = [0, 1] means: 'dimension 0 first, then dimension 1' (normal). orderOfDimensions = [1, 0] means: 'dimension 1 first, then dimension 0' (transposed). The matmul code reads elements using this logical order — so the same physical data appears transposed without moving a single byte. O(1) transpose instead of O(N).

```

void setOrderOfDimsForNewTensor(size_t numberOfDimensions,
size_t *orderOfDimensions) {
for (size_t i = 0; i < numberOfDimensions; i++) {
orderOfDimensions[i] = i;
}
}
void transposeTensor(tensor_t *tensor, size_t dim0Index, size_t dim1Index) {
if (tensor->shape->numberOfDimensions < 2) {
return; // can't transpose a 1D vector
}
size_t temp = tensor->shape->orderOfDimensions[dim0Index];
tensor->shape->orderOfDimensions[dim0Index] =
tensor->shape->orderOfDimensions[dim1Index];
tensor->shape->orderOfDimensions[dim1Index] = temp;
}

```

3-5	setOrderOfDimsForNewTensor: orderOfDimensions[i] = i	Initialize the order array to identity: [0, 1, 2, ...]. This means 'no permutation' — dimension 0 is first, 1 is second, etc. Must be called when creating a new tensor before any transpose.
9-10	if (numberOfDimensions < 2) return	Guard: a 1D vector has no meaningful rows/columns to swap. Early return avoids accessing out-of-bounds indices.
12-1 5	three-way swap of orderOfDimensions	Classic three-step swap using a temporary variable temp. After this: orderOfDimensions[dim0Index] contains the old dim1 value and vice versa. The physical data in tensor->data is NOT touched.

Section 5.11 — Initialization Functions

```

void setTensorValues(tensor_t *tensor, uint8_t *data, shape_t *shape,
quantization_t *quantization, sparsity_t *sparsity) {
tensor->data = data;
tensor->shape = shape;
tensor->quantization = quantization;
tensor->sparsity = sparsity;
}
void setTensorValuesForConversion(uint8_t *data, quantization_t *q,
tensor_t *originalTensor,
tensor_t *outputTensor) {
outputTensor->data = data;
outputTensor->shape = originalTensor->shape;
outputTensor->quantization = q;
outputTensor->sparsity = originalTensor->sparsity;
}
void setParameterValues(parameter_t *parameter,
tensor_t *param, tensor_t *grad) {
parameter->param = param;
parameter->grad = grad;
}
void setShape(shape_t *shape, size_t *dims, size_t numberOfDims,
size_t *orderOfDims) {
shape->dimensions = dims;
shape->numberOfDimensions = numberOfDims;
shape->orderOfDimensions = orderOfDims;
}

```

1-6	setTensorValues	Fills in all four fields of a tensor_t from caller-provided arguments. tensor_t itself contains only POINTERS — the actual data arrays are allocated separately (as static uint8_t buffers on MCU). This function just wires the pointers together.
-----	-----------------	---

9-15	setTensorValuesForConversion	Creates an output tensor that SHARES the shape and sparsity of an existing tensor but uses a different data buffer and quantization type. Used during type conversion (e.g. FLOAT32 → SYM_INT32): the output tensor has the same shape as the input but a new byte buffer for converted values.
17-20	setParameterValues	Wires a parameter_t to its param and grad tensors. After this, parameter->param and parameter->grad point to valid tensor_t structs.
22-27	setShape	Fills in a shape_t. dims is a pointer to an array of dimension sizes. orderOfDims is a pointer to the order array (must be pre-initialized with setOrderOfDimsForNewTensor). All three are pointers — shape_t stores no data itself.

Section 5.12 — Accessor Functions

```

tensor_t *getParamFromParameter(parameter_t *parameter) {
    return parameter->param;
}
tensor_t *getGradFromParameter(parameter_t *parameter) {
    return parameter->grad;
}

```

These are trivial one-liners but they serve an important purpose in C: ENCAPSULATION. Instead of accessing `parameter->param` directly everywhere, callers use `getParamFromParameter()`. Benefits: (1) If the `parameter_t` struct changes internally, only these functions need updating. (2) The function name is self-documenting. (3) Makes the intent clear in code that uses it.

Section 5.13 — Copy Functions: copyData, copyShape, copyQuantization, copyTensor

```

void copyData(tensor_t *dest, tensor_t *src) {
    size_t numberOfValues = calcNumberOfElementsByShape(dest->shape);
    size_t sizeData = calcNumberOfBytesForData(src->quantization, numberOfValues);
    memcpy(dest->data, src->data, sizeData);
    if (src->sparsity != NULL) {
        memcpy(dest->sparsity, src->sparsity, sizeof(sparsity_t));
    }
}

```

2	numberOfValues = calcNumberOfElementsByShape(dest->shape)	Uses the DESTINATION shape — assumes src and dest have the same shape. If shapes differ, this would compute the wrong byte count.
3	sizeData = calcNumberOfBytesForData(src->quantization, numberOfValues)	Uses the SOURCE quantization to compute byte count. This is the correct byte count for the actual data in src.
4	memcpy(dest->data, src->data, sizeData)	Copies exactly sizeData bytes from src->data to dest->data. memcpy is safe for any type (byte-level copy, no aliasing issues). It is typically implemented with SIMD/LDM instructions on ARM for speed.
5-7	if (src->sparsity != NULL) memcpy sparsity	Only copy sparsity if the source has one. Copying a NULL pointer would crash. sizeof(sparsity_t) copies the full sparsity struct (type + config pointer).

```
void copyShape(shape_t *dest, shape_t *src) {
    memcpy(dest->dimensions, src->dimensions,
           src->numberOfDimensions * sizeof(size_t));
    memcpy(dest->orderOfDimensions, src->orderOfDimensions,
           src->numberOfDimensions * sizeof(size_t));
    dest->numberOfDimensions = src->numberOfDimensions;
}
```

2-3	memcpy dimensions array	Copies numberOfDimensions size_t values (each is 4 bytes on 32-bit). src->numberOfDimensions * sizeof(size_t) = total bytes to copy. dest->dimensions must already point to a valid array of at least that size.
4-5	memcpy orderOfDimensions array	Copies the logical order array — preserving any transpositions that were applied. If src was transposed, dest will be transposed in the same way.
6	dest->numberOfDimensions = src->numberOfDimensions	Copy the count last — after the arrays are copied. If this were done first and the memcpy crashed, the count would be wrong.

```
void copyQuantization(quantization_t *dest, quantization_t *src) {
    switch (src->type) {
        case FLOAT32:
            dest->type = FLOAT32;
            dest->qConfig = NULL;
            break;
        case SYM_INT32: {
            dest->type = SYM_INT32;
            symInt32QConfig_t *destQC = dest->qConfig;
            symInt32QConfig_t *srcQC = src->qConfig;
            memcpy(destQC, srcQC, sizeof(symInt32QConfig_t));
            break;
        }
        case B00L:
            dest->type = B00L;
            dest->qConfig = NULL;
            break;
        default:
            PRINT_ERROR("Unknown QType!"); exit(1);
    }
}

// TODO copy sparsity
void copyTensor(tensor_t *dest, tensor_t *src) {
    copyShape(dest->shape, src->shape);
    copyQuantization(dest->quantization, src->quantization);
    copyData(dest, src);
}
```

3-6	case FLOAT32: qConfig = NULL	FLOAT32 has no quantization config (no scale, no zero-point). Just copy the type and set qConfig to NULL. No memcpy of qConfig needed.
7-12	case SYM_INT32: memcpy the config struct	SYM_INT32 has a symInt32QConfig_t with the scale. Cast qConfig pointers to the correct type, then memcpy the full struct. sizeof(symInt32QConfig_t) copies all fields (scale, qMaxBits, qBits). dest->qConfig must already point to an allocated symInt32QConfig_t.
14-16	case B00L: qConfig = NULL	BOOL has no config — just a type flag. Same as FLOAT32 handling.
22	// TODO copy sparsity	A TODO comment in the original code — sparsity copying is not yet implemented. This means copyTensor does NOT preserve the sparsity/mask of the source tensor. For RigL, the mask must be copied separately if needed.

23-2 6	copyTensor: calls three sub-functions	Copies in order: shape, quantization config, data bytes. Shape must be copied before copyData (copyData uses dest->shape for byte count). Quantization must be copied before data (copyData uses src->quantization for bytes).
-----------	---------------------------------------	--

Section 5.14 — printTensor and printShape

```

void printTensor(tensor_t *t) {
    quantization_t *q = t->quantization;
    printf("TENSOR BEGIN \n");
    size_t numValues = calcNumberOfElementsByTensor(t);
    int32_t data[numValues]; // VLA – stack allocation!
    switch (q->type) {
        case INT32:
            printf("INT32Q \n");
            readBytesAsInt32Array(numValues, t->data, data);
            for (size_t i = 0; i < numValues; i++) printf("%i\n", data[i]);
            break;
        case FLOAT32:
            printf("FLOAT32Q \n");
            for (size_t i = 0; i < numValues; i++) {
                size_t byteIndex = i * sizeof(float);
                float currentElement = readBytesAsFloat(&t->data[byteIndex]);
                printf("%f\n", currentElement);
            }
            break;
        case SYM_INT32: {
            symInt32QConfig_t *symQC = q->qConfig;
            printf("SYM_INT32 scale=%e\n", symQC->scale);
            for (size_t i = 0; i < numValues; i++) {
                size_t byteIndex = i * sizeof(int32_t);
                int32_t currentElement = readBytesAsInt32(&t->data[byteIndex]);
                printf("%i\n", currentElement);
            }
            break; }
        case BOOL:
            printf("BOOL\n[");
            for (size_t i = 0; i < numValues; i++) {
                printf("%d", tensorBoolGet(t, i) ? 1 : 0);
                if (i + 1 < numValues) printf(", ");
            }
            printf("]\n");
            break;
        default: printf("WTF\n");
    }
    printf("TENSOR END \n_____ \n");
}

```

2	quantization_t *q = t->quantization	Store pointer locally to avoid repeated t->quantization dereferences. A small readability and micro-optimization.
5	int32_t data[numValues]; // VLA	VLA = Variable Length Array. The size numValues is NOT known at compile time. C99 allows VLAs: the array is allocated on the STACK at runtime. DANGER on MCU: if numValues is large (e.g. 8192 weights × 4 bytes = 32 KB), this silently overflows the stack and corrupts other data. printTensor is a DEBUG function only — never called during training on MCU. In production MCU code, this should be static or replaced with printf per element.
10	readBytesAsInt32Array(numValues, t->data, data)	Reads the raw byte array as int32_t values using memcpy (safe type-punning). Defined in DTypes.c. Fills the VLA 'data' array with numValues integers.

15-1 6	byteIndex = i * sizeof(float); readBytesAsFloat(...)	Each float is 4 bytes. byteIndex steps by 4: 0, 4, 8, 12, ... readBytesAsFloat uses memcpy to safely convert 4 bytes → float.
21-2 2	symInt32QConfig_t *symQC = q->qConfig; scale=%e	Cast qConfig (void*) to the correct struct type. %e format: scientific notation (e.g. 1.234e-03) — appropriate for small scales.
30-3 3	BOOL: tensorBoolGet per element + separator	Cannot use data[] array for BOOL — 1 bit per element, not 1 byte. Must use tensorBoolGet() which does the bit unpacking. i+1 < numValues prevents a trailing comma after the last element.

■ ■ IMPORTANT

The VLA 'int32_t data[numValues]' in printTensor is DANGEROUS on MCU. For a 128×64 weight tensor (8192 elements × 4 bytes = 32 KB), this allocates 32 KB on the stack — far exceeding the typical 4-8 KB MCU stack. Result: stack overflow, silent memory corruption, random crashes. printTensor should only be used on PC (for debugging). On STM32, use static arrays or print element-by-element without the VLA.

```
void printShape(shape_t *shape) {
    printf("NumberOfDims: %lu\n", shape->numberOfDimensions);
    printf("Dims: \n");
    for (size_t i = 0; i < shape->numberOfDimensions; i++)
        printf("%lu\n", shape->dimensions[i]);
    printf("OrderOfDims: \n");
    for (size_t i = 0; i < shape->numberOfDimensions; i++)
        printf("%lu\n", shape->orderOfDims[i]);
    printf("_____\n");
}
void initOrderOfDimensions(size_t *orderOfDims, size_t numberOfDims) {
    for (size_t i = 0; i < numberOfDims; i++)
        orderOfDims[i] = i;
}
```

3-4	printf("%lu", shape->dimensions[i])	%lu format: unsigned long — matches size_t on 32-bit platforms (both are 4 bytes). Prints each dimension size on a separate line.
6-8	OrderOfDims loop	Prints the logical order array — shows any transpositions. Normal: '0 1'. Transposed: '1 0'. This is the key diagnostic for debugging matmul direction issues.
12-1 4	initOrderOfDimensions	Identical to setOrderOfDimsForNewTensor — initializes orderOfDims to identity. Duplicate of setOrderOfDimsForNewTensor (the function appears twice in the codebase). Both set orderOfDims[i] = i for i in 0..numberOfDims-1.

Section 5.16 — parameter_t: Weights and Gradients Together

■ C CONCEPT: Why parameter_t Pairs param and grad

Every trainable variable in a neural network has TWO tensors that must always travel together: 1. param — the weight values W (or bias b) 2. grad — the gradient dL/dW computed during backprop Both tensors always have EXACTLY the same shape. Keeping them in one struct (parameter_t) ensures: - They are passed to SGD together (no mismatch possible) - sgdZeroGrad() can zero grad while preserving param - sgdStep() can update param using grad in one call - RigL can access both W and dL/dW to prune and grow Without parameter_t, every function that needs both would require two separate tensor_t* arguments — and a caller could accidentally pass the wrong gradient for the wrong weight.

```
typedef struct parameter {  
    tensor_t *param; // the weight tensor W (or bias b)  
    tensor_t *grad; // gradient dL/dW (same shape as param)  
} parameter_t;  
  
// ■■ How a linear layer uses parameter_t ████████████████████████████████████████████████████████████████  
// Static buffers (on MCU – no malloc):  
static uint8_t wData[128 * 64 * sizeof(float)]; // 32 KB  
static uint8_t gData[128 * 64 * sizeof(float)]; // 32 KB (gradient)  
static tensor_t wTensor, gTensor;  
static parameter_t weights;  
// Initialize shape (shared: both tensors have the same shape):  
static size_t wDims[2] = {128, 64};  
static size_t wOrder[2];  
static shape_t wShape;  
// Setup:  
setOrderOfDimsForNewTensor(2, wOrder);  
setShape(&wShape, wDims, 2, wOrder);  
// Quantization (float32 for both param and grad in float mode):  
static quantization_t wQ = {.type = FLOAT32, .qConfig = NULL};  
static quantization_t gQ = {.type = FLOAT32, .qConfig = NULL};  
// Wire up tensors:  
setTensorValues(&wTensor, wData, &wShape, &wQ, NULL);  
setTensorValues(&gTensor, gData, &wShape, &gQ, NULL);  
// Bundle into parameter_t:  
setParameterValues(&weights, &wTensor, &gTensor);  
// Access later:  
tensor_t *w = getParamFromParameter(&weights); // pointer to wTensor  
tensor_t *g = getGradFromParameter(&weights); // pointer to gTensor
```

9-10	wData[128*64*sizeof(float)], gData[128*64*sizeof(float)]	Both buffers are the same size: 128 rows × 64 cols × 4 bytes = 32 KB each. Total for one layer's weights+gradients: 64 KB. On STM32 (320 KB RAM) this is a large but manageable allocation.
14	shared shape (wShape used for both wTensor and gTensor)	Both param and grad always have the same shape. Using a SINGLE shape_t for both is fine and saves memory. setShape only stores pointers — wTensor->shape and gTensor->shape both point to the same wShape struct.
23	setTensorValues: sparsity=NULL	Dense tensors (no sparsity mask) pass NULL for the sparsity argument. The mask tensor is stored separately in linearConfig_t for RgL.
27	setParameterValues(&weights, &wTensor, &gTensor)	Wires the parameter_t: weights.param = &wTensor, weights.grad = &gTensor. After this, passing &weights to any SGD or backprop function gives it access to both the weight values and their gradients.

Operation	What it uses	How parameter_t helps
Forward pass	param only	getParamFromParameter() → weight values
Backward pass (grad calc)	param + sets grad	Both accessible from one pointer
sgdZeroGrad	grad only	Zeros grad without touching param
sgdStep	param and grad	$w = w - lr * grad$, both from one parameter_t
RigL prune	param (w) + grad (dL/dw)	Access both from parameter_t->param and ->grad
copyTensor (weight save)	param only	getParamFromParameter() then copyTensor

Section 5.17 — sparsity_t: The RigL Sparsity Hook

```

// In Tensor.h:
typedef enum { SPARSITY_TYPE_1, SPARSITY_TYPE_2 } sparsityType_t;
typedef struct sparsityConfig {
// intentionally empty — placeholder for future fields
} sparsityConfig;
typedef struct sparsity {
sparsityType_t type;
sparsityConfig *config;
} sparsity_t;
// In tensor_t:
typedef struct tensor {
uint8_t *data;
shape_t *shape;
quantization_t *quantization;
sparsity_t *sparsity; // NULL = dense; non-NULL = sparse
} tensor_t;

```

2	typedef enum { SPARSITY_TYPE_1, SPARSITY_TYPE_2 } sparsityType_t	Two placeholder sparsity types — currently no functional difference. Future use: structured sparsity (2:4 pattern) vs unstructured (arbitrary mask). RigL uses UNSTRUCTURED sparsity — any weight can be zero.
4-6	struct sparsityConfig { }	An EMPTY struct — a forward declaration placeholder. In C, an empty struct has size 1 (or 0 in some compilers). Future fields would go here: pointer to the BOOL mask tensor, sparsity ratio, update schedule, etc.
8-11	struct sparsity { type + *config }	The sparsity_t struct itself is small: one enum (4 bytes) + one pointer (4 bytes) = 8 bytes. It is included in tensor_t as a pointer (tensor_t->sparsity). Dense tensors set sparsity=NULL — no allocation needed.
17	sparsity_t *sparsity in tensor_t	The key hook. When sparsity != NULL, arithmetic functions CAN check it. In the current ODT code, the BOOL mask is NOT stored in sparsity_t — it is stored in linearConfig_t->mask (a separate tensor_t*). sparsity_t is the planned integration point for a future unified API.

■ RigL GAP IN THIS FILE

How RigL ACTUALLY uses sparsity in ODT (vs. what sparsity_t provides): Current implementation: linearConfig_t has a 'tensor_t *mask' field — a BOOL tensor with 1 bit per weight. riglMaskedMatmulFloat32() receives the mask tensor directly. The mask is NOT stored in tensor_t->sparsity. What sparsity_t would ideally provide: tensor_t->sparsity->config would contain a pointer to the BOOL mask tensor. Generic arithmetic functions could check tensor->sparsity != NULL and apply the mask automatically without a separate masked function. The sparsity_t field exists as an architectural hook — the full integration is deferred (consistent with the 'TODO copy sparsity' comment in copyTensor).

```
// How to wire sparsity_t to a mask tensor (future / planned pattern):
// Step 1: Create a BOOL mask tensor (1 bit per weight):
static uint8_t maskData[(128 * 64 + 7) / 8]; // 1 KB for 128x64 weights
static tensor_t maskTensor;
// (initialize maskTensor with BOOL quantization and same shape as weights)
// Step 2: Create a sparsityConfig that points to the mask:
// (would require adding fields to sparsityConfig):
// typedef struct sparsityConfig { tensor_t *mask; float ratio; } sparsityConfig;
// Step 3: Wire into the weight tensor's sparsity field:
static sparsityConfig_t sc = { .mask = &maskTensor, .ratio = 0.5f };
static sparsity_t sp = { .type = SPARSITY_TYPE_1, .config = &sc };
wTensor.sparsity = &sp; // now wTensor knows it is sparse
// Currently in ODT: the mask is accessed via linearConfig_t, not sparsity_t:
linearConfig.mask = &maskTensor; // direct field in layer config
```

Section 5.18 — TensorConversion.h / TensorConversion.c

■ WHAT THIS FILE DOES

TensorConversion provides ONE public function: `convertTensor(inputTensor, outputTensor)` It converts any tensor from one quantization type to any other. Internally it uses a 6x6 dispatch table (`conversionMatrix`) of function pointers. Callers never need to know which conversion function runs — just call `convertTensor()` and the right function is selected automatically.

5.18.1 — The Function Pointer Typedef

```
typedef void (*conversionFunction_t)(tensor_t *inputTensor, tensor_t *outputTensor);
```

■ C CONCEPT: Function Pointer Typedef

This line defines a TYPE called `conversionFunction_t`. `void (*conversionFunction_t)(tensor_t*, tensor_t*)` Means: 'a pointer to a function that takes two `tensor_t*` arguments and returns `void`.' The typedef gives this type a short name. Instead of writing: `void (*fn)(tensor_t*, tensor_t*)`; You write: `conversionFunction_t fn`; All 20+ conversion functions (`convertInt32ToFloat`, `convertFloatToSymInt32`, etc.) have exactly this signature — they are all valid `conversionFunction_t` values. Storing them in the `conversionMatrix` array enables O(1) dispatch.

5.18.2 — The conversionMatrix Dispatch Table

```
// Static assert: catches the bug if a new qtype_t is added without
// updating the matrix:
_Static_assert(BOOL + 1 == 6,
"extend conversionMatrix when adding qtype_t entries");
conversionFunction_t conversionMatrix[6][6] = {
  [INT32] = {
    [INT32] = NULL, // same type: handled separately
    [FLOAT32] = convertInt32TensorToFloatTensor,
    [SYM_INT32]= convertInt32TensorToSymInt32Tensor,
    [ASYM] = convertInt32TensorToAsymTensor,
    [SYM] = unsupportedConversionTypes,
    [BOOL] = unsupportedConversionTypes,
  },
  [FLOAT32] = {
    [INT32] = convertFloatTensorToInt32Tensor,
    [FLOAT32] = NULL,
    [SYM_INT32]= convertFloatTensorToSymInt32Tensor,
    [ASYM] = convertFloatTensorToAsymTensor,
    [SYM] = unsupportedConversionTypes,
    [BOOL] = unsupportedConversionTypes,
  },
  [SYM_INT32] = {
    [INT32] = extractInt32TensorFromSymInt32Tensor,
    [FLOAT32] = convertSymInt32TensorToFloat32Tensor,
    [SYM_INT32]= requantSymInt32Tensor, // re-quantize same type
    [ASYM] = convertSymInt32TensorToAsymTensor,
    [SYM] = unsupportedConversionTypes,
    [BOOL] = unsupportedConversionTypes,
  },
  // ... ASYM, SYM, BOOL rows similarly
};
```

3-4	<code>_Static_assert(BOOL + 1 == 6, ...)</code>	A compile-time assertion — evaluated by the compiler, not at runtime. <code>qtype_t</code> enum has 6 values: <code>INT32=0</code> , <code>FLOAT32=1</code> , <code>SYM_INT32=2</code> , <code>SYM=3</code> , <code>ASYM=4</code> , <code>BOOL=5</code> . <code>BOOL+1 = 6</code> confirms the enum still has exactly 6 entries. If a developer adds a new <code>qtype</code> (e.g. <code>BFLOAT16=6</code>), <code>BOOL+1</code> becomes 6 but the matrix is still <code>[6][6]</code> — the assert fires, forcing the developer to extend the matrix.
6	<code>conversionMatrix[6][6]</code>	A 6×6 array of function pointers. Row = input type (index by <code>qtype_t</code> enum value). Column = output type. <code>conversionMatrix[FLOAT32][SYM_INT32] = convertFloatTensorToSymInt32Tensor</code> . Lookup is <code>O(1)</code> — just two array indices.
8	<code>[INT32] = NULL</code> (diagonal)	<code>NULL</code> on the diagonal (same input and output type). <code>convertTensor()</code> checks for same-type BEFORE looking up the matrix and calls <code>convertTensorsWithSameType()</code> instead — so <code>NULL</code> is never dereferenced.
11-1 2	<code>unsupportedConversionTypes</code> entries	For unsupported pairs (e.g. <code>FLOAT32→BOOL</code> , <code>SYM→INT32</code>), the entry points to <code>unsupportedConversionTypes()</code> which prints an error and exits. This is safer than <code>NULL</code> : <code>NULL</code> would cause a crash with no error message; the function gives a clear diagnostic.
7	<code>[INT32] = { ... }</code> designated initializers	C99 designated initializers: <code>[INT32] = { [FLOAT32] = fn, ... }</code> explicitly names each row and column by enum value instead of position. This means the order in the source does not matter — and adding a new enum value does not shift existing entries.

5.18.3 — convertTensor: The Public Entry Point

```

void convertTensor(tensor_t *inputTensor, tensor_t *outputTensor) {
    qtype_t inputDType = inputTensor->quantization->type;
    qtype_t outputDType = outputTensor->quantization->type;
    if (inputDType == outputDType) {
        convertTensorsWithSameType(inputTensor, outputTensor, inputDType);
    } else {
        conversionFunction_t conversionFn =
            conversionMatrix[inputDType][outputDType];
        if (conversionFn == NULL) {
            PRINT_ERROR("No conversion function for %s to %s",
                quantTypeToString(inputDType), quantTypeToString(outputDType));
            exit(1);
        }
        conversionFn(inputTensor, outputTensor);
    }
}

```

2-3	read input and output types from their quantization structs	qtype_t is an enum (INT32=0, FLOAT32=1, ...). Both types are extracted first so the rest of the function is readable.
5-6	if (inputDType == outputDType) → convertTensorsWithSameType	Same-type conversion is a special case: just memcpy the data + copy the scale. This short-circuits BEFORE the matrix lookup — so the NULL diagonal entries are never reached via convertTensor.
8-9	conversionMatrix[inputDType][outputDType]	Array indexing with enum values (which are just integers 0-5). This is O(1) — one multiply (row * 6) + one add (column) + one pointer load. No if/else chain needed, regardless of how many types exist.
10-13	if (conversionFn == NULL) → error	Extra safety: even though the diagonal is handled above, a NULL in a non-diagonal slot would be a programming error (forgot to register a conversion). PRINT_ERROR + exit(1) gives a clear message instead of a null pointer crash.
14	conversionFn(inputTensor, outputTensor)	Call through the function pointer. The CPU fetches the address from the matrix, then jumps to it. Slightly slower than a direct call (one extra memory load for the address) but the code savings are enormous — 20+ if/else cases replaced by one line.

5.18.4 — convertTensorsWithSameType

```

static void convertTensorsWithSameType(tensor_t *inputTensor,
tensor_t *outputTensor,
qtype_t qType) {
size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);
size_t numberOfBytes =
calcNumberOfBytesForData(inputTensor->quantization, numberOfElements);
memmove(outputTensor->data, inputTensor->data, numberOfBytes);
switch (qType) {
case SYM_INT32: {
symInt32QConfig_t *inQC = inputTensor->quantization->qConfig;
symInt32QConfig_t *outQC = outputTensor->quantization->qConfig;
outQC->scale = inQC->scale;
break;
}
case ASYM: {
asymQConfig_t *inQC = inputTensor->quantization->qConfig;
asymQConfig_t *outQC = outputTensor->quantization->qConfig;
outQC->scale = inQC->scale;
outQC->zeroPoint = inQC->zeroPoint;
break;
}
default: break;
}
}
}

```

8	memmove instead of memcpy	memmove is used instead of memcpy because input and output MAY OVERLAP (in-place conversion: inputTensor == outputTensor is allowed). memcpy has UNDEFINED BEHAVIOR with overlapping regions — it may copy garbage. memmove is always safe with overlapping: it detects overlap and copies in the right direction.
10-1 5	SYM_INT32: copy the scale	SYM_INT32 has a scale value that must be copied alongside the data. Data bytes are already copied by memmove. The scale is in the qConfig struct — a separate allocation — so it is NOT copied by memmove on the data buffer.
16-2 2	ASYM: copy scale and zeroPoint	ASYM needs both scale and zeroPoint copied. Same reason: qConfig is pointed to, not embedded in tensor_t.
23	default: break	FLOAT32, INT32, BOOL have no qConfig fields to copy (qConfig is NULL for these types). The memmove alone is sufficient.

5.18.5 — convertFloatTensorToSymInt32Tensor (FQT Quantization)

```

void convertFloatTensorToSymInt32Tensor(tensor_t *inputTensor,
tensor_t *outputTensor) {
size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);
// Pass A: find the absolute maximum float value
float absMax = findAbsMaxFloat(inputTensor->data, numberOfElements);
symInt32QConfig_t *symInt32QC = outputTensor->quantization->qConfig;
uint8_t qMaxBits = symInt32QC->qMaxBits;
const float qMax = powf(2, (float)qMaxBits - 1) - 1; // e.g. 32767 for 16-bit
const float qMin = -powf(2, (float)qMaxBits - 1); // e.g. -32768
// Compute scale: maps absMax to qMax
float scale = (absMax == 0.f) ? 1.f : absMax / qMax;
symInt32QC->scale = scale;
// Pass B: quantize each float to int32 mantissa
int32_t *outputInt32 = (int32_t *)outputTensor->data;
float *inputFloat = (float *)inputTensor->data;
for (size_t i = 0; i < numberOfElements; i++) {
outputInt32[i] = roundByMode(
clamp(inputFloat[i] / scale, qMin, qMax),
symInt32QC->roundingMode);
}
}

```

6	absMax = findAbsMaxFloat(inputTensor->data, numberOfElements)	Pass A: scan all float values to find the maximum absolute value. This determines the scale: the quantization range must cover [-absMax, +absMax]. No writes happen in pass A — it is read-only over the input data.
11	qMax = powf(2, qMaxBits-1) - 1	qMaxBits is the training bit width (typically 16). qMax = 2 ¹⁵ - 1 = 32767. qMin = -32768. These are the representable range of the int32 mantissa during training. The mantissa uses only 16 bits of the 32-bit int32 storage — this leaves headroom for gradient accumulation without overflow.
14	scale = absMax / qMax	The scale factor converts real values ↔ mantissas. real = mantissa × scale. mantissa = real / scale. Mapping absMax → qMax means the largest real value maps exactly to the largest representable mantissa — no clipping for values in the data.
14	(absMax == 0.f) ? 1.f : ...	Guard against division by zero. If all values are 0, scale=1 (arbitrary safe choice — the mantissas will all be 0 regardless of scale).
18-1 9	int32_t* and float* casts from data	The data buffer is uint8_t* (raw bytes). Casting to int32_t* and float* allows direct indexed access. This is valid here because the buffer was allocated aligned for these types. Note: this is the DIRECT CAST pattern — technically only safe if the alignment and type match. DTypes.c uses memcpy for stricter safety.
21-2 4	roundByMode(clamp(inputFloat[i] / scale, qMin, qMax), roundingMode)	Three nested steps: 1. inputFloat[i] / scale: convert real value to mantissa (float) 2. clamp(..., qMin, qMax): ensure it fits in [-32768, 32767] 3. roundByMode(...): round float to nearest int32 (ROUND_HALF_AWAY, ROUND_HALF_EVEN, or SR_HALF_AWAY for stochastic rounding)

5.18.6 — requantSymInt32Tensor: Two-Pass Requantization


```

void requantSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {
    size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);
    symInt32QConfig_t *inputSymInt32QC = inputTensor->quantization->qConfig;
    symInt32QConfig_t *outputSymInt32QC = outputTensor->quantization->qConfig;
    // Latch inScale BEFORE writing to outputSymInt32QC->scale:
    // (in-place call: both pointers alias the same qConfig)
    float inScale = inputSymInt32QC->scale;
    const float qMax = powf(2, (float)outputSymInt32QC->qMaxBits - 1) - 1;
    const float qMin = -powf(2, (float)outputSymInt32QC->qMaxBits - 1);
    int32_t *inputInt32 = (int32_t *)inputTensor->data;
    int32_t *outputInt32 = (int32_t *)outputTensor->data;
    // Pass A: find absMax over dequantized values (READ ONLY)
    float absMax = 0.f;
    for (size_t i = 0; i < numberOfElements; i++) {
        float dequant = fabsf((float)inputInt32[i] * inScale);
        if (dequant > absMax) absMax = dequant;
    }
    float scale = (absMax == 0.f) ? 1.f : absMax / qMax;
    outputSymInt32QC->scale = scale;
    // Pass B: requantize with the fresh scale (READ-THEN-WRITE, in-place safe)
    for (size_t i = 0; i < numberOfElements; i++) {
        outputInt32[i] = roundByMode(
            clamp(((float)inputInt32[i] * inScale) / scale, qMin, qMax),
            outputSymInt32QC->roundingMode);
    }
}

```

9	float inScale = inputSymInt32QC->scale (latched before writing)	CRITICAL in-place safety. When inputTensor == outputTensor (in-place call), both qConfig pointers alias the same struct. If we wrote outputSymInt32QC->scale = newScale before reading inScale, we would use the NEW scale for pass A instead of the OLD scale — wrong results. Latching inScale first (before any write) prevents this aliasing bug.
19-2 1	Pass A: dequant = fabsf(inputInt32[i] * inScale)	Dequantize each mantissa to its real value: real = mantissa × scale. fabsf() takes the absolute value (float version of fabs). We need the maximum absolute real value to compute the new scale. Pass A is READ ONLY — no writes to outputInt32 here.
23-2 4	scale = absMax / qMax, written to outputSymInt32QC->scale	Now safe to write the new scale — pass A is complete. By construction: absMax maps exactly to qMax, so no clamping occurs. This is the Deutel (IEEE TCAD 2025) 'observe range → fresh scale → requantize' idiom.
27-3 1	Pass B: requantize each element	(float)inputInt32[i] * inScale: dequantize to real value (using the OLD scale). / scale: requantize to the NEW scale. clamp(..., qMin, qMax): fit in the representable range. roundByMode: round to integer with the configured rounding mode. In-place safe: each index i is read then overwritten — no index is read after being written.

5.18.7 — requantSymInt32TensorToScale: Fixed-Scale Requantization

```

void requantSymInt32TensorToScale(tensor_t *inputTensor, tensor_t *outputTensor) {
    symInt32QConfig_t *inputQC = inputTensor->quantization->qConfig;
    symInt32QConfig_t *outputQC = outputTensor->quantization->qConfig;
    float inScale = inputQC->scale;
    float targetScale = outputQC->scale; // caller sets this BEFORE the call
    // NaN-robust guard: !(x > 0.f) catches NaN too (unlike x <= 0.f)
    if (!(targetScale > 0.f)) {
        PRINT_ERROR("target scale must be pre-set and > 0, got %f", targetScale);
        exit(1);
    }
    const float qMax = powf(2, (float)outputQC->qMaxBits - 1) - 1;
    const float qMin = -powf(2, (float)outputQC->qMaxBits - 1);
    int32_t *inputInt32 = (int32_t *)inputTensor->data;
    int32_t *outputInt32 = (int32_t *)outputTensor->data;
    // Single pass – no scale computation needed:
    for (size_t i = 0; i < numberOfElements; i++) {
        outputInt32[i] = roundByMode(
            clamp(((float)inputInt32[i] * inScale) / targetScale, qMin, qMax),
            outputQC->roundingMode);
    }
}

```

5	float targetScale = outputQC->scale (caller pre-sets this)	KEY DIFFERENCE from requantSymInt32Tensor: the scale is NOT computed from the data. The caller sets outputQC->scale to the desired target scale BEFORE calling. This function reads it and uses it directly — no pass A needed.
8	if (!(targetScale > 0.f))	NaN-robust guard. The standard way to check for non-positive would be (x <= 0). But NaN has the property that ALL comparisons with NaN return false: NaN <= 0 is false, NaN > 0 is false. So !(x > 0) catches BOTH (x <= 0) AND (x == NaN) in one check.
21	clamp(...) SATURATES by design	Unlike requantSymInt32Tensor which never clips (scale chosen to avoid it), this function CAN clamp values that fall outside [qMin, qMax] at the target scale. This is intentional — the caller accepts saturation as the correct behavior. Example use: requantizing activations/gradients into a known layer scale.

5.18.8 — All Conversion Functions Summary

	Input	Output	Key Step
	INT32	FLOAT32	(float)inputData[i]
2	INT32	SYM_INT32	memcpy data + scale=1
	INT32	ASYM	find min/max → scale/zeroPoint → byteConversion
	FLOAT32	INT32	(int32_t)inputData[i]
2	FLOAT32	SYM_INT32	absMax→scale → clamp/round each value
	FLOAT32	SYM	min/max→scale → clamp/round → memcpy
	FLOAT32	ASYM	min/max→scale/zeroPoint → byteConversion
at32	SYM_INT32	INT32	memcpy raw int32 bytes (scale ignored)
at32	SYM_INT32	FLOAT32	(float)mantissa * scale
or	SYM_INT32	SYM_INT32	Pass A: absMax; Pass B: / newScale
ale	SYM_INT32	SYM_INT32	Single pass with caller-set targetScale
rm	SYM_INT32	ASYM	dequant→float → min/max → byteConversion
	ASYM	INT32	byteConversion(qBits→32) + zeroPoint
	ASYM	FLOAT32	byteConversion + (q + zeroPoint) * scale

32	ASYM	SYM_INT32	byteConversion + zeroPoint → scale from asym
	any	same	memset(data, 0, N*bytesPerElement)
meTy	any	same	memmove data + copy scale/zeroPoint
	any	any	Dispatch via conversionMatrix[in][out]

Section 5.15 — Complete Function Summary

Function	Returns	Purpose
calcNumberOfElementsByShape	size_t	Product of all dimension sizes
calcNumberOfElementsByTensor	size_t	Wrapper: uses tensor->shape
calcNumberOfElementsByParameter	size_t	Wrapper: uses parameter->param->shape
calcBytesPerElement	size_t	Byte size per element (type-dependent)
calcBitsPerElement	size_t	Bit width per element (1 for BOOL, 32 for float)
calcBitsPerTensor	size_t	Total bits in tensor
calcBytesPerTensor	size_t	Total bytes (use calcNumberOfBytesForData for BOOL)
calcNumberOfBytesForData	size_t	Correct byte count with ceiling division for BOOL/SYM/ASYM
tensorBoolGet	bool	Read 1 bit from BOOL tensor by flat index
tensorBoolSet	void	Write 1 bit to BOOL tensor by flat index
getBitmask	uint32_t	Build byte mask with 1s in [startbit, endbit)
readByte	uint8_t	Extract bits [startbit, endbit) from a byte, shift to position 0
writeByte	uint8_t	Insert bits into [startbit, endbit) of a byte
byteConversion	void	Convert N values from dataInBits to dataOutBits per element
print_binary_uint8	void	Print a byte as 8 binary digits (MSB first)
setOrderOfDimsForNewTensor	void	Initialize orderOfDimensions to identity [0,1,2,...]
setTensorValues	void	Wire data/shape/quantization/sparsity pointers into tensor_t
setTensorValuesForConversion	void	Create output tensor sharing shape+sparsity with input
setParameterValues	void	Wire param/grad tensors into parameter_t
setShape	void	Fill shape_t from dims/numberOfDims/orderOfDims
transposeTensor	void	Swap two axes in orderOfDimensions (O(1), no data copy)
getParamFromParameter	tensor_t*	Return parameter->param
getGradFromParameter	tensor_t*	Return parameter->grad
copyData	void	memcpy tensor data bytes + sparsity
copyShape	void	memcpy dimensions and orderOfDimensions arrays
copyQuantization	void	Copy type + qConfig struct (type-specific memcpy)
copyTensor	void	Full tensor copy: shape + quantization + data
printTensor	void	Debug print all values (switch on type; VLA — PC only!)
printShape	void	Print numberOfDimensions, dimensions[], orderOfDimensions[]

initOrderOfDimensions	void	Duplicate of setOrderOfDimsForNewTensor
-----------------------	------	---

The Arithmetic Engine — Complete Deep Dive

DTypes, Indexing, Matmul, Rounding, Sparse Matmul — Every Line Explained

Section 6.1 — How Data Lives in Memory: Endianness, Alignment, Type-Punning

■ C CONCEPT: Endianness — Byte Order in Memory

When a multi-byte value (like a 4-byte float or `int32_t`) is stored in memory, which byte comes first? Little-endian (ARM, x86): the LEAST significant byte is at the lowest address. float 1.0f = 0x3F800000 in IEEE 754 Memory at address A: [00] [00] [80] [3F] (LSB first) Big-endian (network protocols, some old CPUs): MSB first. Memory at address A: [3F] [80] [00] [00] STM32 Cortex-M7 is little-endian. So is a standard x86 PC. This means serializing a model on PC and deserializing on STM32 works without byte-swapping — ONLY because both use little-endian. If you port to a big-endian MCU, the serializer must swap bytes. ODT reads/writes bytes directly: `A->data + i*4` reads 4 bytes. These 4 bytes represent the float/int32 in little-endian format on both platforms.

■ C CONCEPT: Memory Alignment — Why Bytes Must Line Up

The ARM Cortex-M7 is fastest when data is ALIGNED: a 4-byte value (`int32_t`, float) stored at an address that is a multiple of 4. Address 0x20000000: aligned (divisible by 4) → 1 cycle to read Address 0x20000001: UNALIGNED → extra cycles or exception (bus fault) ODT avoids alignment problems by using `uint8_t*` buffers and `memcpy` instead of direct type casting: WRONG (may bus fault if buf is not 4-byte aligned): `float f = *(float*)buf; // casts uint8_t* to float* directly` CORRECT (`memcpy` handles any alignment safely): `float f; memcpy(&f, buf, 4); // readBytesAsFloat` does this Static arrays declared as `uint8_t data[N*4]` are always 1-byte aligned, which is why the `memcpy` approach is essential.

■ C CONCEPT: Type-Punning and Strict Aliasing

Type-punning: reading the same bytes as a different type. Example: reading a float's bit pattern as `uint32_t` to inspect the exponent field. In C, doing this via a pointer cast is UNDEFINED BEHAVIOR (violates strict aliasing rule): `float f = 1.0f; uint32_t bits = *(uint32_t*)&f; // UNDEFINED BEHAVIOR — strict aliasing violation` The strict aliasing rule: the C standard assumes pointers of different types never alias the same memory. This lets the compiler optimize more aggressively but means the above cast can produce wrong results with `-O2` or `-O3`. Safe alternatives: (1) `memcpy`: `float f; uint32_t bits; memcpy(&bits, &f, 4); // always safe` (2) union: `union { float f; uint32_t u; } pun; pun.f = 1.0f; bits = pun.u; // valid in C99+ ODT uses memcpy exclusively. readBytesAsFloat and writeFloatToByteArray are the safe type-punning wrappers used throughout the library.`

```
// DTypes.c – readBytesAsFloat and writeFloatToByteArray (the safe type-punning pattern):
float readBytesAsFloat(const uint8_t *bytes) {
    float f;
    memcpy(&f, bytes, sizeof(float)); // safe: memcpy avoids aliasing UB
    return f;
}
void writeFloatToByteArray(float value, uint8_t *bytes) {
    memcpy(bytes, &value, sizeof(float));
}
int32_t readBytesAsInt32(const uint8_t *bytes) {
    int32_t v;
    memcpy(&v, bytes, sizeof(int32_t));
    return v;
}
void writeInt32ToByteArray(int32_t value, uint8_t *bytes) {
    memcpy(bytes, &value, sizeof(int32_t));
}
```

1	<code>float readBytesAsFloat(const uint8_t *bytes)</code>	Takes a pointer to raw bytes (<code>uint8_t*</code>). The <code>const</code> qualifier promises 'I will not modify these bytes.' Returns a float value.
2-3	<code>float f; memcpy(&f, bytes, sizeof(float));</code>	Declares a local float variable (allocated on stack). <code>memcpy</code> copies exactly 4 bytes from the input buffer into <code>f</code> . This is the ONLY safe way to interpret raw bytes as a float in C.
10	<code>memcpy(bytes, &value, sizeof(float));</code>	Reverse: copies the 4 bytes of 'value' into the output buffer. <code>sizeof(float) = 4</code> on all platforms used by ODT.

Section 6.2 — BOOL Tensors: Bit Manipulation for Masks

```
// DTypes.c – writing/reading individual bits in a byte array:
void writeBitsToByteArray(uint8_t *ba, size_t start, size_t end, uint8_t data) {
    size_t byteIdx = start / 8; // which byte contains 'start'
    uint8_t bitOff = (uint8_t)(start % 8); // bit offset within that byte
    uint8_t numBits = (uint8_t)(end - start); // number of bits to write
    uint8_t mask = (uint8_t)((1u << numBits) - 1u); // e.g. 1 bit: mask=0b00000001
    ba[byteIdx] &= ~(mask << bitOff); // clear the target bits
    ba[byteIdx] |= (data & mask) << bitOff; // set them to 'data'
}
uint8_t readBitsFromByteArray(const uint8_t *ba, size_t start, size_t end) {
    size_t byteIdx = start / 8;
    uint8_t bitOff = (uint8_t)(start % 8);
    uint8_t numBits = (uint8_t)(end - start);
    uint8_t mask = (uint8_t)((1u << numBits) - 1u);
    return (ba[byteIdx] >> bitOff) & mask;
}
// tensorBoolGet / tensorBoolSet use these with start=i, end=i+1 (1 bit):
bool tensorBoolGet(tensor_t *t, size_t i) {
    return (bool)readBitsFromByteArray(t->data, i, i+1);
}
void tensorBoolSet(tensor_t *t, size_t i, bool val) {
    writeBitsToByteArray(t->data, i, i+1, val ? 1u : 0u);
}
```

3	<code>size_t byteIdx = start / 8;</code>	Integer division by 8: bit index -> byte index. Bit 0 is in byte 0. Bit 9 is in byte 1 ($9/8=1$). Bit 15 is in byte 1 ($15/8=1$).
4	<code>uint8_t bitOff = start % 8;</code>	Position within the byte. Bit 9: <code>bitOff = 9 % 8 = 1</code> (second bit of byte 1).
6	<code>mask = (1u << numBits) - 1u;</code>	Creates a bitmask with exactly <code>numBits</code> ones at the low end. 1 bit: $(1 << 1) - 1 = 0b00000001$. 3 bits: $(1 << 3) - 1 = 0b00000111$.

7	<code>ba[byteIdx] &= ~(mask << bitOff);</code>	Clear the target bit positions. ~inverts: <code>~0b00000001 = 0b11111110</code> . AND with the byte: only the target bit is cleared, rest unchanged.
8	<code>ba[byteIdx] = (data & mask) << bitOff;</code>	Set the target bits to 'data'. First mask off any extra bits in data, shift to the correct position, OR into the byte.
12	<code>return (ba[byteIdx] >> bitOff) & mask;</code>	Read the bits: shift right to position 0, mask off surrounding bits. Result: the bit value (0 or 1 for 1-bit reads).

— ■ WORKED EXAMPLE

Reading bit 9 from buffer `[0xFF, 0b10110101]`: `byteIdx = 9/8 = 1`, `bitOff = 9%8 = 1`, `numBits=1`, `mask=0b00000001`
`ba[1] = 0b10110101` `ba[1] >> 1 = 0b01011010` `0b01011010 & 0b00000001 = 0` (bit 9 is 0)

Section 6.3 — getDimensionsByIndex: Logical vs Physical Dimensions

■ C CONCEPT: Logical vs Physical Dimension — The Transpose Problem

A freshly created `[3,4]` tensor has: `dimensions[] = {3, 4}` `orderOfDimensions = {0, 1}` (identity mapping)
`getDimensionsByIndex(t, 0)` returns 3 (rows), `getDimensionsByIndex(t, 1)` returns 4 (cols). After `transposeTensor(t, 0, 1)` (swap dim 0 and 1): `dimensions[] = {4, 3}` (physically swapped in array) `orderOfDimensions = {1, 0}` (logical dim 0 -> physical slot 1)
`getDimensionsByIndex(t, 0)`: Searches `orderOfDimensions` for the slot where value == 0.
`orderOfDimensions[1] == 0` -> returns `dimensions[1] = 3` This correctly returns 3 (the original rows) as logical dimension 0, even though the data is not moved — only the mapping changed.

```
size_t getDimensionsByIndex(tensor_t *tensor, size_t index) {
    size_t numberOfDims = tensor->shape->numberOfDimensions;
    for (size_t i = 0; i < numberOfDims; i++) {
        if (tensor->shape->orderOfDimensions[i] == index) {
            return tensor->shape->dimensions[i];
        }
    }
    PRINT_ERROR("Tensor doesn't have %lu dimensions!", index);
    exit(1);
}
```

1	<code>size_t numberOfDims = tensor->shape->numberOfDimensions;</code>	Cache the number of dimensions to avoid repeated pointer dereferences. Compiler may optimize this away, but it makes the loop condition cleaner.
2-4	<code>for (...) { if (orderOfDimensions[i] == index) return dimensions[i]; }</code>	Linear search: scan each physical slot until we find the one that maps to 'index'. When found, return the size stored at that physical slot. $O(N)$ where N = number of dimensions — fine for $N \leq 4$.
7-8	<code>PRINT_ERROR(...); exit(1);</code>	If index not found: caller asked for a dimension that doesn't exist. <code>PRINT_ERROR</code> is a macro that optionally compiles to <code>printf</code> (debug) or nothing (release). <code>exit(1)</code> is used as an error handler — on STM32, this would trigger a hard fault.

Section 6.4 — calcTensorIndexByIndices: N-D to Flat Index

```
// Converts multi-dimensional indices to a flat (linear) memory index:
size_t calcTensorIndexByIndices(tensor_t *tensor, size_t *indices) {
    size_t numberOfDims = tensor->shape->numberOfDimensions;
    size_t flatIndex = 0;
    size_t stride = 1;
    for (int i = (int)numberOfDims - 1; i >= 0; i--) {
        size_t physI = /* find physical slot for logical dim i */;
        flatIndex += indices[i] * stride;
        stride *= tensor->shape->dimensions[physI];
    }
    return flatIndex;
}
```

■ C CONCEPT: Row-Major (C-order) Indexing

For a 2D tensor [rows, cols], the flat index is: $\text{flat} = \text{row} * \text{cols} + \text{col}$ For a 3D tensor [D0, D1, D2]: $\text{flat} = i_0 * (D_1 * D_2) + i_1 * D_2 + i_2$ The loop computes this from the INNERMOST dimension outward: Start: stride=1, process last dim ($i=D-1$) $\text{flatIndex} += \text{indices}[\text{last}] * 1$ stride = $D[\text{last}]$ Next: $\text{flatIndex} += \text{indices}[\text{second-last}] * D[\text{last}]$ stride = $D[\text{second-last}] * D[\text{last}]$... and so on. After the loop: flatIndex = multi-dim index mapped to a single offset into the 1D data array.

Section 6.5 — calcIndicesByRowIndex: Flat to N-D

```
// Inverse: given a flat index, compute the N-dimensional indices:
void calcIndicesByRowIndex(tensor_t *tensor, size_t rowIndex, size_t *indices) {
    size_t numberOfDims = tensor->shape->numberOfDimensions;
    size_t remaining = rowIndex;
    // Process from LAST dimension (innermost) to FIRST:
    for (int i = (int)numberOfDims - 1; i >= 0; i--) {
        size_t dimSize = getDimensionsByIndex(tensor, (size_t)i);
        indices[i] = remaining % dimSize; // index within this dim
        remaining /= dimSize; // remove this dimension
    }
}
```

1	size_t remaining = rowIndex;	We'll consume the flat index dimension by dimension, extracting one index per iteration.
4	size_t dimSize = getDimensionsByIndex(tensor, i);	Get the SIZE of logical dimension i. Uses getDimensionsByIndex to handle transposed tensors correctly.
5	indices[i] = remaining % dimSize;	The modulo gives the coordinate within dimension i. Example: flat=7 in shape [3,4]: dim 1 (cols=4): $7\%4=3 \rightarrow \text{col}=3$.
6	remaining /= dimSize;	Integer division removes the contribution of dimension i. Continuing: $\text{remaining}=7/4=1 \rightarrow \text{dim 0: } 1\%3=1 \rightarrow \text{row}=1$. Result: flat 7 in [3,4] = [1,3].

Section 6.6 — doDimensionsMatch: Shape Validation Before Operations


```
// Check if two tensors have identical LOGICAL shapes:
bool doDimensionsMatch(tensor_t *a, tensor_t *b) {
    if (a->shape->numberOfDimensions != b->shape->numberOfDimensions)
        return false;
    size_t nDims = a->shape->numberOfDimensions;
    for (size_t i = 0; i < nDims; i++) {
        if (getDimensionsByIndex(a, i) != getDimensionsByIndex(b, i))
            return false;
    }
    return true;
}
```

2-3	numberOfDimensions check	First check: tensors must have the same number of dimensions. A 2D tensor cannot match a 3D tensor regardless of sizes.
5-8	getDimensionsByIndex comparison per dim	Use getDimensionsByIndex (not raw dimensions[]) so transpose is handled. A [3,4] tensor and a transposed [4,3] viewed as [3,4] would match — correct, since they have the same logical shape.
9	return true;	All dimensions match. Called before every element-wise arithmetic operation to ensure operand shapes are compatible.

Section 6.7 — int32PointWiseArithmetic: Function Pointer Dispatch

■ C CONCEPT: Function Pointers in C — The Dispatch Pattern

A function pointer holds the ADDRESS of a function. It allows calling different functions through the same variable:

```
typedef int32_t (*int32BinaryOp_t)(int32_t a, int32_t b);
int32_t addOp(int32_t a, int32_t b) { return a + b; }
int32_t mulOp(int32_t a, int32_t b) { return a * b; }
int32BinaryOp_t op = addOp; // op points to addOp
int32_t result = op(3, 4); // calls addOp(3,4) = 7
op = mulOp; result = op(3, 4); // calls mulOp(3,4) = 12
```

int32PointWiseArithmetic uses this to implement add, subtract, multiply, divide, max with ONE loop body instead of five separate loops. The caller passes the operation function; the loop calls it for each element.

```
typedef int32_t (*int32BinaryOp_t)(int32_t, int32_t);
void int32PointWiseArithmetic(tensor_t *a, tensor_t *b,
    tensor_t *out, int32BinaryOp_t op) {
    size_t N = calcNumberOfElementsByTensor(a);
    if (!doDimensionsMatch(a, b)) {
        PRINT_ERROR("Shape mismatch in pointwise arithmetic");
        return;
    }
    for (size_t i = 0; i < N; i++) {
        int32_t aVal = readBytesAsInt32(a->data + i*4);
        int32_t bVal = readBytesAsInt32(b->data + i*4);
        int32_t res = op(aVal, bVal);
        writeInt32ToByteArray(res, out->data + i*4);
    }
}
// Usage: add two int32 tensors:
int32_t addInt32Op(int32_t a, int32_t b) { return a + b; }
int32PointWiseArithmetic(&tensorA, &tensorB, &out, addInt32Op);
```

1	typedef int32_t (*int32BinaryOp_t)(int32_t, int32_t);	Defines the function pointer type. Any function that takes two int32_t arguments and returns int32_t can be passed as the 'op' parameter.
4	size_t N = calcNumberOfElementsByTensor(a);	Get total element count. Works for any shape.
5-7	if (!doDimensionsMatch(a, b)) { ... return; }	Safety check: a and b must have the same shape. If mismatched, print an error and return without operating. Better than a silent wrong result or crash.

8-12	for loop body	Reads element <i>i</i> from <i>a</i> and <i>b</i> (using safe memcpy wrappers), applies the operation function pointer, writes the result to <i>out</i> . The data pointer arithmetic: <i>a->data + i*4</i> = pointer to the <i>i</i> -th int32 element.
12	int32_t res = op(aVal, bVal);	This is the function pointer call. At runtime, the CPU jumps to whatever function was passed as 'op'. One indirect branch per element — the compiler cannot inline this, so there is a small overhead vs a direct call.

Section 6.8 — Element-Wise Add Variants

```
// 1. Add two INT32 tensors element-wise into 'out':
void addInt32s(tensor_t *a, tensor_t *b, tensor_t *out) {
    size_t N = calcNumberOfElementsByTensor(a);
    for (size_t i = 0; i < N; i++) {
        int32_t aVal = readBytesAsInt32(a->data + i*4);
        int32_t bVal = readBytesAsInt32(b->data + i*4);
        writeInt32ToByteArray(aVal + bVal, out->data + i*4);
    }
}

// 2. Add two FLOAT32 tensors IN-PLACE (a += b):
void addFloat32TensorsInplace(tensor_t *a, tensor_t *b) {
    size_t N = calcNumberOfElementsByTensor(a);
    for (size_t i = 0; i < N; i++) {
        float aVal = readBytesAsFloat(a->data + i*4);
        float bVal = readBytesAsFloat(b->data + i*4);
        writeFloatToByteArray(aVal + bVal, a->data + i*4);
    }
}

// 3. Add INT32 tensor to SYM_INT32 tensor in-place (gradient accumulation):
void addInt32TensorToSymInt32TensorInplace(tensor_t *dst, tensor_t *src) {
    symInt32QConfig_t *qc = (symInt32QConfig_t *)dst->quantization->qConfig;
    size_t N = calcNumberOfElementsByTensor(dst);
    for (size_t i = 0; i < N; i++) {
        int32_t dVal = readBytesAsInt32(dst->data + i*4);
        int32_t sVal = readBytesAsInt32(src->data + i*4);
        // Accumulate: dst mantissa += src mantissa (same scale assumed)
        writeInt32ToByteArray(dVal + sVal, dst->data + i*4);
    }
}
```

11	void addFloat32TensorsInplace(tensor_t *a, tensor_t *b)	'Inplace' means <i>a</i> is BOTH input and output: <i>a</i> = <i>a</i> + <i>b</i> . No separate output buffer needed. Saves memory.
15	writeFloatToByteArray(aVal + bVal, a->data + i*4);	Write back to <i>a->data</i> (same buffer as input). This is safe because we read <i>aVal</i> before writing — no risk of reading overwritten data since the loop processes one element at a time.
19	symInt32QConfig_t *qc = (symInt32QConfig_t *)dst->quantization->qConfig;	Cast void* <i>qConfig</i> to the actual type. This is the standard cast pattern in ODT. The caller guarantees <i>dst</i> is SYM_INT32. The cast is safe because <i>initSymInt32Quantization</i> stored a <i>symInt32QConfig_t*</i> in <i>qConfig</i> .
23-24	dVal + sVal mantissa accumulation	Gradient accumulation: multiple backward passes add their gradients. Mantissas are added directly — valid ONLY if both tensors have the same scale. After all batches in an epoch, the accumulated gradient is divided by <i>N</i> batches.

Section 6.9 — matmulFloat32TensorsWithBias: Complete Line-by-Line

```

void matmulFloat32TensorsWithBias(tensor_t *A, tensor_t *B,
    tensor_t *out, tensor_t *bias) {
    size_t M = getDimensionsByIndex(A, 0); // rows of A (batch size)
    size_t K = getDimensionsByIndex(A, 1); // cols of A = rows of B
    size_t N = getDimensionsByIndex(B, 0); // rows of B (output neurons)
    for (size_t i = 0; i < M; i++) { // loop over batch samples
        for (size_t j = 0; j < N; j++) { // loop over output neurons
            float sum = 0.0f;
            for (size_t k = 0; k < K; k++) { // dot product over K
                float aVal = readBytesAsFloat(A->data + (i*K + k)*4);
                float bVal = readBytesAsFloat(B->data + (j*K + k)*4);
                sum += aVal * bVal;
            }
            float biasVal = readBytesAsFloat(bias->data + j*4);
            writeFloatToByteArray(sum + biasVal, out->data + (i*N + j)*4);
        }
    }
}

```

2-4	M, K, N dimensions	M = batch size (rows of A). K = shared dimension (input features). N = output neurons. B is stored TRANSPOSED: shape [N, K] so that B->data + (j*K+k)*4 gives weight[j][k] (j-th output neuron, k-th input).
5	for i in M	Outer loop: one iteration per sample in the batch. For batch=1 (single sample): runs once.
6	for j in N	For each output neuron, we compute one dot product. Output neuron j receives the weighted sum of all K input features.
7	float sum = 0.0f;	Accumulator initialized to 0 before computing dot product for output[i][j].
8-11	float aVal = ...; float bVal = ...; sum += aVal * bVal;	Inner loop: element-wise multiply and accumulate. A->data + (i*K+k)*4: element at row i, column k of A. B->data + (j*K+k)*4: element at row j, column k of B (B is transposed). On ARM Cortex-M7, the compiler generates VFMA instructions (1 cycle each).
12	float biasVal = readBytesAsFloat(bias->data + j*4);	The bias is a 1D tensor of N values, one per output neuron. bias[j] is added to every sample's j-th output.
13	writeFloatToByteArray(sum + biasVal, out->data + (i*N+j)*4);	Store result at out[i][j]: row i (sample), column j (neuron). Flat index: i*N + j.

Section 6.10 — matmulSymInt32TensorsWithBias: The FQT Matmul Core

■ C CONCEPT: Why int64_t? — Preventing Overflow in Quantized Matmul

In SYM_INT32, each element is an int32_t mantissa. When we multiply two int32_t values: max int32_t = 2,147,483,647 ≈ 2.1 billion $2.1 \text{ billion} \times 2.1 \text{ billion} = 4.6 \times 10^{18}$ max int64_t = 9.2×10^{18} (just fits) max int32_t = 2.1×10^9 (FAR too small) But the accumulator SUMS K products: With qMaxBits=16: each mantissa $\leq 32,767$ $32,767 \times 32,767 = 1,073,676,289 \approx 10^9$ (fits in int32_t) But K=128 such products: $128 \times 10^9 \approx 10^{11}$ — overflows int32_t! int64_t max = $9.2 \times 10^{18} \gg 10^{11}$ — safe for K up to millions. This is why the accumulator is int64_t and qMaxBits=16 (not 31): we need the product of two mantissas to fit in int64_t without overflow.

```

void matmulSymInt32TensorsWithBias(tensor_t *A, tensor_t *B,
tensor_t *out, tensor_t *bias) {
symInt32QConfig_t *qcA = (symInt32QConfig_t *)A->quantization->qConfig;
symInt32QConfig_t *qcB = (symInt32QConfig_t *)B->quantization->qConfig;
float outScale = qcA->scale * qcB->scale; // combined scale
size_t M = getDimensionsByIndex(A, 0);
size_t K = getDimensionsByIndex(A, 1);
size_t N = getDimensionsByIndex(B, 0);
for (size_t i = 0; i < M; i++) {
for (size_t j = 0; j < N; j++) {
int64_t acc = 0; // 64-bit accumulator!
for (size_t k = 0; k < K; k++) {
int32_t aVal = readBytesAsInt32(A->data + (i*K+k)*4);
int32_t bVal = readBytesAsInt32(B->data + (j*K+k)*4);
acc += (int64_t)aVal * (int64_t)bVal; // widening multiply
}
// Read bias (float or symint32), add:
float biasReal = readBytesAsFloat(bias->data + j*4);
float outReal = (float)acc * outScale + biasReal;
// Requantize result to out's quantization type:
symInt32QConfig_t *qcOut = (symInt32QConfig_t *)out->quantization->qConfig;
int32_t outMantissa = (int32_t)(outReal / qcOut->scale);
// Clamp to valid range:
int32_t qMax = (1 << (qcOut->qMaxBits - 1)) - 1;
if (outMantissa > qMax) outMantissa = qMax;
if (outMantissa < -qMax) outMantissa = -qMax;
writeInt32ToByteArray(outMantissa, out->data + (i*N+j)*4);
}
}
}

```

2-3	qcA, qcB – cast qConfig to symInt32QConfig_t*	Extract the scale from each tensor's quantization config. The cast (symInt32QConfig_t*) is safe because the caller guarantees both tensors use SYM_INT32 quantization.
4	float outScale = qcA->scale * qcB->scale;	The scale of the result: $\text{real_out} = (\text{mantissa_A} * \text{mantissa_B}) * (\text{scaleA} * \text{scaleB})$. Scales multiply because: $(\text{mantissa_A} * \text{scaleA}) * (\text{mantissa_B} * \text{scaleB}) = (\text{mantissa_A} * \text{mantissa_B}) * (\text{scaleA} * \text{scaleB})$.
9	int64_t acc = 0;	64-bit accumulator to hold the sum of up to K products. Each product is at most $32767^2 \approx 10^9$. K=128 products: $\text{sum} \leq 128 \times 10^9 \approx 1.3 \times 10^{11}$. int64_t max $\approx 9.2 \times 10^{18}$ — vastly sufficient.
13	acc += (int64_t)aVal * (int64_t)bVal;	The casts (int64_t) widen BEFORE multiplication. Without them: $\text{int32_t} \times \text{int32_t} = \text{int32_t}$ (overflow). With them: $\text{int64_t} \times \text{int64_t} = \text{int64_t}$ (safe). ARM Cortex-M7 has a SMULL instruction for $32 \times 32 \rightarrow 64$ multiply.
16	float outReal = (float)acc * outScale + biasReal;	Convert the accumulated integer result back to real value. (float)acc: widening cast from int64 to float (may lose precision for very large acc). Multiply by combined scale gives the real dot product value.
18-20	Requantize: outReal / qcOut->scale -> outMantissa	Convert back to the output tensor's quantization. outMantissa = outReal / scaleOut. Then clamp to [-qMax, +qMax] to prevent overflow in subsequent operations.

Section 6.11 — findAbsMaxFloat, findMaxInt32, clamp

```
// Find the maximum absolute value in a float tensor (used for requantization Pass A):
float findAbsMaxFloat(tensor_t *tensor) {
    size_t N = calcNumberOfElementsByTensor(tensor);
    float maxVal = 0.0f;
    for (size_t i = 0; i < N; i++) {
        float v = readBytesAsFloat(tensor->data + i*4);
        float absV = (v < 0.0f) ? -v : v; // fabsf equivalent
        if (absV > maxVal) maxVal = absV;
    }
    return maxVal;
}

// Find the maximum int32 value (used for int32 gradient magnitude):
int32_t findMaxInt32(tensor_t *tensor) {
    size_t N = calcNumberOfElementsByTensor(tensor);
    int32_t maxVal = INT32_MIN; // start below any real value
    for (size_t i = 0; i < N; i++) {
        int32_t v = readBytesAsInt32(tensor->data + i*4);
        if (v > maxVal) maxVal = v;
    }
    return maxVal;
}

// Clamp: restrict a value to [minVal, maxVal]:
int32_t clamp(int32_t value, int32_t minVal, int32_t maxVal) {
    if (value < minVal) return minVal;
    if (value > maxVal) return maxVal;
    return value;
}
```

3	float maxVal = 0.0f;	Initialize to 0. Since we're looking for the maximum ABSOLUTE value, 0 is the smallest possible absolute value. If all elements are negative, their absolute values are positive — still larger than 0, so the loop finds the correct maximum.
6	float absV = (v < 0.0f) ? -v : v;	Manual absolute value without calling fabsf. Ternary operator: if v is negative, negate it; otherwise keep it. Used in requantization Pass A: scale = maxAbsVal / qMax.
13	int32_t maxVal = INT32_MIN;	INT32_MIN = -2,147,483,648. Starting from the minimum ensures that the very first element will always be larger, so the loop correctly finds the true maximum even if all values are negative.
21-25	clamp(value, minVal, maxVal)	Clamps value to [minVal, maxVal]. Called after requantization to ensure mantissas stay within [-qMax, qMax]. Without clamping, a very large gradient could overflow int32_t on the next matmul.

Section 6.12 — roundByMode: Rounding Modes and Stochastic Rounding

■ C CONCEPT: Stochastic Rounding — Why FQT Needs It

Standard rounding (round half up): 0.4 -> 0, 0.5 -> 1, 0.6 -> 1. Problem: if a gradient is 0.001 and the scale is 0.01, the quantized gradient is 0.1 -> rounds to 0 EVERY time. The weight NEVER updates. Training stalls. Stochastic rounding (SR_HALF_AWAY): Compute fractional part: $f = 0.1$ (the 0.001/0.01 case) Generate uniform random noise r in $[0, 1]$ If $r < f$ (0.1): round UP -> 1 If $r \geq f$: round DOWN -> 0 Expected value: $E[\text{rounded}] = 0 \times 0.9 + 1 \times 0.1 = 0.1$ = true value. Stochastic rounding is UNBIASED. The expected gradient update is correct even for updates smaller than one quantization step. This is WHY stochastic rounding is essential for FQT convergence: it preserves the mean of small gradient updates that would otherwise vanish.

```

int32_t roundByMode(float realValue, roundingMode_t mode) {
    switch (mode) {
        case ROUND_HALF_AWAY: {
            // Standard: round 0.5 away from zero
            return (realValue >= 0.0f)
                ? (int32_t)(realValue + 0.5f)
                : (int32_t)(realValue - 0.5f);
        }
        case ROUND_HALF_EVEN: {
            // Banker's rounding: round 0.5 to nearest EVEN integer
            float rounded = roundf(realValue); // standard round
            float frac = realValue - floorf(realValue);
            if (frac == 0.5f) {
                // Check if floor is even; if so, floor; else ceil
                float floorVal = floorf(realValue);
                rounded = (fmodf(floorVal, 2.0f) == 0.0f) ? floorVal : ceilf(realValue);
            }
            return (int32_t)rounded;
        }
        case SR_HALF_AWAY: {
            // Stochastic rounding: add uniform noise before rounding
            float noise = rngNextFloat(); // uniform in [0, 1)
            float frac = realValue - floorf(realValue); // fractional part
            float adj = realValue + (noise - frac); // shift value
            return (int32_t)floorf(adj + 0.5f);
        }
        default: return (int32_t)realValue;
    }
}

```

3-6	ROUND_HALF_AWAY case	Adds 0.5 before truncating to integer. Works for positive values. For negative: subtract 0.5 (round away from 0). Example: 2.5 -> (int32_t)(3.0) = 3. -2.5 -> (int32_t)(-3.0) = -3.
8-16	ROUND_HALF_EVEN case (banker's rounding)	Breaks ties (exactly 0.5) by rounding to the nearest EVEN integer. 2.5 -> 2 (floor 2 is even). 3.5 -> 4 (floor 3 is odd -> ceil). Over many roundings, this eliminates systematic bias.
18	float noise = rngNextFloat();	Generate a uniform random number in [0, 1). Uses the ODT xorshift32 PRNG. This is why rngSetSeed matters for reproducibility of FQT training.
19	float frac = realValue - floorf(realValue);	Fractional part of realValue. For 3.7: floorf(3.7)=3.0, frac=0.7. This is the 'probability of rounding up' in stochastic rounding.
20-21	adj = realValue + (noise - frac); return floorf(adj + 0.5f);	The stochastic rounding implementation: if noise < frac: noise-frac is negative, adj < realValue, rounds down. if noise >= frac: adj >= realValue + (1-frac), rounds up. Probability of rounding up = P(noise < frac) = frac. Unbiased.

Section 6.13 — RigL Extension: riglMaskedMatmulFloat32

RigL requires a matmul that skips pruned weight positions. `riglMaskedMatmulFloat32` is the sparse-aware forward pass kernel. It is identical to `matmulFloat32TensorsWithBias` except for the mask check.

```

void riglMaskedMatmulFloat32(tensor_t *A, tensor_t *B,
tensor_t *out, tensor_t *bias,
tensor_t *mask) {
    size_t M = getDimensionsByIndex(A, 0);
    size_t K = getDimensionsByIndex(A, 1);
    size_t N = getDimensionsByIndex(B, 0);
    for (size_t i = 0; i < M; i++) {
        for (size_t j = 0; j < N; j++) {
            float sum = 0.0f;
            for (size_t k = 0; k < K; k++) {
                if (mask != NULL) {
                    size_t flatIdx = j * K + k; // weight [j][k]
                    if (!tensorBoolGet(mask, flatIdx)) continue; // SKIP
                }
                float aVal = readBytesAsFloat(A->data + (i*K+k)*4);
                float bVal = readBytesAsFloat(B->data + (j*K+k)*4);
                sum += aVal * bVal;
            }
            float biasVal = readBytesAsFloat(bias->data + j*4);
            writeFloatToByteArray(sum + biasVal, out->data + (i*N+j)*4);
        }
    }
}

```

10-1 3	mask != NULL check + tensorBoolGet	Dual purpose: if mask is NULL, behave like the dense matmul (no RigL). If mask is provided, check bit [j*K+k] for weight at row j, column k of B. continue skips to next k — the multiplication is never performed for pruned weights.
11	size_t flatIdx = j * K + k;	Map weight matrix coordinates [j][k] to flat bit index in the mask. B has shape [N, K] (N output neurons, K input features). Row j contains all K weights for output neuron j. Flat = j*K + k.
14-1 6	float aVal, bVal; sum += aVal * bVal;	Only reached for active weights (mask=1). With sparsityRatio=0.8: 80% of k iterations hit the continue. Only 20% reach the float multiply — the actual speedup on MCU.

■ RigL GAP IN THIS FILE

Performance note: the mask check (tensorBoolGet + conditional continue) costs ~3-4 cycles per k iteration even when pruned. With sparsityRatio=0.8 and K=128: $128 \times 4 = 512$ extra cycles for the mask checks, but saves $128 \times 0.8 = 102$ multiply-adds ≈ 102 cycles. For large K (512+) the savings dominate. For very small K (<64), consider a CSR (Compressed Sparse Row) format instead.

Function	File	Purpose
readBytesAsFloat	DTypes.c	Safe float read via memcpy (strict aliasing safe)
writeFloatToByteArray	DTypes.c	Safe float write via memcpy
readBytesAsInt32 / writeInt32ToByteArray	DTypes.c	Same but for int32_t
writeBitsToByteArray / readBitsFromByteArray	DTypes.c	Bit-level read/write for BOOL
tensorBoolGet / tensorBoolSet	Tensor.c	Single-bit mask read/write
getDimensionsByIndex	Arithmetic.c	Transpose-aware dimension lookup
calcTensorIndexByIndices	Arithmetic.c	N-D coordinates to flat index
calcIndicesByRawIndex	Arithmetic.c	Flat index to N-D coordinates
doDimensionsMatch	Arithmetic.c	Shape validation before operations
int32PointWiseArithmetic	Arithmetic.c	Function-pointer-dispatch element-wise op

addInt32s	Arithmetic.c	Element-wise INT32 addition
addFloat32TensorsInplace	Arithmetic.c	In-place float32 addition
matmulFloat32TensorsWithBias	Arithmetic.c	Dense float matmul
matmulSymInt32TensorsWithBias	Arithmetic.c	FQT int32 matmul with int64 accum
findAbsMaxFloat	Arithmetic.c	Max absolute value (requantization Pass A)
findMaxInt32	Arithmetic.c	Max int32 value
roundByMode	Rounding.c	ROUND_HALF_AWAY / HALF_EVEN / SR (stochastic)
clamp	Arithmetic.c	Clamp to [min, max]
riglMaskedMatmulFloat32	RigL.c	Sparse float matmul (skips pruned weights)

Deep Dive — Arithmetic.c and Matmul.c

Every Function Explained Line by Line

Section 12.1 — getDimensionsByIndex: Handling Transposed Tensors

getDimensionsByIndex is subtle. It does NOT just return dimensions[index]. It searches the orderOfDimensions array to find which physical slot holds the requested logical dimension. This is needed because after transposeTensor, the mapping between logical and physical dimensions is scrambled.

```
size_t getDimensionsByIndex(tensor_t *tensor, size_t index) {
    size_t numberOfDims = tensor->shape->numberOfDimensions;
    for (size_t i = 0; i < numberOfDims; i++) {
        if (tensor->shape->orderOfDimensions[i] == index) {
            return tensor->shape->dimensions[i];
        }
    }
    PRINT_ERROR("Tensor doesn't have %lu dimensions!", index);
    exit(1);
}
```

■ C CONCEPT: Logical vs Physical Dimension Index

A freshly created [3,4] tensor has: dimensions[] = {3, 4} orderOfDimensions = {0, 1} (identity: logical 0 → physical 0)
 After transposeTensor(t, 0, 1) (makes it a [4,3] transposed view): dimensions[] = {4, 3} (SWAPPED physically)
 orderOfDimensions = {1, 0} (logical 0 → physical 1) getDimensionsByIndex(t, 0): Loops: orderOfDimensions[0]=1 ≠ 0; orderOfDimensions[1]=0 == 0 → returns dimensions[1] = 3 (logical dim 0 has size 3, which was rows before transpose) This correctly gives the LOGICAL dimension size, accounting for the transpose.

1	size_t numberOfDims = tensor->shape->numberOfDimensions;	Cache the number of dimensions to avoid repeated struct field access in the loop.
2-3	for (size_t i = 0; i < numberOfDims; i++) { if (orderOfDimensions[i] == index)	Search for the physical slot whose logical label matches 'index'. In the identity case this finds i==index immediately. After transpose, it may find a different i.
4	return tensor->shape->dimensions[i];	Return the physical dimension size at that slot. This is the correct SIZE for the requested logical dimension.
5-6	PRINT_ERROR(...); exit(1);	If no slot matched, the requested dimension index exceeds the tensor's rank. e.g. asking for dimension 2 of a 2D tensor (valid indices: 0,1).

Section 12.2 — orderDims: Materializing Logical Dimensions

```
void orderDims(tensor_t *tensor, size_t *orderedDims) {
    for (size_t i = 0; i < tensor->shape->numberOfDimensions; i++) {
        orderedDims[i] = getDimensionsByIndex(tensor, i);
    }
}
```

orderDims fills an array with the logical dimension sizes in order [0,1,2,...]. The caller provides a VLA array large enough for numberOfDimensions elements. After this call, orderedDims[0] = logical-size-of-dim-0, etc. This is used by

int32PointWiseArithmetic to compute indices correctly.

— ■ WORKED EXAMPLE

For a transposed [3,4] tensor viewed as [4,3]: dimensions[] = {4, 3} orderOfDimensions = {1, 0} orderDims() calls: getDimensionsByIndex(t, 0) → looks for orderOfDims[i]==0 → i=1 → dimensions[1]=3 getDimensionsByIndex(t, 1) → looks for orderOfDims[i]==1 → i=0 → dimensions[0]=4 orderedDims = {3, 4} — the LOGICAL sizes before transpose. This is used in calcIndicesByRowIndex so that flat-index-to-indices conversion uses the logical sizes, not the physical ones.

Section 12.3 — doDimensionsMatch: Shape Validation

```
bool doDimensionsMatch(tensor_t *a, tensor_t *b) {
    size_t aN = a->shape->numberOfDimensions;
    size_t bN = b->shape->numberOfDimensions;
    if (aN != bN) {
        PRINT_ERROR("Rank mismatch: %lu vs %lu", aN, bN);
        exit(1);
    }
    size_t aOrd[aN], bOrd[bN];
    orderDims(a, aOrd);
    orderDims(b, bOrd);
    for (size_t i = 0; i < aN; i++) {
        if (aOrd[i] != bOrd[i]) {
            PRINT_DEBUG("Dim mismatch at %lu: %lu vs %lu", i, aOrd[i], bOrd[i]);
            return false;
        }
    }
    return true;
}
```

■ C CONCEPT: bool Type in C

C (before C99) had no bool type. C99 introduced `_Bool` and the `<stdbool.h>` header which defines 'bool', 'true' (=1), and 'false' (=0). `#include <stdbool.h> bool flag = true; // flag holds 1 bool other = false; // other holds 0 if (flag) { ... } // checks if flag != 0` Functions returning bool are used for predicates (yes/no questions). `doDimensionsMatch` returns true if shapes are compatible for element-wise operations.

Rank check	<code>if (aN != bN) { PRINT_ERROR(); exit(1); }</code>	Two tensors with different numbers of dimensions cannot be added/multiplied element-wise. e.g. a 2D matrix and a 3D cube have incompatible shapes regardless of individual sizes.
VLA arrays	<code>size_t aOrd[aN], bOrd[bN];</code>	Allocate two temporary arrays on the stack. Size is runtime-determined. Note: <code>aN == bN</code> after the rank check, so <code>aOrd</code> and <code>bOrd</code> are the same size.
orderDims	<code>orderDims(a, aOrd); orderDims(b, bOrd);</code>	Materialize LOGICAL dimension sizes for both tensors. This handles transposed tensors correctly.
Dimension comparison	<code>if (aOrd[i] != bOrd[i]) return false;</code>	Every logical dimension must match. <code>PRINT_DEBUG</code> prints a message but DOES NOT exit — returns false instead, letting the caller decide whether to abort or handle the mismatch.
return true	All checks passed: shapes are element-wise compatible.	The tensors can be used in point-wise arithmetic together.

Section 12.4 — calcElementIndexByIndices: The Transpose-Aware Index

calcElementIndexByIndices converts logical multi-dim indices to a physical flat index, applying the orderOfDimensions permutation. This is the function that makes transposed tensor access correct.

```
size_t calcElementIndexByIndices(size_t numberOfDims, size_t *dims,
size_t *indices, size_t *orderOfDimensions) {
    size_t offset = 1;
    for (size_t i = 0; i < numberOfDims; i++) {
        offset *= dims[i]; // total elements = product of all dims
    }
    // Apply permutation: map logical indices to physical indices
    size_t orderedIndices[numberOfDims];
    for (size_t d = 0; d < numberOfDims; d++) {
        orderedIndices[orderOfDimensions[d]] = indices[d];
    }
    // Row-major index calculation using physical indices
    size_t outputIndex = 0;
    for (size_t i = 0; i < numberOfDims; i++) {
        offset /= dims[i];
        outputIndex += orderedIndices[i] * offset;
    }
    return outputIndex;
}
```

— ■ WORKED EXAMPLE

Example: access element $[i=0, j=1]$ of a transposed $[3,4]$ matrix (transposed from a physical $[4,3]$ layout): Physical tensor: $\text{dims}=\{4,3\}$, $\text{orderOfDimensions}=\{1,0\}$ Logical indices: $\text{indices}=\{0, 1\}$ (logical row 0, logical col 1) Phase 1: permute indices $d=0$: $\text{orderedIndices}[\text{orderOfDims}[0]=1] = \text{indices}[0] = 0 \rightarrow \text{orderedIndices}[1]=0$ $d=1$: $\text{orderedIndices}[\text{orderOfDims}[1]=0] = \text{indices}[1] = 1 \rightarrow \text{orderedIndices}[0]=1$ $\text{orderedIndices} = \{1, 0\}$ Phase 2: row-major index from physical dims $\{4,3\}$ $\text{offset} = 4 \times 3 = 12$ $i=0$: $\text{offset} /= 4 \rightarrow \text{offset}=3$; $\text{outputIndex} += \text{orderedIndices}[0] \times 3 = 1 \times 3 = 3$ $i=1$: $\text{offset} /= 3 \rightarrow \text{offset}=1$; $\text{outputIndex} += \text{orderedIndices}[1] \times 1 = 0 \times 1 = 0$ $\text{outputIndex} = 3$ Physical byte offset = $3 \times \text{sizeof}(\text{int32_t}) = 12$ This accesses the element at row 1, col 0 of the ORIGINAL (untransposed) matrix, which is exactly what row 0, col 1 of the TRANSPOSED matrix should be. ✓

Section 12.5 — int32PointWiseArithmetic: Full Line-by-Line

```
// Full implementation from Arithmetic.c:
void int32PointWiseArithmetic(tensor_t *aTensor, tensor_t *bTensor,
int32ElementArithmeticFunc_t arithmeticFunc, tensor_t *outputTensor) {
if (!doDimensionsMatch(aTensor, bTensor)) {
PRINT_ERROR("Dimensions don't match!"); exit(1);
}
size_t numberOfElements = calcNumberOfElementsByTensor(aTensor);
size_t bytesPerElement = sizeof(int32_t);
size_t numberOfDims = aTensor->shape->numberOfDimensions;
size_t *aDims = aTensor->shape->dimensions;
size_t *bDims = bTensor->shape->dimensions;
size_t aOrderedDims[numberOfDims];
size_t bOrderedDims[numberOfDims];
orderDims(aTensor, aOrderedDims);
orderDims(bTensor, bOrderedDims);
for (size_t i = 0; i < numberOfElements; i++) {
size_t aIndices[numberOfDims];
size_t bIndices[numberOfDims];
calcIndicesByRowIndex(numberOfDims, aOrderedDims, i, aIndices);
size_t aIdx = calcElementIndexByIndices(numberOfDims,
aDims, aIndices, aTensor->shape->orderOfDimensions);
calcIndicesByRowIndex(numberOfDims, bOrderedDims, i, bIndices);
size_t bIdx = calcElementIndexByIndices(numberOfDims,
bDims, bIndices, bTensor->shape->orderOfDimensions);
int32_t aVal = readBytesAsInt32(aTensor->data + aIdx*bytesPerElement);
int32_t bVal = readBytesAsInt32(bTensor->data + bIdx*bytesPerElement);
int32_t result = arithmeticFunc(aVal, bVal);
size_t oIdx = calcElementIndexByIndices(numberOfDims,
outputTensor->shape->dimensions, aIndices,
outputTensor->shape->orderOfDimensions);
writeInt32ToByteArray(result,
outputTensor->data + oIdx*bytesPerElement);
}
}
```

L1-2	doDimensionsMatch check and exit on failure	Correctness guard. Both input tensors must have the same shape (after accounting for transposes). Exits immediately on mismatch because continuing with mismatched shapes would write to wrong memory locations.
L3	numberOfElements = total number of elements in tensor A	This is the product of all dimensions. We iterate i from 0 to numberOfElements-1, covering every element.
L4	bytesPerElement = sizeof(int32_t) = 4	All INT32 elements are 4 bytes. We use this to compute byte offsets.
L5-7	Cache dimension arrays as local pointers	Avoids repeatedly dereferencing tensor->shape->dimensions in the hot loop. Modern compilers may do this anyway, but explicit caching makes intent clear.
L8-11	VLA arrays + orderDims: materialize logical sizes	aOrderedDims[i] = logical size of dimension i for tensor A (accounting for transpose). Used in calcIndicesByRowIndex to convert flat index → logical indices.
L12	Main loop: for each element position i in [0, N)	We process every element of the result tensor.
L13-14	VLA index arrays – new stack allocation per iteration	aIndices[d] = logical index along dimension d for element i. These are inside the loop so a new VLA is created each iteration. Compilers typically optimize this to a single stack frame adjustment.
L15	calcIndicesByRowIndex → convert flat i to logical [row, col, ...]	For a [3,4] tensor, flat index 6 → logical [1,2]. Uses aOrderedDims (the logical sizes) so the result is in LOGICAL space.

L16	<code>calcElementIndexByIndices</code> → convert logical indices to PHYSICAL byte offset	Applies the <code>orderOfDimensions</code> permutation. For a non-transposed tensor this is the same as row-major index. For a transposed tensor, the physical memory offset differs from the logical index.
L17-18	Same process for tensor B	B may have a DIFFERENT <code>orderOfDimensions</code> than A. This correctly handles: A is row-major, B is transposed.
L19-20	<code>readBytesAsInt32</code> to read values from both tensors	The byte offset = index × 4 bytes. <code>readBytesAsInt32</code> copies 4 bytes → <code>int32_t</code> .
L21	<code>arithmeticFunc(aVal, bVal)</code> – the actual operation	This is the function pointer call. For <code>addInt32s</code> : returns <code>aVal + bVal</code> . For <code>mulInt32s</code> : returns <code>aVal × bVal</code> . The loop logic is identical for all operations.
L22	Compute output physical index from the SAME logical indices as A	The output tensor has the same logical shape as A and B. We reuse <code>aIndices</code> (= logical position of element <code>i</code>) to find where to write.
L23	<code>writeInt32ToByteArray</code> – write result	Copies 4 bytes of result into the output tensor's data buffer.

■■ IMPORTANT

Performance note: `int32PointWiseArithmetic` calls `calcIndicesByRowIndex` and `calcElementIndexByIndices` per element. For an N-element tensor: $O(N \times \text{numDims})$ work just for index calculations. This is correct but slow compared to a simple flat-index loop (which would ignore `orderOfDimensions`). If you know your tensors are non-transposed, you can use the inplace variants (`int32PointWiseArithmeticInplace`) which directly iterate flat indices.

Section 12.6 — matmulIntCore: The Matrix Multiply Engine

`matmulIntCore` is the shared implementation for all INT32 matrix multiplications. It handles 1D vectors as well as 2D matrices. An optional bias array can be pre-added to the accumulator.

```

static void matmulIntCore(tensor_t *A, tensor_t *B, tensor_t *out,
const int32_t *biasSeed) {
if (A->shape->numberOfDimensions > 2 || B->shape->numberOfDimensions > 2) {
PRINT_ERROR("Matmul only supports up to 2D Tensors"); exit(1);
}
size_t aRows, aColumns;
if (A->shape->numberOfDimensions < 2) {
aRows = 1; aColumns = getDimensionsByIndex(A, 0);
} else {
aRows = getDimensionsByIndex(A, 0);
aColumns = getDimensionsByIndex(A, 1);
}
size_t bRows = getDimensionsByIndex(B, 0);
size_t bColumns = (B->shape->numberOfDimensions < 2)
? 1 : getDimensionsByIndex(B, 1);
if (aColumns != bRows) {
PRINT_ERROR("Rows don't match Columns"); exit(1);
}
size_t resultCounter = 0;
for (size_t row = 0; row < aRows; row++) {
for (size_t col = 0; col < bColumns; col++) {
int32_t result = biasSeed ? biasSeed[col] : 0;
for (size_t k = 0; k < aColumns; k++) {
size_t aIdx[2] = {row, k};
size_t aPhys = calcElementIndexByIndices(2, A->shape->dimensions,
aIdx, A->shape->orderOfDimensions);
int32_t aV = readBytesAsInt32(A->data + aPhys*4);
size_t bIdx[2] = {k, col};
size_t bPhys = calcElementIndexByIndices(2, B->shape->dimensions,
bIdx, B->shape->orderOfDimensions);
int32_t bV = readBytesAsInt32(B->data + bPhys*4);
result += mulInt32s(aV, bV);
}
writeInt32ToByteArray(result, out->data + resultCounter*4);
resultCounter++;
}
}
}
}

```

1D handling	if (numberOfDimensions < 2) { aRows=1; aColumns=dim0; }	A 1D vector [K] is treated as a [1,K] row vector for matrix multiplication. This enables matmul to handle both vector-matrix and matrix-matrix products.
Dimension check	if (aColumns != bRows) { PRINT_ERROR; exit(1); }	The inner dimension must match: A is [M,K], B must be [K,N]. If K doesn't match, matrix multiplication is undefined.
bias Seed	result = biasSeed ? biasSeed[col] : 0;	Start the accumulator with the bias value for column 'col'. This elegantly fuses the bias addition into the matmul loop, avoiding a separate addition pass over the output.
Inner loop	for k: aPhys = calcElementIndex(A, [row,k]);	Look up A[row][k] using the transpose-aware index function. For linearForward: A has been transposed, so A[row][k] accesses the COLUMN of the weight matrix before the transpose.

result += mullnt32s(aV, bV)	Accumulate in int32_t.	For small K (e.g. 64), the maximum accumulation is $64 \times 32767^2 \approx 68B$ — which OVERFLOWS int32_t (max 2.1B). For safe INT32 accumulation, qMaxBits must be small enough that $K \times qMax^2 < INT32_MAX$. The SYM_INT32 variant uses int64_t to avoid this overflow.
resultCounter++	Output is written in row-major order.	output[row*bColumns + col] = result. resultCounter is a flat index that increments for each output element.

— ■ WORKED EXAMPLE

Trace matmulIntCore for A=[2,3] and B=[3,2] (no bias): A = [[1,2,3],[4,5,6]] B = [[7,8],[9,10],[11,12]] Expected C = A×B = [[1×7+2×9+3×11, 1×8+2×10+3×12], [4×7+5×9+6×11, ...]] = [[58, 64], [139, 154]] Loop trace: row=0, col=0: result=0; k=0: 1×7=7; k=1: +2×9=25; k=2: +3×11=58. Write 58. row=0, col=1: result=0; k=0: 1×8=8; k=1: +2×10=28; k=2: +3×12=64. Write 64. row=1, col=0: result=0; k=0: 4×7=28; k=1: +5×9=73; k=2: +6×11=139. Write 139. row=1, col=1: result=0; k=0: 4×8=32; k=1: +5×10=82; k=2: +6×12=154. Write 154. Output flat: [58, 64, 139, 154] ✓

Section 12.7 — matmulSymInt32TensorsWithBias: The FQT Core

```

void matmulSymInt32TensorsWithBias(tensor_t *A, tensor_t *B,
tensor_t *out, tensor_t *bias) {
symInt32QConfig_t *aCfg = A->quantization->qConfig;
symInt32QConfig_t *bCfg = B->quantization->qConfig;
symInt32QConfig_t *outCfg = out->quantization->qConfig;
symInt32QConfig_t *biasCfg = bias->quantization->qConfig;
float scaleA = aCfg->scale;
float scaleB = bCfg->scale;
float scaleOut = outCfg->scale;
float scaleBias = biasCfg->scale;
int32_t outQMax = (1 << (outCfg->qMaxBits - 1)) - 1; // e.g. 32767
int32_t outQMin = -outQMax;
size_t M = getDimensionsByIndex(A, 0);
size_t K = getDimensionsByIndex(A, 1);
size_t N = getDimensionsByIndex(B, 1);
size_t resultCounter = 0;
for (size_t row = 0; row < M; row++) {
for (size_t col = 0; col < N; col++) {
int64_t acc = 0; // 64-bit to prevent overflow
for (size_t k = 0; k < K; k++) {
// A[row][k] and B[k][col] – respecting orderOfDimensions
int32_t aV = /* readBytesAsInt32 at [row][k] */;
int32_t bV = /* readBytesAsInt32 at [k][col] */;
acc += (int64_t)aV * (int64_t)bV;
}
// Convert from mantissa-product to real value:
float realResult = (float)acc * scaleA * scaleB;
// Add bias (bias is already in real-value units × scaleBias):
float biasReal = readBytesAsInt32(bias->data + col*4) * scaleBias;
realResult += biasReal;
// Quantise to output SYM_INT32:
int32_t q = roundByMode(
clamp(realResult / scaleOut, outQMin, outQMax),
outCfg->roundingMode);
writeInt32ToByteArray(q, out->data + resultCounter*4);
resultCounter++;
}
}
}

```

Real scales	scaleA, scaleB, scaleOut, scaleBias from qConfig	Each tensor has its own scale. $\text{real_A}[i][k] = \text{A_mantissa}[i][k] \times \text{scaleA}$. The product $\text{real_A} \times \text{real_B} = \text{mantissa_A} \times \text{mantissa_B} \times \text{scaleA} \times \text{scaleB}$.
out QMax	$(1 \ll (\text{qMaxBits}-1)) - 1$	$\text{qMaxBits}=16 \rightarrow \text{outQMax} = 2^{15} - 1 = 32767$. The $\ll$ operator is a bit shift: $1 \ll 15 = 32768$. Subtract 1 = 32767. This is the maximum representable mantissa.
int64_t acc = 0	64-bit accumulator for the inner product.	Each product $\text{aV} \times \text{bV}$ fits in <code>int32_t</code> , but K products may overflow. Casting to <code>int64_t</code> before multiply prevents the overflow.
(int64_t)aV × (int64_t)bV	Cast to 64-bit BEFORE the multiply, not after.	Without the cast: $\text{int32} \times \text{int32} = \text{int32}$ (potential overflow). With the cast: $\text{int64} \times \text{int64} = \text{int64}$ (no overflow for K up to $\sim 2\text{M}$).

float realResult = acc × scaleA × scaleB	Convert mantissa-product accumulation to real value.	acc holds $\text{sum}(\text{aM}[k] \times \text{bM}[k])$. $\text{real_result} = \text{sum}(\text{aM}[k] \times \text{scaleA} \times \text{bM}[k] \times \text{scaleB}) = \text{acc} \times \text{scaleA} \times \text{scaleB}$.
biasReal = biasM × scaleBias	Bias is also in SYM_INT32. Convert to real value and add.	The bias is NOT converted to float VLA here — we read one bias element at a time.
clamp(outQMin, outQMax)	Clip the real value to the output's representable range.	If $\text{realResult} / \text{scaleOut} = 40000$ but $\text{outQMax} = 32767$: clamp to 32767. This saturation is INTENTIONAL — values outside the range are clipped.
roundByMode(.., roundingMode)	Round the float to nearest int32_t using the configured rounding mode.	HALF_AWAY: standard round(). SR_HALF_AWAY: add noise before rounding (stochastic rounding for gradient estimation).

Section 12.8 — TRACK_INSTRUCTIONS: Profiling Support

Matmul.c has a conditional compile feature for profiling: if TRACK_INSTRUCTIONS is defined, each matmul call increments a counter. This lets you measure how many multiply-accumulate operations your model uses.

```
// At the top of Matmul.c:
#ifdef TRACK_INSTRUCTIONS
#define MATMUL_FUNC_INT matmulIntTensorsWithInstructionCounter
#else
#define MATMUL_FUNC_INT matmulIntTensors
#endif
// The counter:
size_t matmulInstructionCounter = 0;
// Accessed via:
size_t getMatmulInstructionCounter() { return matmulInstructionCounter; }
// Add.h also has addInstructionCounter:
size_t getAddInstructionCounter() { return addInstructionCounter; }
```

■ C CONCEPT: `#ifdef` — Conditional Compilation

`#ifdef SYMBOL` compiles the following block only if `SYMBOL` is defined. `#endif` ends the conditional block. `#ifndef` compiles only if `SYMBOL` is NOT defined. In `CMakeLists.txt` or `Makefile`:

`add_definitions(-DTRACK_INSTRUCTIONS)` → defines `TRACK_INSTRUCTIONS` (Without this flag, the profiling code is completely absent from the binary) Why conditional? Profiling adds overhead (counter increment per `matmul` call). In production (deployment), you want maximum speed. `#ifdef` lets you enable profiling for benchmarking and remove it for deployment with a single build flag, without changing source code.

Neural Network Layers — Complete Deep Dive

Layer.h Interface, Dispatch Table, Every Layer Forward/Backward, RigL Integration

Section 7.1 — Layer.h: The Interface Contract

■ WHAT THIS FILE DOES

Layer.h defines the INTERFACE that every neural network layer must implement. All layers (linear, ReLU, softmax, dropout, conv1d, flatten) are accessed through the SAME interface. The caller does not need to know which layer type it has — it just calls `layer->forward(cfg, input, output)`. This is the DISPATCH PATTERN (also called polymorphism in object-oriented languages).

```
// Layer.h – the layer interface:
// Enum of all supported layer types:
typedef enum {
    LAYER_LINEAR = 0,
    LAYER_RELU = 1,
    LAYER_SOFTMAX = 2,
    LAYER_DROPOUT = 3,
    LAYER_CONV1D = 4,
    LAYER_FLATTEN = 5,
} layerType_t;
// Union: holds ONE of the possible layer configs (shared memory):
typedef union {
    linearConfig_t *linear;
    reluConfig_t *relu;
    softmaxConfig_t *softmax;
    dropoutConfig_t *dropout;
    conv1dConfig_t *conv1d;
    flattenConfig_t *flatten;
} layerConfig_t;
// Function pointer types for forward and backward passes:
typedef void (*forwardFn_t)(layerConfig_t*, tensor_t*, tensor_t*);
typedef void (*backwardFn_t)(layerConfig_t*, tensor_t*, tensor_t*);
// The layer struct – one instance per layer in the model:
typedef struct {
    layerType_t type; // which layer type
    layerConfig_t config; // pointer to the specific config
    forwardFn_t forward; // function pointer to forward pass
    backwardFn_t backward; // function pointer to backward pass
    tensor_t *forwardInput; // saved input for backprop
} layer_t;
```

■ C CONCEPT: Union in C — Shared Memory for Multiple Types

A union allocates memory for its LARGEST member, and all members share that memory: `typedef union { int i; // 4 bytes float f; // 4 bytes char c[8]; // 8 bytes } MyUnion;` // sizeof(MyUnion) = 8 (size of largest member) Writing one member and reading another gives the same bits interpreted differently. This is intentional in `layerConfig_t`: only ONE field is valid at a time (the one matching `layer->type`). Memory saving: `struct { linear*, relu*, softmax*, dropout*, conv1d*, flatten* } = 24 bytes`. `Union { linear*, relu*, ..., flatten* } = 4-8 bytes (size of one pointer)`. For a model with many layers, this matters on a 320 KB MCU. RULE: Only read the union member that was written. Always check `layer->type` before accessing `layer->config.linear` (vs `.relu`, etc.).

Section 7.2 — The Dispatch Table: forwardFuncs[]

```
// Layer.c – the dispatch table: index = layerType_t, value = function pointer
static forwardFn_t forwardFuncs[] = {
    [LAYER_LINEAR] = linearForwardFloat,
    [LAYER_RELU] = reluForwardFloat,
    [LAYER_SOFTMAX] = softmaxForwardFloat,
    [LAYER_DROPOUT] = dropoutForwardFloat,
    [LAYER_CONV1D] = conv1dForwardFloat,
    [LAYER_FLATTEN] = flattenForwardFloat,
};
// How to call the forward pass for any layer (generic training loop):
void layerForward(layer_t *layer, tensor_t *input, tensor_t *output) {
    layer->forwardInput = input; // save for backprop
    forwardFuncs[layer->type](&layer->config, input, output);
}
// The compiler compiles this as: jump to address in forwardFuncs[layer->type]
// No if-else chain needed. Adding a new layer type = add one entry to the table.
```

1-8	static forwardFn_t forwardFuncs[] = { [LAYER_LINEAR]=... }	Designated initializer syntax: [LAYER_LINEAR]=linearForwardFloat means forwardFuncs[0] = linearForwardFloat (since LAYER_LINEAR=0). This is safer than a plain array: if you reorder the enum, the table stays correct.
10	layer->forwardInput = input;	Save the input tensor pointer for the backward pass. linearCalcWeightGrads needs the forward-pass input ($dL/dW = dL/dY^T @ X$). This is stored in the layer struct so backprop can access it without a separate argument.
11	forwardFuncs[layer->type](&layer->config, input, output);	Indirect function call through the dispatch table. layer->type is the array index. forwardFuncs[layer->type] is the function pointer. Calling it with (...) executes the correct layer's forward function. On ARM Cortex-M7: BLXI instruction, ~1-2 cycles (branch indirect).

Section 7.3 — linearConfig_t: All Fields Explained

```
typedef struct {
    parameter_t weights; // weight tensor + weight gradient tensor
    parameter_t bias; // bias tensor + bias gradient tensor
    size_t inFeatures; // number of input features (K)
    size_t outFeatures; // number of output neurons (N)
    bool useBias; // true if bias should be added
    tensor_t *forwardInput; // pointer to saved input (for backprop)
    tensor_t *mask; // RigL sparsity mask (NULL = dense)
} linearConfig_t;
// parameter_t – bundles weight and gradient tensor together:
typedef struct {
    tensor_t param; // the weight/bias tensor
    tensor_t grad; // the gradient tensor (same shape as param)
} parameter_t;
```

1	parameter_t weights;	NOT a pointer — the parameter_t struct is stored INLINE in linearConfig_t. weights.param = weight tensor (stores actual weights). weights.grad = gradient tensor (stores dL/dW , same shape). Both are tensor_t structs, so all their data pointers must also be set up.
2	parameter_t bias;	Similar to weights but shape [outFeatures] instead of [outFeatures, inFeatures]. bias.param = bias values. bias.grad = bias gradients (dL/db).

3-4	inFeatures, outFeatures	K and N in the matmul $Y = X @ W^T + b$. For a 128->64 layer: inFeatures=128, outFeatures=64. Shape of weight matrix: [outFeatures, inFeatures] = [64, 128].
5	bool useBias;	If false: skip the bias addition in forward pass (equivalent to b=0). Some architectures (batch norm layers) remove bias since BN already adds offset.
6	tensor_t *forwardInput;	Pointer to the input tensor from the most recent forward pass. Set by linearForwardFloat before calling matmul. Used by linearCalcWeightGradsFloat: $dL/dW = dL/dY^T @ forwardInput$.
7	tensor_t *mask;	NULL for dense (standard) layers. For RigL sparse layers: pointer to a BOOL tensor with shape [outFeatures, inFeatures]. mask[j][k]=1 means weight[j][k] is active; 0 means pruned.

Section 7.4 — linearForwardFloat: Complete Line-by-Line

```
void linearForwardFloat(layerConfig_t *cfg, tensor_t *input, tensor_t *output) {
    linearConfig_t *lc = cfg->linear; // access linear config via union
    lc->forwardInput = input; // save for backprop
    tensor_t *w = &lc->weights.param; // weight tensor
    tensor_t *b = &lc->bias.param; // bias tensor
    transposeTensor(w, 0, 1); // W^T: shape [inF, outF]
    if (lc->mask != NULL) {
        riglMaskedMatmulFloat32(input, w, output, b, lc->mask);
    } else {
        matmulFloat32TensorsWithBias(input, w, output, b);
    }
    transposeTensor(w, 0, 1); // restore: shape [outF, inF]
}
```

1	linearConfig_t *lc = cfg->linear;	Access the linear config via the union. cfg is passed as layerConfig_t* (the union). cfg->linear accesses the linearConfig_t* member. lc is now a typed pointer we can use to access all linearConfig fields.
2	lc->forwardInput = input;	Store the input pointer. The backward pass (linearCalcWeightGradsFloat) needs the input to compute $dL/dW = dL/dOutput^T @ Input$. Saving here avoids passing the input through every backward function call.
4	transposeTensor(w, 0, 1);	Transpose the weight matrix in-place (O(1) — just swaps orderOfDimensions). Weights are stored as [outFeatures, inFeatures]. matmul computes $A @ B$ where A is input [batch, inF] and B is [N, K] with B transposed. After transpose: w has shape [inF, outF] logically, but same bytes.
5-8	if mask != NULL: sparse matmul, else dense matmul	Dispatch: use RigL masked matmul for sparse layers, standard matmul for dense. The mask is part of linearConfig_t, set during RigL setup. Dense layers (mask=NULL) have zero overhead from RigL.
9	transposeTensor(w, 0, 1); // restore	Undo the transpose. transposeTensor is an O(1) logical operation — it only swaps indices in orderOfDimensions, not bytes. The restore ensures the weight matrix is always stored as [outF, inF] for the backward pass and weight update.

Section 7.5 — linearForwardSymInt32: Quantized Forward Pass

```
void linearForwardSymInt32(layerConfig_t *cfg, tensor_t *input, tensor_t *output) {
    linearConfig_t *lc = cfg->linear;
    lc->forwardInput = input;
    tensor_t *w = &lc->weights.param; // SYM_INT32 weights
    tensor_t *b = &lc->bias.param; // bias (float or SYM_INT32)
    transposeTensor(w, 0, 1);
    matmulSymInt32TensorsWithBias(input, w, output, b);
    transposeTensor(w, 0, 1);
    // After matmul: output is SYM_INT32; requantize if needed:
    quantizeSymInt32(output); // Pass B: rescale to output's qConfig.scale
}
```

6	matmulSymInt32TensorsWithBias(input, w, output, b);	The FQT matmul. Uses int64_t accumulator. Output scale = input_scale * weight_scale. Result mantissas may be very large before requantization.
8	quantizeSymInt32(output);	Pass B requantization: rescale the output to the designated output scale. Finds max absolute value of result, computes new scale = max_abs / qMax, clamps all mantissas to [-qMax, qMax].

Section 7.6 — Linear Backward: Computing All Three Gradients

■ C CONCEPT: Three Gradients in a Linear Layer

A linear layer $Y = X @ W^T + b$ has three sets of parameters to differentiate: Given the incoming gradient dL/dY (shape [batch, outF]):

1. Weight gradient: $dL/dW = dL/dY^T @ X$ Shape [outF, inF] = [outF, batch] @ [batch, inF] Used to update W: $W = W - lr * dL/dW$
2. Bias gradient: $dL/db = \text{sum over batch of } dL/dY$ Shape [outF] = $\sum_i dL/dY[i,:]$ Used to update b: $b = b - lr * dL/db$
3. Input gradient (for upstream layers): $dL/dX = dL/dY @ W$ Shape [batch, inF] = [batch, outF] @ [outF, inF] Passed to the previous layer's backward function. All three are needed for a correct backpropagation implementation.

```
// linearCalcWeightGradsFloat: compute dL/dW and accumulate
void linearCalcWeightGradsFloat(tensor_t *lossGrad, tensor_t *forwardInput,
    tensor_t *weightGrads) {
    // dL/dW = lossGrad^T @ forwardInput
    // lossGrad: [batch, outF], forwardInput: [batch, inF]
    // lossGrad^T: [outF, batch], result: [outF, inF]
    transposeTensor(lossGrad, 0, 1); // [outF, batch]
    tensor_t tempGrads; /* same shape as weightGrads */
    matmulFloat32TensorsWithBias(lossGrad, forwardInput, &tempGrads, &zeroBias);
    transposeTensor(lossGrad, 0, 1); // restore [batch, outF]
    // ACCUMULATE (don't overwrite): += for multi-batch training:
    addFloat32TensorsInplace(weightGrads, &tempGrads);
}

// linearCalcInputGradsFloat: compute dL/dX to pass to previous layer
void linearCalcInputGradsFloat(tensor_t *lossGrad, tensor_t *w, tensor_t *inputGrads) {
    // dL/dX = lossGrad @ W ([batch,outF] @ [outF,inF] = [batch,inF])
    matmulFloat32TensorsWithBias(lossGrad, w, inputGrads, &zeroBias);
}

// linearCalcBiasGradsFloat: sum dL/dY over batch dimension
void linearCalcBiasGradsFloat(tensor_t *lossGrad, tensor_t *biasGrads) {
    size_t batch = getDimensionsByIndex(lossGrad, 0);
    size_t outF = getDimensionsByIndex(lossGrad, 1);
    for (size_t j = 0; j < outF; j++) {
        float sum = 0.0f;
        for (size_t i = 0; i < batch; i++) {
            sum += readBytesAsFloat(lossGrad->data + (i*outF+j)*4);
        }
        writeFloatToByteArray(sum, biasGrads->data + j*4);
    }
}
```

6	<code>transposeTensor(lossGrad, 0, 1);</code>	Transpose lossGrad from [batch, outF] to [outF, batch]. O(1) operation — just swaps orderOfDimensions, no data movement. After this: <code>matmul</code> sees lossGrad as [outF, batch].
7	<code>matmulFloat32TensorsWithBias(lossGrad, forwardInput, &tempGrads, &zeroBias);</code>	$dL/dW = \text{lossGrad}^T @ X = [\text{outF}, \text{batch}] @ [\text{batch}, \text{inF}] = [\text{outF}, \text{inF}]$. Uses <code>zeroBias</code> (all zeros) since the weight gradient has no bias term.
10	<code>addFloat32TensorsInplace(weightGrads, &tempGrads);</code>	ACCUMULATE — do not overwrite. Over a mini-batch, each sample's gradient is added. After all samples: <code>weightGrads</code> holds the SUM of gradients. The optimizer divides by <code>batch_size</code> implicitly via learning rate scaling.
14-1 6	<code>linearCalcInputGradsFloat: dL/dX = lossGrad @ W</code>	Matrix multiply: $[\text{batch}, \text{outF}] @ [\text{outF}, \text{inF}] = [\text{batch}, \text{inF}]$. This is the gradient that gets passed to the PREVIOUS layer's backward function. Backpropagation: chain of these input gradient computations from output to input.
18-2 7	<code>linearCalcBiasGradsFloat: sum over batch</code>	The bias gradient is the sum of lossGrad over all batch samples for each output neuron. <code>lossGrad[i][j]</code> is the gradient for sample <i>i</i> , neuron <i>j</i> . Sum over <i>i</i> gives $dL/db[j]$.

Section 7.7 — ReLU Layer: Forward, Backward, and Quantized

```
// reluForwardFloat: f(x) = max(0, x) element-wise
void reluForwardFloat(tensor_t *inOut) { // in-place: output = input
    size_t N = calcNumberOfElementsByTensor(inOut);
    for (size_t i = 0; i < N; i++) {
        float v = readBytesAsFloat(inOut->data + i*4);
        writeFloatToByteArray((v > 0.0f) ? v : 0.0f, inOut->data + i*4);
    }
}

// reluBackwardFloat: dL/dX = dL/dY * (Y > 0)
// (gradient is 1 where forward output > 0, 0 where output was clamped to 0)
void reluBackwardFloat(tensor_t *lossGrad, tensor_t *forwardOutput) {
    size_t N = calcNumberOfElementsByTensor(lossGrad);
    for (size_t i = 0; i < N; i++) {
        float fwdOut = readBytesAsFloat(forwardOutput->data + i*4);
        float grad = readBytesAsFloat(lossGrad->data + i*4);
        // Multiply gradient by the derivative of ReLU:
        float maskedGrad = (fwdOut > 0.0f) ? grad : 0.0f;
        writeFloatToByteArray(maskedGrad, lossGrad->data + i*4);
    }
}

// reluForwardSymInt32: ReLU on quantized mantissas (clamp negative mantissas to 0)
void reluForwardSymInt32(tensor_t *inOut) {
    size_t N = calcNumberOfElementsByTensor(inOut);
    for (size_t i = 0; i < N; i++) {
        int32_t v = readBytesAsInt32(inOut->data + i*4);
        writeInt32ToByteArray((v > 0) ? v : 0, inOut->data + i*4);
    }
}
```

5	<code>(v > 0.0f) ? v : 0.0f</code>	Ternary operator: if $v > 0$, keep <i>v</i> ; otherwise output 0. The compiler typically generates a VMAX instruction on Cortex-M7 FPU: 1 cycle.
10	<code>void reluBackwardFloat(tensor_t *lossGrad, tensor_t *forwardOutput)</code>	The backward pass of ReLU uses the FORWARD OUTPUT (not input). We check if the forward output was > 0 (meaning input was also > 0). If yes: gradient passes through unchanged. If no (output was clamped to 0): gradient is zeroed ('dead neuron').
17	<code>float maskedGrad = (fwdOut > 0.0f) ? grad : 0.0f;</code>	This implements $d(\text{ReLU})/dx = 1$ if $x > 0$, 0 if $x \leq 0$. The chain rule: $dL/dX = dL/dY * d(\text{ReLU})/dX = dL/dY * (Y > 0)$. Neurons that were 'off' during forward pass receive zero gradient — they don't learn.

21-2 5	reluForwardSymInt32	Same logic but for int32_t mantissas. A negative mantissa means a negative real value (since scale > 0). Setting it to 0 is correct: ReLU(negative) = 0.
-----------	---------------------	--

Section 7.8 — Softmax: Numerical Stability and Forward Pass

■ C CONCEPT: Numerical Stability in Softmax

$\text{softmax}(x)_i = \exp(x_i) / \sum_j (\exp(x_j))$ Problem: if x_i is large (e.g. 1000), $\exp(1000) = \text{inf}$ (float overflow). Solution: subtract the maximum before computing exp: $\text{softmax}(x)_i = \exp(x_i - \max(x)) / \sum_j (\exp(x_j - \max(x)))$ This is mathematically identical (the max cancels in numerator and denominator) but numerically stable: the largest exponent is $\exp(0) = 1$. All others are $\exp(\text{negative})$ in $(0, 1]$. No overflow possible. Example: $x = [1000, 1001, 999]$
Without shift: $\exp([1000, 1001, 999]) = [\text{inf}, \text{inf}, \text{inf}] \rightarrow \text{NaN}$ With shift (max=1001): $\exp([-1, 0, -2]) = [0.368, 1.0, 0.135]$
sum = 1.503, softmax = [0.245, 0.665, 0.090]

```
void softmaxForwardFloat(tensor_t *inOut) {
    size_t N = calcNumberOfElementsByTensor(inOut);
    // Step 1: find max value for numerical stability
    float maxVal = -FLT_MAX;
    for (size_t i = 0; i < N; i++) {
        float v = readBytesAsFloat(inOut->data + i*4);
        if (v > maxVal) maxVal = v;
    }
    // Step 2: compute exp(x - max) and sum
    float sumExp = 0.0f;
    for (size_t i = 0; i < N; i++) {
        float v = readBytesAsFloat(inOut->data + i*4);
        float eVal = expf(v - maxVal);
        writeFloatToByteArray(eVal, inOut->data + i*4); // temp store
        sumExp += eVal;
    }
    // Step 3: divide by sum to normalize
    for (size_t i = 0; i < N; i++) {
        float eVal = readBytesAsFloat(inOut->data + i*4);
        writeFloatToByteArray(eVal / sumExp, inOut->data + i*4);
    }
}
```

3-7	Step 1: find maxVal	Single pass to find the maximum logit. Initialized to -FLT_MAX (the most negative float) so any real value is larger.
9-14	Step 2: $\exp(v - \text{maxVal})$	$v - \text{maxVal}$: at most 0 (for the max element), always ≤ 0 for others. expf: the single-precision exponential function from math.h. Results stored BACK to inOut temporarily (overwrites logits).
16-1 9	Step 3: divide by sumExp	Each exp value divided by the total gives a probability. After this: all values sum to 1.0 and are in $(0, 1]$. The output IS the probability distribution over N speaker classes.

Section 7.9 — Dropout: Regularization with Bernoulli Masks

■ C CONCEPT: Dropout — What It Does and Why

Dropout randomly zeroes a fraction p of activations during training. Each activation is independently dropped with probability p . Why: prevents co-adaptation — neurons cannot rely on specific other neurons being active. Forces each neuron to learn features that work independently. Reduces overfitting (model memorizing training data, failing on new data). Inverted dropout: kept activations are scaled by $1/(1-p)$. This ensures the EXPECTED output value is the same as without dropout. During inference: dropout is disabled, no scaling needed. Example: $p=0.5$ (drop 50%), input=[1.0, 2.0, 3.0, 4.0] mask = [1, 0, 1, 0] (random) dropped = [1.0, 0.0, 3.0, 0.0] scaled = [2.0, 0.0, 6.0, 0.0] ($\times 1/(1-0.5) = 2$)

```
void dropoutForwardFloat(layerConfig_t *cfg, tensor_t *input, tensor_t *output) {
    dropoutConfig_t *dc = cfg->dropout;
    if (!dc->training) { // inference: pass through unchanged
        // copy input to output (or point output->data = input->data)
        memcpy(output->data, input->data,
            calcNumberOfElementsByTensor(input) * 4);
        return;
    }
    float keepProb = 1.0f - dc->dropProb; // e.g., 0.5 for 50% drop
    float scale = 1.0f / keepProb; // inverted dropout scale
    size_t N = calcNumberOfElementsByTensor(input);
    // Generate Bernoulli mask: each element kept with prob keepProb:
    bernoulliSampleTensor(dc->mask, keepProb);
    // Apply mask and scale:
    for (size_t i = 0; i < N; i++) {
        float v = readBytesAsFloat(input->data + i*4);
        bool keep = tensorBoolGet(dc->mask, i);
        writeFloatToByteArray(keep ? v * scale : 0.0f, output->data + i*4);
    }
}

// dropoutBackwardFloat: pass gradient through where mask was 1, zero where mask was 0:
void dropoutBackwardFloat(layerConfig_t *cfg, tensor_t *lossGrad, tensor_t *outGrad) {
    dropoutConfig_t *dc = cfg->dropout;
    float keepProb = 1.0f - dc->dropProb;
    float scale = 1.0f / keepProb;
    size_t N = calcNumberOfElementsByTensor(lossGrad);
    for (size_t i = 0; i < N; i++) {
        float g = readBytesAsFloat(lossGrad->data + i*4);
        bool kept = tensorBoolGet(dc->mask, i); // SAME mask as forward pass
        writeFloatToByteArray(kept ? g * scale : 0.0f, outGrad->data + i*4);
    }
}
```

2-7	<code>if (!dc->training) { memcpy; return; }</code>	During inference: dropout is disabled. The layer is a pass-through. <code>dc->training</code> must be set to false before inference. forgetting to do so would randomly drop features and give wrong predictions.
12	<code>bernoulliSampleTensor(dc->mask, keepProb);</code>	Generate a new Bernoulli mask: each bit set to 1 with probability <code>keepProb</code> . Uses <code>rngNextFloat() < keepProb</code> . Different mask each forward pass.
15-17	<code>keep ? v * scale : 0.0f</code>	If this position is kept (mask=1): multiply by scale (inverted dropout). If dropped (mask=0): output 0. The scale ensures the expected value equals the no-dropout value.
25	<code>bool kept = tensorBoolGet(dc->mask, i);</code>	Backward uses the SAME mask as the forward pass. The mask is saved in <code>dc->mask</code> between forward and backward calls. Gradient is zeroed where forward output was zeroed, and scaled where it was kept.

Section 7.10 — Conv1D: 1D Convolution for Sequential Data

■ C CONCEPT: 1D Convolution — Mathematical Formula

Conv1D applies a learnable filter (kernel) to a 1D signal. Input: X of shape [batch, inChannels, length] Kernel: K of shape [outChannels, inChannels, kernelSize] Output: Y of shape [batch, outChannels, outLength] $\text{outLength} = (\text{length} - \text{kernelSize}) / \text{stride} + 1$ (no padding) Formula for output element $Y[b, oc, t]$: $Y[b, oc, t] = \sum_{ic=0}^{inC-1} \sum_{k=0}^{kSize-1} X[b, ic, t*stride+k] * K[oc, ic, k] + \text{bias}[oc]$ Intuition: slide the kernel along the time axis. At each position t, compute the dot product of the kernel with a window of the input. Different output channels detect different local patterns.

```
typedef struct {
    parameter_t weights; // kernel shape [outC, inC, kernelSize]
    parameter_t bias; // bias shape [outC]
    size_t inChannels; // C_in
    size_t outChannels; // C_out
    size_t kernelSize; // K (window size)
    size_t stride; // step between windows
    bool useBias;
    tensor_t *forwardInput; // saved for backprop
} conv1dConfig_t;

void conv1dForwardFloat(layerConfig_t *cfg, tensor_t *input, tensor_t *output) {
    conv1dConfig_t *cc = cfg->conv1d;
    size_t batch = getDimensionsByIndex(input, 0);
    size_t inC = cc->inChannels;
    size_t L = getDimensionsByIndex(input, 2); // sequence length
    size_t outC = cc->outChannels;
    size_t kSize = cc->kernelSize;
    size_t stride = cc->stride;
    size_t outLen = (L - kSize) / stride + 1;
    cc->forwardInput = input;
    for (size_t b = 0; b < batch; b++) {
        for (size_t oc = 0; oc < outC; oc++) {
            float bias = cc->useBias ? readBytesAsFloat(cc->bias.param.data + oc*4) : 0.0f;
            for (size_t t = 0; t < outLen; t++) {
                float sum = bias;
                for (size_t ic = 0; ic < inC; ic++) {
                    for (size_t k = 0; k < kSize; k++) {
                        size_t xIdx = (b*inC + ic)*L + (t*stride + k);
                        size_t wIdx = (oc*inC + ic)*kSize + k;
                        float xVal = readBytesAsFloat(input->data + xIdx*4);
                        float wVal = readBytesAsFloat(cc->weights.param.data + wIdx*4);
                        sum += xVal * wVal;
                    }
                }
                size_t outIdx = (b*outC + oc)*outLen + t;
                writeFloatToByteArray(sum, output->data + outIdx*4);
            }
        }
    }
}
```

17	<code>size_t outLen = (L - kSize) / stride + 1;</code>	Output length formula (no padding). L=100, kSize=3, stride=1: <code>outLen=(100-3)/1+1 = 98</code> . L=100, kSize=3, stride=2: <code>outLen=(100-3)/2+1 = 49</code> .
26	<code>float sum = bias;</code>	Initialize accumulator with the bias. Avoids a separate bias-addition loop after the convolution.
27-31	<code>size_t xIdx</code> and <code>wIdx</code>	<code>xIdx = (batch * inChannels + inputChannel) * length + time_offset</code> . <code>wIdx = (outChannel * inChannels + inputChannel) * kernelSize + kernelOffset</code> . These are the flat indices for C-order (row-major) 3D tensors.

Section 7.11 — Flatten Layer: Zero-Copy Reshape

■ C CONCEPT: Zero-Copy Reshape — How Flatten Works

Flatten converts a multi-dimensional tensor (e.g., [batch, channels, length]) into a 2D tensor [batch, channels * length], WITHOUT moving any data. All the data is already in a flat byte array. All we need to do is update the shape metadata: Before: numberOfDimensions=3, dimensions=[1, 4, 8] After: numberOfDimensions=2, dimensions=[1, 32] The data bytes are identical. Only the shape struct changes. This is called a zero-copy operation: O(1) time, 0 bytes copied. Very important for MCU performance — no 32KB memcpY needed to flatten.

```
void flattenForwardFloat(layerConfig_t *cfg, tensor_t *input, tensor_t *output) {
// Point output at the SAME data as input (no copy):
output->data = input->data;
// Compute total elements in all dimensions after dim 0 (batch):
size_t batch = getDimensionsByIndex(input, 0);
size_t flatSize = calcNumberOfElementsByTensor(input) / batch;
// Update shape: [batch, flatSize]
output->shape->numberOfDimensions = 2;
output->shape->dimensions[0] = batch;
output->shape->dimensions[1] = flatSize;
output->shape->orderOfDimensions[0] = 0;
output->shape->orderOfDimensions[1] = 1;
// Quantization and sparsity pointers stay the same:
output->quantization = input->quantization;
}
```

2	output->data = input->data;	Both tensors point to the same byte array. No memcpY. No allocation. Reading output->data[i] reads the same byte as input->data[i]. This is the zero-copy property.
4	size_t flatSize = calcNumberOfElementsByTensor(input) / batch;	Total elements / batch = elements per sample. For [1, 4, 8]: total=32, batch=1, flatSize=32. For [4, 3, 8]: total=96, batch=4, flatSize=24.
6-10	Update shape to [batch, flatSize]	Set numberOfDimensions=2, update dimensions and orderOfDimensions. orderOfDimensions = {0, 1} (identity — no transpose). After this, getDimensionsByIndex(output, 0) = batch, getDimensionsByIndex(output, 1) = flatSize.
12	output->quantization = input->quantization;	The quantization type (FLOAT32, SYM_INT32) doesn't change after flatten. Copy the pointer so the output tensor has the correct quantization metadata.

Section 7.12 — RigL Integration: Per-Layer Mask and Masked SGD

RigL is integrated at the layer level: each linear layer can optionally have a mask. The mask is stored in linearConfig_t and checked in every forward pass. The optimizer (riglMaskedSgdStepFloat) skips pruned weights.

```
// Setting up a sparse linear layer with RigL:
// 1. Allocate the mask buffer (1 bit per weight):
static uint8_t maskBuf[(OUT_F * IN_F + 7) / 8]; // packed bits
// 2. Create the mask tensor (BOOL quantization):
static tensor_t maskTensor;
static boolQConfig_t boolCfg;
quantization_t boolQ;
initBoolQuantization(&boolQ, &boolCfg);
// set up maskTensor shape [OUT_F, IN_F], quantization=boolQ, data=maskBuf
// 3. Assign mask to the linear config:
linearConfig.mask = &maskTensor;
// 4. Initialize RigL config:
riglConfig_t riglCfg = {
    .weights = &linearConfig.weights.param,
    .grads = &linearConfig.weights.grad,
    .mask = &maskTensor,
    .sparsityRatio = 0.8f,
    .updateStep = 100,
    .batchCount = 0,
};
riglInitMask(&riglCfg, 42); // random initial mask, seed=42
// In training loop – mask-aware optimizer step:
riglMaskedSgdStepFloat(&riglCfg, LEARNING_RATE); // skips pruned weights
// In training loop – periodic topology update:
riglUpdate(&riglCfg); // fires every 100 batches (checks internally)
```

■ RigL GAP IN THIS FILE

RigL only modifies linear layers (or other parameterized layers). ReLU, Softmax, Dropout, and Flatten have no weights — no RigL needed. Conv1D can also be made sparse with RigL using the same mask mechanism. The dispatch table (`forwardFuncs[]`) routes to the same `linearForwardFloat`; the mask check inside that function determines sparse or dense behavior.

	Forward Function	Backward Function	RigL Support
	<code>linearForwardFloat</code>	<code>linearCalcWeightGradsFloat</code> + <code>linearCalcInputGradsFloat</code>	Yes (mask field in config)
<code>_INT32)</code>	<code>linearForwardSymInt32</code>	<code>linearCalcWeightGradsSymInt32</code>	Yes (same mask mechanism)
	<code>reluForwardFloat</code>	<code>reluBackwardFloat</code>	N/A (no weights)
	<code>softmaxForwardFloat</code>	N/A (combined with loss)	N/A
	<code>dropoutForwardFloat</code>	<code>dropoutBackwardFloat</code>	N/A
	<code>conv1dForwardFloat</code>	<code>conv1dBackwardFloat</code>	Possible (not in base ODT)
	<code>flattenForwardFloat</code>	N/A (zero-copy)	N/A

Layer Implementations Deep Dive

Every forward and backward pass, line by line

15.1 What Are Neural Network Layers?

A neural network is nothing more than a chain of mathematical operations called **layers**. Each layer takes a tensor in, does some math, and produces a tensor out. The magic is that when you stack many of these layers and tune their internal parameters (weights and biases) using gradient descent, the network learns to map inputs to outputs.

In the ODT library, every layer follows the same interface contract defined in `Layer.h`. This means you can add any new layer type without touching the training loop — you just register three function pointers: forward, backward, and `calcOutputShape`.

■ C CONCEPT: Forward Pass vs Backward Pass

The **forward pass** computes the layer's output from its input — this is inference. The **backward pass** computes gradients — how much does changing each parameter affect the loss? The backward pass flows in reverse through the network, which is why it is called **backpropagation**.

15.2 Layer.h: The Interface Contract

Every layer is described by a `layer_t` struct. Let's look at each piece:

Layer.h — `layer_t` struct

```
typedef struct {
    layerType_t type; // Which kind of layer is this?
    layerConfig_t config; // Pointer to the layer's settings
} layer_t;
```

The `layerType_t` is an enum — an integer constant with a name. ODT defines: LINEAR, RELU, SOFTMAX, CONV1D, DROPOUT, FLATTEN, BATCHNORM, EMBEDDING, ATTENTION, POOLING, LOSS.

The `layerConfig_t` is a **union** — a single block of memory that can hold a pointer to *any* of the 11 config structs. A union is like a parking space: only one car (value) at a time, but any car that fits can park there.

layerConfig_t union — one memory slot for all layer config types

```
typedef union {
    linearConfig_t *linear;
    reluConfig_t *relu;
    softmaxConfig_t *softmax;
    conv1dConfig_t *conv1d;
    dropoutConfig_t *dropout;
    flattenConfig_t *flatten;
    batchNormConfig_t *batchNorm;
    embeddingConfig_t *embedding;
    attentionConfig_t *attention;
    poolingConfig_t *pooling;
    lossConfig_t *loss;
} layerConfig_t;
```

■ C CONCEPT: Why Use a Union Here?

All pointers on a 32-bit CPU are 4 bytes. A union that holds any pointer therefore costs exactly 4 bytes — the same as a single pointer. Without the union, you'd need 11 pointer fields (44 bytes) even though only one is ever used at a time. The union is pure memory efficiency.

15.2.1 The Dispatch Table — `layerFunctions[]`

The dispatch table connects a layer type (an integer) to its three function pointers. It is an array where the index IS the layer type.

Layer.h — dispatch table entry and full table

```
typedef void (*forwardFn_t)(layer_t*, tensor_t*, tensor_t*);
typedef void (*backwardFn_t)(layer_t*, tensor_t*, tensor_t*,
tensor_t*, tensor_t*);
typedef void (*calcOutputShapeFn_t)(layer_t*, shape_t*, shape_t*);
typedef struct {
forwardFn_t forward;
backwardFn_t backward;
calcOutputShapeFn_t calcOutputShape;
} layerFunctions_t;
extern layerFunctions_t layerFunctions[]; // declared in Layer.h
// In Layer.c – definition using designated initializers:
layerFunctions_t layerFunctions[] = {
[LAYER_LINEAR] = { linearForward, linearBackward, linearCalcOutputShape },
[LAYER_RELU] = { reluForward, reluBackward, reluCalcOutputShape },
[LAYER_SOFTMAX] = { softmaxForward, softmaxBackward, softmaxCalcOutputShape },
[LAYER_CONV1D] = { conv1dForward, conv1dBackward, conv1dCalcOutputShape },
[LAYER_DROPOUT] = { dropoutForward, dropoutBackward, dropoutCalcOutputShape },
[LAYER_FLATTEN] = { flattenForward, flattenBackward, flattenCalcOutputShape },
};
```

When the training loop calls `layerFunctions[layer->type].forward(layer, input, output)`, C looks up the function pointer at index `layer->type` and calls it. This is called a **vtable** pattern — it is how object-oriented languages implement virtual methods, but here we do it explicitly with C arrays.

— WORKED EXAMPLE

Calling a layer generically: `layer_t myLayer; // could be linear, relu, anything // The same call works for ANY layer type: layerFunctions[myLayer.type].forward(&myLayer, &inputTensor, &outputTensor); // C resolves the correct function at runtime based on myLayer.type`

15.3 Linear Layer — Full Deep Dive

The Linear (fully connected / dense) layer is the fundamental building block of neural networks. It computes: $\text{output} = \text{input} \times W^T + b$, where W is the weight matrix and b is the bias vector.

Mathematically: if input has shape $[\text{batchSize}, \text{inputFeatures}]$ and W has shape $[\text{outputFeatures}, \text{inputFeatures}]$, then output has shape $[\text{batchSize}, \text{outputFeatures}]$. Each output neuron is a dot product of the input with one row of W .

15.3.1 linearConfig_t — All 7 Fields

Linear.h — linearConfig_t struct <pre>typedef struct { parameter_t weights; // {tensor_t param; tensor_t grad;} parameter_t bias; // bias parameter + its gradient bool hasBias; // true = add bias; false = skip bias bool freezeWeights; // true = don't update weights during backward bool freezeBias; // true = don't update bias during backward bool isTraining; // true during training, false during inference tensor_t *inputCache; // stores input for use in backward pass } linearConfig_t;</pre>		
weights		A parameter_t holds both the value tensor (.param) and its gradient tensor (.grad). For a linear layer mapping 64→32, weights.param has shape [32,64].
bias		Shape [outputFeatures]. Added to each row of the output.
hasBias		If false, the bias addition is skipped entirely — useful when a BatchNorm layer follows (it would cancel the bias anyway).
freezeWeights/freezeBias		Skip gradient accumulation for these parameters. Used in transfer learning where you want to train only part of the network.
inputCache		Pointer to the forward pass input tensor. The backward pass needs this to compute the weight gradient: $dW = d\text{Output}^T \times \text{input}$.

15.3.2 linearForwardFloat — Complete Walkthrough

Linear.c — linearForwardFloat

```
void linearForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {
    linearConfig_t *cfg = layer->config.linear; // Step 1
    cfg->inputCache = input; // Step 2
    // Step 3: transpose input from [batch, in] → [in, batch]
    transposeTensor(input);
    // Step 4: output = weights × inputAT + bias
    // weights shape: [out, in]
    // input (transposed) shape: [in, batch]
    // result shape: [out, batch]
    matmulFloat32TensorsWithBias(
        &cfg->weights.param, // A: [out, in]
        input, // B: [in, batch] (transposed)
        cfg->hasBias ? &cfg->bias.param : NULL,
        output // C: [out, batch]
    );
    // Step 5: un-transpose input (restore caller's tensor)
    transposeTensor(input);
    // Step 6: transpose output from [out, batch] → [batch, out]
    transposeTensor(output);
}
```

Step 1		Cast the generic config union to linearConfig_t*. This is safe because we know this function is only called when layer->type == LAYER_LINEAR.
Step 2		Store the input pointer in inputCache. The backward pass will read this later to compute weight gradients. Note: only the pointer is stored, not a copy. The caller must keep the input tensor alive until backward completes.
Step 3		transposeTensor is O(1) — it just swaps orderOfDimensions[0] and orderOfDimensions[1]. Now what was [batch, in] logically looks like [in, batch] when accessed via tensor index functions.
Step 4		matmulFloat32TensorsWithBias computes A×B+bias. The shapes are [out,in] × [in,batch] = [out,batch]. If hasBias is false, NULL is passed and the matmul skips the bias addition.
Step 5		Restore the input tensor to its original orientation. The caller expects the input to be unchanged after the forward pass.
Step 6		The matmul result is [out,batch]. Transpose to [batch,out] so that the next layer receives data in the conventional [batch, features] format.

— WORKED EXAMPLE

Shape trace through linearForwardFloat: Input: [4, 64] (batch=4, input_features=64) Weights: [32, 64] (output_features=32, input_features=64) Bias: [32] (one bias per output neuron) After transposeTensor(input): logically [64, 4] After matmul([32,64] × [64,4]): result [32, 4] After transposeTensor(output): output is [4, 32] ✓

15.3.3 linearBackward — Computing All Three Gradients

The backward pass receives `dOutput` — the gradient of the loss with respect to this layer's output. It must compute:

1. $dWeights = dOutput^T \times input$ (shape: [out, in])
2. $dBias = \text{sum over batch of } dOutput$ (shape: [out])
3. $dInput = dOutput \times weights$ (shape: [batch, in]) — propagates loss upstream

Linear.c — linearBackward (simplified)

```
void linearBackward(layer_t *layer, tensor_t *input,
tensor_t *dOutput, tensor_t *dInput) {
    linearConfig_t *cfg = layer->config.linear;
    if (!cfg->freezeWeights) {
        // dWeights = dOutput^T x inputCache
        // dOutput shape: [batch, out] → transpose → [out, batch]
        // inputCache shape: [batch, in]
        // [out, batch] x [batch, in] = [out, in] ← same as weights shape
        transposeTensor(dOutput);
        matmulFloat32TensorsWithBias(dOutput, cfg->inputCache,
        NULL, &cfg->weights.grad);
        transposeTensor(dOutput); // restore
    }
    if (cfg->hasBias && !cfg->freezeBias) {
        // dBias[j] = sum over batch of dOutput[batch, j]
        for (size_t j = 0; j < biasSize; j++) {
            float sum = 0.0f;
            for (size_t b = 0; b < batchSize; b++)
                sum += dOutput[b * outFeatures + j];
            biasGradPtr[j] += sum;
        }
    }
    if (dInput != NULL) {
        // dInput = dOutput x weights (no transpose on weights)
        // [batch, out] x [out, in] = [batch, in]
        matmulFloat32TensorsWithBias(dOutput, &cfg->weights.param,
        NULL, dInput);
    }
}
```

■ C CONCEPT: Why += for gradient accumulation?

Gradients use `+=` because a single parameter (like a weight) can affect multiple outputs. Each path through the network contributes a partial gradient, and the total gradient is the sum of all contributions. This is the chain rule applied across a batch.

15.3.4 linearCalcOutputShape

Linear.c — linearCalcOutputShape

```
void linearCalcOutputShape(layer_t *layer, shape_t *inputShape,
shape_t *outputShape) {
    linearConfig_t *cfg = layer->config.linear;
    size_t outFeatures = cfg->weights.param.shape.dimensions[0];
    // Output has same batch dimensions, but last dim changes
    *outputShape = *inputShape; // copy all dims
    outputShape->dimensions[inputShape->numberOfDimensions - 1] = outFeatures;
}
```

This function is used before allocating output tensors. By knowing input shape and layer config, the training infrastructure can pre-allocate the correct output buffer.

15.4 ReLU Layer — Complete Analysis

ReLU stands for **R**ectified **L**inear **U**nit. It is the most widely used activation function in neural networks. The formula is simple: $\text{ReLU}(x) = \max(0, x)$. Negative values become 0; positive values pass through unchanged.

Why is this useful? Without non-linear activation functions like ReLU, stacking multiple linear layers is mathematically equivalent to a single linear layer — the network cannot learn complex patterns. ReLU introduces non-linearity cheaply.

15.4.1 reluForwardFloat — Line by Line

Relu.c — reluForwardFloat

```
void reluForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {
    size_t n = calcNumberOfElementsByShape(&input->shape);
    float *inPtr = (float*) input->data.dataPTR;
    float *outPtr = (float*) output->data.dataPTR;
    for (size_t i = 0; i < n; i++) {
        outPtr[i] = gteFloatValue(inPtr[i], 0.0f) ? inPtr[i] : 0.0f;
    }
}
```

Line 2		<code>calcNumberOfElementsByShape</code> multiplies all dimensions together. For shape [4, 32], n = 128. We loop over every element.
Line 3-4		Cast the void* data pointer to float*. This lets us use array indexing with [i] instead of byte-level pointer arithmetic.
Line 5-7		For each element: if it is ≥ 0 , copy it. Otherwise write 0. The <code>gteFloatValue(a, b)</code> helper returns true if $a \geq b$.

Note that ReLU has no trainable parameters — `reluConfig_t` exists only to hold a pointer to the input tensor cached for the backward pass.

15.4.2 reluForwardSymInt32 — Quantized ReLU

Relu.c — reluForwardSymInt32

```
void reluForwardSymInt32(layer_t *layer, tensor_t *input, tensor_t *output) {
    size_t n = calcNumberOfElementsByShape(&input->shape);
    int32_t *inPtr = (int32_t*) input->data.dataPTR;
    int32_t *outPtr = (int32_t*) output->data.dataPTR;
    for (size_t i = 0; i < n; i++) {
        outPtr[i] = gteSymInt32Zero(inPtr[i]) ? inPtr[i] : 0;
    }
    // Copy scale from input to output (ReLU doesn't change scale)
    symInt32QConfig_t *inQ = input->quantization.qConfig;
    symInt32QConfig_t *outQ = output->quantization.qConfig;
    outQ->scale = inQ->scale;
    outQ->qMaxBits = inQ->qMaxBits;
}
```

gteSymInt32Zero		For SYM_INT32, zero is represented as the integer 0 (because the quantization is symmetric). So this is just checking if the integer mantissa ≥ 0 , which is equivalent to checking if the real value ≥ 0 .
Scale copy		ReLU does not change the magnitude of positive values, so the output has the same quantization scale as the input. This is important — if we forgot to copy the scale, the next layer would decode the tensor wrong.

■ C CONCEPT: Symmetric Quantization and Zero

In SYM_INT32: $\text{real_value} = \text{mantissa} \times \text{scale}$. Zero mantissa $\rightarrow$ zero real value. Checking $\text{mantissa} \geq 0$ is exactly the same as checking $\text{real_value} \geq 0$, because scale is always positive. This is why symmetric quantization is convenient for ReLU.

15.4.3 reluBackward

Relu.c — reluBackward

```
void reluBackward(layer_t *layer, tensor_t *dOutput, tensor_t *dInput) {
    size_t n = calcNumberOfElementsByShape(&dOutput->shape);
    float *dOutPtr = (float*) dOutput->data.dataPTR;
    float *dInPtr = (float*) dInput->data.dataPTR;
    float *cachedIn = (float*) layer->config.relu->inputCache->data.dataPTR;
    for (size_t i = 0; i < n; i++) {
        // If the forward input was > 0, gradient passes through
        // If the forward input was <= 0, gradient is blocked (= 0)
        dInPtr[i] = (cachedIn[i] > 0.0f) ? dOutPtr[i] : 0.0f;
    }
}
```

The ReLU backward pass implements the derivative of $\max(0, x)$: it equals 1 when $x > 0$ and 0 otherwise. By multiplying the upstream gradient by this derivative, gradients only flow back through neurons that were 'on' (positive) during the forward pass. Neurons that were 'off' (negative) get zero gradient — they are 'dead' for this sample.

■■ IMPORTANT

The Dying ReLU Problem: If a neuron always receives negative input, its gradient is always 0, so it never gets updated. This is called a 'dead ReLU'. Solutions include: careful weight initialization, lower learning rates, or using LeakyReLU which allows a small negative slope instead of zero.

15.5 Softmax Layer — Probability Output

Softmax converts a vector of arbitrary real numbers (called **logits**) into a probability distribution — all values are in [0,1] and they sum to 1. It is used as the last layer in classification networks.

The formula: $\text{softmax}(x)[i] = \exp(x[i]) / \sum(\exp(x[j]))$ for all j

■ C CONCEPT: Numerical Stability Trick

Computing $\exp(x)$ directly can overflow for large x values (e.g., $\exp(1000) = \text{inf}$). The trick: subtract the maximum value first. Since $\text{softmax}(x) = \text{softmax}(x - \max(x))$, this shifts all values to ≤ 0 before exponentiating, preventing overflow.

Softmax.c — softmaxForward (simplified)

```
void softmaxForward(layer_t *layer, tensor_t *input, tensor_t *output) {
    size_t batch = input->shape.dimensions[0];
    size_t n = input->shape.dimensions[1]; // number of classes
    float *in = (float*) input->data.dataPTR;
    float *out = (float*) output->data.dataPTR;
    for (size_t b = 0; b < batch; b++) {
        float *row_in = in + b * n;
        float *row_out = out + b * n;
        // Step 1: find max for numerical stability
        float maxVal = row_in[0];
        for (size_t i = 1; i < n; i++)
            if (row_in[i] > maxVal) maxVal = row_in[i];
        // Step 2: compute exp(x - max) and sum
        float sum = 0.0f;
        for (size_t i = 0; i < n; i++) {
            row_out[i] = expf(row_in[i] - maxVal);
            sum += row_out[i];
        }
        // Step 3: divide by sum to normalize
        for (size_t i = 0; i < n; i++)
            row_out[i] /= sum;
    }
}
```

b * n		Pointer arithmetic to jump to the start of batch row b. If each row has n elements, batch b starts at offset b×n from the beginning of the array.
expf		C standard library function for float exponentiation. Use expf (not exp) for float — exp works on double, which is slower and unnecessary on an MCU.
Dividing by sum		This ensures the outputs sum to exactly 1.0, making them valid probabilities.

15.5.1 Why Softmax and Cross-Entropy Are Usually Combined

If you compute softmax backward separately and cross-entropy backward separately, you get a numerically unstable and complicated gradient. But if you combine them, the gradient simplifies beautifully: $d(\text{CE after softmax})/d(\text{logits}) = \text{pred} - \text{target}$.

This is why ODT implements `crossEntropySoftmaxBackward` as a single function that assumes softmax was applied just before cross-entropy. The combined backward is both simpler and more numerically stable.

15.6 Conv1D Layer — 1D Convolution for Sequences

1D convolution processes sequential data — like audio waveforms, time series, or sequences of feature vectors. It slides a small kernel (filter) across the input and computes dot products at each position.

For speaker identification on LibriSpeech audio, Conv1D extracts local temporal patterns — things like the shape of phonemes or short acoustic events that characterize a speaker's voice.

15.6.1 conv1dConfig_t — Every Field Explained

Conv1d.h — conv1dConfig_t

```
typedef struct {
    parameter_t weights; // shape: [outChannels, inChannels, kernelSize]
    parameter_t bias; // shape: [outChannels]
    bool hasBias;
    size_t inChannels; // input feature dimension
    size_t outChannels; // output feature dimension
    kernel_t kernel; // size, padding, stride, dilation
    bool isTraining;
    tensor_t *inputCache;
} conv1dConfig_t;

typedef struct {
    size_t size; // kernel size (e.g. 3, 5, 7)
    paddingType_t paddingType; // VALID, SAME, or EXPLICIT
    size_t stride; // step between kernel positions
    size_t dilation; // spacing between kernel elements
    size_t padding; // explicit padding amount (if EXPLICIT)
} kernel_t;
```

kernel Size		How many input time steps each output value looks at. A kernel of size 3 means each output is computed from 3 consecutive inputs.
stride		How many steps to advance the kernel each time. stride=1 gives maximum overlap (most detailed). stride=2 halves the output length.
dilation		Gaps between kernel elements. dilation=2 means the kernel samples at positions [0, 2, 4] instead of [0, 1, 2]. This gives a larger receptive field without more parameters.
paddingType= SAME		Adds zeros at the edges so the output length equals the input length (assuming stride=1).
paddingType= VALID		No padding. Output length = floor((L-K)/stride)+1 where L is input length and K is kernel size.

15.6.2 Conv1D Mathematical Formula

For each output channel c_{out} , output position t , and batch b :

Conv1D formula

```
output[b, c_out, t] = bias[c_out]
+ sum over c_in, k of:
weights[c_out, c_in, k] * input[b, c_in, t*stride + k*dilation]
```

—■ WORKED EXAMPLE

Conv1D with kernel=3, stride=1, no padding: input: [1, 1, 5] (batch=1, channels=1, length=5): [1, 2, 3, 4, 5]
kernel: [1, 1, 3] (1 out_channel, 1 in_channel, size=3): [0.5, 1.0, 0.5] output[t=0] = $1*0.5 + 2*1.0 + 3*0.5 = 0.5 + 2.0 + 1.5 = 4.0$ output[t=1] = $2*0.5 + 3*1.0 + 4*0.5 = 1.0 + 3.0 + 2.0 = 6.0$ output[t=2] = $3*0.5 + 4*1.0 + 5*0.5 = 1.5 + 4.0 + 2.5 = 8.0$ Output shape: [1, 1, 3] (length reduced from 5 to 3 by VALID padding)

15.7 Dropout Layer — Regularization

Dropout randomly sets a fraction p of the neurons to zero during training. This prevents co-adaptation: neurons cannot rely on specific other neurons, so they must learn more robust features. During inference, dropout is disabled.

ODT uses **inverted dropout**: during training, surviving neurons are scaled by $1/(1-p)$ so that the expected value of the output is the same as without dropout. This means no adjustment is needed at inference time.

15.7.1 dropoutConfig_t Fields

Dropout.h — dropoutConfig_t

```
typedef struct {
    float p; // dropout probability (fraction to zero out)
    bool training; // true = apply dropout; false = pass-through
    tensor_t *mask; // boolean mask (1=keep, 0=drop), saved for backward
    quantization_t *forwardQ; // quantization for mask tensor
    quantization_t *backwardQ;
    bool ownsQuantizations; // true = we allocated forwardQ/backwardQ
} dropoutConfig_t;
```

15.7.2 dropoutForward — Step by Step

Dropout.c — dropoutForward

```
void dropoutForward(layer_t *layer, tensor_t *input, tensor_t *output) {
    dropoutConfig_t *cfg = layer->config.dropout;
    size_t n = calcNumberOfElementsByShape(&input->shape);
    float *in = (float*) input->data.dataPTR;
    float *out = (float*) output->data.dataPTR;
    if (!cfg->training) {
        // Inference: copy input to output unchanged
        memcpy(out, in, n * sizeof(float));
        return;
    }
    // Training: generate random binary mask
    bernoulliSampleTensor(cfg->mask, 1.0f - cfg->p);
    // 1-p = probability of keeping a neuron
    float scale = 1.0f / (1.0f - cfg->p); // inverted dropout scale
    for (size_t i = 0; i < n; i++) {
        bool keep = tensorBoolGet(cfg->mask, i);
        out[i] = keep ? in[i] * scale : 0.0f;
    }
}
```

bernoulliSampleTensor		Fills <code>cfg->mask</code> with random bits: each bit is 1 with probability $(1-p)$ and 0 with probability p . Uses the RNG module.
scale = $1/(1-p)$		For $p=0.5$: $scale=2.0$. Surviving neurons are doubled, so their expected contribution equals the full (no-dropout) case.
tensorBoolGet		Reads a single bit from the packed boolean mask tensor. The mask uses 1 bit per element, so a 512-element mask costs only 64 bytes.
out[i] = 0.0f		Dropped neurons output exactly zero. During backward, zeros also block gradient flow through dropped neurons.

15.7.3 dropoutBackward

Dropout.c — dropoutBackward

```
void dropoutBackward(layer_t *layer, tensor_t *dOutput, tensor_t *dInput) {
    dropoutConfig_t *cfg = layer->config.dropout;
    size_t n = calcNumberOfElementsByShape(&dOutput->shape);
    float *dOut = (float*) dOutput->data.dataPTR;
    float *dIn = (float*) dInput->data.dataPTR;
    float scale = 1.0f / (1.0f - cfg->p);
    for (size_t i = 0; i < n; i++) {
        bool kept = tensorBoolGet(cfg->mask, i);
        dIn[i] = kept ? dOut[i] * scale : 0.0f;
    }
}
```

The backward pass uses the SAME mask as the forward pass (saved in `cfg->mask`). This ensures that gradients only flow through neurons that were active during the forward pass. The scale factor is applied again for consistency.

15.8 Flatten Layer — Zero-Copy Reshape

Flatten converts a multi-dimensional tensor into a 1D vector (per batch item). For example, after a Conv1D layer with output [batch, channels, length], Flatten produces [batch, channels × length]. This is needed before a Linear layer which expects 2D input.

The key insight in ODT's implementation: **no data is copied**. The underlying memory is identical — we just change the shape metadata.

Flatten.c — flattenForward

```
void flattenForward(layer_t *layer, tensor_t *input, tensor_t *output) {
    // Share the data buffer – no copy needed
    output->data.dataPTR = input->data.dataPTR;
    output->data.numberOfElements = input->data.numberOfElements;
    // Compute flat size: product of all dims except batch
    size_t flatSize = 1;
    for (size_t d = 1; d < input->shape.numberOfDimensions; d++)
        flatSize *= input->shape.dimensions[d];
    // Set output shape: [batchSize, flatSize]
    size_t batchSize = input->shape.dimensions[0];
    output->shape.numberOfDimensions = 2;
    output->shape.dimensions[0] = batchSize;
    output->shape.dimensions[1] = flatSize;
    output->shape.orderOfDimensions[0] = 0;
    output->shape.orderOfDimensions[1] = 1;
}
```

output->data.dataPTR = input->data.dataPTR

Both tensors point to the SAME memory. A write through output->data changes what input->data sees too. This is safe here because flatten's output is only read, not written.

flatSize

Multiply dimensions [1], [2], ... together. For Conv1D output [4, 16, 8] (batch=4, channels=16, length=8): flatSize = 16×8 = 128.

orderOfDimensions = {0, 1}

Reset to natural order after flattening. The 2D output is not transposed.

— WORKED EXAMPLE

Flatten shape transformation: Input shape: [4, 16, 8] (batch=4, 16 channels, length=8) Output shape: [4, 128] (batch=4, flat=16×8=128) Data in memory: unchanged — both tensors read the same bytes Memory saved: 0 bytes copied, 0 extra allocations

■ NOTE

Flatten and Memory Safety: Because flatten shares memory, the caller must not free the input tensor's data buffer while the output tensor is still in use. The training loop manages this by keeping all layer activations alive until the backward pass completes.

15.9 Layer Summary Table

Layer	Has Params?	Forward Cost	Key Feature
g_t	Yes (W, b)	$O(\text{in} \times \text{out} \times \text{batch})$	Fully connected
t	No	$O(n)$	Element-wise, i
config_t	No	$O(\text{classes} \times \text{batch})$	Normalizes to p
fig_t	Yes (W, b)	$O(\text{out} \times \text{in} \times K \times L \times \text{batch})$	Local pattern e
config_t	No	$O(n) + \text{RNG}$	Regularization,
g_t	No	$O(1)$	Zero-copy resh

■ RELATIONS WITH OTHER FILES

Chapter 15 connects to: Chapter 6 (Arithmetic) — `matmulFloat32TensorsWithBias`, `gteFloatValue` called by layers · Chapter 5 (Tensor) — `transposeTensor`, `calcNumberOfElementsByShape` used everywhere · Chapter 8 (Training) — `layerFunctions` dispatch table called by training loop · Chapter 9 (RNG) — `bernoulliSampleTensor` used by dropout · Chapter 11 (MCU) — All layers execute on-device, data flows through SRAM buffers

Chapter 8

Training Infrastructure — Mathematics, Code, and Worked Example

Backprop Math, SGD, Cross-Entropy, FQT Math, Worked 3-Class Example, Full RigL Loop

Section 8.1 — Linear Algebra Essentials for Neural Networks

■ C CONCEPT: Vectors and Matrices — What They Represent

Vector: a 1D array of numbers. Represents a point or direction in N-dimensional space. $x = [x_1, x_2, \dots, x_N]$ — the input feature vector (e.g., 128 audio features) Matrix: a 2D grid of numbers. Represents a linear transformation. $W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}$ — a 2x2 weight matrix Matrix-vector product $Y = W \times x$: $Y_i = \sum_j (W_{ij} \times x_j)$ Each output element Y_i is the dot product of row i of W with x . Matrix-matrix product $C = A \times B$: $C_{ij} = \sum_k (A_{ik} \times B_{kj})$ A is $[M \times K]$, B is $[K \times N]$, C is $[M \times N]$. K is the SHARED dimension that gets summed over. Transpose (T): swap rows and columns. A is $[M \times K]$, A^T is $[K \times M]$. $(A^T)_{ij} = A_{ji}$

— ■ WORKED EXAMPLE

Matrix multiplication example: Input $x = [1.0, 2.0]$ (2 features) Weight $W = \begin{bmatrix} 0.5 & 0.3 \\ -0.2 & 0.8 \end{bmatrix}$ (2 output neurons, 2 input features) Bias $b = [0.1, -0.1]$ Output = $W \times x + b$: $Y[0] = 0.5 \times 1.0 + 0.3 \times 2.0 + 0.1 = 0.5 + 0.6 + 0.1 = 1.2$ $Y[1] = -0.2 \times 1.0 + 0.8 \times 2.0 - 0.1 = -0.2 + 1.6 - 0.1 = 1.3$ Output $Y = [1.2, 1.3]$

Section 8.2 — Gradient Descent and the Chain Rule

■ C CONCEPT: What Is a Gradient?

The gradient of a function $f(x)$ is a vector of partial derivatives: $\text{grad}_f = [df/dx_1, df/dx_2, \dots, df/dx_N]$ Each component df/dx_i tells us: 'if we increase x_i by epsilon, how much does f change?' Gradient descent: move x in the OPPOSITE direction of the gradient to reduce f : $x_{\text{new}} = x_{\text{old}} - \text{learning_rate} \times \text{grad}_f(x_{\text{old}})$ For a neural network: x = weight matrix W f = loss function L (cross-entropy, MSE, etc.) $\text{grad}_f = dL/dW$ (gradient of loss w.r.t. weights) Update: $W_{\text{new}} = W_{\text{old}} - lr \times dL/dW$ After many steps: the weights converge to values that minimize the loss.

■ C CONCEPT: The Chain Rule — Engine of Backpropagation

For composed functions $f(g(x))$, the chain rule gives: $df/dx = (df/dg) \times (dg/dx)$ A neural network is a composition of layers: $\text{output} = \text{softmax}(\text{linear2}(\text{relu}(\text{linear1}(\text{input}))))$ To get dL/dW_1 (gradient for the first layer's weights), we apply the chain rule repeatedly: $dL/dW_1 = (dL/d_{\text{softmax_out}}) \times (d_{\text{softmax}}/d_{\text{linear2_out}}) \times \dots \times (d_{\text{relu_out}}/d_{\text{linear1_out}}) \times (d_{\text{linear1_out}}/dW_1)$ Each layer computes its LOCAL gradient and multiplies by the INCOMING gradient. Backpropagation is just the chain rule, applied layer by layer from output to input. In code: each layer's backward function: 1. Multiplies incoming gradient by local Jacobian 2. Returns the result as the 'outgoing' gradient to the previous layer

Section 8.3 — Loss Functions: Cross-Entropy and MSE

■ C CONCEPT: Cross-Entropy Loss — For Classification

Cross-entropy measures how wrong a probability prediction is: $L = -\sum_i (y_{\text{true}_i} \times \log(y_{\text{pred}_i}))$ For one-hot labels (exactly one class is correct): $L = -\log(y_{\text{pred}[\text{correct_class}]})$ If the model predicts [0.9, 0.05, 0.05] and the true class is 0: $L = -\log(0.9) = 0.105$ (good prediction, low loss) If the model predicts [0.1, 0.45, 0.45] and the true class is 0: $L = -\log(0.1) = 2.303$ (wrong prediction, high loss) Gradient of cross-entropy after softmax: $dL/d_{\text{logit}_i} = y_{\text{pred}_i} - y_{\text{true}_i}$. This is the simplest possible gradient — the difference between prediction and truth.

```
// crossEntropyLossFloat: compute loss and gradient simultaneously
float crossEntropyLossFloat(tensor_t *probs, tensor_t *labels, tensor_t *lossGrad) {
    size_t N = calcNumberOfElementsByTensor(probs); // number of classes
    float loss = 0.0f;
    // probs: output of softmax (probabilities summing to 1)
    // labels: one-hot vector (label[correct_class] = 1.0, rest = 0.0)
    for (size_t i = 0; i < N; i++) {
        float p = readBytesAsFloat(probs->data + i*4);
        float y = readBytesAsFloat(labels->data + i*4);
        // Cross-entropy: -y * log(p). Only the true class contributes (y=1 there).
        loss -= y * logf(p + 1e-7f); // 1e-7 prevents log(0) = -inf
        // Gradient: d(CE)/d_logit_i = p_i - y_i (when combined with softmax)
        writeFloatToByteArray(p - y, lossGrad->data + i*4);
    }
    return loss;
}
```

3	<code>float loss = 0.0f;</code>	Accumulate the cross-entropy sum over all classes. For one-hot labels, only one term contributes (where $y=1$).
8	<code>float p = readBytesAsFloat(probs->data + i*4);</code>	Read the predicted probability for class i . This is the softmax output, so p in $(0, 1]$ and all p sum to 1.
10	<code>loss -= y * logf(p + 1e-7f);</code>	The cross-entropy term for class i . For non-true classes: $y=0$, term=0. For the true class: $y=1$, term = $-\log(p)$. $1e-7$ prevents $\log(0)$ which is $-\infty$ if the model is extremely confident and wrong.
12	<code>writeFloatToByteArray(p - y, lossGrad->data + i*4);</code>	The combined cross-entropy + softmax gradient is simply $p - y$. For the true class: $p - 1.0$ (negative, pushes weight to increase p). For wrong classes: $p - 0.0 = p$ (positive, pushes to decrease p).

Section 8.4 — Fixed-Point Quantization Training: The Mathematics

■ C CONCEPT: SYM_INT32 Encoding — How Real Values Become Integers

SYM_INT32 represents a real value as: $\text{real_value} = \text{mantissa} \times \text{scale}$ Where mantissa is an int32_t and scale is a float32. The quantization grid: ..., -2*scale, -scale, 0, scale, 2*scale, ... Smallest representable non-zero: scale. Largest: qMax × scale. Example: scale=0.001, qMax=32767 (for qMaxBits=16): Smallest non-zero = 0.001 Largest = 32767 × 0.001 = 32.767 Resolution = 0.001 (any real value rounded to nearest multiple of 0.001) Why 16-bit qMax for a 32-bit integer? In matmul: $\text{mantissa_A} \times \text{mantissa_B} \leq 32767 \times 32767 \approx 10^9 < 2^{31}$ (int32 max) Sum of K=128 such products: $128 \times 10^9 \approx 1.3 \times 10^{11} > 2^{31}$ But int64_t max = $9.2 \times 10^{18} \gg 1.3 \times 10^{11} \rightarrow$ safe with int64 accumulator If qMaxBits=17: $65535^2 \approx 4.3 \times 10^9 > 2^{31}$ — product itself overflows int32!

■ C CONCEPT: Two-Pass Requantization: Finding the Right Scale

After a matmul or other operation, the output needs a new scale. Pass A (compute scale): 1. Find max absolute value in the result: $\text{maxAbs} = \max(|\text{output}[i]|)$ 2. Set new scale: $\text{scale_new} = \text{maxAbs} / \text{qMax}$ 3. Now $\text{mantissa}[i] = \text{output}[i] / \text{scale_new} \approx \text{output}[i] \times \text{qMax} / \text{maxAbs}$ 4. The largest mantissa = qMax. Fully utilizes the integer range. Pass B (requantize with new scale): Divide each element by scale_new, round to nearest integer, clamp to [-qMax, qMax]. Why two passes? Pass A must see ALL elements before setting the scale. Pass B then uses that scale to convert all elements. Single-pass would need to know the maximum before seeing the data — impossible.

Section 8.5 — SGD: Zero-Gradient, Float Step, SYM_INT32 Step

```
// sgdZeroGrad: reset all gradient tensors to zero before a new batch
void sgdZeroGrad(parameter_t *parameter) {
    size_t paramBytes = calcNumberOfBytesForData(
        parameter->grad.quantization, calcNumberOfElementsByTensor(&parameter->grad));
    memset(parameter->grad.data, 0, paramBytes);
    // CRITICAL for SYM_INT32: restore scale after zeroing
    if (parameter->grad.quantization->type == SYM_INT32) {
        symInt32QConfig_t *qc = (symInt32QConfig_t *)parameter->grad.quantization->qConfig;
        qc->scale = 1.0f; // reset to 1.0; will be set by requantization
    }
}
```

2-4	calcNumberOfBytesForData + memset	Get the exact byte count for this gradient tensor (depends on quantization type), then zero all bytes. memset with 0 correctly zeroes both float32 and int32 mantissas (zero in IEEE 754 float is all-zero bytes; zero int32 = 0x00000000).
6-9	SYM_INT32 scale restore	memset zeros ALL bytes including the scale stored in qConfig. WAIT — scale is stored in symInt32QConfig_t, NOT in the tensor bytes. The memset only zeros tensor->data. The qc->scale field must be restored separately. Setting scale=1.0f after zero is safe: 0 mantissas × any scale = 0.

```

// sgdStepFloat: update float32 weights
void sgdStepFloat(parameter_t *parameter, float learningRate) {
    tensor_t *w = &parameter->param;
    tensor_t *g = &parameter->grad;
    size_t N = calcNumberOfElementsByTensor(w);
    for (size_t i = 0; i < N; i++) {
        float wi = readBytesAsFloat(w->data + i*4);
        float gi = readBytesAsFloat(g->data + i*4);
        writeFloatToByteArray(wi - learningRate * gi, w->data + i*4);
    }
}

// sgdStepSymInt32: update quantized weights via real-value detour
void sgdStepSymInt32(parameter_t *parameter, float learningRate) {
    tensor_t *w = &parameter->param;
    tensor_t *g = &parameter->grad;
    symInt32QConfig_t *qcW = (symInt32QConfig_t *)w->quantization->qConfig;
    symInt32QConfig_t *qcG = (symInt32QConfig_t *)g->quantization->qConfig;
    size_t N = calcNumberOfElementsByTensor(w);
    for (size_t i = 0; i < N; i++) {
        // 1. Convert mantissas to real values:
        float wReal = (float)readBytesAsInt32(w->data+i*4) * qcW->scale;
        float gReal = (float)readBytesAsInt32(g->data+i*4) * qcG->scale;
        // 2. Apply gradient descent in real-value space:
        float wNew = wReal - learningRate * gReal;
        // 3. Requantize back to int32 mantissa with stochastic rounding:
        float newMantissaF = wNew / qcW->scale;
        int32_t newMantissa = roundByMode(newMantissaF, qcW->roundingMode);
        // 4. Clamp to valid range:
        int32_t qMax = (1 << (qcW->qMaxBits - 1)) - 1;
        newMantissa = clamp(newMantissa, -qMax, qMax);
        writeInt32ToByteArray(newMantissa, w->data + i*4);
    }
}

```

7	<code>wi - learningRate * gi</code>	SGD update: $w = w - lr * grad$. Negative gradient direction = direction of loss decrease. Learning rate scales the step size.
18	<code>float wReal = (float)readBytesAsInt32(w->data+i*4) * qcW->scale;</code>	Convert quantized weight to real value. <code>readBytesAsInt32</code> reads the mantissa. Cast to float before multiplying by scale. CRITICAL: cast first! $int32_t \times float \rightarrow$ the int32 is widened to float, then multiplied.
21	<code>float wNew = wReal - learningRate * gReal;</code>	The actual gradient descent step, performed in REAL (float) space. <code>wReal</code> and <code>gReal</code> are both real-valued floats at this point.
23	<code>float newMantissaF = wNew / qcW->scale;</code>	Convert the new real weight back to a mantissa. Division by scale gives the quantized representation.
24	<code>int32_t newMantissa = roundByMode(newMantissaF, qcW->roundingMode);</code>	Round to integer. If mode is <code>SR_HALF_AWAY</code> (stochastic rounding): small updates ($newMantissaF - floor < 1.0$) are rounded probabilistically. This is the key FQT mechanism that preserves small gradient updates.
26	<code>newMantissa = clamp(newMantissa, -qMax, qMax);</code>	Prevent mantissa overflow. If a weight grew too large, clamp prevents it from exceeding the safe range for the next matmul.

Section 8.6 — SGD with Momentum

■ C CONCEPT: Momentum — Accelerating Learning

Vanilla SGD: $w = w - lr * grad$ Problem: on flat regions, tiny gradients → very slow learning. On zig-zag paths, the gradient oscillates and cancels itself. SGD with Momentum: keep a 'velocity' v that accumulates past gradients: $v = momentum * v + (1 - momentum) * grad$ (exponential moving average) $w = w - lr * v$ Effect: (1) Smooths out gradient noise — random fluctuations average out in v . (2) Accelerates in consistent directions — if gradient points the same way repeatedly, v accumulates and the step gets larger. (3) 'Rolls past' shallow local minima. Typical momentum value: 0.9. Means 90% of the velocity is carried over from the previous step, 10% is the new gradient. After 10 steps, the velocity is dominated by recent gradients.

```
// sgdStepMFloat: SGD with momentum for float32 weights
void sgdStepMFloat(parameter_t *parameter, float lr, float momentum,
tensor_t *velocity) {
    tensor_t *w = &parameter->param;
    tensor_t *g = &parameter->grad;
    size_t N = calcNumberOfElementsByTensor(w);
    for (size_t i = 0; i < N; i++) {
        float wi = readBytesAsFloat(w->data + i*4);
        float gi = readBytesAsFloat(g->data + i*4);
        float vi = readBytesAsFloat(velocity->data + i*4);
        // Exponential moving average of gradient:
        float vi_new = momentum * vi + (1.0f - momentum) * gi;
        writeFloatToByteArray(vi_new, velocity->data + i*4);
        // Weight update using smoothed gradient:
        writeFloatToByteArray(wi - lr * vi_new, w->data + i*4);
    }
}
```

1	tensor_t *velocity	A separate tensor of the same shape as the weights to store the velocity. Must be allocated, zero-initialized before training, and persisted between steps. Memory cost: same as the weight tensor itself (doubles the memory for this layer).
10	float vi_new = momentum * vi + (1.0f - momentum) * gi;	Exponential moving average. With momentum=0.9: $vi_new = 0.9 * old_velocity + 0.1 * current_gradient$ If the last 10 gradients were all 1.0: v converges to 1.0. If they alternated +1 and -1: v averages to 0 (oscillation cancelled).
12	wi - lr * vi_new	Weight update with smoothed gradient. When the gradient is consistent, $vi > gi$ (accumulated velocity) → faster learning. When gradient reverses, vi is buffered by old velocity → smoother trajectory.

Section 8.7 — Worked Example: 3-Class Speaker Identification

■ WHAT THIS FILE DOES

A complete, numerical training step for a minimal 3-speaker identification model. Architecture: Linear(4→8) → ReLU → Linear(8→3) → Softmax. One sample, one training step. Every number is computed explicitly.

	Bytes	Description
	16	4 audio feature
	32	After Linear1
	32	After ReLU
	12	After Linear2
	12	After Softmax
	12	One-hot label

	12	Softmax+CE gr
	32	Gradient at Rel
	32	Gradient at Lin
	96	Gradient for W2
	128	Gradient for W2

```

// Input: 4 audio features
input = [0.5, -0.3, 0.8, 0.1]
// Weight matrix W1: shape [8, 4], bias b1: shape [8]
// (using simplified small weights for illustration)
W1[0,:] = [ 0.2, -0.1, 0.3, 0.0]
W1[1,:] = [-0.1, 0.4, -0.2, 0.1]
// ... (8 rows total)
b1 = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
// === FORWARD PASS ===
// Linear1: h1 = W1 @ input + b1
h1[0] = 0.2*0.5 + (-0.1)*(-0.3) + 0.3*0.8 + 0.0*0.1 + 0 = 0.37
h1[1] = (-0.1)*0.5 + 0.4*(-0.3) + (-0.2)*0.8 + 0.1*0.1 = -0.28
// ... etc for h1[2..7]
h1 = [0.37, -0.28, 0.60, 0.05, -0.15, 0.42, -0.08, 0.31]
// ReLU: h2 = max(0, h1)
h2 = [0.37, 0.00, 0.60, 0.05, 0.00, 0.42, 0.00, 0.31]
// ^ kept ^ dead ^ kept ^ dead
// Linear2: logits = W2 @ h2 + b2
logits = [0.25, 0.40, 0.18] // scores for 3 speakers
// Softmax: probs = softmax(logits)
max_logit = 0.40
exp_shifted = [exp(-0.15), exp(0.00), exp(-0.22)] = [0.861, 1.000, 0.803]
sum_exp = 2.664
probs = [0.323, 0.375, 0.302] // sum = 1.0

```



```

// True label: Speaker 1 (second speaker, index 1)
target = [0.0, 1.0, 0.0]
// Cross-entropy loss:
L = -log(probs[1]) = -log(0.375) = 0.981
// === BACKWARD PASS ===
// Gradient of loss w.r.t. logits (softmax + cross-entropy combined):
dL/dlogits = probs - target = [0.323-0.0, 0.375-1.0, 0.302-0.0]
= [0.323, -0.625, 0.302]
// Gradient w.r.t. W2: dL/dW2 = dL/dlogits^T @ h2
// dL/dlogits shape [1,3], h2 shape [1,8]
// dL/dlogits^T shape [3,1], result [3,8]:
dL/dW2[0,:] = 0.323 * h2 = [0.120, 0.000, 0.194, 0.016, 0.000, 0.136, 0.000, 0.100]
dL/dW2[1,:] = -0.625 * h2 = [-0.231, 0.000, -0.375, -0.031, 0.000, -0.263, 0.000, -0.194]
dL/dW2[2,:] = 0.302 * h2
// Gradient w.r.t. h2: dL/dh2 = dL/dlogits @ W2
// [1,3] @ [3,8] = [1,8]
dL/dh2 = sum_j(dL/dlogits[j] * W2[j,:]) = [...] // shape [1,8]
// ReLU backward: zero gradient where h1 was <= 0
dL/dh1 = dL/dh2 * (h1 > 0)
// h1 = [0.37, -0.28, ...]: dead neurons (h1 <= 0) get zero gradient
dL/dh1[1] = 0.0 // h1[1]=-0.28 < 0 -> dead neuron, no gradient
dL/dh1[4] = 0.0 // h1[4]=-0.15 < 0 -> dead neuron
dL/dh1[6] = 0.0 // h1[6]=-0.08 < 0 -> dead neuron
// Gradient w.r.t. W1: dL/dW1 = dL/dh1^T @ input
// [8,1] @ [1,4] = [8,4]
// ... etc.
// === WEIGHT UPDATE (SGD, lr=0.01) ===
W2 = W2 - 0.01 * dL/dW2
W1 = W1 - 0.01 * dL/dW1
b2 = b2 - 0.01 * dL/dlogits // = [0.323, -0.625, 0.302] * 0.01
b1 = b1 - 0.01 * dL/dh1
// After update: probs[1] (correct speaker) will be slightly higher next time.

```

— ■ WORKED EXAMPLE

After one gradient step ($lr=0.01$): $b2[1] = 0.0 - 0.01 * (-0.625) = +0.00625$ This makes logits[1] slightly larger next forward pass. softmax will give slightly higher probability to speaker 1. After thousands of steps, probs[1] approaches 1.0 for this sample. Dead neurons ($h1[1]=-0.28$): $W1[1,:]$ receives ZERO gradient every step. If a neuron stays dead for all training samples, it never learns. Solution: careful weight initialization (Xavier/He initialization) to avoid starting with too many negative pre-activations.

Section 8.8 — Complete Training Loop with RigL Integration

```

// Complete training loop for speaker ID on STM32:
// Architecture: Linear(128→64, sparse) → ReLU → Linear(64→N_SPEAKERS) → Softmax
// === SETUP (once before training) ===
// RigL config for layer 1 (sparse):
riglConfig_t riglCfg1 = {
    .weights = &linear1Cfg.weights.param,
    .grads = &linear1Cfg.weights.grad,
    .mask = &layer1Mask,
    .sparsityRatio = 0.8f,
    .updateStep = 100,
    .batchCount = 0,
};
riglInitMask(&riglCfg1, 42);
// === TRAINING LOOP ===
for (size_t epoch = 0; epoch < NUM_EPOCHS; epoch++) {
    rngShuffleIndices(indices, NUM_SAMPLES); // shuffle training data
    for (size_t batch = 0; batch < NUM_BATCHES; batch++) {
        // 1. Load batch
        dataLoaderGetNextBatch(&loader, &inputBatch, &labelBatch);
        // 2. Zero all gradients
        sgdZeroGrad(&linear1Cfg.weights);
        sgdZeroGrad(&linear1Cfg.bias);
        sgdZeroGrad(&linear2Cfg.weights);
        sgdZeroGrad(&linear2Cfg.bias);
        // 3. Forward pass
        layerForward(&layer1, &inputBatch, &h1); // sparse linear
        layerForward(&relu1, &h1, &h2); // ReLU
        layerForward(&layer2, &h2, &logits); // dense linear
        softmaxForwardFloat(&logits); // in-place softmax
        // 4. Loss
        float loss = crossEntropyLossFloat(&logits, &labelBatch, &lossGrad);
        // 5. Backward pass
        // layer2 backward:
        linearCalcWeightGradsFloat(&lossGrad, &h2, &linear2Cfg.weights.grad);
        linearCalcBiasGradsFloat(&lossGrad, &linear2Cfg.bias.grad);
        linearCalcInputGradsFloat(&lossGrad, &linear2Cfg.weights.param, &dh2);
        // ReLU backward:
        reluBackwardFloat(&dh2, &h1); // dh1 = dh2 * (h1 > 0)
        // layer1 backward (all weights, including pruned, for RigL grow step):
        linearCalcWeightGradsFloat(&dh2, &inputBatch, &linear1Cfg.weights.grad);
        linearCalcBiasGradsFloat(&dh2, &linear1Cfg.bias.grad);
        // 6. Optimizer step
        sgdStepFloat(&linear2Cfg.weights, LEARNING_RATE);
        sgdStepFloat(&linear2Cfg.bias, LEARNING_RATE);
        riglMaskedSgdStepFloat(&riglCfg1, LEARNING_RATE); // sparse layer1
        sgdStepFloat(&linear1Cfg.bias, LEARNING_RATE);
        // 7. RigL topology update (fires every 100 batches)
        riglUpdate(&riglCfg1);
        // Logging:
        if (batch % 10 == 0) PRINT_DEBUG("Epoch %zu Batch %zu Loss: %f", epoch, batch, loss);
    }
}

```

20	<code>rngShuffleIndices(indices, NUM_SAMPLES);</code>	Shuffle training data order at the start of each epoch. Prevents the model from overfitting to the presentation order of samples. Uses Fisher-Yates shuffle with the ODT xorshift32 RNG.
28-30	<code>sgdZeroGrad</code> for all parameters	Zero ALL gradient tensors before each batch. Critical: gradients ACCUMULATE across the backward pass (<code>addFloat32TensorsInplace</code>). Forgetting to zero grad = gradients from previous batch contaminate current batch.
33	<code>layerForward(&layer1, &inputBatch, &h1);</code>	Dispatches to <code>linearForwardFloat</code> via the <code>forwardFuncs</code> table. Since <code>layer1</code> has a RigL mask, <code>riglMaskedMatmulFloat32</code> is used internally.

41	<code>linearCalcWeightGradsFloat(&lossGrad, &h2, &linear2Cfg.weights.grad);</code>	Compute dL/dW2 and ACCUMULATE into the gradient tensor. The gradient of the second linear layer uses h2 (ReLU output) as the forward input.
47	<code>linearCalcWeightGradsFloat(&dh2, &inputBatch, &linear1Cfg.weights.grad);</code>	Compute dL/dW1 for ALL weights (active and pruned). The gradient for pruned weights is computed but not applied by the optimizer. It IS used by riglGrowStep to select the best weights to reactivate.
52	<code>riglMaskedSgdStepFloat(&riglCfg1, LEARNING_RATE);</code>	Mask-aware SGD: updates only active weights (mask=1). Pruned weights (mask=0) are skipped — they stay at 0.0f.
55	<code>riglUpdate(&riglCfg1);</code>	Increments batchCount. Every updateStep batches: prune + grow + zero stale grads. The topology update uses the freshly computed gradients (step 5) to decide which inactive weights to grow.

■ RigL GAP IN THIS FILE

Why compute gradients for pruned weights (step 47 above)? The RigL grow step reads gradients to find which inactive weight would reduce the loss most if re-enabled. We need gradient values at ALL positions — not just active ones. The backward pass naturally computes these ($dL/dW = dL/dY^T @ X$) for the entire matrix. The mask is NOT applied during the backward pass, only during forward and optimizer step.

Mathematical Foundations

The math behind every operation in the ODT library

16.1 Linear Algebra Essentials

Neural networks are, at their core, linear algebra. Every forward pass is a sequence of matrix multiplications and element-wise operations. Understanding the shapes and operations on matrices is essential to understanding the code.

16.1.1 Vectors and Matrices

A **vector** is a list of numbers: $v = [v_1, v_2, \dots, v_n]$. A **matrix** is a 2D grid of numbers with rows and columns. An $m \times n$ matrix has m rows and n columns.

Matrix notation example

```
W = [[1, 2, 3], // row 0
      [4, 5, 6]] // row 1
// W is a 2x3 matrix: 2 rows, 3 columns
// W[i][j] means row i, column j
// W[1][2] = 6
```

In C, a 2D matrix is typically stored as a flat 1D array in **row-major order**: all elements of row 0 come first, then row 1, etc. Element $W[i][j]$ is at index $i \times \text{numCols} + j$.

WORKED EXAMPLE

Row-major memory layout: Matrix W (2 rows, 3 cols) stored in memory: Index: 0 1 2 3 4 5 Value: 1 2 3 4 5 6
↑row0↑ ↑row1↑ $W[1][2] = W_flat[1*3 + 2] = W_flat[5] = 6$ ✓

16.1.2 Matrix Multiplication

If A has shape $[m, k]$ and B has shape $[k, n]$, then $C = A \times B$ has shape $[m, n]$. Each element $C[i][j] = \sum_t A[i][t] \times B[t][j]$. The inner dimension k must match — this is the 'contracted' dimension.

Matrix multiply formula

```
// C[i][j] = sum_{t=0}^{k-1} A[i][t] * B[t][j]
//
// Example: A is [2,3], B is [3,2], C is [2,2]
A = [[1,2,3], [4,5,6]] // shape [2,3]
B = [[7,8], [9,10], [11,12]] // shape [3,2]
C[0][0] = 1*7 + 2*9 + 3*11 = 7+18+33 = 58
C[0][1] = 1*8 + 2*10 + 3*12 = 8+20+36 = 64
C[1][0] = 4*7 + 5*9 + 6*11 = 28+45+66 = 139
C[1][1] = 4*8 + 5*10 + 6*12 = 32+50+72 = 154
```

In ODT, `matmulFloat32TensorsWithBias` computes exactly this, with an optional bias added to each row of the result.

16.1.3 Matrix Transpose

The transpose of A (written A^T) swaps rows and columns: $A^T[i][j] = A[j][i]$. If A has shape $[m, n]$, then A^T has shape $[n, m]$.

In ODT, `transposeTensor` is $O(1)$ — it does not move any data. It swaps the `orderOfDimensions` array, so that when the tensor is accessed logically, dimensions are looked up in swapped order.

WORKED EXAMPLE

Why transpose is needed in Linear forward pass: We want: $\text{output} = \text{input} \times W^T$ (so each row of W is a set of weights for one output neuron) input shape: $[\text{batch}=4, \text{in}=64]$ W shape: $[\text{out}=32, \text{in}=64]$ W^T shape: $[\text{in}=64, \text{out}=32]$ $\text{input} \times W^T = [4,64] \times [64,32] = [4,32]$ ← correct output shape But ODT's `matmul` expects row $\times$ column = scalar, so instead we transpose input and compute $W \times \text{input}^T = [32,64] \times [64,4] = [32,4]$, then transpose result to $[4,32]$

16.2 Gradient Descent and Backpropagation

The goal of training is to find weight values that minimize the loss function. We do this by computing the gradient (direction of steepest increase) and moving the weights in the *opposite* direction.

16.2.1 What Is a Gradient?

For a function $f(x)$, the derivative $f'(x)$ tells us: 'if I increase x by a tiny amount ϵ , f increases by approximately $f'(x) \times \epsilon$ '. The **gradient** is the multidimensional generalization: ∇f is a vector where each component is the partial derivative with respect to one variable.

Gradient intuition

```
// f(w) = w^2 → f'(w) = 2w
// At w=3: f'(3) = 6
// Meaning: if w increases by 0.01, f increases by ~0.06
// To DECREASE f, move w in direction -f'(w) = -6
// Update: w_new = w - lr * f'(w) = 3 - 0.1*6 = 2.4
// Now f(2.4) = 5.76 < f(3) = 9 ✓ loss went down
```

16.2.2 Chain Rule — The Engine of Backpropagation

Neural networks are composed functions: $\text{loss} = L(\text{softmax}(\text{linear}(\text{relu}(\text{linear}(x))))))$. To find how much the loss changes when we change a weight deep in the network, we use the **chain rule**: if $z = f(g(x))$, then $dz/dx = (dz/dg) \times (dg/dx)$.

In a network with layers $A \rightarrow B \rightarrow C \rightarrow \text{loss}$:

Chain rule in backprop

```
// Forward: x → A(x) → B(A(x)) → C(B(A(x))) → loss
// Backward (chain rule):
// dL/dA = dL/dC × dC/dB × dB/dA
//
// In code, each layer's backward receives dOutput = dL/d(this layer's output)
// and must compute dInput = dL/d(this layer's input) = dOutput × d(output)/d(input)
// This dInput becomes dOutput for the previous layer.
```

This is exactly what ODT implements: the training loop calls backward on each layer from last to first, passing the gradient signal backwards through the network.

16.2.3 Stochastic Gradient Descent (SGD)

Pure gradient descent computes gradients over the entire dataset. **Stochastic Gradient Descent (SGD)** computes gradients over small random batches (mini-batches). This is noisier but much faster per update.

SGD update rule

```
// For each parameter w:
// w_new = w - learning_rate * gradient(w)
//
// With momentum (helps escape local minima):
// velocity = momentum * velocity - learning_rate * gradient
// w_new = w + velocity
//
// With weight decay (L2 regularization):
// w_new = w - learning_rate * (gradient + weight_decay * w)
```

ODT's `sgdStepFloat` implements the basic version: $w \leftarrow w - \text{lr} \times \text{grad}$. The `sgd_t` struct stores `momentumFactor` and `weightDecay` fields for optional use in extended implementations.

16.3 Loss Functions — Measuring Error

A loss function measures how wrong the network's predictions are. It takes the network's output and the true target, and returns a single scalar number. Lower loss = better predictions.

16.3.1 Mean Squared Error (MSE)

MSE = $(1/n) \times \sum((\text{predicted} - \text{target})^2)$. Used for regression tasks (predicting a continuous value). The squared term penalizes large errors more heavily.

MSE example

```
// predictions: [2.3, 0.5, 1.8]
// targets: [2.0, 0.0, 2.0]
// errors: [0.3, 0.5, -0.2]
// squared: [0.09, 0.25, 0.04]
// MSE = (0.09 + 0.25 + 0.04) / 3 = 0.127
//
// MSE gradient w.r.t. predictions: 2/n * (pred - target)
// = 2/3 * [0.3, 0.5, -0.2] = [0.2, 0.333, -0.133]
```

16.3.2 Cross-Entropy Loss

Cross-entropy is used for classification. If the network outputs probabilities (after softmax), cross-entropy is: $-\sum(\text{target} \times \log(\text{prediction}))$.

For one-hot targets (only one class is correct), this simplifies to $-\log(\text{probability of the correct class})$. Perfect prediction $\rightarrow$ loss = 0. Wrong prediction $\rightarrow$ loss = infinity.

Cross-entropy example — 3-class classification

```
// targets (one-hot): [0, 1, 0] ← class 1 is correct
// predictions (softmax output): [0.1, 0.7, 0.2]
// CE = -(0*log(0.1) + 1*log(0.7) + 0*log(0.2))
// = -log(0.7)
// = 0.357 (low loss — prediction was fairly good)
//
// If predictions were [0.4, 0.2, 0.4] (wrong):
// CE = -log(0.2) = 1.609 (high loss)
```

The backward gradient through softmax+cross-entropy combined is simply: $dL/d(\text{logits}) = \text{predictions} - \text{targets}$. This is what `crossEntropySoftmaxBackward` computes.

—■ WORKED EXAMPLE

Cross-entropy backward example: predictions: [0.1, 0.7, 0.2] targets: [0, 1, 0] gradient: $[0.1-0, 0.7-1, 0.2-0] = [0.1, -0.3, 0.2]$ Interpretation: the network should increase logit for class 1 (negative gradient means: decreasing this value decreases loss) and decrease logits for classes 0 and 2.

16.4 Fixed-Point Quantization — The Mathematics

Floating-point numbers (float32) are convenient but expensive on MCUs. Quantization maps floating-point values to integers, enabling faster arithmetic with less memory. FQT (Fixed-point Quantization Training) in ODT trains with quantized weights from the start.

16.4.1 SYM_INT32 Encoding

In ODT's SYM_INT32 format: $\text{real_value} = \text{mantissa} \times \text{scale}$, where mantissa is an int32_t and scale is a float. The range of representable values is approximately $[-qMax \times \text{scale}, +qMax \times \text{scale}]$ where $qMax = 2^{(qMaxBits-1)} - 1$.

SYM_INT32 quantization formulas

```
// To quantize a float to SYM_INT32:
mantissa = clamp(round(real_value / scale), -qMax, +qMax)
// To dequantize back to float:
real_value = mantissa * scale
// Example: scale=0.01, qMaxBits=8, qMax=127
// Quantize 0.735: mantissa = round(0.735/0.01) = round(73.5) = 74
// Dequantize: 74 * 0.01 = 0.74 (small rounding error)
```

16.4.2 Choosing Scale — Two-Pass Requantization

To minimize quantization error, we want the scale to be as small as possible while still representing the maximum value in the tensor. The optimal scale is: $\text{scale} = \text{absMax} / qMax$.

ODT's `requantSymInt32Tensor` does this in two passes:

Two-pass requantization algorithm

```
// Pass A: find absolute maximum value in the tensor
float absMax = 0.0f;
for each element e in tensor:
float val = e.mantissa * e.scale; // dequantize
if (fabsf(val) > absMax) absMax = fabsf(val);
// Compute new scale to use the full integer range
float newScale = absMax / qMax;
// Pass B: re-encode every element with the new scale
for each element e in tensor:
float realVal = e.mantissa * e.scale; // old representation
e.mantissa = clamp(round(realVal / newScale), -qMax, qMax);
e.scale = newScale; // update scale (same for whole tensor)
```

■ NOTE

Scale is Per-Tensor, Not Per-Element: In ODT's SYM_INT32, a single scale value is shared across all elements of a tensor. This is called per-tensor quantization and is the simplest scheme. Per-channel quantization (one scale per output channel) can give better accuracy but is more complex to implement.

16.4.3 Integer Arithmetic for Matmul

When multiplying two SYM_INT32 tensors, the result has a combined scale: $\text{scale}_C = \text{scale}_A \times \text{scale}_B$.

SYM_INT32 matmul math

```
// A[i] = mantissa_A[i] * scale_A
// B[j] = mantissa_B[j] * scale_B
// C[i][j] = sum_k(A[i][k] * B[k][j])
// = sum_k(mantissa_A[i][k] * scale_A * mantissa_B[k][j] * scale_B)
// = (sum_k(mantissa_A[i][k] * mantissa_B[k][j])) * scale_A * scale_B
// = int64_accumulator * (scale_A * scale_B)
// Why int64_t? Because int32_t * int32_t can overflow int32_t!
// int32 max = ~2.1 billion
// (2.1e9)^2 = 4.4e18, which overflows int32 but fits in int64
```

This is why `matmulSymInt32TensorsWithBias` uses `int64_t acc = 0` as the accumulator before converting back to int32_t with the new combined scale.

16.4.4 Quantization Noise and Stochastic Rounding

Every quantization step introduces a small error (the **quantization noise**). Regular rounding (round-half-away) is deterministic: the same values always round the same way. Over many training steps, this can cause a systematic bias.

Stochastic rounding (SR_HALF_AWAY in ODT) adds random noise before rounding: **mantissa = round(real/scale + U(-0.5, 0.5))**. This makes rounding errors random rather than systematic, which helps training converge better with low-bit quantization.

16.5 Sparse Training Mathematics (RigL)

Sparse neural networks have many weights set to exactly zero. A network that is 90% sparse (90% zeros) uses 10× less memory and can compute 10× faster if the zeros are exploited.

16.5.1 Magnitude Pruning

The simplest pruning criterion: remove the weights with the smallest absolute value. The intuition: small weights contribute little to the output and can be set to zero without much loss in accuracy.

Magnitude pruning math

```
// Given weights W and target sparsity ratio s (e.g., 0.9 for 90% sparse):  
// 1. Sort |W[i]| in ascending order  
// 2. Set to zero the smallest floor(n*s) weights  
// 3. Keep the remaining ceil(n*(1-s)) weights  
// Example: W = [0.5, -0.1, 0.8, -0.03, 0.2], s=0.4 (prune 40%)  
// |W| sorted: [0.03, 0.1, 0.2, 0.5, 0.8]  
// floor(5*0.4) = 2 → prune 2 smallest: 0.03 and 0.1  
// Result mask: W[-0.1]=0, W[-0.03]=0, others remain
```

16.5.2 Gradient-Based Regrowth (RigL)

Magnitude pruning only prunes. RigL (Rigging the Lottery by Evci et al., 2020) also **grows** new connections where the gradient signal is largest. The criterion for growing: weights with the largest |gradient|.

The gradient of a pruned (zero) weight is meaningful — it tells us 'if this weight were non-zero, how much would it help?' Computing this gradient requires a special backward pass that also propagates through masked weights.

RigL update algorithm

```
// At step T (e.g., every 100 batches):  
//  
// 1. PRUNE: find k weights with smallest |magnitude|  
// mask[i] = 0 for these k weights  
// W[i] = 0 (explicitly zero)  
//  
// 2. GROW: find k positions with largest |gradient|  
// (among currently masked/pruned positions)  
// mask[i] = 1 (activate these weights)  
// W[i] = 0 (start at zero, will grow during training)  
//  
// 3. Continue normal training with the new mask  
// Active weights (mask=1): updated by SGD  
// Pruned weights (mask=0): skipped by SGD
```

■ RigL GAP IN THIS FILE

RigL Gap in ODT: ODT has the foundation (sparsity_t struct stub, BOOL tensor type, RNG for random init) but lacks the rigUpdate() function and the mask-aware matmul and SGD. See Chapter 10 for the complete implementation checklist.

16.5.3 Why Sparse Training Is Hard on Embedded Systems

On a CPU or GPU, sparse operations use special hardware instructions or libraries. On the STM32 MCU, there is no hardware sparse support. The options are:

1. **Dense storage with mask**: Keep W in full [out, in] memory but multiply by the mask during forward/backward. No memory savings, but computation savings if you check the mask before each multiply (skip if mask=0).
2. **Compressed Sparse Row (CSR)**: Store only non-zero values and their indices. Memory-efficient but adds index lookup overhead and more complex C code.

ODT currently targets option 1 (dense + mask) as the simpler starting point. The mask tensor uses 1 bit per weight via tensorBoolSet/Get, costing only n/8 bytes for an n-element weight matrix.

16.6 Why This Math Applies to Speaker Identification

The LibriSpeech speaker identification task requires the network to map audio features to speaker identities. Let's trace the math through one inference:

Full forward pass — mathematical view

```
// Input: x = audio feature vector, shape [1, 64] (MFCC features)
// Layer 1 (Linear 64→128):
// h1 = x × W1T + b1
// shape: [1,64] × [64,128] + [128] = [1,128]
// Layer 2 (ReLU):
// h2 = max(0, h1) element-wise
// shape: [1,128]
// Layer 3 (Linear 128→num_speakers):
// logits = h2 × W2T + b2
// shape: [1,128] × [128,num_speakers] + [num_speakers] = [1,num_speakers]
// Layer 4 (Softmax):
// probs = softmax(logits)
// probs[k] = exp(logits[k]) / sum(exp(logits[:]))
// shape: [1,num_speakers], all values in (0,1), sum=1
// Prediction: speaker = argmax(probs)
```

Cross-entropy loss during training: $CE = -\log(\text{probs}[\text{true_speaker}])$. The network learns to assign high probability to the correct speaker for each audio segment.

After training is complete on the MCU, inference is just the forward pass without any gradient computation — fast and deterministic.

16.7 Numerical Concepts Reference

Concept	Value / Formula	Where Used in ODT
int32_t max	$2,147,483,647 \approx 2.1 \times 10^9$	SYM_INT32 mantissa range
int64_t max	9.2×10^{18}	matmulSymInt32 accumulator
float32 precision	~7 decimal digits	SGD, loss computation
float32 range	$\pm 3.4 \times 10^{38}$	Intermediate activations
exp overflow	$\exp(89) \approx \text{float max}$	Softmax uses max subtraction trick
qMax (8-bit SYM)	$2^7 - 1 = 127$	SYM_INT32 with qMaxBits=8
qMax (16-bit SYM)	$2^{15} - 1 = 32767$	SYM_INT32 with qMaxBits=16
UINT32_MAX	$4,294,967,295 = 2^{32} - 1$	xorshift32 state, rngBounded limit
2^{24} float normalization	16,777,216	rngNextFloat: $x > 8$ then $/2^{24} \rightarrow [0,1)$

■ RELATIONS WITH OTHER FILES

Chapter 16 connects to: Chapter 4 (Quantization) — SYM_INT32 encoding and scale computation · Chapter 6 (Arithmetic) — matmul accumulator in int64_t, roundByMode · Chapter 8 (Training) — SGD update rule, loss functions forward and backward · Chapter 10 (RigL gap) — Pruning and regrowth math that needs to be implemented · Chapter 11 (MCU) — Why integer math is preferred on Cortex-M7

Chapter 17

End-to-End Worked Example

Building, training, and running a 2-layer MLP — step by step with real numbers

17.1 The Task: 3-Class Speaker Identification

To make the example concrete, we will build a tiny network that classifies audio feature vectors into one of 3 speaker identities (A, B, C). This is a simplified version of the full LibriSpeech task.

Network architecture

Input: 4 features (simplified MFCC coefficients)
Linear 1: 4 $\rightarrow$ 8 neurons, with bias
ReLU 1: element-wise activation
Linear 2: 8 $\rightarrow$ 3 neurons (3 speaker classes), with bias
Softmax: normalize to probabilities
Loss: CrossEntropy

Parameter count: Linear1 weights = $8 \times 4 = 32$, bias = 8. Linear2 weights = $3 \times 8 = 24$, bias = 3. Total: 67 parameters. This is tiny — the full model for LibriSpeech would have thousands.

17.2 Step 1 — Memory Layout on the MCU

Before writing any code, plan where each tensor lives in SRAM. The STM32F746ZG has 320 KB RAM. For our tiny example:

	Type	Bytes	Purpose
	float32	128	Linear1 weights
	float32	128	Linear1 weights
	float32	32	Linear1 bias
	float32	32	Linear1 bias gr
	float32	96	Linear2 weights
	float32	96	Linear2 weights
	float32	12	Linear2 bias
	float32	12	Linear2 bias gr
	float32	16	Input feature ve
	float32	32	After Linear1
	float32	32	After ReLU1
	float32	12	After Linear2 (l
	float32	12	After Softmax
	float32	32	Gradient at h2
	float32	32	Gradient at h1
	float32	12	One-hot target
		716 B	Well within 320

17.3 Step 2 — Initialization Code

All tensors and layer configs must be initialized before training begins. Here is the complete initialization sequence:

main.c — initialization

[illegible]

static		All arrays and structs are static — they live in .bss/.data sections in flash/SRAM, not on the stack. This avoids stack overflow for large buffers.
rng Set Seed(&rng, 42)		Initialize the xorshift32 RNG with seed 42 for reproducibility.
Xavier init		A well-known initialization scheme that scales initial weights to prevent activations from vanishing or exploding. Scale = $\sqrt{6/(fan_in+fan_out)}$.

17.4 Step 3 — One Training Step with Real Numbers

Let's trace a complete training step with specific numbers. We use a single sample (batch_size=1) for clarity.

Initial state (after random init, simplified):

Initial weights and input

```
Input (MFCC features): x = [0.5, -0.3, 0.8, 0.1]
W1 (8x4) - showing first 3 rows for brevity:
[0.2, 0.5, -0.3, 0.1] // row 0 → neuron h1[0]
[-0.4, 0.1, 0.6, -0.2] // row 1 → neuron h1[1]
[0.3, -0.2, 0.1, 0.4] // row 2 → neuron h1[2]
... (5 more rows)
b1 (8): [0.1, -0.1, 0.05, 0.0, 0.1, -0.05, 0.02, 0.0]
Target (speaker A = class 0, one-hot): [1, 0, 0]
```

17.4.1 Forward Pass — Layer by Layer

Linear1 forward: $h1 = W1 \times x + b1$

```
h1[0] = 0.2*0.5 + 0.5*(-0.3) + (-0.3)*0.8 + 0.1*0.1 + 0.1
= 0.10 - 0.15 - 0.24 + 0.01 + 0.10
= -0.18
h1[1] = (-0.4)*0.5 + 0.1*(-0.3) + 0.6*0.8 + (-0.2)*0.1 + (-0.1)
= -0.20 - 0.03 + 0.48 - 0.02 - 0.10
= 0.13
h1[2] = 0.3*0.5 + (-0.2)*(-0.3) + 0.1*0.8 + 0.4*0.1 + 0.05
= 0.15 + 0.06 + 0.08 + 0.04 + 0.05
= 0.38
// (complete h1 for all 8 neurons in practice)
h1 = [-0.18, 0.13, 0.38, 0.05, -0.22, 0.31, -0.07, 0.44]
```

ReLU1 forward: $h2 = \max(0, h1)$

```
h2[0] = max(0, -0.18) = 0.00 ← neuron is OFF (dead for this sample)
h2[1] = max(0, 0.13) = 0.13
h2[2] = max(0, 0.38) = 0.38
h2[3] = max(0, 0.05) = 0.05
h2[4] = max(0, -0.22) = 0.00 ← neuron is OFF
h2[5] = max(0, 0.31) = 0.31
h2[6] = max(0, -0.07) = 0.00 ← neuron is OFF
h2[7] = max(0, 0.44) = 0.44
h2 = [0.00, 0.13, 0.38, 0.05, 0.00, 0.31, 0.00, 0.44]
```

Linear2 forward: $\text{logits} = W2 \times h2 + b2$

```
// W2 shape [3,8], h2 shape [8] → logits shape [3]
// Assume W2 and b2 initialized similarly to W1
logits[0] = W2[0,:] · h2 + b2[0] = 0.25 // speaker A score
logits[1] = W2[1,:] · h2 + b2[1] = 0.40 // speaker B score
logits[2] = W2[2,:] · h2 + b2[2] = 0.10 // speaker C score
logits = [0.25, 0.40, 0.10]
```

Softmax forward: $\text{probs} = \text{softmax}(\text{logits})$

```
maxVal = 0.40 (for numerical stability)
shifted = [0.25-0.40, 0.40-0.40, 0.10-0.40]
= [-0.15, 0.00, -0.30]
exp_vals = [exp(-0.15), exp(0.00), exp(-0.30)]
= [0.861, 1.000, 0.741]
sum = 0.861 + 1.000 + 0.741 = 2.602
probs = [0.861/2.602, 1.000/2.602, 0.741/2.602]
= [0.331, 0.384, 0.285]
// Network assigns 33.1% to speaker A, 38.4% to B, 28.5% to C
// True class is A (0) – not great prediction, but we just started training
```


CrossEntropy loss

```
target = [1, 0, 0] (speaker A)
CE = -(1*log(0.331) + 0*log(0.384) + 0*log(0.285))
= -log(0.331)
= 1.107
// High loss – the network is not confident about the correct speaker
```

17.4.2 Backward Pass with Real Numbers

CrossEntropy+Softmax backward: d_logits = probs - target

```
d_logits = [0.331 - 1, 0.384 - 0, 0.285 - 0]
= [-0.669, 0.384, 0.285]
// Meaning: logits[0] should increase (negative gradient → decrease loss)
// logits[1] and [2] should decrease
```

Linear2 backward: dW2, db2, dh2

```
// dW2[i][j] = d_logits[i] * h2[j]
dW2[0][:] = -0.669 * [0.00, 0.13, 0.38, 0.05, 0.00, 0.31, 0.00, 0.44]
= [0.00, -0.087, -0.254, -0.033, 0.00, -0.207, 0.00, -0.294]
dW2[1][:] = 0.384 * h2 = [0.00, 0.050, 0.146, 0.019, 0.00, 0.119, 0.00, 0.169]
dW2[2][:] = 0.285 * h2 = [0.00, 0.037, 0.108, 0.014, 0.00, 0.088, 0.00, 0.125]
db2 = d_logits = [-0.669, 0.384, 0.285]
// dh2 = W2^T × d_logits (propagate loss upstream)
dh2[j] = sum_i(W2[i][j] * d_logits[i]) for j=0..7
// This gives the gradient at each h2 neuron
```

ReLU1 backward: dh1 = dh2 * (h1 > 0)

```
// Only neurons that were ON in forward get gradient
// h1 = [-0.18, 0.13, 0.38, 0.05, -0.22, 0.31, -0.07, 0.44]
// mask = [0, 1, 1, 1, 0, 1, 0, 1]
dh1[0] = dh2[0] * 0 = 0.0 ← h1[0] was negative, gradient blocked
dh1[1] = dh2[1] * 1 = dh2[1] ← passes through
dh1[2] = dh2[2] * 1 = dh2[2]
// etc.
```

Linear1 backward: dW1, db1

```
// dW1[i][j] = dh1[i] * x[j] (input was x = [0.5, -0.3, 0.8, 0.1])
// db1[i] = dh1[i]
// (dInput computed but not used – x is the network input, no upstream layer)
```

17.4.3 SGD Weight Update

SGD update (learning_rate = 0.01)

```
// For each parameter w with gradient dw:
w_new = w - 0.01 * dw
// Example – W1 row 0 first element:
// W1[0][0]_old = 0.20
// dW1[0][0] = dh1[0] * x[0] = 0.0 * 0.5 = 0.0 (neuron 0 was OFF!)
// W1[0][0]_new = 0.20 - 0.01 * 0.0 = 0.20 (no change)
// W1[1][0] (neuron 1 was ON):
// W1[1][0]_old = -0.40
// dW1[1][0] = dh1[1] * x[0] = dh2[1] * 0.5
// W1[1][0]_new = -0.40 - 0.01 * (dh2[1] * 0.5)
// After one step, loss will be slightly lower.
// After many steps (hundreds or thousands), network converges.
```

17.5 Step 4 — The Training Loop in C

Now let's write the actual C training loop that ties everything together:

main.c — training loop

```
void trainEpoch(dataLoader_t *loader, sgd_t *sgd,
layer_t *lin1, layer_t *relu1,
layer_t *lin2, lossFunc_t *lossFn,
tensor_t *input, tensor_t *h1, tensor_t *h2,
tensor_t *output, tensor_t *dh2, tensor_t *dh1,
tensor_t *target) {
    size_t numBatches = loader->getDatasetSize(loader) / loader->batchSize;
    for (size_t batch = 0; batch < numBatches; batch++) {
        // 1. Zero all gradients
        sgdZeroGrad(sgd, lin1->config.linear);
        sgdZeroGrad(sgd, lin2->config.linear);
        // 2. Load one batch
        loader->getBatch(loader, batch, input, target);
        // 3. Forward pass
        layerFunctions[LAYER_LINEAR].forward(lin1, input, h1);
        layerFunctions[LAYER_RELU].forward(relu1, h1, h2);
        layerFunctions[LAYER_LINEAR].forward(lin2, h2, output);
        // 4. Compute loss (includes softmax inside)
        float loss = lossFn->forward(lossFn, output, target);
        // 5. Backward pass (reverse order)
        lossFn->backward(lossFn, output, target, dh2);
        layerFunctions[LAYER_LINEAR].backward(lin2, h2, dh2, dh1);
        layerFunctions[LAYER_RELU].backward(relu1, dh1, dh1); // in-place
        layerFunctions[LAYER_LINEAR].backward(lin1, input, dh1, NULL);
        // 6. Update weights
        sgdStep(sgd, lin1->config.linear);
        sgdStep(sgd, lin2->config.linear);
        if (batch % 10 == 0)
            printf('Batch %zu loss: %.4f\n', batch, loss);
    }
}
```

sgdZeroGrad		CRITICAL: must be called at the start of every batch. Otherwise gradients from previous batches accumulate, corrupting the update.
getBatch		Fills the input and target tensors with data from the dataset. In the DataLoader, this calls getSample in a loop to fill the batch.
NULL for dInput in lin1		The first layer has no upstream layer to send gradients to, so we pass NULL. linearBackward checks for NULL before computing dInput.
printf on MCU		On the STM32, printf can be redirected to UART for debugging. This is usually done by overriding the _write() system call to use HAL_UART_Transmit.

17.6 Step 5 — Serialization and Inference

After training, the weights need to be saved to flash so they survive power cycles, or transmitted to a host PC for analysis.

Saving trained weights with `serializeTensor`

```
// Write trained weights to UART (connected to PC)
// PC-side Python script reads and saves as .npy
void saveModel(layer_t *lin1, layer_t *lin2) {
    serializeTensor(&lin1->config.linear->weights.param);
    serializeTensor(&lin1->config.linear->bias.param);
    serializeTensor(&lin2->config.linear->weights.param);
    serializeTensor(&lin2->config.linear->bias.param);
}

// For inference (no gradients needed):
void runInference(tensor_t *x, tensor_t *probs) {
    lin1cfg.isTraining = false; // disable inputCache saving
    layerFunctions[LAYER_LINEAR].forward(&lin1, x, &h1);
    layerFunctions[LAYER_RELU].forward(&relu1, &h1, &h2);
    layerFunctions[LAYER_LINEAR].forward(&lin2, &h2, &outBuf_t);
    softmaxForward(NULL, &outBuf_t, probs);
    // probs now holds probabilities for each speaker class
}
```

17.7 Common Mistakes and How to Avoid Them

Mistake	Symptom	Fix
Forgot <code>sgdZeroGrad</code>	Loss oscillates, gradients grow	Call <code>sgdZeroGrad</code> before each batch
Wrong layer order in backward	Loss doesn't decrease	Backward must be exact reverse of forward
<code>transposeTensor</code> not in pairs	Wrong matmul shapes, garbage output	Every <code>transposeTensor</code> must be undone
Scale not copied in ReLU	Next layer decodes wrong values	Copy scale from input to output quantization
<code>inputCache</code> is NULL in backward	Segfault / wrong gradients	Check <code>isTraining=true</code> and forward called first
VLA too large	Stack overflow, MCU crashes	Use static buffers for large tensors
float/int32 mix without casting	Compiler warning, wrong results	Explicit cast: <code>(float)(int32_val)</code>
<code>log(0)</code> in cross-entropy	NaN loss	Guard: <code>max(pred, EPSILON)</code> before log

■ RELATIONS WITH OTHER FILES

Chapter 17 connects to: Chapters 7-8 (Layers + Training) — All layer and SGD functions used in the loop · Chapter 6 (Arithmetic) — Matmul and add operations underneath each layer · Chapter 9 (Support) — RNG for weight init, serialization for saving weights · Chapter 11 (MCU) — Static buffer strategy, UART printf, STM32 startup · Chapter 16 (Math) — Xavier init formula, cross-entropy gradient derivation

Chapter 9

Layer 7 — Support Modules

RNG, Serialization, Bernoulli, and NPY Loader

Section 9.1 — RNG.h/c: The Random Number Generator

■ WHAT THIS FILE DOES

RNG.h provides a fast, deterministic pseudo-random number generator (PRNG) based on the xorshift32 algorithm. It is used for: (1) dataset shuffling (rngShuffleIndices), (2) Dropout mask generation (via Bernoulli.c), (3) stochastic rounding in FQT (via Rounding.c). The generator is seeded by rngSetSeed and maintains a single global state. Setting the same seed gives the same sequence every time (reproducibility).

```
// RNG.h – the public interface
typedef struct { uint32_t state; } rng32_t;
// Seed and query
void rngSetSeed(uint32_t seed);
uint32_t rngGetSeed(void);
// Generate a uniform float in [0, 1)
float rngNextFloat(void);
// Fisher-Yates shuffle of an index array
void rngShuffleIndices(size_t *indices, size_t n);
```

9.1.1 xorshift32 — How Random Numbers Are Generated

```
// RNG.c – the internal state and xorshift32 step:
static rng32_t rng = { .state = 1 }; // module-level, persistent
static uint32_t rngNext(rng32_t *rng) {
    uint32_t x = rng->state;
    x ^= x << 13; // step 1: XOR with left-shifted self
    x ^= x >> 17; // step 2: XOR with right-shifted result
    x ^= x << 5; // step 3: XOR with left-shifted result
    rng->state = x;
    return x;
}
```

■ C CONCEPT: XOR-Shift PRNG — Why It Works

A PRNG (Pseudo-Random Number Generator) produces a deterministic sequence that LOOKS random but is fully determined by the initial seed. XOR (exclusive-or, ^): bit-by-bit, output 1 if exactly one input is 1. $0 \wedge 0 = 0$, $0 \wedge 1 = 1$, $1 \wedge 0 = 1$, $1 \wedge 1 = 0$ $x \wedge = x \ll 13$ means: $x = x \text{ XOR } (x \text{ shifted left by 13 bits})$ This mixes bits: each bit becomes a function of multiple original bits. Three shifts (13, 17, 5) are carefully chosen constants that make the generator cycle through all $2^{32} - 1$ non-zero values before repeating (maximal period). The generator is very fast: 3 XOR + 3 shift = 6 instructions, 1 cycle on ARM. This is significantly faster than rand() from the C standard library, which is important on a microcontroller where every cycle counts.

■ WORKED EXAMPLE

Trace with seed=1 (state = 0x00000001 = binary 0000...0001): $x = 0x00000001$ $x \wedge = x \ll 13$: $x = 0x00000001 \text{ XOR } 0x00002000 = 0x00002001$ $x \wedge = x \gg 17$: $x = 0x00002001 \text{ XOR } 0x00000000 = 0x00002001$ (17-bit shift of small number = 0) $x \wedge = x \ll 5$: $x = 0x00002001 \text{ XOR } 0x00040020 = 0x00042021$ state = 0x00042021 = 270369 decimal
Next call: state = 270369, another round of 3 XOR-shifts → new number. Every call produces a different 32-bit integer.

9.1.2 rngBounded — Fair Bounded Range

```
// Get a random uint32_t in [0, bound) with NO modulo bias:
static uint32_t rngBounded(uint32_t bound) {
// Compute largest multiple of bound that fits in uint32_t:
uint32_t limit = UINT32_MAX - (UINT32_MAX % bound);
uint32_t x;
// Rejection sampling: discard values above 'limit'
do {
x = rngNext(&rng);
} while (x >= limit);
return x % bound;
}
```

■ C CONCEPT: Rejection Sampling — Avoiding Modulo Bias

Why not just use 'rngNext() % bound'? Example: bound=3, range=[0,1,2]. $UINT32_MAX+1 = 4,294,967,296$. $4,294,967,296 / 3 = 1,431,655,765$ remainder 1. So values 0 and 1 appear 1 more time than value 2 — NOT uniform! Rejection sampling: discard any generated value above 'limit' (the largest multiple of bound that fits). The remaining values are uniformly distributed when divided by bound. For bound=3: limit = $4,294,967,296 - (4,294,967,296 \% 3) = 4,294,967,295$. We reject nothing — by luck limit = $UINT32_MAX+1$ for bound=3. For bound=2: limit = $UINT32_MAX+1 - 0 = 4,294,967,296$. Even here, values 0..MAX/2-1 map to 0, others to 1 — perfectly balanced.

9.1.3 rngShuffleIndices — Fisher-Yates Shuffle

```
// rngShuffleIndices: Fisher-Yates shuffle of indices [0..n-1]
void rngShuffleIndices(size_t *indices, size_t n) {
// Initialise to identity permutation if needed:
// (caller typically calls setOrderOfDimsForNewTensor to pre-fill)
for (size_t i = n - 1; i >= 1; i--) {
// Pick a random j in [0, i] (inclusive):
size_t j = (size_t)rngBounded((uint32_t)(i + 1));
// Swap indices[i] and indices[j]:
size_t tmp = indices[i];
indices[i] = indices[j];
indices[j] = tmp;
}
}
```

■ C CONCEPT: Fisher-Yates Shuffle — Why This Algorithm?

The Fisher-Yates shuffle produces an UNBIASED permutation: all $N!$ orderings are equally likely. Algorithm (working from end to beginning): for i from $N-1$ down to 1: pick random j in $[0, i]$ swap elements i and j Example: [A, B, C, D] ($N=4$): $i=3$: $j=\text{random}[0..3]=1 \rightarrow$ swap [3] and [1] $\rightarrow$ [A, D, C, B] $i=2$: $j=\text{random}[0..2]=2 \rightarrow$ swap [2] and [2] $\rightarrow$ [A, D, C, B] (no-op) $i=1$: $j=\text{random}[0..1]=0 \rightarrow$ swap [1] and [0] $\rightarrow$ [D, A, C, B] Result: [D, A, C, B] — one of 24 equally likely orderings. Why not a naive shuffle (pick random pair and swap N times)? Naive is biased: some permutations are more likely than others.

9.1.4 rngNextFloat — Uniform Float in [0, 1)

```
float rngNextFloat(void) {
// Take top 24 bits of the 32-bit random number:
uint32_t raw = rngNext(&rng) >> 8; // 24-bit value in [0, 2^24 - 1]
return (float)raw / (float)(1 << 24); // divide by 2^24  $\rightarrow$  [0, 1)
}
```

■ C CONCEPT: 24-Bit Float Resolution

A float32 mantissa has 23 bits of precision (24 bits including the implicit 1). Using more bits would be wasted — they can't be represented exactly in float32. $\text{raw} >> 8$: discard the bottom 8 bits, keep the top 24. Divide by $2^{24} = 16,777,216$ gives a value in $[0, 1)$. Example: $\text{raw} = 0x3A7B12C4 = 979763908 >> 8 \rightarrow 0x003A7B12 = 3833618 / 16777216 = 0.22848...$ (a float in $[0, 1)$)

9.1.5 rngSetSeed — Reproducibility

```
void rngSetSeed(uint32_t seed) {  
    // xorshift32 cannot have state=0 (0 XOR anything = 0 forever)  
    rng.state = (seed == 0) ? UINT32_MAX : seed;  
}
```

■ NOTE

Always set the seed before any training run for reproducibility. With the same seed, the same shuffle order, the same dropout masks, and the same stochastic rounding decisions are made. Different seeds → different results even with identical training data and code. Publishing the seed used in your experiments lets others reproduce your results exactly.

Section 9.2 — Serialize.h/.c and Deserialize.h/.c

■ WHAT THIS FILE DOES

Serialization saves a trained model (tensor weights) to a byte stream. Deserialization reads it back. On the STM32, you train the model, then serialize weights to UART (PC receives them). Later, deserialize from flash/UART to restore the model for inference on the MCU. The format is raw bytes — each tensor is written as its metadata followed by its data.

```
// Serialize.h — save model to a FILE stream:  
void serializeTensor(tensor_t *tensor, FILE *f);  
void serializeParameter(parameter_t *parameter, FILE *f);  
void serializeModel(layer_t **model, size_t sizeModel, FILE *f);  
// Deserialize.h — restore model from a FILE stream:  
void deserializeTensor(tensor_t *tensor, FILE *f);  
void deserializeParameter(parameter_t *parameter, FILE *f);  
void deserializeModel(layer_t **model, size_t sizeModel, FILE *f);
```

■ C CONCEPT: Serialization — Writing Structs to a Byte Stream

serializeTensor writes in order: 1. tensor->quantization->type (4 bytes: qtype_t) 2. qConfig fields (scale, qMaxBits, etc.) 3. shape: numberOfDimensions, dimensions[], orderOfDimensions[] 4. data: the raw weight bytes
deserializeTensor reads in the same order and fills the struct. This only works if both writer and reader use the SAME struct layout — true on one architecture (ARM Cortex-M7 → same ARM PC) but potentially broken across platforms with different endianness or struct packing. The library uses fwrite/fread for simplicity. On the MCU, FILE* maps to UART or flash.

— ■ WORKED EXAMPLE

Serializing a SYM_INT32 tensor with shape [3,4] and 12 elements: Bytes written: [0-3]: qtype = SYM_INT32 = 2 (uint32_t, 4 bytes) [4-7]: scale = 0.001f (float, 4 bytes = 0x6F126E3A in IEEE754) [8]: qMaxBits = 16 (uint8_t, 1 byte) [9]: roundingMode = 0 (uint8_t, 1 byte) [10-13]: numberOfDimensions = 2 (size_t, 4 bytes) [14-17]: dimensions[0] = 3 (4 bytes) [18-21]: dimensions[1] = 4 (4 bytes) [22-25]: orderOfDimensions[0] = 0 (4 bytes) [26-29]: orderOfDimensions[1] = 1 (4 bytes) [30-77]: 12 × 4 = 48 bytes of int32_t mantissas Total: 78 bytes for this tensor's weights.

Section 9.3 — Bernoulli.h/.c: Sampling for Dropout

■ WHAT THIS FILE DOES

Bernoulli.c provides Bernoulli sampling: for a given probability p, generate a BOOL (0 or 1) where P(1) = p. Used by Dropout to generate the random drop mask: each mask element is 0 (drop, prob p) or 1 (keep, prob 1-p). Relies on RNG.c's rngNextFloat.

```
// Bernoulli.h:
void bernoulliSampleTensor(tensor_t *mask, float p);
// Fills all elements of 'mask' (BOOL tensor):
// mask[i] = 1 with probability p
// mask[i] = 0 with probability 1-p
// Implementation (Bernoulli.c):
void bernoulliSampleTensor(tensor_t *mask, float p) {
    size_t n = calcNumberOfElementsByTensor(mask);
    for (size_t i = 0; i < n; i++) {
        bool val = (rngNextFloat() < p); // true with prob p
        tensorBoolSet(mask, i, val);
    }
}
```

— ■ WORKED EXAMPLE

Generating a dropout mask with $p=0.5$ for 8 neurons: `rngNextFloat()` calls: [0.73, 0.12, 0.61, 0.48, 0.92, 0.33, 0.55, 0.09] Compare to $p=0.5$: [F, T, F, T, F, T, F, T] ($T = < 0.5 = \text{keep}$) Wait — Dropout DROPS with prob $p=0.5$, KEEPS with prob 0.5. So Dropout calls `bernoulliSampleTensor(mask, 1-p=0.5)` for the KEEP mask: mask = [0, 1, 0, 1, 0, 1, 0, 1] Activations multiplied by mask: [0, a1, 0, a3, 0, a5, 0, a7] The kept activations are also scaled by $1/(1-p) = 2$ (inverted dropout).

Section 9.4 — NPYLoader.h/.c: Loading NumPy Data Files

■ WHAT THIS FILE DOES

NPYLoader reads NumPy .npy files. NumPy is the standard Python array library. Training data prepared on a PC (audio features, spectrograms) is saved as .npy files and loaded onto the MCU via UART or stored in flash. NPYLoader parses the .npy header (array shape and dtype) and fills a `tensor_t` with the array data.

■ C CONCEPT: The .npy File Format

A .npy file has: Bytes 0-5: magic string 0x93NUMPY Byte 6: major version (1 or 2) Byte 7: minor version (0) Bytes 8-9: header length (uint16_t, little-endian) Bytes 10..(10+header_len-1): Python dict as ASCII text, e.g.: {'descr': '<f4', 'fortran_order': False, 'shape': (1000, 128), } After header: raw array data (C contiguous, little-endian) '<f4' means little-endian float32. '<i4' means little-endian int32. NPYLoader parses this header, extracts the shape and dtype, then reads the raw bytes directly into the tensor's data buffer.

Section 9.5 — Relations With Other Files

File	Direction	Why
RNG.h/.c	→ standalone	No project dependencies — uses only <code>stdint</code> , <code>stddef</code>
Bernoulli.h/.c	→ uses RNG + Tensor	<code>rngNextFloat</code> for sampling, <code>tensorBoolSet</code> to write mask
Dropout.c	← uses Bernoulli	Calls <code>bernoulliSampleTensor</code> to generate the drop mask
Rounding.c	← uses RNG	<code>rngNextFloat</code> for stochastic rounding noise
DataLoader.c	← uses RNG	<code>rngShuffleIndices</code> for epoch-wise shuffle
Serialize.h/.c	← uses Tensor + Layer	Iterates model layers and serializes each tensor
Deserialize.h/.c	← uses Tensor + Layer	Reads bytes and fills <code>tensor_t</code> fields
NPYLoader.h/.c	← uses Tensor	Fills <code>tensor_t.data</code> and <code>tensor_t.shape</code> from .npy bytes

Training loop	← uses all support modules	DataLoader + RNG + Serialize complete the training pipeline
---------------	----------------------------	---

■ RigL GAP IN THIS FILE

RNG.c already provides everything needed for RigL's stochastic elements. RigL itself is deterministic (prune smallest, grow largest gradient) — no extra randomness needed. The Bernoulli sampler is NOT needed for basic RigL (RigL doesn't randomly prune). However, a variant called 'Random pruning + gradient regrow' (simpler than RigL) would use Bernoulli to select which weights to prune randomly.

Chapter 10

Gap Analysis — From ODT to RigL

Every Missing Piece, How It Was Solved, and the Actual C API

Section 10.1 — What RigL Needs (The Complete Picture)

■ C CONCEPT: RigL in One Sentence

RigL (Evci et al., NeurIPS 2020) trains a neural network where a fixed fraction of weights are always zero — but WHICH weights are zero changes periodically: Every T batches: 1. PRUNE: deactivate the S% of active weights with smallest |weight| 2. GROW: reactivate the S% of inactive weights with largest |gradient| Between updates, only active weights participate in forward/backward passes. The gradient of inactive weights is still computed (for the grow step). Inactive weights stay at zero and are never updated by the optimizer.

RigL Component	Needed From ODT	Status Before This Work
Sparsity mask (BOOL tensor)	BOOL quantization + tensorBoolGet/Set	EXISTS in Tensor.c
Masked forward pass	matmul that skips masked weights	MISSING
Full gradient for inactive weights	linearCalcWeightGrads all positions	EXISTS
Mask-aware SGD step	sgdStep skips masked positions	MISSING
riglInitMask()	Initialize mask to target sparsity	MISSING
riglPruneStep()	Sort active by w , deactivate bottom S%	MISSING
riglGrowStep()	Sort inactive by grad , activate top S%	MISSING
riglUpdate()	Orchestrate prune+grow every T batches	MISSING

Section 10.2 — The Mask: riglConfig_t and BOOL Tensors

```
// riglConfig_t – configuration struct for one sparse layer (RigL.h):
typedef struct {
    tensor_t *weights; // the weight tensor being made sparse
    tensor_t *grads; // corresponding gradient tensor
    tensor_t *mask; // BOOL tensor: mask[i]=1 active, mask[i]=0 pruned
    float sparsityRatio; // 0.8 = 80% of weights pruned at all times
    size_t updateStep; // batches between prune+grow updates
    size_t batchCount; // internal counter
} riglConfig_t;

// Mask allocation for a 128x64 linear layer:
// 128 * 64 = 8192 weights -> 8192 bits -> 1024 bytes
static uint8_t maskBuf[1024];
static tensor_t maskTensor;
// set up maskTensor with BOOL quantization, shape [128,64]
riglConfig_t cfg = {
    .weights = &weightTensor,
    .grads = &gradTensor,
    .mask = &maskTensor,
    .sparsityRatio = 0.8f,
    .updateStep = 100,
    .batchCount = 0,
};
```

1	tensor_t *mask;	The mask is a separate tensor_t with BOOL quantization. It stores 1 bit per weight, packed into bytes. tensorBoolGet/Set access individual bits.
2	float sparsityRatio;	Fraction of weights that are PRUNED at any time. 0.8 = 80% zero, 20% active. This stays constant throughout training. Only the PATTERN of which weights are pruned changes.
3	size_t updateStep;	T in the RigL paper — number of training batches between topology updates. Typical value: 100. Too small = unstable; too large = topology cannot adapt.
4	size_t batchCount;	Counter incremented each batch. When batchCount % updateStep == 0, riglUpdate() fires the prune+grow step.

Section 10.3 — Gap #1 Solved: riglInitMask()

The first step is initializing the mask before training begins: exactly $(1 - \text{sparsityRatio}) * N$ weights start active, randomly selected.

```

void riglInitMask(riglConfig_t *cfg, unsigned int seed) {
    tensor_t *mask = cfg->mask;
    size_t N = calcNumberOfElementsByTensor(mask);
    size_t maskBytes = (N + 7) / 8;
    memset(mask->data, 0xFF, maskBytes); // all bits = 1 (all active)
    if (N % 8 != 0) { // fix spare bits in last byte
        size_t validBits = N % 8;
        uint8_t lastMask = (uint8_t)((1u << validBits) - 1);
        mask->data[maskBytes - 1] &= lastMask;
    }
    size_t numToPrune = (size_t)(cfg->sparsityRatio * (float)N);
    srand(seed);
    size_t pruned = 0;
    while (pruned < numToPrune) {
        size_t idx = (size_t)((unsigned)rand() % (unsigned)N);
        if (tensorBoolGet(mask, idx)) {
            tensorBoolSet(mask, idx, false);
            pruned++;
        }
    }
}

```

3	size_t N = calcNumberOfElementsByTensor(mask);	Total weight count. For [128,64]: N=8192.
4	size_t maskBytes = (N + 7) / 8;	Ceiling division: bytes needed for N bits. 8192 bits = 1024 bytes.
6	memset(mask->data, 0xFF, maskBytes);	0xFF = 11111111 binary. Sets every bit to 1 = every weight active.
8-11	if (N % 8 != 0) { ... &= lastMask; }	If N is not a multiple of 8, the last byte has extra 1-bits from memset. (1u << validBits)-1 creates a bitmask for only the valid bits. Example N=10: last byte has 2 valid bits, mask=0b00000011.
13	size_t numToPrune = (size_t)(cfg->sparsityRatio * (float)N);	sparsityRatio=0.8, N=8192 -> prune 6553 weights, keep 1639 active.
16-2 2	while (pruned < numToPrune) { ... rejection sampling ... }	Pick a random index; if it is still active, prune it. Retry if already pruned. Guarantees exactly numToPrune unique positions pruned.

Section 10.4 — Gap #2 Solved: Masked Forward Pass

During training, pruned weights must not contribute to the output. `riglMaskedMatmulFloat32` wraps the standard `matmul` but skips positions where `mask=0`. This is where the MCU speedup comes from.

```

void riglMaskedMatmulFloat32(tensor_t *A, tensor_t *B,
    tensor_t *out, tensor_t *bias,
    tensor_t *mask) {
    size_t M = getDimensionsByIndex(A, 0); // batch size
    size_t K = getDimensionsByIndex(A, 1); // input features
    size_t N = getDimensionsByIndex(B, 0); // output neurons
    for (size_t i = 0; i < M; i++) {
        for (size_t j = 0; j < N; j++) {
            float sum = 0.0f;
            for (size_t k = 0; k < K; k++) {
                if (mask != NULL) {
                    size_t flatIdx = j * K + k;
                    if (!tensorBoolGet(mask, flatIdx)) continue;
                }
                float aVal = readBytesAsFloat(A->data + (i*K+k)*4);
                float bVal = readBytesAsFloat(B->data + (j*K+k)*4);
                sum += aVal * bVal;
            }
            float biasVal = readBytesAsFloat(bias->data + j*4);
            writeFloatToByteArray(sum+biasVal, out->data + (i*N+j)*4);
        }
    }
}

```

3-5	M, K, N dimensions	M=batch rows, K=shared dimension (input features), N=output neurons. For input [1,128] and weight [64,128] (transposed): M=1, K=128, N=64.
9	float sum = 0.0f;	Accumulator reset to zero before computing each output element out[i][j].
11-1 3	if (!tensorBoolGet(mask, flatIdx)) continue;	flatIdx = j*K+k: the linear index of weight [j][k] in the mask. continue skips this k entirely — no load, no multiply. With 80% sparsity: 80% of inner-loop iterations are just a bit-check and skip.
14-1 6	readBytesAsFloat + sum += aVal * bVal	For active weights only: load input element and weight element, multiply, accumulate.
18-1 9	bias + writeFloatToByteArray	Add the per-output-neuron bias and write result to output tensor.

— ■ WORKED EXAMPLE

Speedup example — 128-input, 64-output linear, sparsityRatio=0.8: Dense matmul: 1 x 64 x 128 = 8,192 multiplications Sparse matmul: 1 x 64 x 128 x 0.2 = 1,638 multiplications Saved: 6,554 multiplications (80% fewer)
ARM Cortex-M7: 1 FMA = 1 cycle. Real speedup ~3-4x (not 5x) due to mask check overhead.

Section 10.5 — Gap #3 Solved: riglPruneStep()

The prune step finds the weakest active weights (smallest |w|) and deactivates them. Static buffers avoid stack overflow. qsort() provides the sorting.

```

#define RIGL_MAX_WEIGHTS 16384
static size_t s_activeIdx[RIGL_MAX_WEIGHTS]; // in .bss, not stack
static float s_wFloat[RIGL_MAX_WEIGHTS];
static int cmpAbsWeightAsc(const void *a, const void *b) {
    size_t ia = *(const size_t *)a; size_t ib = *(const size_t *)b;
    float fa = fabsf(s_wFloat[ia]); float fb = fabsf(s_wFloat[ib]);
    return (fa > fb) - (fa < fb); // -1, 0, or +1
}
void riglPruneStep(riglConfig_t *cfg) {
    size_t N = calcNumberOfElementsByTensor(cfg->weights);
    size_t numActive = 0;
    for (size_t i = 0; i < N && numActive < RIGL_MAX_WEIGHTS; i++) {
        if (tensorBoolGet(cfg->mask, i)) {
            s_activeIdx[numActive] = i;
            s_wFloat[numActive] = readBytesAsFloat(cfg->weights->data + i*4);
            numActive++;
        }
    }
    qsort(s_activeIdx, numActive, sizeof(size_t), cmpAbsWeightAsc);
    size_t numToPrune = (size_t)(cfg->sparsityRatio * (float)numActive);
    for (size_t i = 0; i < numToPrune; i++) {
        size_t idx = s_activeIdx[i];
        tensorBoolSet(cfg->mask, idx, false);
        writeFloatToByteArray(0.0f, cfg->weights->data + idx*4);
    }
}

```

1-2	static size_t s_activeIdx[]; static float s_wFloat[];	Static = allocated in .bss (zeroed RAM section), persists between calls. NOT on the stack. A VLA of 8192 size_t = 32 KB would overflow a ~4 KB stack.
5-8	cmpAbsWeightAsc callback	qsort calls this with pointers to two array elements (indices). Cast void* to size_t*, dereference to get index, look up weight in s_wFloat. Returns negative if w[a] < w[b] (a comes first = smaller weights first).
13-18	Collect active indices and weight values	Linear scan of all N positions. For each active weight (mask=1), record its index and float value for sorting.
20	qsort(s_activeIdx, numActive, sizeof(size_t), cmpAbsWeightAsc);	Standard C library sort. After this, s_activeIdx[0] is the index of the SMALLEST active weight.
21-26	Deactivate bottom fraction	The first numToPrune entries are the weakest. Set mask=0 (deactivate) and set weight to 0.0f. These weights will not be updated by the optimizer.

Section 10.6 — Gap #4 Solved: riglGrowStep()

```

static size_t s_inactiveIdx[RIGL_MAX_WEIGHTS];
static float s_gFloat[RIGL_MAX_WEIGHTS];
static int cmpAbsGradDesc(const void *a, const void *b) {
    size_t ia = *(const size_t *)a; size_t ib = *(const size_t *)b;
    float fa = fabsf(s_gFloat[ia]); float fb = fabsf(s_gFloat[ib]);
    return (fb > fa) - (fb < fa); // NOTE: b before a = DESCENDING
}
void riglGrowStep(riglConfig_t *cfg) {
    size_t N = calcNumberOfElementsByTensor(cfg->mask);
    size_t numActive = 0;
    for (size_t i = 0; i < N; i++)
        if (tensorBoolGet(cfg->mask, i)) numActive++;
    size_t numInactive = 0;
    for (size_t i = 0; i < N && numInactive < RIGL_MAX_WEIGHTS; i++) {
        if (!tensorBoolGet(cfg->mask, i)) {
            s_inactiveIdx[numInactive] = i;
            s_gFloat[numInactive] = readBytesAsFloat(cfg->grads->data + i*4);
            numInactive++;
        }
    }
    qsort(s_inactiveIdx, numInactive, sizeof(size_t), cmpAbsGradDesc);
    size_t targetActive = (size_t)((1.0f - cfg->sparsityRatio) * (float)N);
    size_t numToGrow = (targetActive > numActive) ? (targetActive - numActive) : 0;
    for (size_t i = 0; i < numToGrow && i < numInactive; i++) {
        tensorBoolSet(cfg->mask, s_inactiveIdx[i], true);
        // weight stays 0 - grows from gradient updates next T batches
    }
}

```

7	return (fb > fa) - (fb < fa);	Descending sort: b comes before a when fb > fa. After sort, s_inactiveIdx[0] is the inactive weight with the LARGEST gradient .
11-1 3	Count numActive	Count how many weights are currently active. We need this to calculate how many to grow.
15-2 1	Collect inactive indices + gradient magnitudes	For every pruned weight (mask=0), record its index and gradient value.
22	qsort(..., cmpAbsGradDesc)	Sort so position [0] = inactive weight with highest gradient magnitude. This weight would reduce the loss the most if re-enabled.
24-2 5	targetActive and numToGrow	targetActive = N * (1-sparsityRatio) = the desired number of active weights. numToGrow = targetActive - current numActive. After prune removed some, grow restores exact sparsity ratio.
26-2 8	tensorBoolSet(mask, idx, true)	Re-enable the weight. Its value is 0.0f. Starting next batch, the optimizer will update it. Over the next T batches it grows from 0 based on gradient information.

Section 10.7 — Gap #5 Solved: riglUpdate() + riglMaskedSgdStepFloat()

```

// riglUpdate() – call once per batch, fires every updateStep batches:
void riglUpdate(riglConfig_t *cfg) {
    cfg->batchCount++;
    if (cfg->batchCount % cfg->updateStep != 0) return;
    riglPruneStep(cfg);
    riglGrowStep(cfg);
    size_t bytes = calcNumberOfElementsByTensor(cfg->grads) * 4;
    memset(cfg->grads->data, 0, bytes); // zero stale gradients
}
// riglMaskedSgdStepFloat() – mask-aware optimizer step:
void riglMaskedSgdStepFloat(riglConfig_t *cfg, float lr) {
    tensor_t *w = cfg->weights;
    tensor_t *g = cfg->grads;
    tensor_t *mask = cfg->mask;
    size_t N = calcNumberOfElementsByTensor(w);
    for (size_t i = 0; i < N; i++) {
        if (!tensorBoolGet(mask, i)) continue; // skip pruned
        float wi = readBytesAsFloat(w->data + i*4);
        float gi = readBytesAsFloat(g->data + i*4);
        writeFloatToByteArray(wi - lr * gi, w->data + i*4);
    }
}

```

3	cfg->batchCount++;	Incremented every batch. riglUpdate is called unconditionally each batch but returns early if not yet time for a topology update.
4	if (batchCount % updateStep != 0) return;	The modulo check: only proceed every updateStep batches. For updateStep=100: triggers at batch 100, 200, 300, ...
5-6	riglPruneStep(); riglGrowStep();	Prune first, then grow. Order matters: if we grew first, newly activated weights (value=0) would immediately be the smallest and get pruned in the same step.
7-8	memset(grads, 0, bytes)	Zero all gradients after topology change. The gradient computed under the OLD topology is not valid for the NEW topology. Stale gradients would corrupt the NEXT grow step's selection.
17	if (!tensorBoolGet(mask, i)) continue;	The single most important line for RigL correctness: pruned weights (mask=0) are never updated by the optimizer. They stay at 0.0f. Only the grow step can reactivate them.

Section 10.8 — Complete RigL Training Loop


```

// One-time setup:
riglConfig_t riglCfg = { &linearWeights, &linearWeightGrads,
&linearMask, 0.8f, 100, 0 };
riglInitMask(&riglCfg, 42); // seed=42, reproducible initial mask
// Training loop:
for (size_t epoch = 0; epoch < NUM_EPOCHS; epoch++) {
for (size_t batch = 0; batch < NUM_BATCHES; batch++) {
// 1. Zero gradients
memset(linearWeightGrads.data, 0, gradBytes);
// 2. Forward pass (sparse matmul skips pruned weights)
riglMaskedMatmulFloat32(&inputBatch, &linearWeightsT,
&outputBatch, &linearBias, &linearMask);
reluForwardFloat(&outputBatch);
// 3. Loss
float loss = crossEntropyLossFloat(&outputBatch, &labelBatch, &lossGrad);
// 4. Backward (gradients computed for ALL weights including pruned!)
reluBackwardFloat(&lossGrad, &outputBatch);
linearCalcWeightGradsFloat(&lossGrad, &inputBatch, &linearWeightGrads);
// 5. Optimizer (only updates active weights)
riglMaskedSgdStepFloat(&riglCfg, LEARNING_RATE);
// 6. RigL update (fires every 100 batches automatically)
riglUpdate(&riglCfg);
}
}

```

■ RigL GAP IN THIS FILE

Key insight: `linearCalcWeightGradsFloat` computes gradients for ALL N positions, even pruned ones. This is not a bug — it is intentional. The grow step needs the gradient at pruned positions to decide which to reactivate. We deliberately compute more gradients than the optimizer uses, because those extra gradients drive the topology evolution.

■ NOTE

Chapter 21 contains the complete, compilable `RigL.h` and `RigL.c` with all functions fully implemented. This chapter explained WHY each piece was needed. Chapter 21 shows the complete HOW.

Chapter 11

Embedded Systems Deep Dive — STM32 & ARM Cortex-M7

Hardware Architecture, Memory Management, Fragmentation, Arena, Startup, UART, FPU

Section 11.1 — What Is an Embedded System?

■ C CONCEPT: Embedded System vs General-Purpose Computer

A general-purpose computer (PC, laptop) runs an operating system (Windows, Linux, macOS). The OS provides: virtual memory, processes, threads, filesystems, dynamic allocation (malloc), device drivers, networking, and more. An embedded system runs ONE application directly on hardware (bare-metal). NO operating system (or a very minimal RTOS). Fixed amount of RAM — no virtual memory, no paging, no swap. No heap (or a tiny heap used very carefully). No filesystem (unless external flash/SD card added). Tight power and cost constraints. The STM32 Nucleo-F746ZG is a bare-metal embedded system: - 320 KB SRAM. ALL program data lives here. No more. - 1 MB flash. Code and constants live here permanently. - 216 MHz ARM Cortex-M7 CPU. - FPU for fast floating-point arithmetic. - UART for serial communication with a PC.

Resource	Amount	Implication for ODT
SRAM	320 KB	All tensors + activations + stack must fit. No dynamic expansion.
Flash	1 MB	Code (.text) + initialized data (.data) + constants (.rodata).
CPU	216 MHz ARM Cortex-M7	~216M integer ops/s. ~216M float ops/s (FPU).
FPU	Double-precision optional	Single-precision FMA in 1 cycle. Critical for float training.
UART	4.5 Mbps max	Load training data from PC; save trained weights back.
RTOS	None (bare-metal)	No scheduler, no multitasking, no file I/O.

Section 11.2 — ARM Cortex-M7 Architecture: Registers and Pipeline

■ C CONCEPT: ARM Cortex-M7 Registers

The CPU has 16 32-bit integer registers (R0-R15): R0-R3: Function arguments and return values R4-R11: Callee-saved general purpose registers R12: Intra-procedure-call scratch (IP) R13: Stack Pointer (SP) — points to top of stack R14: Link Register (LR) — stores return address for function calls R15: Program Counter (PC) — address of next instruction And 32 32-bit floating-point registers (S0-S31, also viewable as 64-bit D0-D15): S0-S15: FPU scratch registers S16-S31: Callee-saved FPU registers Also: xPSR (processor status register) with N, Z, C, V flags (Negative, Zero, Carry, oVerflow) — set by arithmetic instructions, read by conditional branches.

■ C CONCEPT: The 6-Stage Pipeline

The Cortex-M7 has a 6-stage superscalar pipeline: 1. Fetch: load instruction from cache/flash 2. Decode: interpret the instruction 3. Issue: send to execution unit 4. Execute: perform the operation 5. Memory: load/store data 6. Write-back: store result to register In the ideal case, one instruction completes per cycle (pipelining). Pipeline stalls occur with: - Data hazards: instruction needs result of previous instruction not yet complete - Branch misprediction: conditional branch jumps unexpectedly - Cache miss: needed data not in L1 cache → wait for SRAM access For ODT matmul inner loops: sequential memory accesses (cache-friendly) + no branches (no conditionals inside the ReLU-free matmul) → close to 1 instruction per cycle sustained throughput.

Section 11.3 — Memory Sections: .text, .data, .bss, Stack, Heap

Memory Section	Lifetime	Example in ODT
Embedded machine code + constants	Permanent (survives reset)	All functions: m...
Only initialized data	Permanent	const char* erro...
Uninitialized global/static variables	Copied to RAM at boot	static rng32_t r...
Zeroed global/static variables	Zeroed at boot	static size_t s...
Local variables, function call frames, return addresses	Function scope	float sum, size...
Dynamic allocations	Until free()	NOT used in O...

```
// Memory map example for ODT on STM32F746:
// FLASH (1 MB, starting at 0x08000000):
// 0x08000000: vector table (interrupt handlers)
// 0x08000200: startup code (ResetHandler)
// 0x08001000: .text (all compiled functions)
// 0x08080000: .rodata (string constants, lookup tables)
// 0x080A0000: .data initial values (copied to RAM at boot)
// SRAM (320 KB, starting at 0x20000000):
// 0x20000000: .data (copied from flash by startup code)
// 0x20001000: .bss (zeroed by startup code)
// 0x20002000: static arrays (tensors, buffers)
// 0x2004F000: stack (grows DOWN from 0x20050000)
// 0x20050000: top of SRAM (stack starts here)
// Total SRAM used by ODT (128->64->N speaker model):
// Weights (128*64*4 + 64*N*4) + Grads (same) + Activations + Mask
// = ~130 KB for a typical model → fits in 320 KB with room to spare
```

Section 11.4 — Memory Management: Fragmentation, Arena, Pool, Static

■ C CONCEPT: Heap Fragmentation — Why Embedded Systems Avoid malloc

Heap fragmentation occurs when malloc/free is used repeatedly: the heap becomes Swiss cheese — lots of small free chunks that cannot serve large requests. Example sequence: Initial heap: [free: 100 KB] malloc(40 KB): [A: 40 KB | free: 60 KB] malloc(30 KB): [A: 40 KB | B: 30 KB | free: 30 KB] free(A): [free: 40 KB | B: 30 KB | free: 30 KB] malloc(50 KB): FAILS — no contiguous 50 KB block! (only 40+30=70 total but not contiguous) This is EXTERNAL FRAGMENTATION. The heap has enough total free memory but not in a single contiguous block. On a 320 KB MCU: one failed malloc crashes the program (hard fault or silent corruption). There is no way to recover — no swap, no compaction, no virtual memory. This is why ODT uses NO malloc.

■ C CONCEPT: Arena Allocator (Bump Allocator)

An arena (also called bump allocator or linear allocator) is the simplest possible allocator: Start: a large contiguous buffer and a pointer to its start. Allocate N bytes: return current pointer, advance pointer by N. Free all: reset the pointer to the start (free the entire arena at once). static uint8_t arenaBuffer[256 * 1024]; // 256 KB arena static uint8_t *arenaPtr = arenaBuffer; // bump pointer void* arenaAlloc(size_t n) { void *result = arenaPtr; arenaPtr += n; return result; } void arenaReset() { arenaPtr = arenaBuffer; } // free everything Properties: - O(1) allocation (just a pointer increment) - Zero fragmentation (allocations are contiguous) - Cannot free individual allocations — only the whole arena - Perfect for neural network activations: allocate for forward pass, reset for next batch ODT does not implement a formal arena, but the static array pattern achieves the same fragmentation-free benefit.

■ C CONCEPT: Memory Pool Allocator

A memory pool allocates fixed-size slots from a pre-allocated buffer. `#define POOL_SLOT_SIZE 32 // bytes per slot`
`#define POOL_NUM_SLOTS 64 // total slots`
`uint8_t poolBuffer[POOL_SLOT_SIZE * POOL_NUM_SLOTS];`
`bool slotUsed[POOL_NUM_SLOTS] = {false};`
`void* poolAlloc() { for (int i=0; i<POOL_NUM_SLOTS; i++) if (!slotUsed[i]) { slotUsed[i]=true; return &poolBuffer[i*POOL_SLOT_SIZE]; } return NULL; // pool exhausted }`
`void poolFree(void *p) { int i = ((uint8_t*)p - poolBuffer) / POOL_SLOT_SIZE; slotUsed[i] = false; }`
Properties: - O(1) allocation and deallocation - No external fragmentation (fixed slots) - Can free individual allocations - Only works for fixed-size allocations (padding needed for smaller objects) - Used for `layer_t` instances if dynamic layer creation is needed

■ C CONCEPT: ODT's Approach: Static Allocation

ODT avoids all allocators by using static arrays: `static uint8_t w1Data[128*64*4];` // COMPILE TIME: exactly 32 KB
`static uint8_t g1Data[128*64*4];` // COMPILE TIME: exactly 32 KB
`static tensor_t weightTensor1;` // struct in .bss
Properties of static allocation: - ZERO runtime allocation overhead (sizes fixed at compile time) - ZERO fragmentation (no dynamic allocation = no fragmentation) - ZERO risk of out-of-memory at runtime (linker checks at build time!) - INFLEXIBLE: model architecture is fixed at compile time - No runtime error: if arrays don't fit, the LINKER fails, not the MCU The linker error 'region SRAM overflowed by N bytes' means: your static arrays exceed 320 KB. Reduce model size or use a different MCU. RigL's static sort buffers (`s_activeldx[]`, `s_wFloat[]`) follow this pattern: allocated at .bss, no heap, linker checks the total at build time.

■■ IMPORTANT

Stack overflow is the embedded programmer's nightmare. Declaring a large array inside a function (not static) puts it on the STACK: `void riglSort() { size_t indices[8192]; ... }` // WRONG: 8192*4=32 KB on stack! The stack on STM32 is ~4-8 KB by default. 32 KB overflows it silently: other variables, return addresses, and function frames are corrupted. The MCU crashes or produces wrong results with NO clear error message. Fix: add 'static' keyword: `static size_t indices[8192];` — now in .bss, not stack. Cost: the array persists between function calls, but that is fine for sort buffers.

Section 11.5 — Startup Sequence: Before main()

On a PC, the OS loads your program, sets up the environment, and calls `main()`. On bare-metal STM32, `startup.c` (provided by ST/ARM) does this manually.

```
// Startup sequence on STM32 (simplified from startup_stm32f746xx.s):
// 1. CPU starts at reset vector (address in vector table at 0x08000000+4):
void ResetHandler(void) {
// 2. Initialize .data: copy initialized globals from flash to RAM
uint32_t *src = &__data_start_flash; // flash address of initial values
uint32_t *dst = &__data_start_ram; // RAM destination
while (dst < &__data_end_ram) *dst++ = *src++;
// 3. Zero .bss: zero-initialize uninitialized globals
uint32_t *bss = &__bss_start;
while (bss < &__bss_end) *bss++ = 0;
// 4. Initialize C++ static constructors (not used in ODT C code):
// __libc_init_array();
// 5. Configure clocks (PLL to 216 MHz):
SystemClock_Config(); // HAL function
// 6. Enable FPU (CPACR register):
SCB->CPACR |= (0xF << 20); // enable CP10 and CP11 (FPU coprocessors)
// 7. Enable instruction and data cache:
SCB_EnableICache();
SCB_EnableDCache();
// 8. Call application entry point:
main();
// 9. If main() returns (it shouldn't), loop forever:
while (1);
}
```

5-7	.data copy from flash to RAM	Global variables with initializers (e.g., static rng32_t rng = {.state=1}) have their initial values stored in flash. The startup copies them to RAM. After this: rng.state == 1 in RAM.
9-11	.bss zero-initialization	All global/static variables without initializers are guaranteed to be 0 by C standard. The startup writes zero to every word in .bss. This includes all ODT's static sort buffers: s_activeldx[], s_wFloat[], etc.
16	SCB->CPACR = (0xF << 20);	Enable the Floating Point Unit. Without this, any float instruction causes an exception. SCB = System Control Block (ARM hardware register). Bits 20-23: CP10 (single-precision FPU) and CP11 (double-precision). Setting both to 0b11 = full access from privileged and unprivileged modes.
18-19	SCB_EnableICache + SCB_EnableDCache	Enable L1 instruction cache and data cache. Without cache: every instruction fetch and data access hits the slow SRAM bus. With cache: repeated accesses to the same data are served from fast cache (1 cycle vs 3-5).

Section 11.6 — The FPU: Why Floating-Point Is Fast on Cortex-M7

■ C CONCEPT: Floating-Point Without FPU vs With FPU

Without FPU (soft-float): every float operation is emulated in software. float add: ~20-50 cycles (a library function call) float multiply: ~30-100 cycles matmul inner loop (1 multiply + 1 add): ~50-150 cycles With FPU (hard-float, enabled in startup): VADD.F32 (float add): 1-2 cycles VMUL.F32 (float multiply): 1-2 cycles VFMA.F32 (fused multiply-add, $a*b+c$): 1-2 cycles The matmul inner loop: 1-2 cycles Speedup for ODT training: 20-50x faster with FPU enabled. ODT compiles with -mfpu=fpv5-d16 -mfloat-abi=hard (hard-float ABI). FPU also has dedicated float registers (S0-S31) — float values stay in FPU registers during computation instead of being loaded/stored to integer registers.

```
// Example: the matmul inner loop as compiled assembly with FPU enabled:
// C code:
// sum += aVal * bVal;
// ARMv7-M with FPU (-O2 optimization):
// VLDR S0, [R0, #offset] ; load aVal from memory into FPU register S0
// VLDR S1, [R1, #offset] ; load bVal from memory into FPU register S1
// VFMA.F32 S2, S0, S1 ; S2 = S2 + S0 * S1 (fused multiply-add)
// ; S2 is 'sum', updated in 1 cycle (ignoring memory latency)
// Without FPU (-mfloat-abi=soft):
// PUSH {r0-r3, lr}
// BL __aeabi_fmul ; software float multiply (20+ cycles)
// BL __aeabi_fadd ; software float add (20+ cycles)
// POP {r0-r3, pc}
```

Section 11.7 — UART: Loading Training Data and Saving Weights

■ C CONCEPT: UART — Universal Asynchronous Receiver-Transmitter

UART is a serial communication protocol for point-to-point data transfer. No clock signal — sender and receiver must agree on baud rate beforehand. Frame format (8N1, most common): 1 start bit (always 0) | 8 data bits (LSB first) | 1 stop bit (always 1) Total: 10 bits per byte transmitted. At 4.5 Mbps (bits per second): Actual data rate = $4.5\text{M} / 10 = 450\text{ KB/s}$ Time to transfer 32 KB weight tensor: $32*1024 / 450*1024 \approx 71\text{ ms}$ Time to load 1000 audio samples of 128 features each: $1000*128*4 = 512\text{ KB} \rightarrow 512/450 \approx 1.1\text{ seconds}$ In ODT: UART is used for: 1. Receiving training data (.npy batches from PC) 2. Receiving model weights (pre-trained initialization) 3. Sending trained weights back to PC after training 4. Debug output (PRINT_DEBUG macro via UART)

```

// HAL UART usage in ODT (simplified):
// Receive N bytes from PC (blocking):
HAL_StatusTypeDef receiveBytes(uint8_t *buf, uint32_t nBytes) {
// HAL_UART_Receive blocks until nBytes received or timeout:
return HAL_UART_Receive(&huart3, buf, nBytes, HAL_MAX_DELAY);
}
// Send N bytes to PC (blocking):
HAL_StatusTypeDef sendBytes(const uint8_t *buf, uint32_t nBytes) {
return HAL_UART_Transmit(&huart3, (uint8_t*)buf, nBytes, HAL_MAX_DELAY);
}
// Load a trained weight tensor from PC via UART:
void loadTensorFromUART(tensor_t *tensor) {
size_t bytes = calcNumberOfBytesForData(
tensor->quantization, calcNumberOfElementsByTensor(tensor));
receiveBytes(tensor->data, (uint32_t)bytes);
}
// After training: send all weights to PC for evaluation:
void serializeModelToUART(layer_t **model, size_t numLayers) {
for (size_t i = 0; i < numLayers; i++) {
// ... serialize each layer's weights and bias ...
serializeTensor(&model[i]->config.linear->weights.param, NULL);
}
}

```

Section 11.8 — Interrupts and the NVIC

■ C CONCEPT: Interrupts — Asynchronous Event Handling

An interrupt is a hardware signal that pauses the main program and runs a special function (ISR = Interrupt Service Routine). Example: UART receives a byte. Without interrupt: Main code must constantly poll: `while(!uartDataReady) { /* spin */ }` CPU does nothing useful while waiting (wastes cycles). With interrupt: 1. UART hardware raises an interrupt signal when a byte arrives. 2. CPU saves current state (all registers) to stack. 3. CPU jumps to the ISR function (UART_IRQHandler). 4. ISR reads the byte and stores it in a buffer. 5. CPU restores saved state and continues the main program. The NVIC (Nested Vectored Interrupt Controller) manages: - Which interrupts are enabled/disabled - Priority (higher priority ISR can interrupt lower priority ISR) - Pending state (interrupt arrived but not yet handled) In ODT: UART interrupt is typically used for UART receive. DMA (Direct Memory Access) can also be used to receive large amounts of data without CPU involvement.

```

// ODT UART interrupt handling (simplified):
// Ring buffer for incoming data:
#define RING_BUF_SIZE 1024
static volatile uint8_t ringBuf[RING_BUF_SIZE];
static volatile uint32_t ringHead = 0, ringTail = 0;
// ISR: called automatically when UART receives a byte:
void USART3_IRQHandler(void) {
if (USART3->ISR & USART_ISR_RXNE) { // data received?
uint8_t byte = (uint8_t)USART3->RDR; // read clears flag
ringBuf[ringHead % RING_BUF_SIZE] = byte;
ringHead++;
}
}
// volatile keyword: tells compiler this variable CAN change outside normal code flow
// (it is written by the ISR, which is called by hardware, not by our code directly).
// Without volatile: compiler might cache ringHead in a register and never re-read it.

```

4-5 static volatile uint8_t ringBuf[] +
volatile indices

volatile: tells the compiler 'do NOT optimize away reads/writes to this variable.' Without volatile, the compiler might cache ringHead in a register. The ISR writes ringHead in hardware-triggered code, so the main code must re-read from memory every time.

8-9	USART3->ISR & USART_ISR_RXNE	Read the USART status register (ISR). RXNE bit = 'Receive Register Not Empty' — set by hardware when a byte arrives. Reading USART3->RDR (Receive Data Register) clears RXNE automatically.
10-12	ringBuf[ringHead % RING_BUF_SIZE]	Circular (ring) buffer: when ringHead reaches RING_BUF_SIZE, modulo wraps it back to 0. Buffer never overflows (old data is overwritten). Main code reads from ringTail; ISR writes to ringHead.

Section 11.9 — Instruction and Data Cache

■ C CONCEPT: Why Caches Matter for Neural Network Inference

The Cortex-M7 has: - 16 KB instruction cache (L1 ICache) - 16 KB data cache (L1 DCache) Without cache: SRAM access: 3-5 cycles (wait states for AHB bus) Flash access (for code): 5-10 cycles (flash wait states) With cache: Cache hit: 1 cycle Cache miss (must fetch from SRAM/flash): 3-10 cycles + cache fill For neural network matmul: Input vector [128 floats] = 512 bytes: fits entirely in cache → all accesses hit. Weight matrix [64×128] = 32 KB: exceeds 16 KB cache → many cache misses. Cache-friendly access pattern: sequential (row-by-row) access reuses cache lines. Random access (sparse indirect indexing) causes cache thrashing. This is why sparse matmul with mask checks can be slower than expected on cache-limited hardware — the mask access pattern is often random.

Section 11.10 — The Linker Script: Defining Memory Regions

```
/* STM32F746ZG linker script (STM32F746ZGTx_FLASH.ld) — simplified: */
MEMORY {
FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 1024K /* 1 MB flash */
SRAM (xrw) : ORIGIN = 0x20000000, LENGTH = 320K /* 320 KB SRAM */
}
SECTIONS {
.text : { /* code section → flash */
*(.text*) /* all .text from all object files */
*(.rodata*) /* read-only data */
} > FLASH
.data : { /* initialized data → copied from flash to SRAM at boot */
*(.data*)
} > SRAM AT> FLASH /* AT>FLASH: initial values stored in flash */
.bss : { /* zero-initialized data → SRAM, no flash cost */
*(.bss*)
*(COMMON)
} > SRAM
/* Stack: top of SRAM */
_estack = ORIGIN(SRAM) + LENGTH(SRAM); /* 0x20000000 + 0x50000 = 0x20050000 */
}
```

2-4	MEMORY { FLASH ... SRAM ... }	Defines the physical memory regions available on this MCU. rx = readable + executable (flash). xrw = executable + readable + writable (SRAM). LENGTH must match the actual MCU: wrong values = linker places code that doesn't fit.
7-10	.text : { *(.text*) } > FLASH	All compiled code goes to flash. *(.text*): wildcard matches .text, .text.funcname, etc. from all object files. The linker places them all contiguously in flash, starting at 0x08000000.
12-14	.data: initial values in FLASH, runtime copy in SRAM	AT>FLASH means initial values are stored in flash (no extra RAM needed). At boot, startup.c copies them to SRAM. This is how 'static rng32_t rng = {state=1}' gets its initial value.
20	_estack = ORIGIN(SRAM) + LENGTH(SRAM);	Stack pointer initialized to the TOP of SRAM. Stack grows downward: each function call decrements SP. If SP reaches .bss or .data = stack overflow (silent corruption).

■ NOTE

When the linker reports 'region SRAM overflowed by 12345 bytes', it means .data + .bss + static arrays exceed 320 KB. Solutions: reduce model size, increase RIGL_MAX_WEIGHTS budget, or move less-critical data to flash (const arrays).

Section 11.11 — Complete ODT Memory Budget for Speaker ID Model

Size	Section	Notes
32 KB	.bss (static)	SYM_INT32 or float32 mantissas
32 KB	.bss (static)	float32 or SYM_INT32
256 B	.bss (static)	float32
256 B	.bss (static)	float32
Varies	.bss	16KB for N=64 speakers
Varies	.bss	Same as W2
N*4 B	.bss	Tiny
1 KB	.bss	1 bit/weight, packed
128 KB	.bss	s_activeIdx + s_wFloat + inactive + s_gFloat at 16384 × REDUCE if needed.
~4 KB	stack or .bss	input, h1, h2, logits, probs
~4 KB	stack or .bss	lossGrad, dh2, dh1
8 KB	SRAM top	Function call frames, local vars
~217 KB	< 320 KB	Fits with ~100 KB margin

■ RigL GAP IN THIS FILE

The RigL sort buffers (128 KB for RIGL_MAX_WEIGHTS=16384) are the largest overhead. For a 128×64 layer (N=8192 weights), reduce to RIGL_MAX_WEIGHTS=8192: $4 \times 8192 \times 4 = 128 \text{ KB} \rightarrow 64 \text{ KB}$. The savings (64 KB) give more room for larger models or more speaker classes. Define RIGL_MAX_WEIGHTS to match your actual largest layer size.

Embedded Systems Deep Dive

STM32, ARM Cortex-M7, bare-metal C, and how the MCU executes your code

18.1 What Is an Embedded System?

An **embedded system** is a computer designed to perform a specific task, built into a larger device. Unlike a laptop or desktop, it typically has no operating system (or a very minimal one), limited resources, and must run reliably for years without human intervention.

The STM32 Nucleo-F746ZG is a development board containing an **STM32F746ZG microcontroller** — a single chip with CPU, RAM, flash, and peripherals all integrated together. Your C code runs directly on the metal.

Component	Specification	Relevance to ODT
CPU	ARM Cortex-M7 @ 216 MHz	Executes all C code
RAM (SRAM)	320 KB total	All tensors, stack, globals
Flash	1 MB	Code + constant data
FPU	Double-precision FPU	Float operations without software emulation
DSP	DSP instructions	SIMD-style operations (not used in ODT)
UART	Multiple UARTs	Serial communication, printf debugging
Timers	Multiple 16/32-bit	Training loop timing
DMA	DMA controller	Memory-to-memory transfers
I-Cache/D-Cache	16 KB instruction, 16 KB data	Speeds up repeated code/data access
Bus width	32-bit AHB/APB	All memory accesses are 32-bit aligned

18.2 ARM Cortex-M7 Architecture

The ARM Cortex-M7 is a 32-bit RISC (Reduced Instruction Set Computer) processor. Understanding its architecture helps explain why ODT makes certain design choices.

18.2.1 Registers

The CPU has 16 general-purpose 32-bit registers (R0–R15) plus status registers. All computation happens in registers — data must be loaded from memory into a register before it can be used, then stored back.

ARM Cortex-M7 key registers

R0–R3: Function arguments and return values
R4–R11: Callee-saved registers (function must preserve these)
R12: Intra-procedure-call scratch register
R13 (SP): Stack Pointer — points to top of current stack frame
R14 (LR): Link Register — return address for function calls
R15 (PC): Program Counter — address of next instruction to execute
xPSR: Status register — zero flag, negative flag, carry flag, etc.
FPU registers (for float operations):
S0–S31: 32 single-precision float registers (32-bit each)
D0–D15: 16 double-precision float registers (64-bit, alias S pairs)
FPSCR: FPU status/control register

When the compiler generates code for `float a = b * c`, it emits VLDR instructions to load b and c into S0 and S1, then VMUL.F32 to multiply them and store the result in another S register, then VSTR to save back to RAM.

18.2.2 The Pipeline

The Cortex-M7 has a 6-stage pipeline: Fetch → Decode → Issue → Execute → Write-back → Retire. While one instruction is executing, the next is being decoded, and the one after that is being fetched. This overlap is called pipelining.

Pipeline hazards occur when an instruction depends on the result of the previous one. The CPU must stall (wait) for the first result. This is why tight loops with data dependencies (like accumulating a sum) can be slower than expected.

WORKED EXAMPLE

Pipeline in matrix multiply inner loop: `acc += A[i][k] * B[k][j]`
Cycle 1: Load A[i][k] into S0
Cycle 2: Load B[k][j] into S1, (waiting for S0 to be ready)
Cycle 3: VMUL.F32 S2, S0, S1 (multiply)
Cycle 4: VADD.F32 S3, S3, S2 (accumulate — must wait for VMUL result)
Cycle 5: Increment k, check loop condition
Total: ~5 cycles per inner loop iteration at 216 MHz → ~43M inner iterations/second

18.2.3 Memory Architecture

The Cortex-M7 uses a Harvard architecture variant with separate instruction and data buses. This means it can fetch the next instruction while loading/storing data for the current instruction simultaneously.

STM32F746ZG memory map (simplified)

0x00000000 - 0x000FFFFF: Flash (1 MB) — code + const data
0x20000000 - 0x2004FFFF: DTCM SRAM (320 KB) — fastest RAM
0x40000000 - 0x5FFFFFFF: Peripheral registers (UART, timers, GPIO, etc.)
0xE0000000 - 0xFFFFFFFF: System/debug registers
Stack grows DOWN from top of SRAM (0x2004FFFF → 0x20000000)
.bss + .data grow UP from bottom of SRAM (0x20000000 → ...)

18.3 Memory Sections in Detail

Your compiled C program is divided into named memory sections. Understanding these is critical for embedded programming.

18.3.1 The .text Section (Flash)

All compiled C functions and constant data go into the .text section, stored in flash memory. Flash is read-only at runtime (you can't write to it during normal execution). It survives power cycles.

What lives in .text

```
// All function code:
void matmulFloat32TensorsWithBias(...) { ... } // compiled to ARM opcodes in flash
void linearForwardFloat(...) { ... }
// String literals:
const char *name = 'ODT'; // 'ODT\0' stored in flash
// Constant arrays:
const float lut[256] = { 0.1, 0.2, ... }; // lookup table in flash
```

18.3.2 The .data Section (RAM — Initialized Globals)

Global and static variables that have initial values go into .data. The initial values are stored in flash and copied to RAM at startup.

.data example

```
float learningRate = 0.01f; // in .data — initial value 0.01
int epochCount = 0; // in .bss — zero initial value
// At startup, the MCU's reset handler copies the flash image
// of .data into RAM. This is why your globals have their initial values.
```

18.3.3 The .bss Section (RAM — Zero-Initialized)

Variables declared without an explicit initial value default to zero. These go in .bss (Block Started by Symbol). At startup, the reset handler calls `memset(0)` over the entire .bss region. This is cheap in flash because we only store the size, not 320KB of zeros.

.bss example — ODT static buffers

```
// All of these go in .bss — zero at startup, no flash storage needed
static float w1Data[8*4]; // 128 bytes in .bss
static float g1Data[8*4]; // 128 bytes in .bss
static tensor_t w1; // tensor metadata in .bss
static linearConfig_t lin1cfg; // layer config in .bss
// Cost in flash: ~4 bytes to record the size
// Cost in RAM: 32*4 = 128 bytes at runtime
```

18.3.4 The Stack

Every function call creates a **stack frame** — a region of memory for local variables and saved registers. The stack grows downward from the top of SRAM. Stack frames are automatically created on function entry and destroyed on return.

Stack frame example

```
void myFunction(int x) {  
    float localArray[64]; // 256 bytes on stack (VLA style if runtime-sized)  
    int count = 0; // 4 bytes on stack  
    // ...  
} // stack frame is automatically freed here  
// Total stack depth example for ODT training loop:  
main() → 64 bytes  
trainEpoch() → 32 bytes  
layerFunctions[].forward() → 32 bytes  
matmulFloat32TensorsWithBias() → 128 bytes  
matmulIntCore() → 48 bytes  
// Total stack depth: ~304 bytes  
// STM32 has 320KB RAM – stack and static buffers must not overlap
```

■■ IMPORTANT

Stack Overflow on MCUs: On a desktop OS, stack overflow triggers a segfault. On the STM32, stack overflow silently corrupts other data (it overwrites .bss or .data). This causes bizarre behavior that is very hard to debug. Always calculate maximum stack depth for embedded code.

18.4 Startup Sequence — Before main()

Most developers assume C programs start at main(). On embedded systems, a lot happens before main() is called.

STM32 startup sequence

```
// 1. CPU resets: PC = address stored in vector table at 0x00000004
// (this is the Reset_Handler address)
// 2. Reset_Handler (in startup_stm32f746zg.s, written in assembly):
Reset_Handler:
// Set stack pointer to top of SRAM
LDR R0, =_estack
MSR MSP, R0
// Copy .data section from flash to RAM
LDR R0, =_sdata // destination in RAM
LDR R1, =_edata // end of .data in RAM
LDR R2, =_sidata // source in flash
CopyLoop:
LDR R3, [R2], #4
STR R3, [R0], #4
CMP R0, R1
BLT CopyLoop
// Zero out .bss section
LDR R0, =_sbss
LDR R1, =_ebss
MOV R2, #0
ZeroLoop:
STR R2, [R0], #4
CMP R0, R1
BLT ZeroLoop
// 3. Call SystemInit() – configure PLL, flash wait states
// 4. Call main()
BL main
```

This is why your static float w1Data[] array is automatically zeroed — the startup code clears .bss before main() runs.

18.5 Clock Configuration — Running at 216 MHz

The STM32F746ZG has an internal 16 MHz RC oscillator as the default clock. To reach 216 MHz, it uses a Phase-Locked Loop (PLL) which multiplies the input frequency.

SystemClock_Config (STM32CubeMX generated)

```
void SystemClock_Config(void) {
    RCC_OscInitTypeDef osc = {0};
    RCC_ClkInitTypeDef clk = {0};
    // Enable HSE (external crystal, usually 8 or 25 MHz)
    osc.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    osc.HSEState = RCC_HSE_BYPASS;
    osc.PLL.PLLState = RCC_PLL_ON;
    osc.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    // PLL multiplier: 8 MHz * (N/M) / P = 216 MHz
    osc.PLL.PLLM = 4; // divide by 4 → 2 MHz
    osc.PLL.PLLN = 216; // multiply by 216 → 432 MHz
    osc.PLL.PLLP = 2; // divide by 2 → 216 MHz
    osc.PLL.PLLQ = 9; // USB clock = 432/9 = 48 MHz
    HAL_RCC_OscConfig(&osc);
    // Set flash wait states (needed at high clock speeds)
    __HAL_FLASH_SET_LATENCY(FLASH_LATENCY_7);
    clk.ClockType = RCC_CLOCKTYPE_SYSCLK | ...
    clk.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    clk.AHBCLKDivider = RCC_SYSCLK_DIV1; // AHB = 216 MHz
    clk.APB1CLKDivider = RCC_HCLK_DIV4; // APB1 = 54 MHz
    clk.APB2CLKDivider = RCC_HCLK_DIV2; // APB2 = 108 MHz
    HAL_RCC_ClockConfig(&clk, FLASH_LATENCY_7);
}
```

HAL

Hardware Abstraction Layer — STM32's C driver library. HAL functions have names like HAL_RCC_OscConfig, HAL_UART_Transmit. They wrap the raw register writes into readable function calls.

18.6 UART — Debugging and Data Transfer

UART (Universal Asynchronous Receiver/Transmitter) is a serial communication protocol. On the Nucleo board, one UART is connected to the ST-Link debugger chip, which bridges to USB. This creates a virtual COM port on the PC.

This is how ODT's `serializeTensor` sends weight data to the PC, and how `printf` outputs debugging information.

UART setup and use

```
static UART_HandleTypeDef huart3; // UART3 connected to ST-Link on Nucleo
void MX_USART3_UART_Init(void) {
    huart3.Instance = USART3;
    huart3.Init.BaudRate = 115200; // 115200 bits per second
    huart3.Init.WordLength = UART_WORDLENGTH_8B;
    huart3.Init.StopBits = UART_STOPBITS_1;
    huart3.Init.Parity = UART_PARITY_NONE;
    huart3.Init.Mode = UART_MODE_TX_RX;
    HAL_UART_Init(&huart3);
}
// Send data:
uint8_t data[] = {0x01, 0x02, 0xFF};
HAL_UART_Transmit(&huart3, data, 3, HAL_MAX_DELAY);
// 115200 baud → ~11520 bytes/second
// Sending 128-byte weight matrix: ~11 ms
// Redirect printf to UART:
int _write(int file, char *ptr, int len) {
    HAL_UART_Transmit(&huart3, (uint8_t*)ptr, len, HAL_MAX_DELAY);
    return len;
}
// Now printf works over UART!
```

115
200
baud

Baud rate = bits per second. With 8 data bits + 1 stop bit = 9 bits per byte. At 115200 baud: $115200/9 \approx 12800$ bytes/second.

HAL
_M
AX_
DEL
AY

Wait until the transmission is complete. On an MCU without an OS, blocking is usually fine for debugging.

18.7 Interrupts and the NVIC

An **interrupt** is a hardware signal that pauses the CPU's current task and calls a special function (the **Interrupt Service Routine**, or ISR). Common interrupt sources: UART receive, timer overflow, GPIO edge.

ODT's training loop is designed to run in the main loop (not in an ISR). But the data loader could use a UART receive interrupt to gather audio samples while computation runs.

Timer interrupt example — measure training loop time

```
volatile uint32_t timerTicks = 0; // volatile! Modified by ISR
// ISR — called every 1 ms by Timer 6
void TIM6_DAC_IRQHandler(void) {
    if (__HAL_TIM_GET_FLAG(&htim6, TIM_FLAG_UPDATE)) {
        __HAL_TIM_CLEAR_FLAG(&htim6, TIM_FLAG_UPDATE);
        timerTicks++;
    }
}
// In main loop:
uint32_t start = timerTicks;
layerFunctions[LAYER_LINEAR].forward(&lin1, &input, &h1);
uint32_t elapsed = timerTicks - start;
printf('Linear forward: %u ms\n', elapsed);
```


■ **C CONCEPT: The volatile Keyword**

When a variable can be modified by an interrupt (or any external event), it must be declared volatile. Without volatile, the compiler may cache the variable in a register and never re-read it from RAM, missing updates made by the ISR. Always use volatile for ISR-shared variables.

18.8 The FPU — Why Float Is Fast on Cortex-M7

Older Cortex-M0 and M0+ processors have no hardware FPU — float operations are emulated in software, taking ~100 cycles each. The Cortex-M7 has a double-precision FPU that handles float operations in 1-3 cycles.

FPU enabling (in system_stm32f7xx.c)

```
// Enable FPU – must be done before any float operations
#define FPU_CPACR (*((volatile uint32_t*)0xE000ED88))
void FPU_Enable(void) {
    FPU_CPACR |= (0xF << 20); // CP10 and CP11 = full access
    __DSB(); // Data Synchronization Barrier
    __ISB(); // Instruction Synchronization Barrier
}
// With FPU enabled:
// float a = b * c; → VMUL.F32 S0, S1, S2 (1-3 cycles)
// Without FPU (software emulation):
// float a = b * c; → BL __aeabi_fmul (~80-100 cycles)
```

The FPU is why ODT can run full float32 neural networks on the STM32. Without it, even a small matmul would be prohibitively slow.

18.9 Instruction and Data Cache

The Cortex-M7 has separate 16 KB instruction cache (I-cache) and 16 KB data cache (D-cache). Flash has 7-cycle wait states at 216 MHz, so without the I-cache, instruction fetches would take 7× longer.

The D-cache is especially important for ODT: the matmul inner loop repeatedly reads the same rows of the weight matrix. With D-cache, these reads come from fast cache (1-4 cycles) rather than slow SRAM (3-4 cycles, but still faster than flash at 216 MHz).

Enabling caches (in main.c)

```
SCB_EnableICache(); // Enable instruction cache
SCB_EnableDCache(); // Enable data cache
// These are usually the first two lines in main()
// Typical speedup: 2-4x for instruction-heavy code
```

■■ IMPORTANT

Cache Coherency: If you use DMA (Direct Memory Access) to transfer data without going through the CPU, the D-cache can become stale. You must explicitly invalidate or clean the cache after DMA transfers. ODT uses memcpy (CPU-driven), so cache coherency is not an issue in the current implementation.

18.10 Linking and the Linker Script

The `<b>linker script</b>` (STM32F746ZGTx_FLASH.ld) tells the linker where to place each section in the memory map. This is what ensures your code goes to flash and your buffers go to SRAM.

Linker script excerpt (STM32F746ZGTx_FLASH.ld)

```
MEMORY {
  RAM (xrw) : ORIGIN = 0x20000000, LENGTH = 320K
  FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 1024K
}
SECTIONS {
  /* Code and read-only data go to flash */
  .text : {
    KEEP(*(.isr_vector)) /* Vector table must be first */
    *(.text) /* All .text sections */
    *(.rodata) /* Constant data */
  } > FLASH
  /* Initialized globals: stored in flash, copied to RAM at startup */
  .data : {
    _sdata = .;
    *(.data)
    _edata = .;
  } > RAM AT > FLASH
  /* Zero-initialized globals: only in RAM */
  .bss : {
    _sbss = .;
    *(.bss)
    *(COMMON)
    _ebss = .;
  } > RAM
  /* Stack at top of RAM */
  _estack = ORIGIN(RAM) + LENGTH(RAM);
}
```

AT
> FL
AS
H

The .data section lives in RAM at runtime (> RAM) but is stored in flash during programming (AT > FLASH). The startup code copies it.

KEE
P*(.
isr_
vect
or))

The vector table (interrupt handler addresses) must be at address 0x08000000 (start of flash). KEEP prevents the linker from removing it even if it looks unused.

_sd
ata,
_ed
ata,
_sb
ss,
_eb
ss,
_est
ack

These symbols are used by the Reset_Handler in startup assembly to know where to copy .data and zero .bss.

18.11 Summary: Why Embedded C Is Different

Feature	Desktop C	Embedded C (STM32)
Memory	Virtual memory, GBs available	Physical only, 320 KB fixed
OS	Linux/Windows manages resources	Bare-metal, you manage everything
malloc/free	Generally safe	Avoid — fragmentation causes crashes
printf	Writes to terminal	Must redirect to UART
Stack overflow	Segfault + OS recovery	Silent corruption, very hard to debug
Interrupts	Rarely need to think about	Must use volatile, avoid race conditions
Float operations	Always hardware-accelerated	Need to enable FPU explicitly
Startup	OS handles initialization	Reset_Handler copies .data, zeros .bss
Debugging	gdb, IDE, print statements	SWD/JTAG probe, UART printf, blinking LED
Power	Not a concern	May need low-power modes for battery life

■ RELATIONS WITH OTHER FILES

Chapter 18 connects to: Chapter 11 (MCU chapter) — Static buffers, startup sequence, SRAM layout · Chapter 13 (C language) — volatile, static, memory sections in C · Chapter 17 (Worked example) — printf via UART, static buffers for tensors · Chapter 9 (Support/RNG) — rng32_t using DTCM SRAM for fast access · Chapter 6 (Arithmetic) — FPU acceleration for matmul float operations

Chapter 13

C Language Deep Dive for Embedded Systems

Pointers, Memory Layout, Structs, and All C Patterns Used in ODT

Section 13.1 — Pointers: The Most Important C Concept

■ C CONCEPT: What is a Pointer?

A pointer is a variable that holds a MEMORY ADDRESS (a location in RAM). Instead of storing a value, it stores the address WHERE a value is stored. `int x = 42;` // x holds the VALUE 42, lives at some address (say 0x2000) `int *p = &x;` // p holds the ADDRESS 0x2000 (& = 'address of') `int y = *p;` // *p = dereference: READ the value at address 0x2000 = 42 `*p = 100;` // write 100 to address 0x2000 → x is now 100 Types of pointers in ODT: `uint8_t *data` → points to a byte array (tensor storage) `size_t *dimensions` → points to an array of sizes `tensor_t *tensor` → points to a `tensor_t` struct `void *qConfig` → generic pointer to any config struct `forwardFn_t forward` → pointer to a function

```
// Pointer examples from the ODT library:
// 1. tensor_t has a pointer to its data:
tensor_t t;
uint8_t buf[48]; // 12 float32 elements
t.data = buf; // t.data now holds the address of buf[0]
t.data[0] = 0x3F; // write byte to buf[0] (same as buf[0] = 0x3F)
// 2. Pointer arithmetic – move a pointer forward by N bytes:
uint8_t *elem3 = t.data + 3 * sizeof(float); // points to element 3
// This is equivalent to &t.data[3 * sizeof(float)]
// 3. Arrow operator → : dereference + field access in one step:
tensor_t *pt = &t;
size_t n = pt->shape->numberOfDimensions; // same as (*pt).shape->numberOfDimensions
// 4. NULL pointer: a pointer that doesn't point to anything valid:
tensor_t *nothing = NULL;
// Accessing nothing->data would crash (segfault / HardFault on STM32)
if (nothing != NULL) { // ALWAYS check before using
size_t x = nothing->shape->numberOfDimensions;
}
// 5. const pointer: can't modify what it points to:
const uint8_t *readOnly = buf; // can read readOnly[0] but NOT write it
// readOnly[0] = 5; ← compiler error
```

■ C CONCEPT: Why Pointers are Essential for ODT

On a microcontroller with 320 KB of RAM, copying data wastes memory and time. ODT passes `tensor_t*` everywhere instead of `tensor_t` (by value) because: 1. `sizeof(tensor_t)` = ~16 bytes (4 pointers). Passing by pointer: 4 bytes on 32-bit ARM. 2. Modification: functions that update tensors (forward pass writes output) MUST receive a pointer to modify the original, not a copy. 3. Large buffers: `tensor->data` points to potentially megabytes of weight data. Copying by value would copy that too — catastrophic. Rule: if the function READS a tensor, pass `const tensor_t*`. If it MODIFIES a tensor, pass `tensor_t*`.

Section 13.2 — Structs: Grouping Related Data

```
// Struct basics:
typedef struct point {
int x;
int y;
} point_t;
// Creating and using:
point_t p; // p is on the stack, uninitialized
p.x = 3; // set field x
p.y = 4;
int dist = p.x * p.x + p.y * p.y; // = 25
// Struct on the heap (ODT avoids this, uses static instead):
point_t *p2 = malloc(sizeof(point_t));
p2->x = 5; // arrow: dereference then access
free(p2);
// Struct layout in memory (no padding for same-sized fields):
// [x: 4 bytes][y: 4 bytes] = 8 bytes total
// sizeof(point_t) = 8
// Struct with pointer fields (the ODT pattern):
typedef struct shape {
size_t numberOfDimensions; // 4 or 8 bytes
size_t *dimensions; // 4 bytes (pointer)
size_t *orderOfDimensions; // 4 bytes (pointer)
} shape_t;
// sizeof(shape_t) ≈ 12 bytes on 32-bit ARM
// The actual dimension data is ELSEWHERE, pointed to by dimensions*
```

■ C CONCEPT: Struct vs Union vs Enum

struct: ALL fields exist simultaneously. Total size = sum of field sizes (+ alignment). struct { int a; float b; } → 8 bytes, both a and b are valid. union: ONLY ONE field valid at a time. Total size = LARGEST field size. union { linearConfig_t *linear; reluConfig_t *relu; } → 4 bytes (one pointer). Reading the wrong field is UNDEFINED BEHAVIOR. enum: named integer constants. enum {A, B, C} → A=0, B=1, C=2. sizeof(enum) = sizeof(int) = 4 bytes on ARM. ODT uses all three: - tensor_t, shape_t, linearConfig_t: structs (all fields always present) - layerConfig_t: union (holds one of 11 config pointers) - qtype_t, layerType_t, lossFuncType_t: enums (small integer sets)

Section 13.3 — Arrays, Pointer Arithmetic, and Multi-Dimensional Access

```
// Arrays in C are CONTIGUOUS blocks of memory:
int arr[5] = {10, 20, 30, 40, 50};
// [0] [1] [2] [3] [4]
// Memory: 10 20 30 40 50 (each int = 4 bytes)
// Address of arr[0]: 0x1000
// Address of arr[1]: 0x1004 (0x1000 + 4)
// Address of arr[2]: 0x1008 etc.
// Pointer arithmetic: arr+i points to arr[i]
int *p = arr; // p points to arr[0]
p++; // p now points to arr[1] (advances by sizeof(int) = 4 bytes)
// *(p+2) = arr[3] (two ints forward from arr[1])
// ODT tensor access: element i at byte offset i*sizeof(int32_t):
int32_t val = readBytesAsInt32(tensor->data + i * sizeof(int32_t));
// tensor->data is uint8_t*
// Adding i*4 gives the byte address of the i-th int32_t element
// 2D array as flat 1D (row-major):
// Matrix [3][4] stored as: [row0: 4 ints][row1: 4 ints][row2: 4 ints]
// Element [r][c] at flat index r*4 + c
// In bytes: (r*4 + c) * sizeof(int32_t)
// The ODT equivalent using calcElementIndexByIndices:
size_t indices[2] = {r, c};
size_t flatIdx = calcElementIndexByIndices(2, dims, indices, order);
int32_t v = readBytesAsInt32(tensor->data + flatIdx * 4);
```

— ■ WORKED EXAMPLE

Memory layout of a [3×4] INT32 tensor with data [0,1,...,11]: Address: 0x2000 0x2004 0x2008 0x200C 0x2010 ...
Value: 0 1 2 3 4 ... Meaning: [0][0] [0][1] [0][2] [0][3] [1][0] ... tensor->data = 0x2000 (pointer to start) Element [1][2] =
flat index 1×4 + 2 = 6 Byte offset = 6 × 4 = 24 bytes Address = 0x2000 + 24 = 0x2018 Value at 0x2018 = 6 ✓

Section 13.4 — Function Pointers: C's Dispatch Mechanism

```
// A function pointer stores the ADDRESS of a function.
// Step 1: define the function signature type:
typedef int32_t (*int32BinaryOp_t)(int32_t a, int32_t b);
// This says: 'int32BinaryOp_t is a pointer to any function
// that takes two int32_t and returns int32_t'
// Step 2: write concrete functions matching this signature:
int32_t addInt32s(int32_t a, int32_t b) { return a + b; }
int32_t mulInt32s(int32_t a, int32_t b) { return a * b; }
int32_t maxInt32s(int32_t a, int32_t b) { return (a > b) ? a : b; }
// Step 3: store or pass a function pointer:
int32BinaryOp_t op = addInt32s; // op now holds the address of addInt32s
int32_t result = op(3, 4); // calls addInt32s(3,4) = 7
op = mulInt32s;
result = op(3, 4); // calls mulInt32s(3,4) = 12
// Step 4: pass as argument (like int32PointWiseArithmetic):
void applyToAll(int32_t *arr, size_t n, int32_t x, int32BinaryOp_t op) {
    for (size_t i = 0; i < n; i++)
        arr[i] = op(arr[i], x);
}
// Usage:
int32_t data[4] = {1, 2, 3, 4};
applyToAll(data, 4, 10, addInt32s); // adds 10 to each: {11,12,13,14}
applyToAll(data, 4, 2, mulInt32s); // multiplies by 2: {22,24,26,28}
```

■ C CONCEPT: Function Pointer Typedef Syntax — The Tricky Part

typedef void (*forwardFn_t)(layer_t*, tensor_t*, tensor_t*); Read this inside-out: *forwardFn_t → it's a pointer (*forwardFn_t) → to a function (layer_t*, tensor_t*, tensor_t*) → taking these parameters void → returning void So forwardFn_t is a type alias for 'pointer to function(layer_t*, tensor_t*, tensor_t*) returning void'. An alternative syntax (C23/GCC extension) using __typeof__ or function alias macros exists but is non-standard. The typedef form above is portable C99.

Section 13.5 — Storage Classes: static, extern, and Global Variables

```
// 1. LOCAL (automatic) – lives on the stack, dies when function returns:
void foo() {
    int x = 42; // created on stack entry, freed on return
}
// 2. STATIC (file-scope) – lives in BSS/data section for entire program:
static uint8_t weightData[32768]; // declared outside functions
// OR static inside a function:
void bar() {
    static int count = 0; // persists between calls!
    count++;
}
// 3. EXTERN – 'this variable is defined in another .c file':
// In Layer.h:
extern layerFunctions_t layerFunctions[];
// This declares that layerFunctions[] is DEFINED in Layer.c
// Any .c file that includes Layer.h can use layerFunctions[] without redefining it
// In Layer.c:
layerFunctions_t layerFunctions[] = { // ← the actual DEFINITION
    [LINEAR] = { linearForward, linearBackward, linearCalcOutputShape },
    // ...
};
// 4. STATIC + function scope = module-level state (hidden from other files):
// In RNG.c:
static rng32_t rng = { .state = 1 }; // only RNG.c can access this
// Other files cannot accidentally modify rng – it's encapsulated in RNG.c
```

■ C CONCEPT: Why static at File Scope?

In C, a function or variable declared static at file scope is PRIVATE to that .c file. Other files cannot see or use it (the linker doesn't export it). This is C's primary encapsulation mechanism: static rng32_t rng → only RNG.c's functions can modify the RNG state static uint32_t rngNext(...) → only RNG.c can call rngNext; public API is rngNextFloat Without static, rng would be visible to ALL .c files (global scope). Any file could accidentally reset the seed to 0 and corrupt training.

Section 13.6 — Memory Sections on ARM Cortex-M7

Location	Contents	Lifetime
Flash (1MB)	Compiled code, string literals, const arrays	Forever (read-only)
SRAM (copied from flash at startup)	Initialized global/static variables (non-zero initial value)	Forever
SRAM (zero-initialized at startup)	Uninitialized global/static variables (zeroed by startup)	Forever
SRAM (grows downward)	Function call frames, local variables, VLAs	Duration of function
SRAM (between .bss and stack)	malloc'd memory	Until free()

```
// Example: where does each variable live?
static uint8_t weightData[32768]; // → .bss (large, initialized to 0)
static float lr = 0.001f; // → .data (non-zero initial value)
const char *msg = "hello"; // msg ptr → .data; "hello" string → .text
void train() {
    int epoch = 0; // → stack
    float tmpBuf[1024]; // → stack (VLA: 4KB! careful with stack size)
    float *grad = malloc(N*4); // → heap (should use static instead!)
    // After train() returns:
    // epoch and tmpBuf are gone (stack frame freed)
    // grad is LEAKED unless free(grad) is called
    free(grad);
}
// Memory map at runtime (typical Cortex-M7):
// 0x00000000 - 0x001FFFFFF: Flash (1MB)
// .text + .rodata (.bss/.data initializers)
// 0x20000000 - 0x2004FFFF: SRAM (320KB)
// .data → .bss → heap → (gap) → stack (grows down)
```

Section 13.7 — Preprocessor Directives in Detail


```
// The preprocessor runs BEFORE compilation. It does text substitution.
// #define: simple text replacement
#define MAX_LAYERS 16
layer_t *layers[MAX_LAYERS]; // becomes: layer_t *layers[16];
// #define with arguments (function-like macro):
#define ABS(x) ((x) < 0 ? -(x) : (x))
int a = ABS(-5); // becomes: int a = ((-5) < 0 ? -(-5) : (-5)) = 5
// Note the extra parentheses around x: ABS(a+b) → ((a+b) < 0 ? -(a+b) : (a+b))
// Without them: ABS(a+b) → (a+b < 0 ? -a+b : a+b) – WRONG due to precedence!
// #include: paste file contents
#include <stdint.h> // system header (angle brackets: search system paths)
#include "Tensor.h" // project header (quotes: search current dir first)
// #ifdef / #ifndef / #endif: conditional compilation
#ifndef ODT_TENSOR_H // if ODT_TENSOR_H is NOT defined:
#define ODT_TENSOR_H // define it (marking this file as included)
// ... header content ...
#endif // end of conditional
// #pragma: compiler-specific directives (non-portable):
#pragma once // alternative to header guards (GCC/Clang extension)
// __VA_ARGS__: variadic macros
#define PRINT_DEBUG(level, fmt, ...) \
do { if (DLEVEL >= level) printf(fmt, ##__VA_ARGS__); } while(0)
// ## before __VA_ARGS__: removes the comma if no args follow fmt
PRINT_DEBUG(2, "Loss: %.4f", loss); // → printf("Loss: %.4f", loss)
PRINT_DEBUG(1, "starting"); // → printf("starting") (no comma issue)
```

Section 13.8 — Type Casting in C

```
// Explicit cast: (type)expression
// 1. Widening cast – safe, no information loss:
int32_t a = 32767;
int64_t b = (int64_t)a; // 32-bit → 64-bit, always safe
// 2. Narrowing cast – may lose information:
int64_t big = 100000000000LL;
int32_t small = (int32_t)big; // TRUNCATION – only lower 32 bits kept
// Always clamp before narrowing!
// 3. Float ↔ integer cast:
float f = 3.14f;
int32_t i = (int32_t)f; // rounds TOWARD ZERO: i = 3
// Use roundf() to round to nearest: (int32_t)roundf(f) = 3
// 4. Pointer cast – the most dangerous!
uint8_t *bytes = ...
float *floats = (float *)bytes; // ONLY safe if bytes is properly aligned!
// On ARM, unaligned float access may trigger HardFault.
// Safe alternative: memcpy into a float variable.
// 5. void* cast – used for qConfig:
void *ptr = &symInt32QCfg;
symInt32QConfig_t *cfg = (symInt32QConfig_t *)ptr; // explicit cast
// In C (not C++), void* can be implicitly assigned to any pointer.
// But explicit cast makes intent clear.
// 6. The safe cast pattern from ODT:
// WRONG:
float *wArr = (float *)w->data; // only safe for FLOAT32 tensors!
// RIGHT:
if (w->quantization->type == FLOAT32) {
float *wArr = (float *)w->data; // now guaranteed safe
}
```

Section 13.9 — Embedded C Patterns Specific to ODT

13.9.1 Designated Initializers

```
// Designated initializer: initialize specific fields by name
static rng32_t rng = { .state = 1 };
// Equivalent to: rng.state = 1; (all other fields = 0)
// Array designated initializer (used in dispatch table):
layerFunctions_t layerFunctions[] = {
    [LINEAR] = { linearForward, linearBackward, linearCalcOutputShape },
    [RELU] = { reluForward, ... },
    [SOFTMAX] = { softmaxForward, ... },
};
// [LINEAR] = 0, [RELU] = 1, etc. (from the enum values)
// Slots not initialized (e.g. indices between 0 and LINEAR) are zero-filled
// This syntax requires C99 or later.
```

13.9.2 Compound Literals

```
// Compound literal: create a temporary struct/array inline:
size_t aIdx[2] = {row, k}; // stack array
// Or inline (C99):
calcElementIndexByIndices(2, dims, (size_t[]){row, k}, order);
// (size_t[]){row, k} creates a temporary 2-element array
// Used in matmulIntCore:
size_t aIndices[] = {rowIndex, i}; // local array, no separate declaration
```

13.9.3 Flexible Array Members and Zero-Length Arrays

```
// The ODT dispatch tables use a flexible-size approach:
layerFunctions_t layerFunctions[]; // no size specified = extern (defined elsewhere)
// In the definition file, size is determined by the initializer:
layerFunctions_t layerFunctions[] = {
    [LINEAR] = ..., [RELU] = ..., ..., [LAYERNORM] = ...
};
// The compiler counts the initializers and sets the array size automatically.
// sizeof(layerFunctions) / sizeof(layerFunctions[0]) = number of layer types
```

13.9.4 exit(1) vs Return Error Codes

```
// ODT uses exit(1) for fatal errors:
if (aColumns != bRows) {
    PRINT_ERROR("Dimension mismatch");
    exit(1);
}
// Alternative: return error code (not used in ODT for simplicity):
int matmul(...) {
    if (aColumns != bRows) return -1; // error
    // ... compute ...
    return 0; // success
}
// Caller must check: if (matmul(...) != 0) { /* handle error */ }
// On bare-metal MCU, exit(1) typically calls _exit() which loops forever
// (or resets the MCU, depending on the startup code).
// This is acceptable: a dimension mismatch is a programming error,
// not a runtime condition to recover from.
// For production code, a reset might be more appropriate:
// NVIC_SystemReset(); // ARM reset via CMSIS
```

Section 13.10 — Common C Mistakes in Embedded Systems

■■ IMPORTANT

MISTAKE 1: Integer overflow in accumulation. `int32_t acc += a * b; // a*b may overflow before being added to acc`
FIX: `acc += (int64_t)a * (int64_t)b; — cast BEFORE multiply`

■■ IMPORTANT

MISTAKE 2: Signed/unsigned comparison. `for (int i = n-1; i >= 0; i--) // works fine for (size_t i = n-1; i >= 0; i--) //`
INFINITE LOOP! `size_t` is unsigned, wraps to HUGE after 0 FIX: use 'int' for loop counters that must go negative.

■■ IMPORTANT

MISTAKE 3: Dangling pointer. `tensor_t *t = &localTensor; // t points to a local variable return t; // localTensor is freed on return! t now points to garbage` FIX: use static/global arrays or heap allocation for tensors that must outlive functions.

■■ IMPORTANT

MISTAKE 4: Missing volatile for hardware registers. `while (!(UART->SR & 0x40)) {} // busy-wait for UART ready`
Without 'volatile', the compiler may cache `UART->SR` and loop forever. FIX: hardware registers are declared volatile in STM32 HAL headers.

■■ IMPORTANT

MISTAKE 5: Forgetting NULL check before pointer dereference. `tensor->sparsity->config->mask // if sparsity==NULL: HardFault! FIX: always check: if (tensor->sparsity != NULL && tensor->sparsity->config != NULL)`

Chapter 14

Complete Function Reference Index

Every Public Function in the ODT Library

Section 14.1 — DTypes Module

Function	Signature	Purpose
readBytesAsInt32	int32_t f(const uint8_t *bytes)	Copy 4 bytes into int32_t via memcpy. Used to read int32 from tensor byte buffer.
readBytesAsFloat	float f(const uint8_t *bytes)	Copy 4 bytes into float via memcpy. Used to read float32 from tensor byte buffer.
writeInt32ToByteArray	void f(int32_t value, uint8_t *bytes)	Copy int32_t into 4 bytes via memcpy. Used to write int32 results into tensor.
writeFloatToByteArray	void f(float value, uint8_t *bytes)	Copy float into 4 bytes via memcpy. Used to write float results into tensor.
writeBitsToByteArray	void f(uint8_t *ba, size_t start, size_t end, uint8_t data)	Write sub-byte bits into a byte array. Used for BOOL tensors and sparse masks.
readBitsFromByteArray	uint8_t f(const uint8_t *ba, size_t start, size_t end)	Read sub-byte bits from a byte array. Companion to writeBitsToByteArray.

Section 14.2 — Quantization Module

Function	Signature	Purpose
initInt32Quantization	void f(quantization_t *q)	Set type=INT32, qConfig=NULL.
initFloat32Quantization	void f(quantization_t *q)	Set type=FLOAT32, qConfig=NULL.
initSymInt32Quantization	void f(quantization_t *q, symInt32QConfig_t *cfg)	Set type=SYM_INT32, qConfig=cfg.
initBoolQuantization	void f(quantization_t *q, boolQConfig_t *cfg)	Set type=BOOL, qConfig=cfg.
initInt32QConfig	void f(int32QConfig_t *cfg)	Initialize empty INT32 config (no-op, placeholder).
initFloat32QConfig	void f(float32QConfig_t *cfg)	Initialize empty FLOAT32 config.
initSymInt32QConfig	void f(roundingMode_t rm, symInt32QConfig_t *cfg)	Set scale=1.0, qMaxBits=16, roundingMode=rm.
calcNumberOfBytesForData	size_t f(quantization_t *q, size_t N)	Return bytes needed for N elements of type q->type.
calcBitsPerElement	size_t f(quantization_t *q)	Return bits per element for the given quantization type.

Section 14.3 — Tensor Module

Function	Signature	Purpose
setTensorValues	void f(tensor_t*, uint8_t*, shape_t*, quantization_t*, sparsity_t*)	Link all tensor fields. No allocation — stores pointers.
setParameterValues	void f(parameter_t*, tensor_t* param, tensor_t* grad)	Link weight and gradient into parameter_t.
setShape	void f(shape_t*, size_t* dims, size_t N, size_t* order)	Link shape fields. No allocation.

setOrderOfDimsForNewTensor	void f(size_t N, size_t* order)	Fill order with identity {0,1,...,N-1}.
transposeTensor	void f(tensor_t*, size_t dim0, size_t dim1)	Swap two axes in orderOfDimensions (O(1)).
copyTensor	void f(tensor_t* dest, tensor_t* src)	memcpy data bytes from src to dest.
calcNumberOfElementsByShape	size_t f(shape_t*)	Product of all dimensions.
calcNumberOfElementsByTensor	size_t f(tensor_t*)	Calls calcNumberOfElementsByShape on tensor->shape.
calcBytesPerElement	size_t f(quantization_t*)	Bytes per element based on quantization type.
calcBytesPerTensor	size_t f(tensor_t*)	Total bytes = elements × bytesPerElement.
tensorBoolGet	bool f(tensor_t*, size_t flatIndex)	Read 1 bit from BOOL tensor at flatIndex.
tensorBoolSet	void f(tensor_t*, size_t flatIndex, bool val)	Write 1 bit to BOOL tensor at flatIndex.
getParamFromParameter	tensor_t* f(parameter_t*)	Return p->param (weight tensor).
getGradFromParameter	tensor_t* f(parameter_t*)	Return p->grad (gradient tensor).
printTensor	void f(tensor_t*)	Print all values to UART (debug use).
printShape	void f(shape_t*)	Print shape info to UART.

Section 14.4 — TensorConversion Module

Function	Signature	Purpose
convertTensor	void f(tensor_t* in, tensor_t* out)	Convert in to out's quantization type. Dispatches via conversionMatrix[fromType][toType].
requantSymInt32Tensor	void f(tensor_t* in, tensor_t* out)	Two-pass SYM_INT32 → SYM_INT32 with auto-computed scale. In-place safe.
requantSymInt32TensorToScale	void f(tensor_t* in, tensor_t* out)	SYM_INT32 → SYM_INT32 using pre-set target scale. Caller must set out->qConfig->scale first.
quantTypeToString	char* f(qtype_t t)	Return type name as string (for debug).
conversionMatrix[6][6]	conversionFunction_t table	6×6 table of type conversion functions. Index by [fromType][toType].

Section 14.5 — Arithmetic Module

Function	Signature	Purpose
getDimensionsByIndex	size_t f(tensor_t*, size_t index)	Logical dimension size at axis 'index', accounting for transpose.
orderDims	void f(tensor_t*, size_t* out)	Fill out[] with logical dimension sizes [0..N-1].
doDimensionsMatch	bool f(tensor_t* a, tensor_t* b)	True if all logical dimensions match (accounts for transpose).
calcTensorIndexByIndices	size_t f(size_t N, size_t* dims, size_t* indices)	Row-major flat index from multi-dim indices (no transpose).
calcIndicesByRawIndex	void f(size_t N, size_t* dims, size_t raw, size_t* out)	Reverse: flat raw index → multi-dim logical indices.
calcElementIndexByIndices	size_t f(size_t N, size_t* dims, size_t* idx, size_t* order)	Physical flat index from logical indices, applying orderOfDimensions permutation.
int32PointWiseArithmetic	void f(tensor_t* a, tensor_t* b, fn, tensor_t* out)	Apply binary fn to each pair of elements. Transpose-aware.
floatPointWiseArithmetic	void f(tensor_t* a, tensor_t* b, fn, tensor_t* out)	Float version of point-wise arithmetic.

int32PointWiseArithmeticInplace	void f(tensor_t* a, tensor_t* b, fn)	Inplace: $a[i] = fn(a[i], b[i])$. Faster (no output tensor).
int32ElementWithTensorArithmetic	void f(tensor_t* a, int32_t x, fn, tensor_t* out)	Apply fn to each tensor element and scalar x.

Section 14.6 — Add / Sub / Mul / Div Modules

Function	Signature	Purpose
addInt32s / addFloat32s	int32_t/float f(a, b)	Primitive element operations: return a+b.
addInt32Tensors	void f(tensor_t* a, tensor_t* b, tensor_t* out)	Element-wise INT32 add.
addFloat32TensorsInplace	void f(tensor_t* a, tensor_t* b)	$a[i] += b[i]$ for all i, FLOAT32.
addInt32TensorToSymInt32TensorInplace	void f(tensor_t* sym, tensor_t* int32)	$sym_mantissa[i] += int32[i]$ for all i (mixed-type accumulation).
addFloat32TensorToSymInt32TensorInplace	void f(tensor_t* sym, tensor_t* f32)	$sym[i] += f32[i] / scale$ (converts float grad to SYM_INT32 units).
addSymInt32Tensors / addSymInt32TensorsInplace	void f(tensor_t* a, tensor_t* b, ...)	SYM_INT32 addition with scale matching.
subInt32Tensors / subFloat32Tensors / subSymInt32Tensors	void f(a, b, out)	Element-wise subtraction variants.
mulInt32s / mulFloat32s	int32_t/float f(a, b)	Primitive element multiplication.
mulInt32Tensors / mulFloat32Tensors / mulSymInt32Tensors	void f(a, b, out)	Element-wise tensor multiplication.
divInt32Tensors / divFloat32Tensors	void f(a, b, out)	Element-wise tensor division.
getAddInstructionCounter	size_t f()	Return count of add operations (for profiling).

Section 14.7 — Matmul, MinMax, Sum, Log, Rounding, Comparison

Function	Signature	Purpose
matmulInt32Tensors	void f(a, b, out)	INT32 matrix multiply without bias.
matmulFloat32Tensors	void f(a, b, out)	FLOAT32 matrix multiply without bias.
matmulSymInt32Tensors	void f(a, b, out)	SYM_INT32 matrix multiply without bias.
matmulFloat32TensorsWithBias	void f(a, b, out, bias)	FLOAT32 $Y = X @ W + b$.
matmulSymInt32TensorsWithBias	void f(a, b, out, bias)	SYM_INT32 $Y = X @ W + b$ with scale handling.
getMatmulInstructionCounter	size_t f()	Return MACs counter.
findAbsMaxFloat	float f(uint8_t* data, size_t N)	Max $ x[i] $ over N FLOAT32 elements.
findMaxFloat / findMinFloat	float f(data, N)	Max/min over N FLOAT32 elements.
findMaxInt32 / findMinInt32	int32_t f(data, N)	Max/min over N INT32 elements.
sumint32 / sumFloat	int32_t/float f(values, N)	Sum of N values.
logFloat	float f(float x)	Natural log. Used in cross-entropy loss.
roundByMode	int32_t f(float x, roundingMode_t)	Round float to int32 with HALF_AWAY or SR_HALF_AWAY.
clamp	float f(float x, float min, float max)	Clip x to [min, max].
gteFloatValue	void f(in, float thresh, float belowVal, out)	Output = input where input $\geq$ thresh, else belowVal.

gteInt32Value	void f(in, int32_t thresh, int32_t belowVal, out)	INT32 threshold gate.
gteSymInt32Zero	void f(in, out)	SYM_INT32 ReLU: zero mantissas below 0, copy scale.

Section 14.8 — Layer Module

Function	Signature	Purpose
initLayer	void f(layer_t*, layerType_t, layerConfig_t*)	Set layer->type and layer->config.
linearInitConfig	void f(linearConfig_t*, weights, bias, forwardQ, ...)	Fill all linearConfig_t fields.
linearForward / linearForwardFloat / linearForwardSymInt32	void f(layer, in, out)	Forward pass: transpose W, matmul, transpose W back.
linearBackward / backwardFloat / backwardSymInt32	void f(layer, fwdIn, loss, propLoss)	Backward pass: compute weight grads, bias grads, propagated loss.
linearCalcWeightGradsFloat32 / SymInt32	void f(loss, fwdIn, weightGrads)	dL/dW = loss^T @ forwardInput. Accumulates (+=) into weightGrads.
linearCalcBiasGradsFloat32 / SymInt32	void f(biasGrads, loss)	dL/db = sum(loss, axis=batch). Accumulates.
linearCalcPropLossFloat32 / SymInt32	void f(weights, loss, propLoss)	dL/dX = loss @ W (propagated gradient for layer below).
linearCalcOutputShape	void f(layer, inShape, outShape)	outShape[0]=inShape[0]=batch; outShape[1]=weights.dims[0]=out_features.
reluInitConfig / reluInit / reluForward / reluBackward	...	ReLU config + dispatch forward/backward.
reluCalcOutputShape	void f(layer, in, out)	Output shape = input shape (ReLU is identity shape).
softmaxInitConfig / softmaxForward / softmaxBackward	...	Softmax layer forward (exp normalization) and backward.
conv1dForward / conv1dBackward / conv1dCalcOutputShape	...	1D convolution forward and backward.
dropoutForward / dropoutBackward	...	Dropout: random mask during training, identity during eval.
flattenForward / flattenBackward / flattenCalcOutputShape	...	Flatten: reshape metadata without copying data.
initKernel / initKernelExplicit	void f(kernel_t*, size, padding, stride, dilation)	Set up convolution kernel parameters.

Section 14.9 — Training Infrastructure

Function	Signature	Purpose
defaultLossConfig	lossConfig_t f(lossFuncType_t)	Returns {funcType, REDUCTION_MEAN, NULL}. Quick setup.
lossFunctions[MSE].forward	float f(out, label, reduction)	MSE loss: mean((out-label)^2).
lossFunctions[CE].forward	float f(out, label, reduction)	Cross-entropy: -sum(label * log(out)).
lossFunctions[CE].backward	void f(out, label, result)	CE+softmax grad: result = out - label.

lossFunctions[X].computeMeanScale	float f(totalSamples, out)	Scale factor for REDUCTION_MEAN gradient.
sgdInit	void f(sgd_t*, float lr, float mom, float wd)	Initialize SGD with learning rate, momentum, weight decay.
sgdStep	void f(sgd_t*, parameter_t*, quantization_t*)	Update: $w \leftarrow lr * grad$. Dispatches float or SYM_INT32 path.
sgdStepM	void f(sgd_t*, parameter_t*, quantization_t*, state)	SGD+Momentum: $v = m*v + g$; $w \leftarrow lr*v$.
sgdZeroGrad	void f(parameter_t*)	memset grad data to 0. Reset SYM_INT32 scale to 1.0.
initDataLoader	void f(dataLoader_t*, getSample, getSize, getBatch, batchSize, ...)	Set up DataLoader with callbacks and config.
optimizerFunctions[SGD].step / .zeroGrad	stepFn_t / zeroFn_t	Dispatch table for optimizer variants.

Section 14.10 — Support Modules

Function	Signature	Purpose
rngSetSeed	void f(uint32_t seed)	Set xorshift32 state. seed=0 → UINT32_MAX (state=0 is illegal).
rngGetSeed	uint32_t f(void)	Return current RNG state.
rngNextFloat	float f(void)	Uniform float in [0,1). Uses top 24 bits of xorshift32.
rngShuffleIndices	void f(size_t* indices, size_t n)	Fisher-Yates shuffle of indices[0..n-1].
bernoulliSampleTensor	void f(tensor_t* mask, float p)	Fill BOOL mask: each bit = 1 with prob p, 0 with prob 1-p.
serializeTensor	void f(tensor_t*, FILE*)	Write tensor (quantization + shape + data) to byte stream.
serializeParameter	void f(parameter_t*, FILE*)	Serialize both param and grad tensors.
serializeModel	void f(layer_t** model, size_t N, FILE*)	Serialize all N layers' parameters.
deserializeTensor	void f(tensor_t*, FILE*)	Read tensor from byte stream into pre-allocated tensor_t.
deserializeModel	void f(layer_t** model, size_t N, FILE*)	Restore all N layers from byte stream.

Section 14.11 — Quick Lookup: Data Type Sizes

	Size (ARM 32-bit)	Value Range	Used For
	1 byte	0 to 255	Tensor data buffer, byte arrays
	4 bytes	-2,147,483,648 to +2,147,483,647	INT32 and SYM_INT32 mantissas, indices
	8 bytes	-9.2e18 to +9.2e18	Matmul accumulator (avoids overflow)
	4 bytes	0 to 4,294,967,295	RNG state, bitmasks
	4 bytes	~1.2e-38 to ~3.4e38 (7 sig. digits)	FLOAT32 tensors, scales, learning rate
	4 bytes (32-bit)	0 to 4,294,967,295	Sizes, indices, counts
	1 byte (implementation)	0 or 1	Boolean flags, BOOL tensor elements (in C)
	4 bytes (32-bit ARM)	0x00000000 to 0xFFFFFFFF	tensor_t*, function pointers, all references

Section 14.12 — Quick Lookup: All Struct Fields

tensor_t — The central data structure:

Field	Type	Purpose
data	uint8_t*	Raw bytes. Size = calcBytesPerTensor(tensor).

shape	shape_t*	Dimensions and axis permutation.
quantization	quantization_t*	Storage format (type + config).
sparsity	sparsity_t*	Sparse pattern (currently a stub, NULL for dense).

shape_t — Dimension descriptor:

Field	Type	Purpose
numberOfDimensions	size_t	Tensor rank (number of axes).
dimensions	size_t*	Physical size of each dimension slot.
orderOfDimensions	size_t*	Permutation mapping logical → physical axes.

quantization_t — Quantization descriptor:

Field	Type	Purpose
type	qtype_t	INT32, FLOAT32, SYM_INT32, SYM, ASYM, or BOOL.
qConfig	void*	Pointer to type-specific config struct. NULL for INT32/FLOAT32.

symInt32QConfig_t — SYM_INT32 parameters:

Field	Type	Purpose
scale	float	real = mantissa × scale. Updated by requantSymInt32Tensor.
qMaxBits	uint8_t	Mantissa range: $[-2^{(qMaxBits-1)}+1, +2^{(qMaxBits-1)}-1]$.
roundingMode	uint8_t	HALF_AWAY or SR_HALF_AWAY.

Chapter 19

Debugging, Testing, and Extending ODT

How to find bugs, validate correctness, and add new layers or features

19.1 Debugging Embedded C — General Strategy

Debugging embedded code is harder than debugging desktop code because you cannot easily inspect memory or stop the program without special hardware. The main tools are: UART printing, LED blinking, hardware debugger (SWD/JTAG), and the ODT-specific PRINT_DEBUG/PRINT_ERROR macros.

19.1.1 PRINT_DEBUG and PRINT_ERROR

ODT's `Common.h` defines conditional print macros. When `#define DEBUG` is set in the build, they expand to `printf`. In release builds (no `DEBUG`), they expand to nothing — zero overhead.

Common.h — debug macros

```
#ifndef DEBUG
#define PRINT_DEBUG(fmt, ...) printf('[DEBUG] ' fmt, ##__VA_ARGS__)
#define PRINT_ERROR(fmt, ...) printf('[ERROR] ' fmt, ##__VA_ARGS__)
#else
#define PRINT_DEBUG(fmt, ...) do { } while(0)
#define PRINT_ERROR(fmt, ...) do { } while(0)
#endif
// Usage in code:
PRINT_DEBUG('matmul: A[%zu,%zu] B[%zu,%zu]\n',
A->shape.dimensions[0], A->shape.dimensions[1],
B->shape.dimensions[0], B->shape.dimensions[1]);
```

**#ifd
ef D
EB
UG**

The compiler checks if `DEBUG` was defined. If using GCC: 'gcc -DDEBUG file.c' defines it. In STM32CubeIDE: add `DEBUG` to Project Properties → C/C++ Build → Settings → Preprocessor → Defined symbols.

**##_
_VA
_AR
GS_
—**

The `##` before `__VA_ARGS__` suppresses the trailing comma if no variadic arguments are passed — allows `PRINT_DEBUG('message')` without `','`.

19.1.2 Adding Debug Prints to Trace a Bug

When a layer produces wrong output, add debug prints to trace the data flow:

Debug tracing example

```
// Add to linearForwardFloat to trace the matmul inputs:
void linearForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {
    linearConfig_t *cfg = layer->config.linear;
    PRINT_DEBUG('Linear forward: input[%zu,%zu] weights[%zu,%zu]\n',
input->shape.dimensions[0], input->shape.dimensions[1],
cfg->weights.param.shape.dimensions[0],
cfg->weights.param.shape.dimensions[1]);
    float *inPtr = (float*) input->data.dataPTR;
    PRINT_DEBUG(' input[0] = %.4f, input[1] = %.4f\n',
inPtr[0], inPtr[1]);
    // ... rest of function
    float *outPtr = (float*) output->data.dataPTR;
    PRINT_DEBUG(' output[0] = %.4f\n', outPtr[0]);
}
```

Common causes of wrong output: wrong shape in `setShape` (e.g., rows and cols swapped), data pointer pointing to wrong buffer, scale not initialized (stays 0 → all values 0), or `transposeTensor` called odd number of times.

19.2 The TRACK_INSTRUCTIONS Macro

ODT includes a special debug mode for counting arithmetic operations. When `#define TRACK_INSTRUCTIONS` is active, the code counts multiplications and additions per forward pass.

Arithmetic.c — TRACK_INSTRUCTIONS usage

```
#ifndef TRACK_INSTRUCTIONS
static size_t g_numMul = 0;
static size_t g_numAdd = 0;
#endif
// In matmul inner loop:
for (size_t k = 0; k < K; k++) {
    acc += (int64_t)A_val * B_val;
    #ifdef TRACK_INSTRUCTIONS
    g_numMul++;
    g_numAdd++;
    #endif
}
// After forward pass:
#ifdef TRACK_INSTRUCTIONS
PRINT_DEBUG('Operations: %zu mul, %zu add\n', g_numMul, g_numAdd);
#endif
```

This is used to estimate how many sparse operations RigL would save (if mask zeros are skipped, that many multiplications are avoided).

19.3 Hardware Debugger — SWD and GDB

The Nucleo board includes an ST-Link debugger. It connects to the STM32's SWD (Serial Wire Debug) pins and lets you use GDB to inspect the MCU's state while it is running.

GDB commands for embedded debugging

```
# Connect to target (in STM32CubeIDE or arm-none-eabi-gdb):
(gdb) target remote localhost:3333 # openocd bridge
(gdb) monitor reset halt
# Set breakpoints:
(gdb) break linearForwardFloat
(gdb) break Matmul.c:45
# Inspect variables:
(gdb) print input->shape.dimensions[0]
(gdb) print *(float*)input->data.dataPTR @ 8 # print 8 floats
# Step through code:
(gdb) next # step over (don't enter functions)
(gdb) step # step into (enter function calls)
(gdb) continue # run until next breakpoint
# Inspect memory:
(gdb) x/32f w1Data # print 32 floats starting at w1Data
(gdb) x/8xb &cfg # print 8 bytes in hex at cfg address
```

19.4 Testing ODT Code — Unit Tests on Desktop

A key advantage of ODT's design: the C code is standard C99 with no MCU-specific headers in the core library. This means you can compile and test it on a desktop Linux/Mac/Windows machine using gcc.

19.4.1 Writing a Unit Test for matmul

test_matmul.c — unit test

```
#include <stdio.h>
#include <assert.h>
#include <math.h>
#include 'Tensor.h'
#include 'Matmul.h'

void test_matmulFloat_2x3_times_3x2(void) {
    // A = [[1,2,3],[4,5,6]] shape [2,3]
    // B = [[7,8],[9,10],[11,12]] shape [3,2]
    // Expected C = A*B = [[58,64],[139,154]] shape [2,2]
    float aData[] = {1,2,3, 4,5,6};
    float bData[] = {7,8, 9,10, 11,12};
    float cData[4] = {0};
    tensor_t A, B, C;
    setShape(&A.shape, 2, (size_t[]){2,3});
    setShape(&B.shape, 2, (size_t[]){3,2});
    setShape(&C.shape, 2, (size_t[]){2,2});
    A.data.dataPTR = aData; A.data.numberOfElements = 6;
    B.data.dataPTR = bData; B.data.numberOfElements = 6;
    C.data.dataPTR = cData; C.data.numberOfElements = 4;
    matmulFloat32TensorsWithBias(&A, &B, NULL, &C);
    float *result = (float*) C.data.dataPTR;
    assert(fabsf(result[0] - 58.0f) < 0.001f);
    assert(fabsf(result[1] - 64.0f) < 0.001f);
    assert(fabsf(result[2] - 139.0f) < 0.001f);
    assert(fabsf(result[3] - 154.0f) < 0.001f);
    printf('test_matmulFloat_2x3_times_3x2: PASS\n');
}

int main(void) {
    test_matmulFloat_2x3_times_3x2();
    return 0;
}
```

Compiling and running on desktop

```
# Compile:
gcc -o test_matmul test_matmul.c Matmul.c Tensor.c Arithmetic.c DTypes.c -lm
# Run:
./test_matmul
# Output: test_matmulFloat_2x3_times_3x2: PASS
```

Testing on desktop first is much faster than flashing to the MCU for every change. Only deploy to the MCU when the logic is proven correct on desktop.

19.4.2 Comparing MCU Output to Python

A powerful validation technique: run the same computation in Python (with NumPy) and compare results with the MCU output.

Python reference computation (run on PC)

```
import numpy as np
# Same weights as MCU (loaded from serialized output)
W1 = np.array([[0.2, 0.5, -0.3, 0.1],
               [-0.4, 0.1, 0.6, -0.2]], dtype=np.float32)
b1 = np.array([0.1, -0.1], dtype=np.float32)
x = np.array([0.5, -0.3, 0.8, 0.1], dtype=np.float32)
# Forward pass in NumPy
h1_py = x @ W1.T + b1
h2_py = np.maximum(0, h1_py) # ReLU
print('h1:', h1_py)
print('h2:', h2_py)
# Compare with MCU UART output
# If they match (within float32 precision), implementation is correct
```

x @
W1.
T

NumPy matrix multiply with transpose. Equivalent to ODT's transpose-matmul-untranspose sequence. Should give identical float32 results.

19.5 Adding a New Layer to ODT

ODT's dispatch table design makes it easy to add new layer types. Here is the complete procedure for adding a new layer (e.g., Tanh activation).

Step 1 — Add to layerType_t enum (Layer.h)

```
typedef enum {
    LAYER_LINEAR = 0,
    LAYER_RELU,
    LAYER_SOFTMAX,
    LAYER_CONV1D,
    LAYER_DROPOUT,
    LAYER_FLATTEN,
    LAYER_TANH, // ← ADD HERE
    LAYER_COUNT
} layerType_t;
```

Step 2 — Add config struct and union field (Layer.h)

```
typedef struct {
    tensor_t *inputCache; // for backward pass
} tanhConfig_t;
// In layerConfig_t union, add:
tanhConfig_t *tanh;
```

Step 3 — Create Tanh.c and Tanh.h

```
// Tanh.h
void tanhForward(layer_t *layer, tensor_t *input, tensor_t *output);
void tanhBackward(layer_t *layer, tensor_t *dOutput, tensor_t *dInput);
void tanhCalcOutputShape(layer_t *layer, shape_t *in, shape_t *out);
// Tanh.c
#include <math.h>
void tanhForward(layer_t *layer, tensor_t *input, tensor_t *output) {
    tanhConfig_t *cfg = layer->config.tanh;
    cfg->inputCache = input;
    size_t n = calcNumberOfElementsByShape(&input->shape);
    float *in = (float*) input->data.dataPTR;
    float *out = (float*) output->data.dataPTR;
    for (size_t i = 0; i < n; i++)
        out[i] = tanhf(in[i]); // tanhf = float version of tanh
}
void tanhBackward(layer_t *layer, tensor_t *dOutput, tensor_t *dInput) {
    tanhConfig_t *cfg = layer->config.tanh;
    size_t n = calcNumberOfElementsByShape(&dOutput->shape);
    float *dOut = (float*) dOutput->data.dataPTR;
    float *dIn = (float*) dInput->data.dataPTR;
    float *cachedIn = (float*) cfg->inputCache->data.dataPTR;
    for (size_t i = 0; i < n; i++) {
        float t = tanhf(cachedIn[i]);
        dIn[i] = dOut[i] * (1.0f - t * t); // d/dx tanh(x) = 1 - tanh(x)^2
    }
}
void tanhCalcOutputShape(layer_t *l, shape_t *in, shape_t *out) {
    *out = *in; // tanh doesn't change shape
}
```

Step 4 — Register in layerFunctions[] dispatch table (Layer.c)

```
layerFunctions_t layerFunctions[] = {
    [LAYER_LINEAR] = { linearForward, linearBackward, linearCalcOutputShape },
    [LAYER_RELU] = { reluForward, reluBackward, reluCalcOutputShape },
    [LAYER_SOFTMAX] = { softmaxForward, softmaxBackward, softmaxCalcOutputShape },
    [LAYER_TANH] = { tanhForward, tanhBackward, tanhCalcOutputShape }, // NEW
    // ...
};
```

After these 4 steps, you can use `LAYER_TANH` anywhere in the training loop exactly like `LAYER_RELU` — the dispatch table handles the routing.

19.6 Adding a New Optimizer

ODT currently only implements SGD. Adding Adam (Adaptive Moment Estimation) is a common next step. Adam requires storing two additional state vectors per parameter (first and second moment estimates).

Adam optimizer struct

```
typedef struct {
    float lr; // learning rate (default 0.001)
    float beta1; // first moment decay (default 0.9)
    float beta2; // second moment decay (default 0.999)
    float epsilon; // numerical stability (default 1e-8)
    size_t t; // step counter (for bias correction)
    tensor_t m; // first moment (mean of gradients)
    tensor_t v; // second moment (mean of squared gradients)
} adam_t;

// Adam update step:
void adamStep(adam_t *adam, parameter_t *param) {
    adam->t++;
    float *w = (float*) param->param.data.dataPTR;
    float *g = (float*) param->grad.data.dataPTR;
    float *m = (float*) adam->m.data.dataPTR;
    float *v = (float*) adam->v.data.dataPTR;
    size_t n = param->param.data.numberOfElements;
    // Bias correction factors
    float bc1 = 1.0f - powf(adam->beta1, adam->t);
    float bc2 = 1.0f - powf(adam->beta2, adam->t);
    for (size_t i = 0; i < n; i++) {
        // Update biased first moment
        m[i] = adam->beta1 * m[i] + (1.0f - adam->beta1) * g[i];
        // Update biased second moment
        v[i] = adam->beta2 * v[i] + (1.0f - adam->beta2) * g[i] * g[i];
        // Bias-corrected moments
        float m_hat = m[i] / bc1;
        float v_hat = v[i] / bc2;
        // Update weight
        w[i] -= adam->lr * m_hat / (sqrtf(v_hat) + adam->epsilon);
    }
}
```

■ NOTE

Adam Memory Cost: Adam requires 2 extra float tensors per parameter tensor (m and v). For a 64×128 weight matrix: $2 \times 64 \times 128 \times 4 \text{ bytes} = 65,536 \text{ extra bytes}$. This is significant on the STM32's 320 KB RAM. SGD needs no extra state.

19.7 Adding a New Loss Function

The `lossFunctions[]` dispatch table works the same way as `layerFunctions[]`. Here's how to add Mean Absolute Error (MAE) loss:

MAE loss implementation

```
// MAE.c
float maeForward(lossFunc_t *loss, tensor_t *pred, tensor_t *target) {
    size_t n = pred->data.numberElements;
    float *p = (float*) pred->data.dataPTR;
    float *t = (float*) target->data.dataPTR;
    float sum = 0.0f;
    for (size_t i = 0; i < n; i++)
        sum += fabsf(p[i] - t[i]);
    return sum / n;
}

void maeBackward(lossFunc_t *loss, tensor_t *pred, tensor_t *target,
    tensor_t *dPred) {
    size_t n = pred->data.numberElements;
    float *p = (float*) pred->data.dataPTR;
    float *t = (float*) target->data.dataPTR;
    float *dp = (float*) dPred->data.dataPTR;
    for (size_t i = 0; i < n; i++) {
        // d/dp |p-t| = sign(p-t)
        dp[i] = (p[i] > t[i]) ? 1.0f/n : -1.0f/n;
    }
}
```

19.8 Performance Profiling on the STM32

To measure exactly how long each part of the training loop takes, use the DWT (Data Watchpoint and Trace) cycle counter built into Cortex-M7.

Cycle counter profiling

```
// Enable DWT cycle counter (one-time setup in main.c):
CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
DWT->CYCCNT = 0;
DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
// Measure a function:
#define CYCLES_START() (DWT->CYCCNT)
#define CYCLES_END(start) (DWT->CYCCNT - (start))
uint32_t t0 = CYCLES_START();
matmulFloat32TensorsWithBias(&W, &x, NULL, &output);
uint32_t cycles = CYCLES_END(t0);
// At 216 MHz: 1 cycle = 4.63 ns
float time_us = (float)cycles / 216.0f; // microseconds
printf('matmul: %u cycles (%.1f us)\n', cycles, time_us);
```

—■ WORKED EXAMPLE

Measured performance on STM32F746ZG (estimated): Operation | Cycles | Time (216 MHz) Linear forward [64→32], f32 | ~18,000 | ~83 μs ReLU forward [32], f32 | ~200 | ~1 μs Softmax forward [10 classes] | ~300 | ~1.4 μs Cross-entropy backward | ~80 | ~0.37 μs SGD step [64×32 weights] | ~2,200 | ~10 μs Full training step (1 sample) | ~42,000 | ~194 μs Throughput: ~5,000 training samples/second

19.9 Checklist: Before Flashing to MCU

Check	How to Verify
All shapes correct	Add PRINT_DEBUG for shape in each layer init
Scale initialized	Check qConfig->scale != 0.0f before first use
sgdZeroGrad called	Add assert(gradBuffer[0] == 0) after sgdZeroGrad
transposeTensor pairs	Count calls: must be even for each tensor
No VLA larger than ~4KB	Calculate: sizeof(type) * maxDim1 * maxDim2
Stack depth estimated	Add up all local variables in the call chain
Loss decreasing	Print loss every 10 batches — should trend down
Gradients not NaN/inf	Print grad[0] every batch; NaN means overflow
Weights updated	Print W[0] before/after sgdStep — should differ

■ RELATIONS WITH OTHER FILES

Chapter 19 connects to: Chapter 2 (Common.h) — PRINT_DEBUG, PRINT_ERROR, TRACK_INSTRUCTIONS macros · Chapter 7 (Layers) — Adding a new layer follows the dispatch table pattern · Chapter 8 (Training) — Adding new optimizers and loss functions · Chapter 13 (C language) — assert, volatile, test compilation on desktop · Chapter 18 (Embedded) — DWT cycle counter, SWD debugger, UART printf

Chapter 20

SGD, DataLoader, and the Training Infrastructure

How the optimizer, data pipeline, and training loop work together

Section 20.1 — SGD Optimizer: Inside sgdStep

The Stochastic Gradient Descent optimizer is the simplest but most fundamental optimizer in deep learning. ODT implements it in Sgd.h and Sgd.c. Understanding every line of sgdStep reveals what 'training' really means at the lowest level.

Sgd.h — sgd_t struct

```
// sgd_t: the optimizer's persistent state
typedef struct {
    float learningRate; //  $\alpha$  — how large each update step is
    float momentumFactor; //  $\beta$  — fraction of velocity to carry forward (0 = no momentum)
    float weightDecay; //  $\lambda$  — L2 regularization strength (0 = no decay)
} sgd_t;
```

Three fields, three different training behaviours. The learning rate (α) controls the size of each gradient step. Momentum (β) smooths updates by combining the current gradient with a running average of past gradients — like a ball rolling downhill, it builds speed. Weight decay (λ) adds a penalty proportional to the weight magnitude, preventing any single weight from becoming too large (L2 regularisation).

Sgd.c — sgdStepFloat line-by-line

```
// Sgd.c — sgdStepFloat (annotated)
void sgdStepFloat(sgd_t *sgd, parameter_t *param) {
    size_t n = param->param.data.numberOfElements;
    float *w = (float*) param->param.data.dataPTR; // weight array
    float *g = (float*) param->grad.data.dataPTR; // gradient array
    for (size_t i = 0; i < n; i++) {
        // Core SGD update: w = w - lr * grad
        w[i] -= sgd->learningRate * g[i];
        // Optional weight decay: w = w - lr * lambda * w
        if (sgd->weightDecay != 0.0f)
            w[i] -= sgd->learningRate * sgd->weightDecay * w[i];
    }
}
```

n	param.data.numberOfElements	Total number of elements in this parameter tensor. For a [32,64] weight matrix: n=2048.
w	(float*) param.param.data.dataPTR	Cast the void* buffer pointer to float* so we can index it as an array. This is the actual weight values that get updated.
g	(float*) param.grad.data.dataPTR	Cast the gradient buffer. These were computed and accumulated during backward pass.
w[i] -= lr * g[i]	w[i] -= sgd->learningRate * g[i];	THE CORE OF TRAINING. Subtract (learning rate × gradient) from each weight. If the gradient is positive (increasing weight → increasing loss), we subtract to decrease the weight, which should decrease the loss.
weight Decay	w[i] -= lr * weightDecay * w[i];	If non-zero, also subtract $lr \times \lambda \times w[i]$. This is equivalent to adding a term $\lambda \times w ^2$ to the loss function, which penalises large weights and encourages simpler (smaller-weight) models — a form of regularisation.

Notice what sgdStepFloat does NOT do: it does not zero the gradients. That is the job of sgdZeroGrad, which must be called at the start of every batch.

Sgd.c — sgdZeroGrad: why scale must be reset

```
// Sgd.c — sgdZeroGrad
void sgdZeroGrad(sgd_t *sgd, linearConfig_t *cfg) {
    // Zero the weight gradient buffer
    size_t wBytes = cfg->weights.grad.data.numberOfElements * sizeof(float);
    memset(cfg->weights.grad.data.dataPTR, 0, wBytes);
    // Reset scale to 1.0 (important for SYM_INT32 — memset alone sets scale to 0)
    if (cfg->weights.grad.quantization.type == SYM_INT32) {
        symInt32QConfig_t *qcfg = cfg->weights.grad.quantization.qConfig;
        qcfg->scale = 1.0f;
    }
    // Zero the bias gradient buffer if present
    if (cfg->hasBias) {
        size_t bBytes = cfg->bias.grad.data.numberOfElements * sizeof(float);
        memset(cfg->bias.grad.data.dataPTR, 0, bBytes);
        if (cfg->bias.grad.quantization.type == SYM_INT32) {
            symInt32QConfig_t *qcfg = cfg->bias.grad.quantization.qConfig;
            qcfg->scale = 1.0f;
        }
    }
}
```

■■ IMPORTANT

The scale=1.0 reset after memset: memset(ptr, 0, n) sets all bytes to zero. For a float, 0x00000000 = 0.0f — correct. But for SYM_INT32, the scale field is also a float stored in the qConfig struct. If the qConfig struct is inside the zeroed region, scale becomes 0.0f. Then the first dequantize call divides by scale = 0.0f → infinity → NaN → disaster. Always reset scale to 1.0f after zeroing quantized tensors.

Section 20.2 — sgdStepSymInt32: Training with Quantized Weights

When weights are stored as SYM_INT32 (fixed-point quantised), the SGD step must convert to float, update, and re-quantise. This is the core of FQT (Fixed-point Quantisation Training) — training happens in float, but weights are stored and used for forward/backward passes in integer format.

Sgd.c — sgdStepSymInt32: dequantise → update → requantise

```
// Sgd.c — sgdStepSymInt32 (simplified)
void sgdStepSymInt32(sgd_t *sgd, parameter_t *param) {
    size_t n = param->param.data.numberOfElements;
    int32_t *w = (int32_t*) param->param.data.dataPTR;
    float *g = (float*) param->grad.data.dataPTR; // grad always float
    symInt32QConfig_t *qcfg = param->param.quantization.qConfig;
    float scale = qcfg->scale;
    int32_t qMax = (1 << (qcfg->qMaxBits - 1)) - 1;
    // VLA: temporary float buffer on the stack
    float wFloat[n]; // e.g. 2048 floats = 8 KB if n=2048
    // Step 1: dequantise all weights to float
    for (size_t i = 0; i < n; i++)
        wFloat[i] = (float)w[i] * scale;
    // Step 2: SGD update in float
    for (size_t i = 0; i < n; i++)
        wFloat[i] -= sgd->learningRate * g[i];
    // Step 3: re-quantise back to int32
    // Find new abs-max for optimal scale
    float absMax = findAbsMaxFloat(wFloat, n);
    float newScale = (absMax > 0) ? absMax / (float)qMax : scale;
    qcfg->scale = newScale;
    for (size_t i = 0; i < n; i++) {
        int32_t q = roundByMode(wFloat[i] / newScale, qcfg->roundingMode);
        w[i] = clamp(q, -qMax, qMax);
    }
}
```

■ C CONCEPT: VLA — Variable Length Array

float wFloat[n]; declares a VLA. The size n is only known at runtime. VLAs are allocated on the STACK — the compiler adjusts the stack pointer by n*4 bytes. For n=2048: 8KB of stack. On the STM32 with 320KB RAM, the stack might only be 8-16KB total. This is why large tensors MUST use static buffers instead of VLAs. For small layers (n ≤ 256), VLAs are safe.

— ■ WORKED EXAMPLE

SYM_INT32 SGD step trace (scale=0.01, qMaxBits=8, qMax=127, lr=0.001) w[0] = 73 (real value = $73 \times 0.01 = 0.73$) g[0] = 2.5 (gradient) Step 1 (dequantise): wFloat[0] = $73 \times 0.01 = 0.730$ Step 2 (update): wFloat[0] = $0.730 - 0.001 \times 2.5 = 0.7275$ Step 3 (requantise): absMax found = 0.9 → newScale = $0.9/127 = 0.00709$ q = round($0.7275 / 0.00709$) = round(102.6) = 103 w[0] = clamp(103, -127, 127) = 103 Real value stored: $103 \times 0.00709 = 0.730$ (very close to 0.7275 — tiny quantisation error)

Section 20.3 — DataLoader: Every Field Explained

The DataLoader is the bridge between the raw dataset and the training loop. It handles batching, shuffling, and calling user-provided data retrieval functions. ODT's DataLoader uses function pointers so that any data source (SD card, UART, hardcoded array, etc.) can be plugged in without modifying the training loop.

DataLoader.h — complete dataLoader_t struct

[illegible]

get Sample	<code>void (*)(dataLoader_t*, size_t idx, tensor_t* out, tensor_t* target)</code>	Fetches a single sample at position <code>idx</code> and writes it into <code>out</code> and <code>target</code> . The user implements this — it might read from a hardcoded array, an SD card, or a ring buffer filled by a UART interrupt.
get Dataset Size	<code>size_t (*)(dataLoader_t*)</code>	Returns the total count of samples. Used to compute the number of batches per epoch: <code>numBatches = getDatasetSize() / batchSize</code> .
get Batch	<code>void (*)(dataLoader_t*, size_t batchIdx, tensor_t* out, tensor_t* target)</code>	Higher-level: fills a full batch. The default implementation calls <code>getSample</code> <code>batchSize</code> times, stacking results into the batch tensor.
batchSize	<code>size_t</code>	How many samples per gradient update. Larger batches = stabler gradients but more memory and slower per-update. On the STM32 with limited RAM, small batches (1-8) are typical.
dropLast	<code>bool</code>	If the dataset size is not divisible by <code>batchSize</code> , the last batch is incomplete. <code>dropLast=true</code> discards it; <code>dropLast=false</code> uses it (smaller gradients for last batch).
transform	<code>void (*)(tensor_t*)</code>	Optional: applied to each input after <code>getSample</code> . E.g., normalise audio features to zero mean. If <code>NULL</code> , no transform is applied.
shuffle	<code>bool</code>	If true, the indices array is shuffled using <code>rngShuffleIndices</code> at the start of each epoch. This randomises the order in which samples are seen, preventing the network from memorising the presentation order.
indices	<code>size_t*</code>	Array of size <code>getDatasetSize()</code> , holding sample indices. When <code>shuffle=true</code> , Fisher-Yates permutes this array. <code>getSample</code> then accesses <code>dataset[indices[i]]</code> instead of <code>dataset[i]</code> . Caller must allocate and pass this buffer.

Complete DataLoader setup example

```
// Typical DataLoader initialisation for a small hardcoded dataset
// User-defined dataset
#define NUM_SAMPLES 120
static float audioFeatures[NUM_SAMPLES][64]; // 120 samples, 64 MFCC features
static int speakerLabels[NUM_SAMPLES]; // 0, 1, or 2 for 3 speakers
static size_t shuffleIndices[NUM_SAMPLES];
// Callback: get one sample
void myGetSample(dataLoader_t *dl, size_t idx, tensor_t *x, tensor_t *y) {
    float *xPtr = (float*) x->data.dataPTR;
    float *yPtr = (float*) y->data.dataPTR;
    // Copy features
    memcpy(xPtr, audioFeatures[idx], 64 * sizeof(float));
    // One-hot encode label
    memset(yPtr, 0, 3 * sizeof(float));
    yPtr[speakerLabels[idx]] = 1.0f;
}
// Callback: dataset size
size_t myGetSize(dataLoader_t *dl) { return NUM_SAMPLES; }
// Initialise DataLoader
dataLoader_t loader;
initDataLoader(&loader, myGetSample, myGetSize, defaultGetBatch,
4, // batchSize = 4
true, // dropLast = true
NULL, NULL, // no transforms
true, // shuffle = true
42, // shuffleSeed
shuffleIndices);
```


Section 20.4 — Training Loop Analysis: Every Line Explained

The training loop ties everything together. Here is the complete loop with annotations explaining WHY each step is required and what happens if you skip it.

Complete annotated training loop

[illegible]

Step 1 — SGDZero Grad	Gradient buffers accumulate via +=. Without zeroing, gradients from all previous batches add up. After 1000 batches, the effective gradient would be 1000× too large, causing wildly incorrect weight updates. MUST be the very first step.
-----------------------	---

Step 2 — getBatch		Loads batchSize samples from the dataset into the input activation buffer. The target tensor is also filled (one-hot encoded labels). With shuffle=true, sample order is randomised each epoch.
Step 3 — Forward pass		Sequentially calls each layer's forward function: Linear→ReLU→Linear→Softmax (example). Each activation is stored in activations[l+1]. The final activation is the network output.
Step 4 — Compute loss		Compares the network output to the target. Returns a scalar loss value. For cross-entropy: $\text{loss} = -\log(\text{probability assigned to true class})$. Lower is better. Also computes the initial gradient (dL/d_output).
Step 5 — Backward pass		Propagates gradients in REVERSE layer order. Each layer receives dOutput (gradient flowing in) and computes: dInput (to pass upstream) and dWeights/dBias (accumulated into gradient buffers). The chain rule in action.
Step 6 — Update weights		For each LINEAR layer: $w := lr * \text{gradient}$. This is the actual learning step. Note: only LINEAR layers have learnable parameters. ReLU, Softmax, Flatten have none.

■ NOTE

Why the Loop Iterates Batches, Not Individual Samples A batch of 4 samples produces 4 sets of gradients. The backward pass accumulates them all (via +=) into the gradient buffers. The SGD step then updates weights based on the AVERAGE gradient over the batch. This is more numerically stable than updating after every single sample (1-sample batches = pure stochastic gradient descent, very noisy).

Section 20.5 — Loss Functions: Forward and Backward in Detail

ODT's loss function dispatch table works exactly like the layer dispatch table. A `lossFunctions_t` struct holds forward, backward, and `computeMeanScale` function pointers.

LossFunction.h — dispatch table

```
// LossFunction.h
typedef float (*lossFwdFn_t)(lossFunc_t*, tensor_t* pred, tensor_t* target);
typedef void (*lossBwdFn_t)(lossFunc_t*, tensor_t* pred, tensor_t* target,
tensor_t* dPred);
typedef struct {
    lossFwdFn_t forward;
    lossBwdFn_t backward;
    // Optional: scale gradient by batch mean (mean reduction)
    computeMeanScaleFn_t computeMeanScale;
} lossFunctions_t;
extern lossFunctions_t lossFunctions[]; // indexed by lossFuncType_t
// Registration:
lossFunctions_t lossFunctions[] = {
    [LOSS_MSE] = { mseForward, mseBackward, ... },
    [LOSS_CROSS_ENTROPY] = { crossEntropyFwdFloat, crossEntropySMBBackFloat, ... },
};
```

crossEntropyForwardFloat — annotated

```
// LossFunction.c – crossEntropyForwardFloat
float crossEntropyForwardFloat(lossFunc_t *loss,
tensor_t *pred, tensor_t *target) {
    size_t batch = pred->shape.dimensions[0];
    size_t classes = pred->shape.dimensions[1];
    float *p = (float*) pred->data.dataPTR;
    float *t = (float*) target->data.dataPTR;
    float totalLoss = 0.0f;
    for (size_t b = 0; b < batch; b++) {
        for (size_t c = 0; c < classes; c++) {
            float predVal = p[b * classes + c];
            float targVal = t[b * classes + c];
            if (targVal > 0.0f) {
                // Guard against log(0) = -infinity
                float safeP = (predVal < 1e-7f) ? 1e-7f : predVal;
                totalLoss -= targVal * logf(safeP);
            }
        }
    }
    return totalLoss / (float)batch; // mean over batch
}
```

targVal > 0.0f check	For one-hot targets, only one class is non-zero. This check skips the log computation for all zero-target classes (saves n-1 logf calls per sample).
predVal < 1e-7f guard	log(0) = -infinity. If the network is very confident and wrong (predicts 0 for the true class), we clamp to 1e-7 to prevent NaN loss. In practice, after softmax all outputs are > 0, but numerical precision can push them to ~0.

/ (float) batch	Divide by batch size to get mean loss. This makes the loss comparable across different batch sizes (a batch of 4 and a batch of 8 give similar loss magnitudes).
-----------------	--

crossEntropySoftmaxBackwardFloat — the 1-line gradient

```
// LossFunction.c – crossEntropySoftmaxBackwardFloat
// This implements the combined softmax + cross-entropy backward pass
// Math result: dL/d(logit_i) = softmax_output_i - target_i
void crossEntropySoftmaxBackwardFloat(
    lossFunc_t *loss, tensor_t *pred, tensor_t *target, tensor_t *dPred) {
    size_t batch = pred->shape.dimensions[0];
    size_t classes = pred->shape.dimensions[1];
    float *p = (float*) pred->data.dataPTR; // softmax outputs
    float *t = (float*) target->data.dataPTR; // one-hot targets
    float *dp = (float*) dPred->data.dataPTR; // output gradient buffer
    for (size_t i = 0; i < batch * classes; i++)
        dp[i] = (p[i] - t[i]) / (float)batch;
    // That's it. One line. The combined derivative is beautiful.
}
```

■ C CONCEPT: Why softmax+CE backward is so simple

Separately, softmax backward is complex (involves the Jacobian matrix). Cross-entropy backward is $-\text{target}/\text{pred}$. Combined, almost everything cancels: $d(\text{CE} + \text{Softmax})/d(\text{logit}_i) = \text{softmax}_i - \text{target}_i$. This is why ODT (and all major frameworks) fuse these two operations. The result: if the network predicted 0.7 for the true class ($\text{target}=1.0$), the gradient is $0.7 - 1.0 = -0.3 \rightarrow$ the logit should increase.

■ RELATIONS WITH OTHER FILES

Chapter 20 uses: Sgd.h/c (sgdStep, sgdZeroGrad) · DataLoader.h/c (initDataLoader, getBatch) · LossFunction.h/c (crossEntropyForward, crossEntropySoftmaxBackward) · Layer.h (layerFunctions dispatch) · Rng.h/c (rngShuffleIndices for shuffle) · Tensor.h (tensor_t for all data) · Chapter 17 worked example shows these functions called together.

Chapter 21

Full RigL C Implementation

Complete compilable code for rigl.h and rigl.c — every line explained

Section 21.1 — What We Are Building

■ WHAT THIS FILE DOES

This chapter presents a COMPLETE, COMPILABLE C implementation of RigL (Rigging the Lottery, Evci et al. NeurIPS 2020) for the ODT library. Every function is presented in full with line-by-line explanations. This is not pseudocode — this is the actual code you would add to the repository. Files to create: RigL.h (header), RigL.c (implementation). Files to modify: Tensor.h (sparsityConfig fields), Sgd.c (mask-aware step), Linear.c (masked forward), Matmul.c (masked multiply).

Function	File	Purpose
riglConfig_t (struct)	RigL.h	Configuration: mask tensor, sparsity ratio, update interval
riglInitMask()	RigL.c	Randomly initialize mask: set sparsityRatio fraction to pruned
riglComputeCountToPrune()	RigL.c	Helper: how many weights to prune/grow this step

<code>riglPruneStep()</code>	RigL.c	Deactivate weights with smallest $ w $
<code>riglGrowStep()</code>	RigL.c	Reactivate pruned weights with largest $ \text{gradient} $
<code>riglUpdate()</code>	RigL.c	Combined: check if it's time to update, then prune+grow
<code>riglMaskedMatmulFloat32()</code>	Matmul.c (modified)	Matmul that skips pruned weights
<code>riglMaskedSgdStepFloat()</code>	Sgd.c (modified)	SGD update that skips pruned weights
<code>riglGetSparsityRatio()</code>	RigL.c	Compute actual current sparsity ratio
<code>riglPrintStats()</code>	RigL.c	Debug: print how many weights active/pruned

Section 21.2 — RigL.h: The Complete Header File

[illegible]

1	<pre>#ifndef RIGL_H / #define RIGL_H / #endif</pre>	Include guard: prevents the header from being processed more than once if multiple .c files include RigL.h. The preprocessor checks: if RIGL_H is not yet defined, define it and process the file. On a second inclusion: RIGL_H is already defined, so the entire file is skipped.
2	<pre>#include <stdbool.h></pre>	Provides the 'bool', 'true', and 'false' keywords for C (prior to C99). In C, there is no built-in boolean type — stdbool.h defines bool as _Bool, true as 1, false as 0. tensorBoolGet returns bool, so we need this.
3	<pre>tensor_t *mask;</pre>	Pointer to a BOOL tensor with the same shape as the weight tensor. mask[i] = 1 (true) → weight i is ACTIVE (participates in computation). mask[i] = 0 (false) → weight i is PRUNED (forced to zero, skip in matmul).
4	<pre>float sparsityRatio;</pre>	Target fraction of pruned weights. sparsityRatio=0.8 → 80% of weights pruned, 20% active. The SAME sparsity ratio is maintained throughout training. Only the pattern changes (which 20% are active).
5	<pre>size_t updateStep;</pre>	How many batches between topology updates. Typical: 100. Every 100 batches, call riglUpdate() to prune+grow. Too frequent: topology oscillates before weights converge. Too rare: suboptimal topology persists too long.

6	<code>size_t batchCount;</code>	Internal counter incremented each batch. Do not set this manually. <code>riglUpdate()</code> uses <code>batchCount % updateStep</code> to know when to run the prune+grow cycle.
---	---------------------------------	--

Section 21.3 — riglInitMask(): Initialize the Sparsity Pattern

Before training, the mask must be initialized so that exactly `sparsityRatio` of the weights are pruned and the rest are active. We use a simple random approach: set all bits to 1 (all active), then randomly deactivate $\text{sparsityRatio} \times N$ of them.

```
// RigL.c – riglInitMask
#include "RigL.h"
#include "DTypes.h" // tensorBoolGet, tensorBoolSet
#include "Tensor.h" // calcNumberOfElementsByTensor
#include <string.h> // memset
#include <stdlib.h> // srand, rand
void riglInitMask(riglConfig_t *cfg, unsigned int seed) {
    tensor_t *mask = cfg->mask;
    size_t N = calcNumberOfElementsByTensor(mask);
    // Step 1: Activate ALL weights (set every mask bit to 1)
    size_t maskBytes = (N + 7) / 8; // ceil(N/8)
    memset(mask->data, 0xFF, maskBytes);
    // Handle non-multiple-of-8: clear the padding bits in the last byte
    if (N % 8 != 0) {
        size_t validBits = N % 8;
        uint8_t lastMask = (uint8_t)((1u << validBits) - 1);
        mask->data[maskBytes - 1] &= lastMask;
    }
    // Step 2: Randomly prune sparsityRatio fraction
    size_t numToPrune = (size_t)(cfg->sparsityRatio * (float)N);
    srand(seed);
    size_t pruned = 0;
    while (pruned < numToPrune) {
        // Pick a random weight index
        size_t idx = (size_t)((unsigned)rand() % (unsigned)N);
        // If it is still active, prune it
        if (tensorBoolGet(mask, idx)) {
            tensorBoolSet(mask, idx, false);
            pruned++;
        }
    }
}
```

1	<code>size_t N = calcNumberOfElementsByTensor(mask);</code>	Total weight count. For a linear layer [128, 64]: $N = 128 \times 64 = 8192$.
2	<code>size_t maskBytes = (N + 7) / 8;</code>	A BOOL tensor stores 1 bit per weight. N bits need $\text{ceil}(N/8)$ bytes. $(N + 7) / 8$ is integer division that rounds UP. For $N=8192$: $(8192+7)/8 = 8199/8 = 1024.875 \rightarrow$ integer division = 1024 bytes.
3	<code>memset(mask->data, 0xFF, maskBytes);</code>	<code>0xFF = 0b11111111</code> = all 8 bits set to 1. <code>memset</code> fills every byte with <code>0xFF</code> , setting all mask bits to 1 (all weights active). This is the starting point before we randomly prune.
4	<code>if (N % 8 != 0) { ... }</code>	If N is not a multiple of 8, the last byte has some unused bits. <code>memset</code> sets them all to 1, but they represent indices beyond N-1 (non-existent weights). We clear those padding bits to avoid <code>tensorBoolGet</code> returning true for invalid indices.
5	<code>uint8_t lastMask = (uint8_t)((1u << validBits) - 1);</code>	Create a bitmask with only the valid bits set. Example: $N=10$, so last byte has 2 valid bits (indices 8 and 9). $\text{validBits}=2$, $1u \ll 2=4$, $4-1=3=0b00000011$. ANDing with this keeps only bits 0 and 1, clearing bits 2-7.

6	<pre>size_t numToPrune = (size_t)(cfg->sparsityRatio * (float)N);</pre>	How many weights to prune. For sparsityRatio=0.8, N=8192: numToPrune = (size_t)(0.8 × 8192.0) = (size_t)(6553.6) = 6553. The cast truncates (not rounds), so actual sparsity may be slightly less than 80%.
7	<pre>size_t idx = (size_t)((unsigned)rand() % (unsigned)N);</pre>	Generate a random index in [0, N-1]. rand() returns int in [0, RAND_MAX]. % N gives the remainder, mapping to [0, N-1]. Casting to unsigned before % avoids signed-modulo issues on some platforms.
8	<pre>if (tensorBoolGet(mask, idx)) { tensorBoolSet(mask, idx, false); pruned++; }</pre>	Only prune if not already pruned. Without this check, the same index could be picked twice, and pruned++ would count an already-pruned weight. The while loop retries until it finds an unpruned weight.

■■ IMPORTANT

The random-retry approach is simple but slow if sparsityRatio is very high (>95%). At 95% sparsity, only 5% of random picks find an unset bit — many retries. A more efficient approach: build a shuffled index array (Fisher-Yates), then prune the first numToPrune entries. rngShuffleIndices from RNG.h does this.

Section 21.4 — riglPruneStep(): Remove Smallest Weights

The prune step deactivates the sparsityRatio fraction of currently active weights that have the smallest absolute value. Small-magnitude weights contribute little to the output, so removing them has minimal impact on accuracy.

■ C CONCEPT: Sorting by Absolute Value

To prune the 'bottom k%' of active weights by $|w|$, we need to sort them. C's standard library provides `qsort()` — the quicksort algorithm. `qsort(array, count, element_size, comparison_function)` The comparison function receives two `const void*` pointers to elements and must return: negative if $a < b$, 0 if $a == b$, positive if $a > b$. For our case: we sort an array of INDICES (not weights) by the $|weight|$ at each index. This lets us sort cheaply and then use the sorted indices to both prune and grow.

```
// RigL.c – sort helper for prune step
// Global pointer used by the qsort comparison function
// (qsort callback cannot receive extra arguments)
static float *g_sortWeights = NULL;
// Compare two indices by |weight| ascending (smallest first)
static int cmpAbsWeightAsc(const void *a, const void *b) {
    size_t ia = *(const size_t*)a;
    size_t ib = *(const size_t*)b;
    float fa = g_sortWeights[ia];
    float fb = g_sortWeights[ib];
    if (fa < 0.0f) fa = -fa; // |fa|
    if (fb < 0.0f) fb = -fb; // |fb|
    if (fa < fb) return -1;
    if (fa > fb) return 1;
    return 0;
}
// Compare two indices by |gradient| descending (largest first)
static float *g_sortGrads = NULL;
static int cmpAbsGradDesc(const void *a, const void *b) {
    size_t ia = *(const size_t*)a;
    size_t ib = *(const size_t*)b;
    float fa = g_sortGrads[ia];
    float fb = g_sortGrads[ib];
    if (fa < 0.0f) fa = -fa;
    if (fb < 0.0f) fb = -fb;
    if (fa > fb) return -1; // reversed: descending
    if (fa < fb) return 1;
    return 0;
}
```

1	<code>static float *g_sortWeights = NULL;</code>	A global (module-level) pointer to the weight array. 'static' limits its scope to <code>RigL.c</code> — it is not visible to other files. The <code>qsort</code> comparison function cannot receive extra arguments (it's a fixed signature), so we pass the weight array via this global pointer. This is NOT thread-safe, but the MCU runs single-threaded.
2	<code>size_t ia = *(const size_t*)a;</code>	Cast the <code>void*</code> pointer to <code>size_t*</code> and dereference to get the actual index. <code>qsort</code> passes <code>&array[i]</code> (address of element), not <code>array[i]</code> (value). We need to dereference (*) to get the index value.
3	<code>if (fa < 0.0f) fa = -fa;</code>	Manual absolute value. Equivalent to <code>fabsf(fa)</code> . We avoid including <code>math.h</code> for this single operation.
4	<code>if (fa < fb) return -1;</code>	<code>qsort</code> convention: return negative if <code>a</code> should come BEFORE <code>b</code> . For ascending sort: smaller absolute value comes first (return -1 when <code>fa < fb</code>).

5	<pre> if (fa > fb) return -1; (in cmpAbsGradDesc) </pre>	For DESCENDING sort: LARGER absolute gradient comes first. Reversed: return -1 (a before b) when fa > fb (a is larger).
<pre> // RigL.c – riglPruneStep // Static buffers: avoid stack overflow on MCU // Max supported weight count for one layer #define RIGL_MAX_WEIGHTS 16384 static size_t s_activeIdx[RIGL_MAX_WEIGHTS]; static float s_wFloat[RIGL_MAX_WEIGHTS]; void riglPruneStep(parameter_t *parameter, riglConfig_t *cfg) { tensor_t *w = getParamFromParameter(parameter); tensor_t *mask = cfg->mask; size_t N = calcNumberOfElementsByTensor(w); if (N > RIGL_MAX_WEIGHTS) { PRINT_ERROR("riglPruneStep: layer too large\n"); return; } // === Collect active indices and their float weights === size_t numActive = 0; for (size_t i = 0; i < N; i++) { s_wFloat[i] = readBytesAsFloat(w->data + i*4); if (tensorBoolGet(mask, i)) { s_activeIdx[numActive++] = i; } } // === Sort active indices by w ascending === g_sortWeights = s_wFloat; qsort(s_activeIdx, numActive, sizeof(size_t), cmpAbsWeightAsc); // === Prune the bottom sparsityRatio fraction === size_t numToPrune = (size_t)(cfg->sparsityRatio * (float)numActive); for (size_t i = 0; i < numToPrune; i++) { size_t idx = s_activeIdx[i]; tensorBoolSet(mask, idx, false); writeFloatToByteArray(0.0f, w->data + idx*4); } } </pre>		
1	<pre> static size_t s_activeIdx[RIGL_MAX_WEIGHTS]; static float s_wFloat[RIGL_MAX_WEIGHTS]; </pre>	Static arrays at file scope (NOT on the stack). On MCU, the stack is ~4-8 KB. A 16384-element size_t array would be 64 KB on the stack — fatal overflow. Static arrays live in .bss or .data section (RAM, but not the stack). They persist between calls; this is safe because riglPruneStep is called one layer at a time.
2	<pre> #define RIGL_MAX_WEIGHTS 16384 </pre>	Compile-time constant limiting layer size. For STM32 with 320 KB RAM, a 16384-weight layer uses 64 KB for weights alone (float32). If you need larger layers, increase this constant and allocate more static buffer.
3	<pre> if (N > RIGL_MAX_WEIGHTS) { PRINT_ERROR(...); return; } </pre>	Guard against buffer overflow. If the layer is larger than our static buffer, print an error and return without crashing. A silent buffer overflow would corrupt memory unpredictably.
4	<pre> for (size_t i = 0; i < N; i++) { s_wFloat[i] = readBytesAsFloat(...); ...} </pre>	Build s_wFloat: a float copy of all weights (for the sort comparison function). Simultaneously collect s_activeIdx: indices of all active (mask=1) weights.
5	<pre> g_sortWeights = s_wFloat; qsort(s_activeIdx, numActive, sizeof(size_t), cmpAbsWeightAsc); </pre>	Set the global pointer, then sort. qsort rearranges s_activeIdx so that s_activeIdx[0] holds the index of the smallest- w active weight, s_activeIdx[1] holds the second smallest, etc.
6	<pre> size_t numToPrune = (size_t)(cfg->sparsityRatio * (float)numActive); </pre>	How many active weights to prune this step. Note: we prune relative to CURRENTLY ACTIVE count (numActive), not total N. This maintains the target sparsity ratio precisely.

7	<code>tensorBoolSet(mask, idx, false);</code>	Clear the mask bit: this weight is now pruned.
8	<code>writeFloatToByteArray(0.0f, w->data + idx*4);</code>	Zero the weight value. Pruned weights must be 0 so the forward pass can skip them (or if the forward pass doesn't check the mask, multiplying by 0 contributes nothing to the output).

Section 21.5 — riglGrowStep(): Reactivate High-Gradient Weights

The grow step reactivates the same number of weights that were just pruned. It selects from currently-pruned weights, choosing those with the largest gradient magnitude. A large gradient at a pruned weight means: 'if this weight existed, it would receive a large update — it would be useful.'

```
// RigL.c — riglGrowStep
static size_t s_inactiveIdx[RIGL_MAX_WEIGHTS];
static float s_gFloat[RIGL_MAX_WEIGHTS];
void riglGrowStep(parameter_t *parameter, riglConfig_t *cfg) {
    tensor_t *g = getGradFromParameter(parameter);
    tensor_t *w = getParamFromParameter(parameter);
    tensor_t *mask = cfg->mask;
    size_t N = calcNumberOfElementsByTensor(g);
    // === Collect inactive indices and their gradient magnitudes ===
    size_t numInactive = 0;
    size_t numActive = 0;
    for (size_t i = 0; i < N; i++) {
        s_gFloat[i] = readBytesAsFloat(g->data + i*4);
        if (!tensorBoolGet(mask, i)) {
            s_inactiveIdx[numInactive++] = i;
        } else {
            numActive++;
        }
    }
    // === Sort inactive indices by |g| descending ===
    g_sortGrads = s_gFloat;
    qsort(s_inactiveIdx, numInactive, sizeof(size_t), cmpAbsGradDesc);
    // === Regrow the same number that was pruned ===
    // numToPrune was (sparsityRatio * numActive_before_prune)
    // After prune: numActive_now = numActive (already reduced by prune step)
    // We want to restore the count: grow back what was pruned
    size_t targetActive = (size_t)((1.0f - cfg->sparsityRatio) * (float)N);
    size_t numToGrow = (numActive < targetActive) ?
        (targetActive - numActive) : 0;
    for (size_t i = 0; i < numToGrow && i < numInactive; i++) {
        size_t idx = s_inactiveIdx[i];
        tensorBoolSet(mask, idx, true);
        // Weight starts at 0 — new gradients will grow it
        // DO NOT copy the old gradient into the weight
        // (the gradient tells us DIRECTION, not the VALUE the weight should have)
    }
}
```

1	tensor_t *g = getGradFromParameter(parameter);	We need the GRADIENT tensor, not the weight tensor. The grow step selects which pruned weights have the highest gradient magnitude. These gradients were computed during the backward pass — even for pruned weights (because linearCalcWeightGrads computes dL/dW for ALL positions, not just active ones).
2	if (!tensorBoolGet(mask, i)) { s_inactiveIdx[numInactive++] = i; }	Collect INACTIVE (pruned) indices. !tensorBoolGet returns true when the bit is 0 (pruned). We store these indices in s_inactiveIdx for sorting.
3	size_t targetActive = (size_t)((1.0f - cfg->sparsityRatio) * (float)N);	How many weights should be active after the grow step. For sparsityRatio=0.8, N=8192: targetActive = 0.2 × 8192 = 1638. We want exactly 20% of weights active after prune+grow.
4	size_t numToGrow = (numActive < targetActive) ? (targetActive - numActive) : 0;	How many to grow: the deficit between current active count and target. If prune reduced active from 1638 to 0 (extreme), we grow back 1638. If prune reduced by 500 (e.g., 1638→1138), we grow back 500. Ternary operator: condition ? value_if_true : value_if_false.

5	<code>tensorBoolSet(mask, idx, true);</code>	Set the mask bit to 1 (active). This weight will now participate in the forward pass, receive gradient updates in the backward pass, and be updated by SGD.
6	<code>// Weight starts at 0 – new gradients will grow it</code>	The weight was pruned (set to 0) in the prune step. We leave it at 0 after reactivation. The very next forward+backward will compute a gradient for it, and SGD will update it to a non-zero value. This is correct: RigL paper initializes regrown weights at 0.

Section 21.6 — riglUpdate(): The Main Entry Point

```
// RigL.c – riglUpdate
// Call this every batch. It internally decides when to run prune+grow.
void riglUpdate(parameter_t *parameter, riglConfig_t *cfg) {
    // Increment batch counter
    cfg->batchCount++;
    // Only run prune+grow every updateStep batches
    if (cfg->batchCount % cfg->updateStep != 0) {
        return; // not time yet
    }
    PRINT_DEBUG("RigL update at batch %zu\n", cfg->batchCount);
    // Run prune step first, then grow step
    riglPruneStep(parameter, cfg);
    riglGrowStep(parameter, cfg);
    // Zero gradients: the stale gradients that triggered grow
    // should not be applied as weight updates
    sgdZeroGrad(parameter);
}
```

1	<code>cfg->batchCount++;</code>	Increment the counter every time <code>riglUpdate</code> is called (every batch). This counter persists across batches because <code>cfg</code> is a pointer to a struct that lives in the caller's scope.
2	<code>if (cfg->batchCount % cfg->updateStep != 0) return;</code>	<code>%</code> is the modulo operator: remainder after division. <code>batchCount % 100 == 0</code> when <code>batchCount</code> is 100, 200, 300, etc. So the <code>prune+grow</code> runs every 100 batches (or whatever <code>updateStep</code> is set to). On all other batches, return immediately.
3	<code>riglPruneStep(parameter, cfg);</code> <code>riglGrowStep(parameter, cfg);</code>	Always prune BEFORE growing. Pruning sets some mask bits to 0 and zeros their weights. Growing then sets different bits to 1 and leaves their weights at 0. The net effect: the active set shifts toward high-gradient weights.
4	<code>sgdZeroGrad(parameter);</code>	CRITICAL: zero gradients after the topology change. The gradients that the grow step used to select weights are now stale: they were computed for the OLD topology. If not zeroed, the next SGD step would apply these old gradients, potentially undoing the grow step (by updating the freshly-regrown weights with a gradient computed when they were inactive and had value 0).

Section 21.7 — Utility Functions: Statistics and Debug

```

// RigL.c – utility functions
// Compute fraction of weights currently active (1-sparsity)
float riglGetSparsityRatio(riglConfig_t *cfg, size_t totalWeights) {
    size_t activeCount = 0;
    for (size_t i = 0; i < totalWeights; i++) {
        if (tensorBoolGet(cfg->mask, i)) activeCount++;
    }
    // Return the PRUNED fraction (0.8 = 80% pruned = 20% active)
    return 1.0f - (float)activeCount / (float)totalWeights;
}
// Print statistics for debugging
void riglPrintStats(riglConfig_t *cfg, size_t totalWeights) {
    float actualSparsity = riglGetSparsityRatio(cfg, totalWeights);
    size_t activeCount = (size_t)((1.0f - actualSparsity) * totalWeights);
    PRINT_DEBUG(
        "RigL stats: batch=%zu, active=%zu/%zu, sparsity=%.1f%%\n",
        cfg->batchCount,
        activeCount, totalWeights,
        actualSparsity * 100.0f
    );
}

```

■ NOTE

riglGetSparsityRatio is O(N): it scans every bit in the mask. For a 16384-weight layer, this is 16384 bit reads — cheap (each is one bit operation). Call this once per epoch for monitoring; do not call inside the inner training loop.

Section 21.8 — riglMaskedSgdStepFloat(): Mask-Aware Weight Update

This function modifies the `sgdStepFloat` from `Sgd.c` to respect the sparsity mask. Active weights (`mask=1`) are updated normally. Pruned weights (`mask=0`) are skipped — their value remains 0.

```
// Sgd.c – riglMaskedSgdStepFloat
// (Add this function to Sgd.c; call from sgdStep when mask is non-NULL)
void riglMaskedSgdStepFloat(sgd_t *sgd, parameter_t *param,
    riglConfig_t *cfg) {
    tensor_t *w = getParamFromParameter(param);
    tensor_t *g = getGradFromParameter(param);
    tensor_t *mask = cfg->mask;
    size_t n = calcNumberOfElementsByTensor(w);
    float lr = sgd->learningRate;
    float wd = sgd->weightDecay;
    for (size_t i = 0; i < n; i++) {
        // Skip pruned weights – do not update them
        if (!tensorBoolGet(mask, i)) {
            continue;
        }
        float wi = readBytesAsFloat(w->data + i*4);
        float gi = readBytesAsFloat(g->data + i*4);
        float grad_with_wd = gi + wd * wi;
        float wNew = wi - lr * grad_with_wd;
        writeFloatToByteArray(wNew, w->data + i*4);
    }
}
```

1	<pre>if (!tensorBoolGet(mask, i)) { continue; }</pre>	The key RigL modification to SGD. 'continue' skips the rest of the current loop iteration and jumps to the next i. For pruned weights (<code>mask=0</code>): the weight stays at 0, gradient is ignored. For active weights (<code>mask=1</code>): normal gradient descent update applies.
2	<pre>float gi = readBytesAsFloat(g->data + i*4);</pre>	The gradient for this weight IS computed (by <code>linearCalcWeightGrads</code>) even if the weight was pruned. But we use the gradient for the GROW STEP decision (in <code>riglGrowStep</code>), not for weight updates. The 'continue' above prevents this gradient from actually being applied to the (zero) weight value.

Section 21.9 — Integration: Putting It All Together

Here is a complete setup and training loop showing exactly how to integrate RigL into an existing ODT training script for a speaker ID model.

```
// ===== Complete RigL integration example =====
// 1. Static buffers for a single linear layer [128, 64]
#define IN_FEATURES 64
#define OUT_FEATURES 128
#define N_WEIGHTS (IN_FEATURES * OUT_FEATURES) // 8192
// Mask: ceil(8192 / 8) = 1024 bytes
static uint8_t maskData[(N_WEIGHTS + 7) / 8];
static tensor_t maskTensor;
// Mask quantization (BOOL type)
static quantization_t maskQ;
// 2. Setup (called once before training)
void setupRigL(void) {
    // 2a. Init mask quantization
    initBoolQuantization(&maskQ);
    // 2b. Mask shape (same as weight shape)
    static size_t maskDims[2] = {OUT_FEATURES, IN_FEATURES};
    static size_t maskOrder[2] = {0, 1};
    static shape_t maskShape;
    setShape(&maskShape, maskDims, 2, maskOrder);
    // 2c. Build mask tensor
    setTensorValues(&maskTensor, maskData, &maskShape, &maskQ, NULL);
    // 2d. RigL config
    static riglConfig_t riglCfg = {
        .mask = &maskTensor,
        .sparsityRatio = 0.8f,
        .updateStep = 100,
        .batchCount = 0,
    };
    // 2e. Initialize mask randomly (80% pruned)
    riglInitMask(&riglCfg, 42); // seed=42 for reproducibility
}
// 3. Training loop
void trainWithRigL(int numEpochs) {
    for (int epoch = 0; epoch < numEpochs; epoch++) {
        for (int batch = 0; batch < numBatches; batch++) {
            // Standard forward/backward/update
            sgdZeroGrad(&weightParam);
            dataLoaderGetNextBatch(&loader, &batchInput, &batchLabels);
            layerForward(&linearLayer, &batchInput, &linear_out);
            // ... rest of forward pass ...
            float loss = crossEntropyLossFloat(...);
            layerBackward(&linearLayer, ...);
            // Use masked SGD (skip pruned weights)
            riglMaskedSgdStepFloat(&sgd, &weightParam, &riglCfg);
            // RigL topology update (every 100 batches)
            riglUpdate(&weightParam, &riglCfg);
        }
        // Optional: print sparsity stats each epoch
        riglPrintStats(&riglCfg, N_WEIGHTS);
    }
}
```

1	static uint8_t maskData[(N_WEIGHTS + 7) / 8];	Static array for the mask bits. On MCU, this must be static (not stack). 8192 weights → 1024 bytes for the mask. Negligible memory overhead.
2	initBoolQuantization(&maskQ);	Set up the BOOL quantization type for the mask tensor. BOOL tensors pack 8 bits per byte, enabling tensorBoolGet/Set operations.

3	<code>.sparsityRatio = 0.8f, .updateStep = 100, .batchCount = 0</code>	Configuration: 80% sparsity, update topology every 100 batches, counter starts at 0. The compound literal initializer sets all fields by name — order doesn't matter.
4	<code>riglInitMask(&riglCfg, 42);</code>	Initialize mask with 80% bits set to 0 (pruned), 20% to 1 (active). seed=42 for reproducibility — same seed gives same initial pattern every run.
5	<code>riglMaskedSgdStepFloat(&sgd, &weightParam, &riglCfg);</code>	Replace the normal <code>sgdStepFloat</code> with the mask-aware version. Only active weights get gradient updates.
6	<code>riglUpdate(&weightParam, &riglCfg);</code>	Called every batch. Internally: only runs prune+grow when <code>batchCount % 100 == 0</code> . Increments <code>batchCount</code> every call.

■ WORKED EXAMPLE

Expected training behaviour with RigL (sparsity=80%, updateStep=100): Batches 0-99: Train with initial random sparse topology. Loss decreases. Batch 100: `riglUpdate`: prune 20% smallest active, grow 20% best-gradient inactive. Topology shifts. Loss may temporarily increase (new weights start at 0). Batches 101-199: New topology trains. The regrown weights adapt quickly. Batch 200: Another prune+grow. Topology converges toward better patterns. ... After 500+ updates: topology stabilises. 20% of weights carry nearly all signal. Final model: 80% of weights are 0. Inference skips those. ~5× speedup on MCU.

Section 21.10 — Memory Budget on STM32 Nucleo-F746ZG

Component	Size (linear [128,64])	Notes
Weight tensor (dense)	$8192 \times 4 = 32 \text{ KB}$	float32 or SYM_INT32
Gradient tensor	$8192 \times 4 = 32 \text{ KB}$	Same shape as weight
RigL mask tensor	$(8192+7)/8 = 1024 \text{ B} \approx 1 \text{ KB}$	BOOL, 1 bit per weight
Sort buffer (s_activeIdx)	$16384 \times 4 = 64 \text{ KB}$ (static)	Shared across all layers, reused
Sort buffer (s_wFloat)	$16384 \times 4 = 64 \text{ KB}$ (static)	Reused per call
riglConfig_t struct	$\approx 16 \text{ bytes}$	Mask pointer + float + $2 \times \text{size_t}$
Total overhead vs dense	+1 KB mask + shared sort bufs	Sort bufs reuse existing space
Total RAM usage	$\sim 193 \text{ KB}$ for this layer alone	Leaves 127 KB for other layers

■ ■ IMPORTANT

The static sort buffers (s_activeIdx, s_wFloat, s_inactiveIdx, s_gFloat) are the largest RigL overhead: $4 \times \text{RIGL_MAX_WEIGHTS} \times 4 \text{ bytes} = 256 \text{ KB}$ for $\text{RIGL_MAX_WEIGHTS}=16384$. This exceeds the STM32's 320 KB RAM if used naively. Optimization: reduce RIGL_MAX_WEIGHTS to match the actual largest layer size, OR reuse the activation buffer (which is not in use during the RigL update phase).

■ NOTE

The gradient buffer is already allocated for training (it is the parameter_t.grad tensor). So the gradient sort buffer (s_gFloat) can be replaced by reading directly from g->data in the comparison function, eliminating the 64 KB copy. The g_sortGrads global would then point to (float*)g->data directly.

C Language Deep Dive

Every concept used in ODT explained: cast, pointer, struct, memory, and more

Section 22.1 — Why Learn C for Embedded ML?

All ODT source code is written in C. Unlike Python, C gives you direct control over every byte in memory. On a microcontroller with 320 KB of RAM and no operating system, this control is not optional — it is required. This chapter explains every C concept that appears in the ODT library, from the most basic (variables, types) to the advanced (function pointers, bit operations).

Section 22.2 — Integer Types and Their Sizes

■ C CONCEPT: Why Exact-Width Integer Types?

In standard C, 'int' is NOT guaranteed to be 32 bits. On a 16-bit platform, int is 16 bits. On a 64-bit platform, it might be 64 bits. This makes code non-portable and causes subtle bugs. The solution: use exact-width types from <stdint.h>: int8_t — exactly 8 bits signed (-128 to +127) uint8_t — exactly 8 bits unsigned (0 to 255) int16_t — exactly 16 bits signed (-32768 to +32767) uint16_t — exactly 16 bits unsigned (0 to 65535) int32_t — exactly 32 bits signed (-2,147,483,648 to +2,147,483,647) uint32_t — exactly 32 bits unsigned (0 to 4,294,967,295) int64_t — exactly 64 bits signed (large range) size_t — unsigned, large enough for any array index (32-bit on STM32)

Value	Max Value	Use in ODT
	255	Raw bytes (data)
	+127	Small signed values
	65,535	qBits, qMaxBits
	+2.1B	SYM_INT32 max
	4.3B	Unsigned 32-bit
negative	very positive	Matmul accumulator
38	~+3.4e38	FLOAT32 weight
	4.3B	Array indices, sizes

— ■ WORKED EXAMPLE

Choosing the right type: uint8_t qBits = 8; // fine: 8 fits in [0, 255] uint8_t qMaxBits = 16; // fine: 16 fits in [0, 255] // qMaxBits=16 means 'use 16-bit quantization range', not 'this field is 16 bits' int32_t mantissa = 32767; // fine: 32767 fits in int32_t int64_t acc = 0; // needed: sum of int32×int32 products can overflow int32 size_t n = calcNumberOfElements(tensor); // correct: n is always positive

Section 22.3 — Type Casting: What It Is and When to Use It

■ C CONCEPT: What is a Cast?

A cast tells the compiler: 'treat this value as if it has this other type'. Syntax: (target_type) expression Examples: int a = 65; char c = (char)a; // cast int to char: reinterpret 65 as 'A' float f = 3.7f; int i = (int)f; // cast float to int: truncates decimal → i = 3 void *ptr = buffer; float *fp = (float*)ptr; // cast void* to float*: pointer reinterpretation

There are four situations where casts appear in ODT:

22.3.1 Void Pointer Casts (the most common in ODT)

```
// tensor->data is void* or uint8_t* – raw bytes, no type
// To read elements, you must cast to the correct pointer type:
// Reading float values:
float *fptr = (float*)tensor->data;
float val = fptr[5]; // element 5 as float
// OR using readBytesAsFloat (which does the cast internally):
float val2 = readBytesAsFloat(tensor->data + 5*4);
// Reading int32_t values:
int32_t *iptr = (int32_t*)tensor->data;
int32_t ival = iptr[5];
// Accessing quantization config (void* qConfig):
symInt32QConfig_t *cfg = (symInt32QConfig_t*)tensor->quantization->qConfig;
float scale = cfg->scale;
// DO NOT cast incorrectly:
// Treating a float tensor as int32:
// int32_t wrong = ((int32_t*)floatTensor->data)[0]; // reads float bits as int!
```

■■ IMPORTANT

A cast does NOT convert the data — it reinterprets the SAME bits. float x = 3.14f stores as 0x4048F5C3 in memory (IEEE 754). Casting that memory to int32_t gives -1010580029 — the same 4 bytes, wrong meaning. Only cast to a type whose memory layout matches the actual stored data.

22.3.2 Numeric Casts (widen or narrow)

```
// Widening: smaller type → larger type (safe, automatic)
int32_t a = 32767;
int64_t b = (int64_t)a; // explicit is better – shows intent
// Without cast: int32_t * int32_t computed in 32-bit (may overflow)
// With cast: multiplication happens in 64-bit (safe)
int64_t product = (int64_t)a * (int64_t)a; // 32767^2 = 1,073,676,289 (fits!)
// Narrowing: larger type → smaller type (potentially lossy)
float f = 3.9999f;
int32_t n = (int32_t)f; // truncates decimal: n = 3 (NOT rounded!)
// Use roundf() to round before truncating:
int32_t rounded = (int32_t)roundf(f); // rounded = 4
// Integer to float
size_t count = 1000;
float ratio = (float)pruned / (float)count; // 0.0f–1.0f
// Without casts: integer division 100/1000 = 0 (truncated!)
// With casts: float division 100.0/1000.0 = 0.1f (correct!)
```

1	int64_t product = (int64_t)a * (int64_t)a;	Cast BOTH operands before multiplication. If only one is cast: (int64_t)a * b — C promotes the other operand too (integer promotion rules). Casting both is explicit and always correct.
2	int32_t n = (int32_t)f;	Truncation: the decimal is dropped, NOT rounded. 3.9 → 3, -3.9 → -3 (always toward zero). If you want rounding, use roundf(f) first, then cast.

3	<pre>float ratio = (float)pruned / (float)count;</pre>	When dividing two integers and expecting a float result, cast at least one operand to float. Integer division drops the remainder: <code>pruned=1, count=3</code> → <code>1/3=0</code> (int), but <code>(float)1/(float)3=0.333f</code> (correct).
---	--	---

Section 22.4 — Pointers and Pointer Arithmetic

■ C CONCEPT: What is a Pointer?

A pointer is a variable that stores the MEMORY ADDRESS of another variable. `int x = 42; // x is an integer, lives at some address in RAM` `int *p = &x; // p stores the ADDRESS of x (&x = 'address of x')` `int val = *p; // *p = 'value at the address p holds' = 42` Memory diagram: Address: 0x20001000 → value 42 ← x Address: 0x20001004 → value 0x20001000 ← p (stores address of x) Dereferencing (*p): follow the pointer to read/write the value it points to. Address-of (&x): get the memory address where x is stored. In ODT, pointers are everywhere: `void *dataPTR;` → points to the raw tensor data buffer `tensor_t *input;` → points to the input tensor struct parameter `t *param;` → points to the weight+gradient pair

22.4.1 Pointer Arithmetic

```
// Pointer arithmetic: adding to a pointer moves by sizeof(type) bytes
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr; // p points to arr[0] (address 0x1000, say)
p + 1; // address 0x1004 (1 int = 4 bytes forward)
p + 2; // address 0x1008 (2 ints = 8 bytes forward)
*(p + 2); // value at 0x1008 = 30
p[2]; // same as *(p+2) – array notation is just shorthand
// In ODT: how readBytesAsFloat is called
// tensor->data is uint8_t* (1 byte per step)
float val = readBytesAsFloat(tensor->data + i * sizeof(float));
// tensor->data + i*4 = address of element i
// (uint8_t* + N) moves N bytes forward
// If tensor->data were float*, + i would move i*4 bytes – same result
// Why uint8_t* and not float*?
// Because the tensor might hold int32, float, or BOOL data.
// uint8_t* is the universal 'raw bytes' pointer – works for any type.
// The caller decides how to interpret the bytes via a cast.
```

■ C CONCEPT: sizeof() — The Size Operator

`sizeof(type)` returns the number of bytes a type occupies. `sizeof(expression)` returns the number of bytes the result would occupy. `sizeof(uint8_t) = 1` `sizeof(int32_t) = 4` `sizeof(float) = 4` `sizeof(int64_t) = 8` `sizeof(tensor_t) = sum of all fields (may include padding)` `sizeof` is evaluated at COMPILE TIME — no runtime cost. Always use `sizeof(type)` rather than hardcoding 4 — it adapts if you change the type.

22.4.2 Passing by Pointer vs Passing by Value

```
// Pass by value: function gets a COPY – changes don't affect original
void addOne_value(int x) { x += 1; } // modifies LOCAL copy
int a = 5;
addOne_value(a); // a is still 5
// Pass by pointer: function gets ADDRESS – can modify original
void addOne_pointer(int *x) { *x += 1; } // modifies the REAL variable
int b = 5;
addOne_pointer(&b); // b is now 6
// In ODT: ALL tensor functions take tensor_t* (pointer)
// Reason: tensor_t is a large struct. Copying it on every call would waste
// 20-40 bytes per call. Using a pointer is 0(1) regardless of tensor size.
void matmulFloat32TensorsWithBias(
    tensor_t *A, // pointer: no copy of A's struct
    tensor_t *B, // pointer: no copy of B's struct
    tensor_t *out, // pointer: writes result directly into caller's tensor
    tensor_t *bias
);
```


—■ WORKED EXAMPLE

Why ODT uses pointers everywhere: `tensor_t` has fields: `data` (4B pointer), `shape` (4B pointer), `quantization` (4B pointer), `sparsity` (4B pointer) Total struct size: ~16 bytes If `matmulFloat32TensorsWithBias` took `tensor_t` by VALUE: Each call copies 4 `tensor_t` structs = 64 bytes pushed onto the stack. Plus, any modifications inside the function would NOT be visible to the caller. Using pointers (`tensor_t*`): Each call passes 4 pointers = 16 bytes (4 × 4 bytes). Modifications (writing to `out->data`) are visible to the caller immediately. The actual tensor data (potentially MB) is never copied — just the address.

Section 22.5 — Structs, Typedef, and Memory Layout

■ C CONCEPT: struct in C

A struct groups related variables into a single named type. `struct point { float x; float y; }; struct point p; // declare a variable of type 'struct point' p.x = 3.0f; // access field x via '.' operator p.y = 4.0f; struct point *pp = &p; // pointer to p pp->x = 3.0f; // access field x via pointer using '->' operator` The '->' operator is shorthand for `(*pp).x` — dereference then access field.

```
// typedef: create an alias so you don't need 'struct' every time
// Without typedef:
struct tensor { uint8_t *data; ... };
struct tensor t; // must write 'struct tensor' every time
// With typedef (ODT approach):
typedef struct tensor {
    uint8_t *data;
    shape_t *shape;
    quantization_t *quantization;
    sparsity_t *sparsity;
} tensor_t;
// Now: tensor_t t; (cleaner)
// Accessing fields:
tensor_t *t = &myTensor;
t->data // the raw data pointer
t->shape // the shape pointer
t->quantization // the quantization pointer
// Accessing nested structs:
t->shape->dimensions[0] // first dimension of the shape
// t->shape is a shape_t* pointer, so we use -> again
// ->dimensions[0] is the first element of the dimensions array
```

■ C CONCEPT: Struct Memory Layout and Padding

Structs in C are laid out in memory in field order, but the compiler may add 'padding' bytes between fields to ensure alignment. `typedef struct { uint8_t a; // 1 byte // 3 bytes padding (to align b to 4-byte boundary) int32_t b; // 4 bytes uint8_t c; // 1 byte // 3 bytes padding (to align next to 4-byte boundary) } example_t; // sizeof = 12 bytes (not 1+4+1=6!)` Rule of thumb: members should be ordered largest-first to minimize padding. In ODT, all struct fields are pointers (4 bytes each), so no padding occurs.

Section 22.6 — Enums: Named Integer Constants

```
// enum: a set of named integer constants
// From Quantization.h:
typedef enum {
INT32 = 0, // explicit values (optional)
FLOAT32 = 1,
SYM_INT32= 2,
SYM = 3,
ASYM = 4,
BOOL = 5
} quantizationType_t;
// Usage:
quantizationType_t t = FLOAT32; // t = 1
if (t == SYM_INT32) {
// handle SYM_INT32 case
}
switch (t) {
case FLOAT32: ...; break;
case SYM_INT32:...; break;
default: ...; break;
}
// WITHOUT enum, you'd write:
// if (t == 1) { ... } – not readable!
// WITH enum: if (t == FLOAT32) { ... } – self-documenting
```

■ NOTE

Enum values are ints. You can compare them with ==, use them in switch statements, and use them as array indices (as in the layer dispatch table: `forwardFuncs[RELU]`). Without the explicit = 0, 1, 2 assignments, enum values are automatically 0, 1, 2, ...

Section 22.7 — Memory Sections: Stack, Heap, .data, .bss

■ C CONCEPT: Where Variables Live in Memory

C variables can live in different memory regions: .text (ROM/Flash): → Function code, string literals, const global arrays. → Read-only. On STM32: 1 MB Flash. .data (RAM): → Global/static variables WITH initial values. → Example: static float lr = 0.01f; (initialized to 0.01) → Copied from Flash to RAM at startup. .bss (RAM): → Global/static variables WITHOUT initial values (initialized to 0 by startup). → Example: static size_t counter; (automatically = 0) → No copy needed from Flash; just zeroed in RAM. Stack (RAM, grows downward): → Local variables inside functions. Automatically allocated on function entry, freed on return. Fast but limited (4-8 KB on STM32). → VLAs (Variable-Length Arrays) also go here. Heap (RAM): → Dynamically allocated memory via malloc(). ODT avoids this. → Problem: malloc on MCU causes fragmentation; free() order matters. → Better: use static arrays (predictable, no fragmentation).

```
// Examples of memory section placement:
// .text (Flash) – function code
void myFunc(void) { ... }
// .data (RAM, initialized) – global with initial value
float learningRate = 0.01f;
// .bss (RAM, zero-initialized) – global without initial value
size_t batchCount; // automatically 0 at startup
// Stack – local variable inside function
void trainStep(void) {
    float sum = 0.0f; // on stack, freed when trainStep returns
    size_t i; // on stack
}
// Static local – like .bss but scoped to function
void rigLUpdate(void) {
    static size_t count = 0; // .data or .bss, persists across calls
    count++;
}
// Static array (RigL buffer) – .bss, avoids stack overflow
static size_t s_activeIdx[16384]; // 64 KB in .bss, safe
// size_t s_activeIdx[16384]; // This would be on stack – 64 KB CRASH!
```

1	float learningRate = 0.01f;	Global with initialiser → .data section. At startup, the MCU's startup code copies this from Flash to RAM. ODT avoids many globals because they consume precious RAM permanently.
2	size_t batchCount;	.bss: global with no initialiser → automatically 0. ODT's rigLConfig_t has batchCount as a struct field, set to 0 in the initialiser.
3	float sum = 0.0f;	Local variable → stack. Allocated when the function is called, freed when it returns. If you recurse deeply enough, stack overflow.
4	static size_t count = 0;	Static local: one allocation, shared across ALL calls to this function. count persists between calls (unlike a normal local which resets each call). First call: count=0. After count++: count=1. Second call: count=1 (not 0).
5	static size_t s_activeIdx[16384];	Static array: 64 KB in .bss, NOT on the stack. ODT's RigL buffers MUST be static for this reason. A non-static 64 KB local array would immediately cause a stack overflow on STM32.

Section 22.8 — Bit Operations (used in DTypes.c and BOOL tensors)

■ C CONCEPT: Bitwise Operators in C

C provides direct bit manipulation operators: & — bitwise AND: $1010 \& 1100 = 1000$ | — bitwise OR: $1010 | 1100 = 1110$ ^ — bitwise XOR: $1010 \wedge 1100 = 0110$ ~ — bitwise NOT: $\sim 1010 = 0101$ (inverts all bits) << — left shift: $0001 \ll 3 = 1000$ (multiply by $2^3 = 8$) >> — right shift: $1000 \gg 2 = 0010$ (divide by $2^2 = 4$) These are used in tensorBoolGet/Set to pack 8 booleans per byte.

```
// tensorBoolGet: read bit 'flatIndex' from a BOOL tensor
bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex) {
    size_t byteIdx = flatIndex / 8; // which byte?
    size_t bitIdx = flatIndex % 8; // which bit in that byte?
    return (tensor->data[byteIdx] >> bitIdx) & 1u;
}

// tensorBoolSet: write bit 'flatIndex' to a BOOL tensor
void tensorBoolSet(tensor_t *tensor, size_t flatIndex, bool value) {
    size_t byteIdx = flatIndex / 8;
    size_t bitIdx = flatIndex % 8;
    if (value)
        tensor->data[byteIdx] |= (1u << bitIdx); // set bit
    else
        tensor->data[byteIdx] &= ~(1u << bitIdx); // clear bit
}
```

1	<code>size_t byteIdx = flatIndex / 8;</code>	Integer division: which byte contains this bit? Element 0–7 are in byte 0, 8–15 in byte 1, etc. flatIndex=13: byteIdx = 13/8 = 1.
2	<code>size_t bitIdx = flatIndex % 8;</code>	Modulo: which bit within that byte? flatIndex=13: bitIdx = 13%8 = 5. Bit 5 of byte 1.
3	<code>(tensor->data[byteIdx] >> bitIdx) & 1u</code>	Right-shift the byte by bitIdx positions, then AND with 1. Example: byte=0b10100000, shift right by 5 → 0b00000101. AND with 1: 0b00000001 → result is 1 (true). The bit was set.
4	<code>(1u << bitIdx)</code>	Create a mask with ONLY bit bitIdx set. 1u=0b00000001, << 5 → 0b00100000. This is the 'bitmask' for bit 5.
5	<code>data[byteIdx] = (1u << bitIdx);</code>	Set the bit: OR with the bitmask. 0b10000000 0b00100000 = 0b10100000. Bit 5 is now set.
6	<code>data[byteIdx] &= ~(1u << bitIdx);</code>	Clear the bit: AND with the INVERTED bitmask. ~0b00100000 = 0b11011111. 0b10100000 & 0b11011111 = 0b10000000. Bit 5 is cleared. Other bits unchanged.

Section 22.9 — Macros and the C Preprocessor

■ C CONCEPT: The C Preprocessor

Before compiling C code, the compiler runs the PREPROCESSOR. The preprocessor handles lines starting with #: `#include <file.h>` — paste the contents of file.h here `#define NAME value` — replace all occurrences of NAME with value `#ifdef NAME` — compile the following block only if NAME is defined `#ifndef NAME` — compile the following block only if NAME is NOT defined `#endif` — end an `#ifdef` or `#ifndef` block The preprocessor runs BEFORE the compiler and produces modified C source. Macros are text substitution — they are not functions and have no types.

```
// From Common.h — debug macros using ifdef
#ifdef DEBUG
#define PRINT_DEBUG(fmt, ...) printf("[DEBUG] " fmt, ##__VA_ARGS__)
#define PRINT_ERROR(fmt, ...) printf("[ERROR] " fmt, ##__VA_ARGS__)
#else
#define PRINT_DEBUG(fmt, ...) do { } while(0)
#define PRINT_ERROR(fmt, ...) do { } while(0)
#endif
// Usage:
PRINT_DEBUG("weight[%zu] = %.4f\n", i, val);
// In DEBUG build, expands to: printf("[DEBUG] weight[%zu] = %.4f\n", i, val);
// In release build, expands to: do { } while(0); (nothing — zero overhead)
// Function-like macro with argument:
#define MAX(a, b) ((a) > (b) ? (a) : (b))
int result = MAX(x, y);
// Expands to: int result = ((x) > (y) ? (x) : (y));
// Parentheses around args prevent operator-precedence bugs:
// MAX(1+2, 3) without parens: 1+2 > 3 ? 1+2 : 3 → ambiguous precedence
// With parens: ((1+2)) > ((3)) ? ((1+2)) : ((3)) → correct
// Constant macro:
#define RIGL_MAX_WEIGHTS 16384
// All occurrences of RIGL_MAX_WEIGHTS in the code are replaced with 16384
// Change this once → all array sizes and checks update automatically
```

1	<code>#ifdef DEBUG ... #else ... #endif</code>	Conditional compilation: the <code>PRINT_DEBUG</code> macro includes real <code>printf</code> only when <code>DEBUG</code> is defined (e.g., via <code>gcc -DDEBUG</code>). In release builds, <code>PRINT_DEBUG</code> expands to an empty <code>do-while</code> — the compiler optimises it away, leaving zero overhead.
2	<code>##__VA_ARGS__</code>	Variadic macro argument. <code>'...'</code> accepts any number of arguments. <code>__VA_ARGS__</code> pastes them into the expansion. <code>##</code> before it suppresses the trailing comma when no variadic args are given.
3	<code>do { } while(0)</code>	The 'do nothing' idiom for empty macros. Using just an empty <code>;</code> would cause issues in <code>if/else</code> without braces. <code>do { } while(0)</code> is a safe, complete statement.
4	<code>#define MAX(a,b) ((a) > (b) ? (a) : (b))</code>	A function-like macro. Note: <code>'a'</code> is evaluated TWICE. <code>MAX(x++, y++)</code> would increment <code>x</code> and <code>y</code> each twice — dangerous. For simple values (no side effects), macros are fine.

Section 22.10 — const, static, and extern Qualifiers

Applies to	Meaning	Example in ODT
variables, pointers	Value cannot be changed after initialization. Compiler may place in Flash (ROM) to save RAM.	const float PI = 3.14159f;
local variables	Variable persists across function calls (in .bss or .data). Not on the stack.	static size_t s_activeIdx[16384]; in RtgL.c
global variables, functions	Limits scope to the current file. Not visible from other .c files.	static int32_t addInt32s(int32_t a, int32_t b)
variables, functions	Declares that the variable/function is DEFINED elsewhere. Tells linker to resolve the symbol.	extern float learningRate; in a header
variables	Tells compiler: this variable can change outside normal code flow (hardware register, interrupt). Do not cache in register.	volatile uint32_t *DWT_CYCCNT;

```
// const examples:
const float SCALE = 0.001f; // cannot assign SCALE = 0.002f
void foo(const tensor_t *t) { // cannot modify t->data inside foo
// t->data = NULL; // COMPILE ERROR
}
// static (local) – persists between calls:
void countCalls(void) {
static int n = 0; // initialized once, persists
n++;
PRINT_DEBUG("Called %d times\n", n);
}
// static (global) – private to file:
// In Sgd.c:
static void sgdStepFloat(sgd_t *sgd, parameter_t *p) { ... }
// sgdStepFloat cannot be called from Linear.c – it is private to Sgd.c
// extern – declare, not define:
// In a .h file:
extern float g_learningRate; // declared: 'it exists somewhere'
// In a .c file:
float g_learningRate = 0.01f; // defined: 'it actually lives here'
```

Section 22.11 — Integer Overflow and Promotion Rules

■ C CONCEPT: Integer Overflow in C

Signed integer overflow is UNDEFINED BEHAVIOUR in C — the result is unpredictable. Unsigned integer overflow wraps around (modular arithmetic) — well defined. `int32_t a = 2147483647; // INT32_MAX` `int32_t b = a + 1; // UNDEFINED BEHAVIOUR (signed overflow)` `uint32_t c = 4294967295; // UINT32_MAX` `uint32_t d = c + 1; // 0` (wraps around — defined!) In ODT matmul: using `int64_t` for accumulation prevents overflow: Max product: $32767 \times 32767 = 1,073,676,289 < \text{INT32_MAX}$ (2,147,483,647) ONE product fits. But 100 such products: $107,367,628,900 > \text{INT32_MAX}$ → OVERFLOW `int64_t` max: 9,223,372,036,854,775,807 → thousands of products fit safely

```
// Integer promotion rules:
// When two operands have different types, the 'narrower' is promoted
// Case 1: int32_t * int32_t stays int32_t
int32_t a = 32767, b = 32767;
int32_t c = a * b; // 1,073,676,289 – fits! (but if K products are summed → risk)
// Case 2: cast before multiply to use 64-bit
int64_t d = (int64_t)a * b; // b is promoted to int64_t → 64-bit multiply
// Case 3: int / int = int (truncated!)
int x = 5, y = 2;
int result = x / y; // 2 (not 2.5 – decimal truncated)
float ratio = (float)x / y; // 2.5f (x promoted to float first)
// Case 4: uint8_t + uint8_t: both promoted to int first
uint8_t a8 = 200, b8 = 100;
uint8_t sum = a8 + b8; // WARNING: 300 doesn't fit in uint8_t!
// 300 % 256 = 44 (wraps around – silent bug!)
uint16_t safeSum = (uint16_t)a8 + b8; // correct: 300
```

■■ IMPORTANT

`uint8_t` arithmetic silently wraps around! $200 + 100 = 44$ (not 300) when stored in `uint8_t`. In ODT, this is why we use `int32_t` for mantissas (not `int8_t`) and `int64_t` for accumulation (not `int32_t`). Always think about the maximum value an expression can produce before choosing the result type.

Section 22.12 — memset and memcpy: Raw Memory Operations

```
// memset: fill N bytes with a given byte value
// Signature: void *memset(void *ptr, int value, size_t count)
uint8_t buf[10];
memset(buf, 0, 10); // set all 10 bytes to 0x00
memset(buf, 0xFF, 10); // set all 10 bytes to 0xFF
// In sgdZeroGrad:
memset(grad->data, 0, bytes); // zero all gradient mantissas
// In riglInitMask:
memset(mask->data, 0xFF, maskBytes); // set all bits to 1 (all active)
// WARNING: memset uses BYTE values (0-255)
// memset(arr, 1, 4*n); does NOT set int32 elements to 1!
// It sets each BYTE to 1: each int32 = 0x01010101 = 16,843,009
// To zero int32 array: memset(arr, 0, 4*n) – works because 0x00000000 = 0
// memcpy: copy N bytes from source to destination
// Signature: void *memcpy(void *dst, const void *src, size_t count)
float src_val = 3.14f;
float dst_val;
memcpy(&dst_val, &src_val, sizeof(float)); // copy 4 bytes
// dst_val is now 3.14f
// In readBytesAsFloat (DTypes.c):
float readBytesAsFloat(const uint8_t *bytes) {
    float value;
    memcpy(&value, bytes, sizeof(float)); // copy 4 bytes from buffer into float
    return value;
}
// WHY memcpy and not (float*)bytes?
// Direct cast via pointer (float*)bytes is undefined behaviour when bytes
// is not float-aligned. memcpy is always safe regardless of alignment.
```

1	<code>memset(buf, 0, 10);</code>	Fill 10 bytes with 0x00. The 'value' argument (0) is an int but only the lowest byte (0x00) is written. Safe for zeroing int32 arrays because 0x00000000 = 0 in all integer representations.
2	<code>memset(mask->data, 0xFF, maskBytes);</code>	0xFF = 0b11111111: sets all 8 bits of each byte to 1. For a BOOL tensor: this marks all elements as active (bit=1 means active).
3	<code>memcpy(&value, bytes, sizeof(float));</code>	Copy 4 bytes from bytes[] into the float variable. memcpy avoids the undefined-behaviour of direct pointer casting. The sizeof(float) = 4 bytes copied exactly reconstruct the original float.

Section 22.13 — Quick Reference: C Concepts in ODT

Concept	Symbol/Syntax	Where in ODT
Pointer declaration	type *name;	tensor_t *input everywhere
Dereference	*ptr	*cfg = access value at pointer
Address-of	&var	&cfg to pass struct by pointer
Arrow operator	ptr->field	layer->config.linear, tensor->data
Cast (pointer)	(type*)ptr	(float*)tensor->data, (symInt32QConfig_t*)qConfig
Cast (numeric)	(type)value	(int64_t)aVal * bVal, (float)count
sizeof	sizeof(type)	sizeof(float)=4, sizeof(int32_t)=4
Enum	typedef enum { A,B } t;	quantizationType_t, layerType_t
Struct	typedef struct { } t;	tensor_t, shape_t, riglConfig_t
Union	typedef union { } t;	layerConfig_t (one pointer for any layer)
Function pointer	typedef ret (*name)(args);	layerForwardFunc_t, int32ArithFunc_t
Static local	static type name;	RigL sort buffers, batch counter
memset	memset(ptr, val, n)	sgdZeroGrad, riglInitMask
memcpy	memcpy(dst, src, n)	readBytesAsFloat, writeFloatToByteArray
Bitwise ops	& ^ ~ << >>	tensorBoolGet/Set in DTypes.c
Modulo	a % b	riglUpdate batch counter, byteIdx for BOOL
Ternary	cond ? a : b	riglBackward: dX = (x>0) ? dY : 0
VLA	type arr[n];	Index arrays in Arithmetic.c (nDim small)
#define	#define NAME value	RIGL_MAX_WEIGHTS, DEBUG macros
#ifdef	#ifdef NAME ... #endif	PRINT_DEBUG only in debug builds

Complete Glossary

Every Technical Term Used in This Book, Explained in Plain Language

This glossary defines every term, abbreviation, and concept introduced in the book. Entries are grouped by topic so related terms appear together. References to ODT source files are given where relevant.

A — C Language Fundamentals

Term	Definition
.bss	The Block Started by Symbol section. A region of RAM for global and static variables that are zero-initialized at program start. Costs no flash (linker fills it at boot). ODT uses .bss for static sort buffers in RigL.c.
.data	A RAM section for initialized global/static variables (e.g., float x = 3.14f). The initial values are stored in flash and copied to RAM at boot.
.text	The code section — where compiled machine instructions live. Stored in flash. Read-only at runtime.
address-of operator (&)	& before a variable returns its memory address as a pointer. int x = 5; int *p = &x; — p now holds the address where x lives.
bit field	A struct field declared with a width in bits: uint8_t flag : 1; Packs multiple flags into one byte. Used in mask representations.

cast (type cast)	(target_type)expression — instructs the compiler to treat a value as a different type. (float)x converts integer x to float. (symint32QConfig_t*)qConfig converts void* to the actual config type. Widening cast (int to long) is always safe. Narrowing cast may lose data.
const	Tells the compiler this value must not change. const float PI = 3.14159f; Placed in flash (.rodata) by the compiler. const tensor_t *t means 't points to a tensor that cannot be modified.'
dereference (*)	*pointer reads or writes the value at the address stored in pointer. int *p = &x; int y = *p; — y gets the value of x.
enum	An enumeration — a named set of integer constants. typedef enum { FLOAT32=0, INT32=1, SYM_INT32=2, BOOL=3 } quantizationType_t; Makes code readable (FLOAT32 instead of 0).
extern	Declares a symbol defined in ANOTHER translation unit. Tells the compiler 'this exists somewhere, the linker will find it.' Used to share global variables or function declarations across .c files.
fused-multiply-accumulate (FMA)	A hardware instruction that computes $a*b + c$ in ONE step, with only one rounding. ARM Cortex-M7 has an FPU with single-precision FMA. matmul inner loop: <code>acc += aVal * bVal</code> — compiled to VFMA on Cortex-M7.

function pointer	A variable holding the address of a function. typedef void (*forwardFn_t)(linearConfig_t*, tensor_t*, tensor_t*); Allows dispatching different layer types through the same interface. Used in Layer.c's forwardFuncs[] dispatch table.
guard (include guard)	#ifndef FILENAME_H / #define FILENAME_H / ... / #endif. Prevents a header file from being included more than once per compilation unit.
heap	Dynamic memory region managed by malloc/free. Exists on hosted systems (PC). On bare-metal STM32 with ODT: malloc is NOT used. All memory is static or stack.
macro	A preprocessor text substitution. #define MAX(a,b) ((a)>(b)?(a):(b)). Expanded before compilation. No type safety. PRINT_DEBUG in ODT is a macro that can be compiled away with #ifdef.
memcpy	void memcpy(dst, src, n_bytes) — copies exactly n_bytes from src to dst. Used in readBytesAsFloat and writeFloatToByteArray for safe type-punning. Faster than element-wise copy for large buffers.
memset	void memset(dst, byte_value, n_bytes) — fills n_bytes at dst with byte_value. memset(buf, 0, N*4) zeros N float32 values. memset(mask, 0xFF, N/8) sets all mask bits to 1 (all active).

NULL	A null pointer constant (typically 0). Represents 'no valid pointer.' Dereferencing NULL causes a hard fault on STM32. Always check <code>!= NULL</code> before use.
pointer arithmetic	Adding an integer to a pointer moves it by <code>integer * sizeof(element)</code> bytes. float *p = buf; *(p+3) reads the 4th float. In ODT: data + i*4 moves i*4 bytes into the byte buffer.
signed vs unsigned	signed int holds -2^{31} to $2^{31}-1$. unsigned int holds 0 to $2^{32}-1$. Overflow of signed int is undefined behavior in C. Overflow of unsigned int wraps modulo 2^{32} (defined).
size_t	An unsigned integer type large enough to hold any array index. On 32-bit STM32: <code>size_t = uint32_t = 4</code> bytes. Used for all array sizes, loop indices, and byte counts in ODT.
stack	Automatic memory for local variables and function call frames. Grows downward in memory. On STM32: typically 4-8 KB. Stack overflow (e.g., large VLAs) causes hard fault with no warning.
static	Two meanings: (1) At file scope: symbol is private to this .c file (not linked elsewhere). (2) At function scope: variable persists between calls, lives in .bss/.data. ODT's <code>s_activeldx</code> in <code>RigL.c</code> is static to avoid stack overflow.

struct	A composite type grouping named fields: struct Point { int x; int y; }. Fields are laid out sequentially in memory (with possible alignment padding). tensor_t, layer_t, riglConfig_t, etc. are all structs.
typedef	Creates a type alias. typedef unsigned char uint8_t; Makes types more readable and portable.
union	Like a struct but ALL fields share the SAME memory location. Size = largest member. Used in layerConfig_t to hold either linearConfig_t* or reluConfig_t* without extra memory per layer type.
uint8_t / uint16_t / uint32_t / uint64_t	Fixed-width unsigned integers. uint8_t = 1 byte (0-255), uint16_t = 2 bytes (0-65535), uint32_t = 4 bytes (0 - 4.29 billion), uint64_t = 8 bytes. Defined in stdint.h. Essential for embedded code where int sizes vary by platform.
void*	A generic (typeless) pointer. Can point to anything. Must be cast to the actual type before dereferencing. Used in qConfig fields and qsort callbacks to accept any type.
VLA (Variable Length Array)	int arr[n] inside a function with non-constant n. Allocated on the stack at runtime. DANGEROUS on embedded: no size check, stack overflow crashes silently.
XOR (^)	Exclusive OR bitwise operation. a ^ b: output 1 if exactly one input is 1. 0 ^ 0=0, 0 ^ 1=1, 1 ^ 0=1, 1 ^ 1=0. Used in xorshift32 RNG and bit-manipulation operations.

Term	Definition
activation function	A non-linear function applied after a linear layer. Without it, stacked linear layers collapse to one linear transformation. ODT includes ReLU, Softmax, Dropout.
backpropagation (backprop)	The algorithm to compute dL/dW for all weights in a neural network. Applies the chain rule layer by layer from the loss back to the input. ODT: <code>linearCalcWeightGradsFloat + linearCalcInputGradsFloat</code> implement this.
batch	A subset of training samples processed together. Batch size = 1 (single sample per step). Larger batches give more accurate gradient estimates but use more memory.
bias	An additive term in a linear layer: $Y = X @ W + b$. The bias allows the layer to shift its output independently of the input. ODT stores bias as a separate tensor in <code>linearConfig_t</code> .
chain rule	$d(f(g(x)))/dx = f'(g(x)) * g'(x)$. Backpropagation is the chain rule applied recursively through layers. Each layer's backward pass multiplies the incoming gradient by the local Jacobian.
cross-entropy loss	$-\sum(y_{true} * \log(y_{pred}))$. Measures distance between predicted probability distribution and true one-hot label. Gradient w.r.t. softmax output: $y_{pred} - y_{true}$.

dense (fully-connected) layer	A layer where every input connects to every output neuron. Computation: $Y = X @ W^T + b$. Weight matrix shape: [out_features, in_features]. Called 'linear layer' or 'FC layer' in code.
dropout	Randomly sets a fraction of activations to zero during training. Probability p = drop rate (e.g., 0.5 = drop 50%). Reduces overfitting by preventing co-adaptation of neurons. Disabled during inference (dropoutForwardFloat is a no-op in inference mode).
epoch	One complete pass through the entire training dataset. Training typically runs for 10-100 epochs.
forward pass	Computing the network output from input to loss. Each layer applies its transformation in sequence. The result is a loss value and predicted output.
gradient	The partial derivative of the loss with respect to a parameter. Tells us: if we increase weight w by a tiny epsilon, how much does the loss change? SGD moves weights in the negative gradient direction to reduce loss.
gradient descent	Optimization algorithm: $w = w - lr * dL/dw$. Repeatedly moves each weight in the direction that reduces the loss. ODT implements vanilla SGD (no momentum, no Adam).
learning rate (lr)	Step size for gradient descent. Too large: training diverges. Too small: training is very slow. Typical values: 0.01, 0.001. Requires tuning per problem.

logits	The raw unnormalized scores output by the last linear layer, before softmax. Logits can be any real number.
loss	A scalar measure of how wrong the model's predictions are. We minimize the loss during training. ODT uses cross-entropy loss for speaker ID classification.
one-hot encoding	A label vector where only one element is 1.0 and the rest are 0.0. Speaker 3 of 10: [0, 0, 1, 0, 0, 0, 0, 0, 0, 0].
overfitting	When a model memorizes training data but fails to generalize to new data. Signs: low training loss, high validation loss. Mitigated by dropout, weight decay, or collecting more data.
ReLU (Rectified Linear Unit)	$f(x) = \max(0, x)$. The most common activation function. Simple, fast to compute, avoids vanishing gradients. ODT: reluForwardFloat, reluBackwardFloat.
softmax	Converts a vector of logits to a probability distribution. $\text{softmax}(x)_i = \exp(x_i) / \sum(\exp(x_j))$. Output sums to 1.0. ODT subtracts $\max(x)$ before exp for numerical stability.
speaker identification	The task of determining WHICH speaker produced a given audio clip. A classification task: N speakers = N output classes. The thesis uses LibriSpeech data: 2484 speakers.
weight	A trainable parameter in a neural network. Stored as a float32 or SYM_INT32 (quantized) value in ODT. Updated by the optimizer each training step.

weight gradient (dL/dW)	How much the loss changes per unit change in a weight. Computed by backpropagation. ODT stores gradients in parameter_t->grad tensor.
-------------------------	---

C — Quantization and Fixed-Point Training (FQT)

Term	Definition
FQT (Fixed-point Quantization Training)	Training directly with quantized weights (integers with a scale factor) instead of full float32. Deutel-style: maintain SYM_INT32 weights, compute forward/backward in quantized form. Benefit: smaller model, faster inference on MCU without float hardware.
INT32	A 32-bit signed integer. Range -2^{31} to $2^{31}-1$. Used as the underlying type for SYM_INT32 mantissas in ODT.
int64_t accumulator	In SYM_INT32 matmul, $\text{int32} * \text{int32}$ can produce up to 2^{62} . To avoid overflow during accumulation, the running sum is stored in int64_t. The final result is scaled back to int32 at the end.
mantissa	The integer part of a SYM_INT32 value. $\text{real_value} = \text{mantissa} * \text{scale}$. Example: mantissa=-1503, scale=0.001 -> real_value=-1.503. Stored in tensor data as int32_t bytes.
qBits	The quantization bit width for storage/deployment. qBits=8 means weights are stored as int8 values (range -127 to +127). Determines how many distinct values can be represented.

qMax	The maximum representable quantized value: $2^{(qMaxBits-1)} - 1$. qMaxBits=16 -> qMax=32767. Weights are clamped to [-qMax, +qMax] during training.
qMaxBits	The bit width used during training for the RANGE check. Default: 16 bits (qMax=32767). Why 16: $32767 * 32767 = 1.07 \text{ billion} < \text{int32_t max (2.1 billion)}$. Products of 16-bit mantissas fit safely in a 32-bit accumulator without overflow.
quantization	The process of approximating real-valued numbers with a discrete set of values. float32 has 2^{32} possible values. int8 has only 256. The trade-off: less memory and faster arithmetic vs slightly lower accuracy.
requantization	Rescaling a quantized tensor after arithmetic operations. After matmul, the accumulated sum may have a different scale. Requantization adjusts scale and clamps to valid range. ODT: <code>quantizeSymInt32()</code> performs this.
rounding mode	How to round a real value to the nearest integer. Options: <code>ROUND_HALF_EVEN</code> (banker's rounding), <code>ROUND_HALF_AWAY</code> (round 0.5 up), <code>SR_HALF_AWAY</code> (stochastic rounding). Stochastic rounding is essential for FQT convergence.
scale	Multiplier that converts a quantized integer to a real value. $\text{real} = \text{mantissa} * \text{scale}$. Scale is learned/set per tensor. Smaller scale = finer resolution.

stochastic rounding (SR_HALF_AWAY)	Add uniform random noise in [0, 0.5) before rounding. Expected value of rounded result = true real value. Preserves small gradient updates that would otherwise round to zero. Critical for FQT: without SR, small gradients have no effect on quantized weights.
symmetric quantization (SYM_INT32)	Quantization where the zero point is always 0. Range is symmetric around 0: [-qMax, +qMax]. Simpler than asymmetric (no zero-point offset needed in matmul).
type-punning	Reading the same bytes as a different type. E.g., reading 4 bytes of a float as a uint32_t to examine its bits. In C, only safe via memcpy (direct casts via pointer violate strict aliasing). ODT uses readBytesAsFloat/writeFloatToByteArray for safe type-punning.

D — RigL and Sparse Training

Term	Definition
active weight	A weight where mask=1. Participates in forward pass computation. Updated by the optimizer. Currently non-zero (or allowed to be non-zero).
dynamic sparsity	Sparse training where the PATTERN of zero weights changes during training. RigL is dynamic sparse. Opposite: static sparsity (mask fixed after init).
Evci et al. 2020	The paper 'Rigging the Lottery: Making All Tickets Winners' by Utku Evci et al., NeurIPS 2020. Introduced the RigL algorithm. Available at: https://arxiv.org/abs/1911.11134

grow step	The second half of a RigL topology update. Selects inactive weights with the highest gradient magnitude and reactivates them. Newly grown weights start with value 0.0 and grow from future gradient steps.
lottery ticket hypothesis	The observation (Frankle & Carlin, 2019) that dense networks contain sparse subnetworks ('winning tickets') that can train to full accuracy. RigL's title 'Rigging the Lottery' refers to finding these tickets during training.
mask	A BOOL tensor (one bit per weight) indicating which weights are active (1) or pruned (0). <code>riglConfig_t->mask</code> in ODT.
masked matmul	Matrix multiplication that skips positions where <code>mask=0</code> . <code>riglMaskedMatmulFloat32()</code> in <code>RigL.c</code> . Reduces FLOPS proportionally to sparsity ratio.
prune step	The first half of a RigL topology update. Deactivates the active weights with the smallest absolute value. Sets their <code>mask=0</code> and <code>weight value=0</code> .
pruned weight	A weight where <code>mask=0</code> . Excluded from forward pass. Not updated by optimizer. Set to 0.0. Its gradient is still computed (needed for grow step).
rejection sampling	A technique to sample from a distribution by generating candidates and discarding those outside the target domain. Used in <code>riglInitMask</code> to ensure unique pruned positions.

RigL (Rigging the Lottery)	Sparse training algorithm: maintain fixed sparsity ratio, periodically update which weights are zero by pruning small weights and growing high-gradient weights.
sparsity pattern	Which specific weight positions are zero at a given point in training. In RigL, the pattern evolves every T batches while sparsity ratio stays constant.
sparsity ratio	Fraction of weights that are zero (pruned). sparsityRatio=0.8 means 80% of weights are zero. Also called 'density' (1-sparsityRatio) or 'fraction of non-zeros.'
static sparsity	Sparse model where the zero positions are fixed (set once and never changed). Simpler than RigL but typically lower accuracy for a given sparsity ratio.
T (update interval)	Number of training batches between consecutive RigL topology updates. <code>riglC onfig_t->updateStep</code> . Typical value: 100. RigL in the paper uses a cosine decay schedule for T.
topology	The structure of the sparse network — which weights are active. RigL updates the topology every T batches by prune+grow.

E — Embedded Systems and STM32

Term	Definition
------	------------

ARM Cortex-M7	A 32-bit RISC processor core used in the STM32F746. Features: branch predictor, instruction cache, data cache, double-precision FPU (optional), SIMD DSP instructions. Maximum clock: 216 MHz on the F746.
bare-metal	Running code directly on hardware with no operating system. No processes, no virtual memory, no filesystem (without additional hardware). The programmer manages all resources directly.
CMSIS (Cortex Microcontroller Software Interface Standard)	A vendor-independent hardware abstraction layer for ARM Cortex-M processors. Provides standardized access to CPU registers, system timer (SysTick), and NVIC.
flash memory	Non-volatile (persistent) storage on the STM32. Holds the program code (.text), read-only constants (.rodata), and initialized variable values (.data init copy). STM32F746: 1 MB flash.
FPU (Floating Point Unit)	Hardware that executes floating-point operations (add, mul, div) without software emulation. Cortex-M7 has a full double-precision FPU. Single-precision FMA executes in 1 clock cycle.
HAL (Hardware Abstraction Layer)	STM32CubeHAL is ST's library of functions for configuring peripherals (UART, GPIO, I2C, SPI, etc.) without writing register-level code. ODT uses HAL_UART for transmitting trained weights to the PC.

hard fault	An unrecoverable processor exception on ARM. Causes: NULL pointer dereference, stack overflow, unaligned memory access, divide by zero (in integer code). On bare-metal with no handler: the MCU freezes or resets.
LibriSpeech	A corpus of 1000 hours of English audiobook speech from LibriVox. The thesis uses a subset: 2484 speakers. Audio features (MFCCs or spectrograms) are extracted on PC, stored as .npy files.
MFCC (Mel-Frequency Cepstral Coefficients)	Audio features commonly used for speech tasks. Computed by: FFT -> mel filterbank -> log -> DCT. Produces a compact representation of the vocal tract shape. A 128-dimensional MFCC vector is a typical input feature for speaker ID.
Nucleo-F746ZG	An STM32 development board with: STM32F746ZG MCU, USB, UART, LEDs, buttons, Arduino headers. The target hardware for the thesis experiments.
NVIC (Nested Vectored Interrupt Controller)	ARM hardware that manages interrupts. In ODT bare-metal usage: interrupts are mostly disabled or unused.
SRAM (Static Random Access Memory)	Fast volatile (loses data on power-off) memory for runtime data storage. STM32F746: 320 KB SRAM. Holds stack, heap (if used), and all ODT tensor buffers.
STM32CubeIDE	ST Microelectronics' integrated development environment for STM32. Based on Eclipse + GCC. Used to compile, flash, and debug ODT on the board.

UART (Universal Asynchronous Receiver-Transmitter)	A serial communication protocol. Used in ODT to stream training data from PC to STM32 and to send trained model weights back to the PC for evaluation.
--	--

F — ODT Library: Key Structs and Functions

Term	Definition
calcNumberOfElementsByTensor()	Returns the total number of elements in a tensor (product of all dimensions). Shape [3,4,5] -> 60 elements.
calcNumberOfBytesForData()	Returns byte count for N elements of a given quantization type. FLOAT32: N*4. INT32: N*4. SYM_INT32: N*4. BOOL: ceil(N/8).
getDimensionsByIndex()	Returns the size of a LOGICAL dimension index, accounting for any transposes (orderOfDimensions).
layer_t	A struct representing one neural network layer. Fields: layerType, config (union of layer-specific configs), forwardInput (pointer to last input for backprop).
layerConfig_t	A union holding a pointer to the specific config for each layer type: linearConfig_t* OR reluConfig_t* OR softmaxConfig_t* OR dropoutConfig_t*.
linearConfig_t	Config for a linear (fully-connected) layer. Fields: weights (parameter_t), bias (parameter_t).
parameter_t	Bundles a weight tensor and its gradient tensor together. Fields: param (tensor_t for weights), grad (tensor_t for gradients).
quantization_t	Describes how tensor values are encoded. Fields: type (quantizationType_t enum), qConfig (void* to specific config).

readBytesAsFloat()	Safely reads 4 bytes from a uint8_t* buffer as a float via memcpy. Avoids strict aliasing undefined behavior.
setTensorValues()	Initializes all fields of a tensor_t: data, shape, quantization, sparsity.
shape_t	Describes tensor dimensions. Fields: numberOfDimensions, dimensions[], orderOfDimensions[].
symInt32QConfig_t	Configuration for SYM_INT32 quantization. Fields: scale (float), qMaxBits (uint8_t), roundingMode.
tensor_t	The central data structure: a multi-dimensional array. Fields: data (uint8_t* raw bytes), shape (shape_t*), quantization (quantization_t*), sparsity (sparsity_t* for RgL mask).
tensorBoolGet()	Reads one bit from a BOOL tensor at a given linear index.
tensorBoolSet()	Writes one bit in a BOOL tensor at a given linear index.
transposeTensor()	Swaps two dimensions in orderOfDimensions. Does NOT move data — just changes the index mapping. O(1) operation.
writeFloatToByteArray()	Safely writes a float as 4 bytes into a uint8_t* buffer via memcpy.

G — Mathematics and Algorithms

Term	Definition
Fisher-Yates shuffle	An O(N) algorithm for uniformly random permutations. For i from N-1 down to 1: swap element i with random element j in [0,i]. All N! orderings equally likely. Used in rngShuffleIndices().

IEEE 754	The standard for floating-point arithmetic. float32: 1 sign bit, 8 exponent bits, 23 mantissa bits. Defines special values: +/-inf, NaN. All modern CPUs and the STM32 FPU implement this standard.
matmul (matrix multiplication)	$\text{Output}[i][j] = \sum_k (\text{A}[i][k] * \text{B}[k][j])$. $O(M*N*K)$ multiplications. The dominant operation in neural network forward and backward passes.
modulo operator (%)	$a \% b = \text{remainder when } a \text{ is divided by } b$. $5 \% 3 = 2$. Used for wrap-around counters (batchCount % updateStep).
numerical stability	Techniques to avoid floating-point errors from large/small intermediate values. Softmax: subtract max before exp to avoid overflow. SYM_INT32 matmul: int64 accumulator to avoid int32 overflow.
qsort()	Standard C library sort. $O(N \log N)$ average. Signature: void qsort(void *base, size_t n, size_t size, int(*cmp)(void*,void*)). cmp callback returns negative/0/positive for $a < b/a = b/a > b$.
uniform distribution	All values in a range [a,b] are equally likely. rngNextFloat() returns values uniformly distributed in [0,1).
xorshift32	A fast PRNG based on XOR and bit-shift operations. Period $2^{32}-1$ (all non-zero 32-bit values). 3 operations: $x \wedge= x << 13$; $x \wedge= x >> 17$; $x \wedge= x << 5$.

■ NOTE

This glossary covers every major term in the book. If a term still seems unclear after reading its definition, check the chapter indicated in brackets, or search the ODT source code for the struct or function name — the code itself is always the ground truth.